TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

ĐẠI HỌC QUỐC GIA TP.HCM

BÁO CÁO BÀI TẬP TUẦN

Môn học: Kỹ thuật lập trình (MTH10107)

Giảng viên hướng dẫn: Phạm Thi Vương

Nhóm sinh viên thực hiện:

Phạm Trần Tấn Phát	20280070
Nguyễn Thị Ngọc Anh	23280037
Ngô Văn Phước	24280041

TP.HCM, 2025

Mục lục

1 Tuần 1: File I/O + String cơ bản + Pipeline M0 11

1.1 Đọc và đếm dòng file [Cơ bản] . . . 11
1.1.1 Phân tích & Thiết kế (M0) . . . 11
1.1.2 Mã nguồn C++11 (Tách module) . . . 11
1.1.3 Kế hoạch kiểm thử (Test Cases) . . . 13
1.2 Tách trường CSV 1 hàng [Cơ bản] . . . 13
1.2.1 Phân tích & Thiết kế (M0) . . . 13
1.2.2 Mã nguồn C++11 (Tách module) . . . 14
1.2.3 Kế hoạch kiểm thử (Test Cases) . . . 15
1.3 Bài 3. Phát hiện missing token [Cơ bản] . . . 16
1.3.1 Phân tích & Thiết kế (M0) . . . 16
1.3.2 Mã nguồn C++11 (Tách module) . . . 17
1.3.3 Kế hoạch kiểm thử (Test Cases) . . . 19
1.4 Thống kê cột số từ CSV [Trung bình] . . . 20
1.4.1 Phân tích & Thiết kế (M0) . . . 20
1.4.2 Mã nguồn C++11 (Tách module) . . . 21
1.4.3 Kế hoạch kiểm thử (Test Cases) . . . 24
1.5 Ghi report CSV nhiều cột [Trung bình] . . . 24
1.5.1 Phân tích & Thiết kế (M0) . . . 24
1.5.2 Mã nguồn C++11 (Tách module) . . . 26
1.5.3 Kế hoạch kiểm thử (Test Cases) . . . 30
1.6 Đếm từ trong file text [Cơ bản] . . . 31
1.6.1 Phân tích & Thiết kế (M0) . . . 31
1.6.2 Mã nguồn C++11 (Tách module) . . . 31
1.6.3 Kế hoạch kiểm thử (Test Cases) . . . 33
1.7 Kiểm tra file CSV có header [Cơ bản] . . . 34
1.7.1 Phân tích & Thiết kế (M0) . . . 34
1.7.2 Mã nguồn C++11 (Tách module) . . . 35
1.7.3 Kế hoạch kiểm thử (Test Cases) . . . 37
1.8 Lọc dòng theo điều kiện [Trung bình] . . . 37
1.8.1 Phân tích & Thiết kế (M0) . . . 37
1.8.2 Mã nguồn C++11 (Tách module) . . . 38
1.8.3 Kế hoạch kiểm thử (Test Cases) . . . 41
1.9 Merge 2 file CSV theo id [Trung bình] . . . 42
1.9.1 Phân tích & Thiết kế (M0) . . . 42
1.9.2 Mã nguồn C++11 (Tách module) . . . 44
1.9.3 Kế hoạch kiểm thử (Test Cases) . . . 47

1.10 Phân tích log file [Nâng cao] 49
1.10.1 Phân tích & Thiết kế (M0) 49
1.10.2 Mã nguồn C++11 (Tách module) 50
1.10.3 Kế hoạch kiểm thử (Test Cases) 52

2 Tuần 2: Mảng 1D + Đa thức + Thống kê mô tả 53
2.1 Tìm min, max, mean mảng [Cơ bản] 53
2.1.1 Phân tích & Thiết kế (M0) 53
2.1.2 Mã nguồn C++11 (Tách module) 53
2.1.3 Kế hoạch kiểm thử (Test Cases) 55
2.2 Đếm phần tử thỏa điều kiện [Cơ bản] 56
2.2.1 Phân tích & Thiết kế (M0) 56
2.2.2 Mã nguồn C++11 (Tách module) 57
2.2.3 Kế hoạch kiểm thử (Test Cases) 59
2.3 Tính đa thức tại nhiều điểm [Cơ bản] 59
2.3.1 Phân tích & Thiết kế (M0) 59
2.3.2 Mã nguồn C++11 (Tách module) 60
2.3.3 Kế hoạch kiểm thử (Test Cases) 63
2.4 Variance và std deviation [Trung bình] 64
2.4.1 Phân tích & Thiết kế (M0) 64
2.4.2 Mã nguồn C++11 (Tách module) 65
2.4.3 Kế hoạch kiểm thử (Test Cases) 67
2.5 Đạo hàm đa thức [Trung bình] 67
2.5.1 Phân tích & Thiết kế (M0) 67
2.5.2 Mã nguồn C++11 (Tách module) 68
2.5.3 Kế hoạch kiểm thử (Test Cases) 71
2.6 Tìm vị trí phần tử lớn nhất [Cơ bản] 72
2.6.1 Phân tích & Thiết kế (M0) 72
2.6.2 Mã nguồn C++11 (Tách module) 73
2.6.3 Kế hoạch kiểm thử (Test Cases) 74
2.7 Bài 7: Chuẩn hoá mảng Min-Max [Cơ bản] 75
2.7.1 Phân tích & Thiết kế (M0) 75
2.7.2 Mã nguồn C++11 (Tách module) 76
2.7.3 Kế hoạch kiểm thử (Test Cases) 77
2.8 Impute missing bằng median [Trung bình] 78
2.8.1 Phân tích & Thiết kế (M0) 78
2.8.2 Mã nguồn C++11 (Tách module) 79
2.8.3 Kế hoạch kiểm thử (Test Cases) 82
2.9 Histogram đơn giản [Trung bình] 83
2.9.1 Phân tích & Thiết kế (M0) 83
2.9.2 Mã nguồn C++11 (Tách module) 84
2.9.3 Kế hoạch kiểm thử (Test Cases) 86
2.10 Nội suy tuyến tính [Nâng cao] 87
2.10.1 Phân tích & Thiết kế (M0) 87
2.10.2 Mã nguồn C++11 (Tách module) 88
2.10.3 Kế hoạch kiểm thử (Test Cases) 91

3 Tuần 3: Prefix Sum 1D + Tối ưu truy vấn 93
3.1 Xây dựng prefix sum [Cơ bản] . . . 93
3.1.1 Phân tích & Thiết kế (M0) . . . 93
3.1.2 Mã nguồn C++11 (Tách module) . . . 93
3.1.3 Kế hoạch kiểm thử (Test Cases) . . . 94
3.2 Trả lời truy vấn tổng đoạn [Cơ bản] . . . 95
3.2.1 Phân tích & Thiết kế (M0) . . . 95
3.2.2 Mã nguồn C++11 (Tách module) . . . 96
3.2.3 Kế hoạch kiểm thử (Test Cases) . . . 96
3.3 Đếm phần tử không âm trong đoạn [Cơ bản] . . . 97
3.3.1 Phân tích & Thiết kế (M0) . . . 97
3.3.2 Mã nguồn C++11 (Tách module) . . . 98
3.3.3 Kế hoạch kiểm thử (Test Cases) . . . 98
3.4 Tổng đoạn trung bình [Trung bình] . . . 99
3.4.1 Phân tích & Thiết kế (M0) . . . 99
3.4.2 Mã nguồn C++11 (Tách module) . . . 100
3.4.3 Kế hoạch kiểm thử (Test Cases) . . . 100
3.5 Đoạn có tổng bằng K [Trung bình] . . . 101
3.5.1 Phân tích & Thiết kế (M0) . . . 101
3.5.2 Mã nguồn C++11 (Tách module) . . . 102
3.5.3 Kế hoạch kiểm thử (Test Cases) . . . 103
3.6 Phần tử xuất hiện nhiều nhất trong đoạn [Trung bình] . . . 103
3.6.1 Phân tích & Thiết kế (M0) . . . 103
3.6.2 Mã nguồn C++11 (Tách module) . . . 104
3.6.3 Kế hoạch kiểm thử (Test Cases) . . . 105
3.7 Tổng đoạn lớn nhất (Prefix) [Trung bình] . . . 105
3.7.1 Phân tích & Thiết kế (M0) . . . 105
3.7.2 Mã nguồn C++11 (Tách module) . . . 106
3.7.3 Kế hoạch kiểm thử (Test Cases) . . . 107
3.8 Prefix product modulo M [Trung bình] . . . 107
3.8.1 Phân tích & Thiết kế (M0) . . . 107
3.8.2 Mã nguồn C++11 (Tách module) . . . 108
3.8.3 Kế hoạch kiểm thử (Test Cases) . . . 109
3.9 Sliding prefix: cập nhật 1 điểm [Nâng cao] . . . 110
3.9.1 Phân tích & Thiết kế (M0) . . . 110
3.9.2 Mã nguồn C++11 (Tách module) . . . 111
3.9.3 Kế hoạch kiểm thử (Test Cases) . . . 111
3.10 Đoạn đẹp nhất theo nhiều tiêu chí [Nâng cao] . . . 112
3.10.1 Phân tích & Thiết kế (M0) . . . 112
3.10.2 Mã nguồn C++11 (Tách module) . . . 113
3.10.3 Kế hoạch kiểm thử (Test Cases) . . . 114

4 Tuần 4: Kadane + LIS O(n²) + Tư duy dãy con 115
4.1 Max subarray sum (Kadane cơ bản) [Cơ bản] . . . 115
4.1.1 Phân tích & Thiết kế (M0) . . . 115
4.1.2 Mã nguồn C++11 (Tách module) . . . 115

4.1.3 Kế hoạch kiểm thử (Test Cases) 116
4.2 Kadane có truy vết l, r [Cơ bản] 117
4.2.1 Phân tích & Thiết kế (M0) 117
4.2.2 Mã nguồn C++11 (Tách module) 118
4.2.3 Kế hoạch kiểm thử (Test Cases) 119
4.3 LIS độ dài O(n^2) [Cơ bản] 119
4.3.1 Phân tích & Thiết kế (M0) 119
4.3.2 Mã nguồn C++11 (Tách module) 120
4.3.3 Kế hoạch kiểm thử (Test Cases) 121
4.4 Min subarray sum [Trung bình] 121
4.4.1 Phân tích & Thiết kế (M0) 121
4.4.2 Mã nguồn C++11 (Tách module) 122
4.4.3 Kế hoạch kiểm thử (Test Cases) 123
4.5 LIS truy vết dãy con [Trung bình] 124
4.5.1 Phân tích & Thiết kế (M0) 124
4.5.2 Mã nguồn C++11 (Tách module) 124
4.5.3 Kế hoạch kiểm thử (Test Cases) 125
4.6 Đoạn có tổng lớn nhất độ dài đúng k [Trung bình] 126
4.6.1 Phân tích & Thiết kế (M0) 126
4.6.2 Mã nguồn C++11 (Tách module) 127
4.6.3 Kế hoạch kiểm thử (Test Cases) 127
4.7 Kadane với nhiều bộ test (Prefix) [Trung bình] 128
4.7.1 Phân tích & Thiết kế (M0) 128
4.7.2 Mã nguồn C++11 (Tách module) 129
4.7.3 Kế hoạch kiểm thử (Test Cases) 129
4.8 Longest Non-Decreasing Subsequence [Trung bình] 130
4.8.1 Phân tích & Thiết kế (M0) 130
4.8.2 Mã nguồn C++11 (Tách module) 131
4.8.3 Kế hoạch kiểm thử (Test Cases) 132
4.9 Maximum circular subarray [Nâng cao] 132
4.9.1 Phân tích & Thiết kế (M0) 132
4.9.2 Mã nguồn C++11 (Tách module) 133
4.9.3 Kế hoạch kiểm thử (Test Cases) 134
4.10 Đếm số dãy con tăng có độ dài = L [Nâng cao] 134
4.10.1 Phân tích & Thiết kế (M0) 134
4.10.2 Mã nguồn C++11 (Tách module) 135
4.10.3 Kế hoạch kiểm thử (Test Cases) 136

5 Tuần 5: Two Pointers / Sliding Window + Rolling Mean 138
5.1 Đoạn ngắn nhất tổng $\geq S$ [Cơ bản] 138
5.1.1 Phân tích & Thiết kế (M0) 138
5.1.2 Mã nguồn C++11 (Tách module) 139
5.1.3 Kế hoạch kiểm thử (Test Cases) 140
5.2 Rolling mean cửa sổ k [Cơ bản] 141
5.2.1 Phân tích & Thiết kế (M0) 141
5.2.2 Mã nguồn C++11 (Tách module) 142

5.2.3 Kế hoạch kiểm thử (Test Cases) 143
5.3 Đoạn dài nhất không vượt quá S [Cơ bản] 144
5.3.1 Phân tích & Thiết kế (M0) 144
5.3.2 Mã nguồn C++11 (Tách module) 145
5.3.3 Kế hoạch kiểm thử (Test Cases) 147
5.4 Rolling max cửa sổ k [Trung bình] 147
5.4.1 Phân tích & Thiết kế (M0) 147
5.4.2 Mã nguồn C++11 (Tách module) 148
5.4.3 Kế hoạch kiểm thử (Test Cases) 150
5.5 Hai con trỏ: tổng 2 phần tử bằng K [Trung bình] 151
5.5.1 Phân tích & Thiết kế (M0) 151
5.5.2 Mã nguồn C++11 (Tách module) 152
5.5.3 Kế hoạch kiểm thử (Test Cases) 153
5.6 Rolling standard deviation [Trung bình] 154
5.6.1 Phân tích & Thiết kế (M0) 154
5.6.2 Mã nguồn C++11 (Tách module) 155
5.6.3 Kế hoạch kiểm thử (Test Cases) 157
5.7 Đoạn không âm dài nhất [Cơ bản] 158
5.7.1 Phân tích & Thiết kế (M0) 158
5.7.2 Mã nguồn C++11 (Tách module) 158
5.7.3 Kế hoạch kiểm thử (Test Cases) 160
5.8 Cặp tổng gần K nhất [Trung bình] 161
5.8.1 Phân tích & Thiết kế (M0) 161
5.8.2 Mã nguồn C++11 (Tách module) 162
5.8.3 Kế hoạch kiểm thử (Test Cases) 163
5.9 Dãy không giảm dài nhất liên tiếp [Cơ bản] 164
5.9.1 Phân tích & Thiết kế (M0) 164
5.9.2 Mã nguồn C++11 (Tách module) 165
5.9.3 Kế hoạch kiểm thử (Test Cases) 166
5.10 Smallest window chứa tất cả K giá trị phân biệt [Nâng cao] 167
5.10.1 Phân tích & Thiết kế (M0) 167
5.10.2 Mã nguồn C++11 (Tách module) 168
5.10.3 Kế hoạch kiểm thử (Test Cases) 170

6 Tuần 6: Ma trận 2D + Prefix Sum 2D 171
6.1 Tổng hàng và cột ma trận [Cơ bản] 171
6.1.1 Phân tích & Thiết kế (M0) 171
6.1.2 Mã nguồn C++11 (Tách module) 172
6.1.3 Kế hoạch kiểm thử (Test Cases) 173
6.2 Kiểm tra ma trận đối xứng [Cơ bản] 174
6.2.1 Phân tích & Thiết kế (M0) 174
6.2.2 Mã nguồn C++11 (Tách module) 175
6.2.3 Kế hoạch kiểm thử (Test Cases) 177
6.3 Xây dựng prefix sum 2D [Cơ bản] 177
6.3.1 Phân tích & Thiết kế (M0) 177
6.3.2 Mã nguồn C++11 (Tách module) 178

6.3.3 Kế hoạch kiểm thử (Test Cases) 180
6.4 Trả lời truy vấn tổng vùng [Trung bình] 180
6.4.1 Phân tích & Thiết kế (M0) 180
6.4.2 Mã nguồn C++11 (Tách module) 181
6.4.3 Kế hoạch kiểm thử (Test Cases) 183
6.5 Ma trận đường chéo [Trung bình] 184
6.5.1 Phân tích & Thiết kế (M0) 184
6.5.2 Mã nguồn C++11 (Tách module) 185
6.5.3 Kế hoạch kiểm thử (Test Cases) 187
6.6 Nhân hai ma trận [Trung bình] 187
6.6.1 Phân tích & Thiết kế (M0) 187
6.6.2 Mã nguồn C++11 (Tách module) 188
6.6.3 Kế hoạch kiểm thử (Test Cases) 190
6.7 Vùng con tổng lớn nhất (brute force) [Trung bình] 191
6.7.1 Phân tích & Thiết kế (M0) 191
6.7.2 Mã nguồn C++11 (Tách module) 191
6.7.3 Kế hoạch kiểm thử (Test Cases) 193
6.8 Kiểm tra ma trận tam giác [Trung bình] 194
6.8.1 Phân tích & Thiết kế (M0) 194
6.8.2 Mã nguồn C++11 (Tách module) 195
6.8.3 Kế hoạch kiểm thử (Test Cases) 196
6.9 Xoay ma trận 90 độ theo chiều kim đồng hồ [Trung bình] 197
6.9.1 Phân tích & Thiết kế (M0) 197
6.9.2 Mã nguồn C++11 (Tách module) 197
6.9.3 Kế hoạch kiểm thử (Test Cases) 199
6.10 Đếm vùng con tổng bằng K [Nâng cao] 200
6.10.1 Phân tích & Thiết kế (M0) 200
6.10.2 Mã nguồn C++11 (Tách module) 201
6.10.3 Kế hoạch kiểm thử (Test Cases) 202

7 Tuần 7: Struct Toán: Phân số (Fraction) 204
7.1 Chuẩn hoá phân số [Cơ bản] 204
7.1.1 Phân tích & Thiết kế (M0) 204
7.1.2 Mã nguồn C++11 (Tách module) 205
7.1.3 Kế hoạch kiểm thử (Test Cases) 207
7.2 Cộng và trừ 2 phân số [Cơ bản] 208
7.2.1 Phân tích & Thiết kế (M0) 208
7.2.2 Mã nguồn C++11 (Tách module) 209
7.2.3 Kế hoạch kiểm thử (Test Cases) 211
7.3 Sắp xếp danh sách phân số [Cơ bản] 212
7.3.1 Phân tích & Thiết kế (M0) 212
7.3.2 Mã nguồn C++11 (Tách module) 213
7.3.3 Kế hoạch kiểm thử (Test Cases) 215
7.4 Tổng tất cả phân số hợp lệ [Trung bình] 216
7.4.1 Phân tích & Thiết kế (M0) 216
7.4.2 Mã nguồn C++11 (Tách module) 217

7.4.3 Kế hoạch kiểm thử (Test Cases) 219
7.5 Lọc phân số trong khoảng [a, b][Trung bình] 220
7.5.1 Phân tích & Thiết kế (M0) 220
7.5.2 Mã nguồn C++11 (Tách module) 221
7.5.3 Kế hoạch kiểm thử (Test Cases) 225
7.6 Nhân và chia 2 phân số [Cơ bản] 226
7.6.1 Phân tích & Thiết kế (M0) 226
7.6.2 Mã nguồn C++11 (Tách module) 227
7.6.3 Kế hoạch kiểm thử (Test Cases) 229
7.7 Phân số lớn nhất và nhỏ nhất [Cơ bản] 230
7.7.1 Phân tích & Thiết kế (M0) 230
7.7.2 Mã nguồn C++11 (Tách module) 231
7.7.3 Kế hoạch kiểm thử (Test Cases) 234
7.8 Trung bình các phân số [Trung bình] 234
7.8.1 Phân tích & Thiết kế (M0) 234
7.8.2 Mã nguồn C++11 (Tách module) 235
7.8.3 Kế hoạch kiểm thử (Test Cases) 238
7.9 Kiểm tra phân số là số nguyên [Trung bình] 238
7.9.1 Phân tích & Thiết kế (M0) 238
7.9.2 Mã nguồn C++11 (Tách module) 239
7.9.3 Kế hoạch kiểm thử (Test Cases) 242
7.10 Dãy Farey bậc N [Nâng cao] 243
7.10.1 Phân tích & Thiết kế (M0) 243
7.10.2 Mã nguồn C++11 (Tách module) 244
7.10.3 Kế hoạch kiểm thử (Test Cases) 246

8 Tuần 8: File + Struct: Đa thức Record + Ma trận Phân số 247
8.1 Đọc danh sách đa thức từ CSV [Cơ bản] 247
8.1.1 Phân tích & Thiết kế (M0) 247
8.1.2 Mã nguồn C++11 (Tách module) 248
8.1.3 Kế hoạch kiểm thử (Test Cases) 250
8.2 Tính P(x) và ghi report [Cơ bản] 251
8.2.1 Phân tích & Thiết kế (M0) 251
8.2.2 Mã nguồn C++11 (Tách module) 252
8.2.3 Kế hoạch kiểm thử (Test Cases) 255
8.3 Thống kê hệ số đa thức [Cơ bản] 256
8.3.1 Phân tích & Thiết kế (M0) 256
8.3.2 Mã nguồn C++11 (Tách module) 257
8.3.3 Kế hoạch kiểm thử (Test Cases) 259
8.4 Cộng hai đa thức [Trung bình] 260
8.4.1 Phân tích & Thiết kế (M0) 260
8.4.2 Mã nguồn C++11 (Tách module) 261
8.4.3 Kế hoạch kiểm thử (Test Cases) 263
8.5 Đạo hàm và đánh giá đa thức [Cơ bản] 264
8.5.1 Phân tích & Thiết kế (M0) 264
8.5.2 Mã nguồn C++11 (Tách module) 265

8.5.3 Kế hoạch kiểm thử (Test Cases) 268
8.6 Đạo hàm và đánh giá đa thức [Cơ bản] 269
8.6.1 Phân tích & Thiết kế (M0) 269
8.6.2 Mã nguồn C++11 (Tách module) 270
8.6.3 Kế hoạch kiểm thử (Test Cases) 273
8.7 Tìm nghiệm nguyên của đa thức [Trung bình] 274
8.7.1 Phân tích & Thiết kế (M0) 274
8.7.2 Mã nguồn C++11 (Tách module) 275
8.7.3 Kế hoạch kiểm thử (Test Cases) 278
8.8 Ma trận đơn vị phân số [Trung bình] 278
8.8.1 Phân tích & Thiết kế (M0) 278
8.8.2 Mã nguồn C++11 (Tách module) 279
8.8.3 Kế hoạch kiểm thử (Test Cases) 283
8.9 Đa thức Lagrange nội suy [Nâng cao] 284
8.9.1 Phân tích & Thiết kế (M0) 284
8.9.2 Mã nguồn C++11 (Tách module) 285
8.9.3 Kế hoạch kiểm thử (Test Cases) 287
8.10 Phân tích bình phương tổng [Nâng cao] 288
8.10.1 Phân tích & Thiết kế (M0) 288
8.10.2 Mã nguồn C++11 (Tách module) 290
8.10.3 Kế hoạch kiểm thử (Test Cases) 294

9 Tuần 9: Chuỗi nâng cao: Normalize, Tokenize, Frequency 295
9.1 Normalize chuỗi ASCII [Cơ bản] 295
9.1.1 Phân tích & Thiết kế (M0) 295
9.1.2 Mã nguồn C++11 (Tách module) 296
9.1.3 Kế hoạch kiểm thử (Test Cases) 298
9.2 Tokenize và đếm từ [Cơ bản] 298
9.2.1 Phân tích & Thiết kế (M0) 298
9.2.2 Mã nguồn C++11 (Tách module) 299
9.2.3 Kế hoạch kiểm thử (Test Cases) 302
9.3 Top-K từ xuất hiện nhiều nhất [Cơ bản] 302
9.3.1 Phân tích & Thiết kế (M0) 302
9.3.2 Mã nguồn C++11 (Tách module) 304
9.3.3 Kế hoạch kiểm thử (Test Cases) 307
9.4 Lọc stop-words [Trung bình] 308
9.4.1 Phân tích & Thiết kế (M0) 308
9.4.2 Mã nguồn C++11 (Tách module) 310
9.4.3 Kế hoạch kiểm thử (Test Cases) 313
9.5 So sánh 2 file văn bản [Trung bình] 315
9.5.1 Phân tích & Thiết kế (M0) 315
9.5.2 Mã nguồn C++11 (Tách module) 316
9.5.3 Kế hoạch kiểm thử (Test Cases) 321
9.6 Đếm tần suất bigram [Trung bình] 322
9.6.1 Phân tích & Thiết kế (M0) 322
9.6.2 Mã nguồn C++11 (Tách module) 324

9.6.3 Kế hoạch kiểm thử (Test Cases) 327
9.7 Phát hiện từ lặp liên tiếp [Cơ bản] 328
9.7.1 Phân tích & Thiết kế (M0) 328
9.7.2 Mã nguồn C++11 (Tách module) 329
9.7.3 Kế hoạch kiểm thử (Test Cases) 333
9.8 Tần suất ký tự [Cơ bản] 333
9.8.1 Phân tích & Thiết kế (M0) 333
9.8.2 Mã nguồn C++11 (Tách module) 335
9.8.3 Kế hoạch kiểm thử (Test Cases) 336
9.9 Nhận diện ngôn ngữ đơn giản [Trung bình] 338
9.9.1 Phân tích & Thiết kế (M0) 338
9.9.2 Mã nguồn C++11 (Tách module) 339
9.9.3 Kế hoạch kiểm thử (Test Cases) 341
9.10 TF-IDF đơn giản trên tập file [Nâng cao] 342
9.10.1 Phân tích & Thiết kế (M0) 342
9.10.2 Mã nguồn C++11 (Tách module) 343
9.10.3 Kế hoạch kiểm thử (Test Cases) 349

10 Tuần 10: Tổng hợp M0: Mini-project + Module hoá 351
10.1 Pipeline M0 hoàn chỉnh: CSV Cleaning [Cơ bản] 351
10.1.1 Phân tích & Thiết kế (M0) 351
10.1.2 Mã nguồn C++11 (Tách module) 352
10.1.3 Kế hoạch kiểm thử (Test Cases) 356
10.2 Fraction List tổng hợp [Cơ bản] 357
10.2.1 Phân tích & Thiết kế (M0) 357
10.2.2 Mã nguồn C++11 (Tách module) 359
10.2.3 Kế hoạch kiểm thử (Test Cases) 362
10.3 Polynomial records + evaluate [Cơ bản] 363
10.3.1 Phân tích & Thiết kế (M0) 363
10.3.2 Mã nguồn C++11 (Tách module) 365
10.3.3 Kế hoạch kiểm thử (Test Cases) 368
10.4 Multi-query prefix sum tổng hợp [Trung bình] 370
10.4.1 Phân tích & Thiết kế (M0) 370
10.4.2 Mã nguồn C++11 (Tách module) 371
10.4.3 Kế hoạch kiểm thử (Test Cases) 376
10.5 Text analysis pipeline [Trung bình] 377
10.5.1 Phân tích & Thiết kế (M0) 377
10.5.2 Mã nguồn C++11 (Tách module) 379
10.5.3 Kế hoạch kiểm thử (Test Cases) 384
10.6 Dashboard tổng hợp 1 file [Trung bình] 385
10.6.1 Phân tích & Thiết kế (M0) 385
10.6.2 Mã nguồn C++11 (Tách module) 386
10.6.3 Kế hoạch kiểm thử (Test Cases) 392
10.7 Sliding window multi-stat [Trung bình] 393
10.7.1 Phân tích & Thiết kế (M0) 393
10.7.2 Mã nguồn C++11 (Tách module) 394

10.7.3 Kế hoạch kiểm thử (Test Cases) 398
10.8 Sort + search tổng hợp [Trung bình] 399
10.8.1 Phân tích & Thiết kế (M0) 399
10.8.2 Mã nguồn C++11 (Tách module) 400
10.8.3 Kế hoạch kiểm thử (Test Cases) 404
10.9 Phát hiện outlier IQR [Nâng cao] 405
10.9.1 Phân tích & Thiết kế (M0) 405
10.9.2 Mã nguồn C++11 (Tách module) 406
10.9.3 Kế hoạch kiểm thử (Test Cases) 410
10.10 Mini-project tự do [Nâng cao] 411
10.10.1 Chọn hướng C (Text Processing Pipeline) 411
10.10.2 Mục tiêu project 411
10.10.3 Checklist M0 412
10.10.4 Cấu trúc project 412
10.10.5 Phân tích & Thiết kế 413
10.10.6 Module 1 – io.h 414
10.10.7 Module 2 – io.cpp 414
10.10.8 Module 3 – text_processing.h 414
10.10.9 Module 4 – text_processing.cpp 415
10.10.10 Module 5 – tfidf.h 415
10.10.11 Module 6 – tfidf.cpp 416
10.10.12 Module 7 – similarity.h 417
10.10.13 Module 8 – similarity.cpp 418
10.10.14 Module 9 – main.cpp 419
10.10.15 README 420
10.10.16 Kế hoạch kiểm thử 421

1 Tuần 1: File I/O + String cơ bản + Pipeline M0

1.1 Đọc và đếm dòng file [Cơ bản]

1.1.1 Phân tích & Thiết kế (M0)

- **Input**: File text.txt chứa nhiều dòng (có thể có dòng rỗng, dòng chỉ chứa khoảng trắng)
- **Xử lý**:
  - **Case biên:** File rỗng → `non_empty = 0`, `empty = 0`
  - **Module 1 (Đọc file):**
    - Hàm `readLines(path, vector<string>&)`
    - Đọc từng dòng bằng `getline`
  - **Module 2 (Xử lý):**
    - Trim mỗi dòng (bỏ khoảng trắng đầu/cuối)
    - Nếu sau trim: rỗng → `empty++`; ngược lại → `non_empty++`
  - **Module 3 (Ghi file):** Hàm `writeOutput(path, non_empty, empty)`
- **Output:**

```
non_empty=<n> / empty=<m>
```

1.1.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <algorithm>

using namespace std;

// MODULE: Trim string
string trim(const string& s) {
```

```
        size_t start = s.find_first_not_of(" \t\r\n");
        size_t end = s.find_last_not_of(" \t\r\n");

        if (start == string::npos) return "";
        return s.substr(start, end - start + 1);
    }

    // MODULE 1: Read file
    bool readLines(string path, vector<string>& lines) {
        ifstream file(path);
        if (!file.is_open()) return false;

        string line;
        while (getline(file, line)) {
            lines.push_back(line);
        }

        file.close();
        return true;
    }

    // MODULE 2: Count lines
    void countLines(const vector<string>& lines, int& non_empty, int&
    empty) {
        non_empty = 0;
        empty = 0;

        for (const string& line : lines) {
            string t = trim(line);
            if (t.empty()) empty++;
            else non_empty++;
        }
    }

    // MODULE 3: Write output
    bool writeOutput(string path, int non_empty, int empty) {
        ofstream file(path);
        if (!file.is_open()) return false;

        file << "non_empty=" << non_empty
            << " / empty=" << empty;

        file.close();
        return true;
    }

    // MAIN
    int main() {
        vector<string> lines;
```

```
        int non_empty, empty;

        if (!readLines("text.txt", lines)) {
            cerr << "Khong mo duoc file text.txt\n";
            return 1;
        }

        countLines(lines, non_empty, empty);

        if (!writeOutput("output.txt", non_empty, empty)) {
            cerr << "Khong ghi duoc file output.txt\n";
            return 1;
        }

        return 0;
    }

```

1.1.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (text.txt):

```
Hello

World
```

Expected Output:

```
non_empty=2 / empty=1
```

Test Case 2 (Case biên – file rỗng)
Input (text.txt):

```

```

Expected Output:

```
non_empty=0 / empty=0
```

1.2 Tách trường CSV 1 hàng [Cơ bản]

1.2.1 Phân tích & Thiết kế (M0)

- **Input:** 1 dòng CSV (từ stdin hoặc file), ví dụ:

```
        101, Nguyen Van A, , NA, 88.5

```

- **Xử lý:**
  - **Case biên:**
    - Dòng rỗng → 1 trường rỗng
    - Dòng chỉ chứa dấu phẩy → 2 trường rỗng
  - **Module 1 (Trim):** Xóa khoảng trắng đầu/cuối mỗi field
  - **Module 2 (Split CSV):**
    - Dùng `stringstream + getline(delimiter=',')`
    - Trả về `vector<string>`
  - **Module 3 (Output):**
    - In:
      - số trường
      - từng field trên 1 dòng dạng `'value'`
- **Output:**

```
            5
            '101'
            'Nguyen Van A'
            ''
            'NA'
            '88.5'
```

1.2.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <string>
#include <sstream>

using namespace std;

// MODULE: Trim
string trim(const string& s) {
    size_t start = s.find_first_not_of(" \t\r\n");
    size_t end = s.find_last_not_of(" \t\r\n");

    if (start == string::npos) return "";
    return s.substr(start, end - start + 1);
}

// MODULE: Split CSV
vector<string> splitCSV(const string& line) {
    vector<string> result;

    \begin{minted}{cpp}
```

```
        // Case: line is empty
        \end{minted}
        if (line.empty()) {
            result.push_back("");
            return result;
        }

        stringstream ss(line);
        string token;

        while (getline(ss, token, ',')) {
            result.push_back(trim(token));
        }

        \begin{minted}{cpp}
        // Case: line ends with a comma
        \end{minted}
        if (!line.empty() && line.back() == ',') {
            result.push_back("");
        }

        return result;
    }

    // MODULE: Output
    void printResult(const vector<string>& fields) {
        cout << fields.size() << endl;
        for (const string& f : fields) {
            cout << "'" << f << "'" << endl;
        }
    }

    // MAIN
    int main() {
        string line;
        getline(cin, line);

        vector<string> fields = splitCSV(line);

        printResult(fields);

        return 0;
    }
```

1.2.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường) Input:

```
101, KTLT, , NA, 88.5
```

Expected Output:

```
5
'101'
'KTLT'
''
'NA'
'88.5'
```

Test Case 2 (Case biên – dòng rỗng)
Input:

```

```

Expected Output:

```
1
''
```

Test Case 3 (Case biên – chỉ dấu phẩy)
Input:

```
,
```

Expected Output:

```
2
''
''
```

1.3 Bài 3. Phát hiện missing token [Cơ bản]

1.3.1 Phân tích & Thiết kế (M0)

- **Input:**
  - File `data.csv` (1 cột), mỗi dòng là một token
- **Xử lý:**
  - **Case biên:**
    * Token rỗng hoặc chỉ chứa khoảng trắng → missing
    * Token `"na"`, `"nan"`, `"n/a"` (không phân biệt hoa/thường) → missing
    * Token `"0"` → **không phải missing**
    * File rỗng → không có dòng nào bị missing
  - **Module 1 (Chuẩn hóa token):**
    * Hàm `trim(string)`: bỏ khoảng trắng đầu/cuối
    * Hàm `toLowerStr(string)`: chuyển về chữ thường

- **Module 2 (Kiểm tra missing):**
  - Hàm `isMissingToken(string t)`
  - Quy trình:
    - trim → tolower
    - nếu rỗng hoặc thuộc {`na, nan, n/a`} → true
    - ngược lại → false
- **Module 3 (Đọc file & phát hiện):**
  - Hàm `findMissingLines(path, vector<int>&)`
  - Đọc từng dòng bằng `getline`
  - Nếu dòng là missing → lưu chỉ số dòng (0-based)
- **Module 4 (Ghi output):**
  - Hàm `writeOutput(path, indices)`
  - In danh sách chỉ số dòng bị missing

- **Output:**

  Danh sách chỉ số dòng (0-based) bị missing, phân tách bằng khoảng trắng

1.3.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <algorithm>

using namespace std;

// MODULE: Trim
string trim(const string& s) {
    size_t start = s.find_first_not_of(" \t\r\n");
    size_t end = s.find_last_not_of(" \t\r\n");

    if (start == string::npos) return "";
    return s.substr(start, end - start + 1);
}

// MODULE: To lower
string toLowerStr(string s) {
    for (char& c : s) {
        c = (char)tolower((unsigned char)c);
    }
    return s;
}

```

```
// MODULE 1: Check missing token
bool isMissingToken(string t) {
    t = trim(t);
    t = toLowerStr(t);

    if (t.empty()) return true;
    if (t == "na" || t == "nan" || t == "n/a") return true;

    return false;
}

// MODULE 2: Read file & detect missing lines
bool findMissingLines(string path, vector<int>& indices) {
    ifstream file(path);
    if (!file.is_open()) return false;

    string line;
    int idx = 0;

    while (getline(file, line)) {
        if (isMissingToken(line)) {
            indices.push_back(idx);
        }
        idx++;
    }

    file.close();
    return true;
}

// MODULE 3: Write output
bool writeOutput(string path, const vector<int>& indices) {
    ofstream file(path);
    if (!file.is_open()) return false;

    for (size_t i = 1; i < indices.size(); i++) {
        file << indices[i]+1;
        if (i != indices.size() - 1) file << " ";
    }

    file.close();
    return true;
}

// MAIN
int main() {
    vector<int> indices;

    if (!findMissingLines("data.csv", indices)) {
```

```
            cerr << "Khong mo duoc file data.csv\n";
            return 1;
        }

        if (!writeOutput("output.txt", indices)) {
            cerr << "Khong ghi duoc file output.txt\n";
            return 1;
        }

        return 0;
    }
```

1.3.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (`data.csv`):

```
10
NA
hello
nan
N/A
0
```

Expected Output:

```
2 4 5
```

Test Case 2 (Không có missing)
Input (`data.csv`):

```
1
2
3
abc
0
```

Expected Output:

```
```

Test Case 3 (Case biên – file rỗng)
Input (`data.csv`):

```
```

Expected Output:

```
```

1.4 Thống kê cột số từ CSV [Trung bình]

1.4.1 Phân tích & Thiết kế (M0)

- **Input:**
  - File `data.csv` gồm 1 cột số thực
  - Có thể có missing; missing được hiểu là ô trống hoặc `NA`/`NaN` (không phân biệt hoa/thường)
- **Xử lý:**
  - **Case biên:**
    * Toàn bộ dòng đều missing → `count = 0`, `missing = số dòng`, và `min/max/mean = N/A`
    * File rỗng → `count = 0`, `missing = 0`, và `min/max/mean = N/A`
    * Chỉ có 1 giá trị hợp lệ ($n = 1$) → `min = max = mean = giá trị đó`
  - **Module 1 (Chuẩn hóa token):**
    * Hàm `trim(string)`: bỏ khoảng trắng đầu/cuối
    * Hàm `toLowerStr(string)`: chuyển về chữ thường
    * Hàm `isMissingToken(string)`: kiểm tra token có phải missing hay không
  - **Module 2 (Đọc CSV 1 cột):**
    * Hàm `readCSV1Col(path, vector<double>&, int& missing)`
    * Đọc từng dòng bằng `getline`
    * Nếu dòng là missing → tăng `missing`
    * Nếu parse được thành số thực → đưa vào vector hợp lệ
    * Nếu token không phải missing nhưng không parse được thành số → bỏ qua để chương trình chạy an toàn
  - **Module 3 (Tính thống kê):**
    * Hàm `computeStats(vector<double>)`
    * Tính:
      · `count`: số giá trị hợp lệ
      · `min`: giá trị nhỏ nhất
      · `max`: giá trị lớn nhất
      · `mean`: trung bình cộng
    * Dùng `long double` để tích lũy tổng, tránh mất độ chính xác
  - **Module 4 (Ghi output):**
    * Hàm `writeOutput(path, stats, missing)`
    * Nếu `count = 0` → ghi `N/A` cho `min/max/mean`

* Nếu `count > 0` → ghi số thực với `fixed << setprecision(2)`

- **Output:**

```
count=<n> / missing=<m> / min=<f> / max=<f> / mean=<f>
```

hoặc nếu không có giá trị hợp lệ:

```
count=0 / missing=<m> / min=N/A / max=N/A / mean=N/A
```

1.4.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <iomanip>

using namespace std;

struct Stats {
    int count;
    double minVal;
    double maxVal;
    double mean;
};

// MODULE: Trim
string trim(const string& s) {
    size_t start = s.find_first_not_of(" \t\r\n");
    size_t end = s.find_last_not_of(" \t\r\n");

    if (start == string::npos) return "";
    return s.substr(start, end - start + 1);
}

// MODULE: To lower
string toLowerStr(string s) {
    for (char& c : s) {
        if ('A' <= c && c <= 'Z') c = c - 'A' + 'a';
    }
    return s;
}

// MODULE: Check missing token
bool isMissingToken(string t) {
    t = toLowerStr(trim(t));
    return t.empty() || t == "na" || t == "nan" || t == "n/a";
}
```

```

// MODULE: Parse double safely
bool parseDouble(const string& s, double& value) {
    string t = trim(s);
    if (t.empty()) return false;

    try {
        size_t idx;
        value = stod(t, &idx);
        return idx == t.size();
    } catch (...) {
        return false;
    }
}

// MODULE 1: Read CSV 1 column
bool readCSV1Col(string path, vector<double>& data, int& missing) {
    ifstream file(path);
    if (!file.is_open()) return false;

    missing = 0;
    string line;

    while (getline(file, line)) {
        if (isMissingToken(line)) {
            missing++;
        } else {
            double value;
            if (parseDouble(line, value)) {
                data.push_back(value);
            }
        }
    }

    file.close();
    return true;
}

// MODULE 2: Compute stats
Stats computeStats(const vector<double>& data) {
    Stats st;
    st.count = (int)data.size();
    st.minVal = 0.0;
    st.maxVal = 0.0;
    st.mean = 0.0;

    if (data.empty()) return st;

    st.minVal = data[0];
```

```
        st.maxVal = data[0];

        long double sum = 0.0;
        for (double x : data) {
            if (x < st.minVal) st.minVal = x;
            if (x > st.maxVal) st.maxVal = x;
            sum += x;
        }

        st.mean = (double)(sum / data.size());
        return st;
    }

    // MODULE 3: Write output
    bool writeOutput(string path, const Stats& st, int missing) {
        ofstream file(path);
        if (!file.is_open()) return false;

        file << "count=" << st.count
            << " / missing=" << missing
            << " / ";

        if (st.count == 0) {
            file << "min=N/A / max=N/A / mean=N/A";
        } else {
            file << fixed << setprecision(2);
            file << "min=" << st.minVal
                << " / max=" << st.maxVal
                << " / mean=" << st.mean;
        }

        file.close();
        return true;
    }

    // MAIN
    int main() {
        vector<double> data;
        int missing = 0;

        if (!readCSV1Col("data.csv", data, missing)) {
            cerr << "Khong mo duoc file data.csv\n";
            return 1;
        }

        Stats st = computeStats(data);

        if (!writeOutput("output.txt", st, missing)) {
            cerr << "Khong ghi duoc file output.txt\n";
```

```
            return 1;
        }

        return 0;
    }
```

1.4.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (`data.csv`):

```
10
20.5
NA
30
NaN
15.5
```

Expected Output:

```
count=4 / missing=2 / min=10.00 / max=30.00 / mean=19.00
```

Test Case 2 (Case biên – toàn missing)
Input (`data.csv`):

```
NA

nan
NA
```

Expected Output:

```
count=0 / missing=4 / min=N/A / max=N/A / mean=N/A
```

Test Case 3 (Case biên – chỉ có 1 giá trị hợp lệ)
Input (`data.csv`):

```
NA
42.5
NaN
```

Expected Output:

```
count=1 / missing=2 / min=42.50 / max=42.50 / mean=42.50
```

1.5 Ghi report CSV nhiều cột [Trung bình]

1.5.1 Phân tích & Thiết kế (M0)

- **Input:**

- File `scores.csv`, mỗi dòng gồm 4 cột:

  `id, math, lit, sci`

- Các cột điểm có thể missing
- Missing được hiểu là ô trống hoặc `NA`/`NaN` (không phân biệt hoa/thường)

- **Xử lý:**
  - **Case biên:**
    * Missing môn → coi là `0` khi tính tổng, nhưng vẫn đánh dấu trong record
    * Tất cả môn đều missing → `total = 0` → xếp loại `Yeu`
    * `id` rỗng → ghi `UNKNOWN`
    * Dòng không đủ 4 cột → bỏ qua để chương trình chạy an toàn
    * File rỗng → `result.csv` chỉ có header và dòng summary với tất cả count bằng 0
  - **Module 1 (Chuẩn hóa token):**
    * Hàm `trim(string)`: bỏ khoảng trắng đầu/cuối
    * Hàm `toLowerStr(string)`: chuyển về chữ thường
    * Hàm `isMissingToken(string)`: kiểm tra missing token
  - **Module 2 (Tách & parse 1 dòng CSV):**
    * Hàm `splitCSV(line, vector<string>&)`: tách dòng theo dấu phẩy
    * Hàm `parseScore(cell, value, isMissing)`:
      · nếu missing → gán `value = 0`, `isMissing = true`
      · nếu parse được số → giữ giá trị thật
      · nếu token không hợp lệ → xem như dòng lỗi
    * Hàm `parseRow(cells, Record& rec)` tạo ra một record hoàn chỉnh
  - **Module 3 (Tính tổng & xếp loại):**
    * Tổng:

      `total = math + lit + sci`

    * Hàm `computeRank(total)`:
      · `total` $\geq$ `24` → `Gioi`
      · `total` $\geq$ `18` → `Kha`
      · `total` $\geq$ `12` → `Trung binh`
      · `total` < `12` → `Yeu`
  - **Module 4 (Đọc toàn bộ file):**
    * Hàm `readScores(path, vector<Record>&)`
    * Đọc từng dòng bằng `getline`
    * Với mỗi dòng hợp lệ:
      · parse record

        - tính tổng
        - gán rank
        - đưa vào danh sách kết quả
    - **Module 5 (Ghi report & summary):**
        * Hàm `writeResult(path, records)`
        * Ghi file `result.csv` gồm các cột:

`id, total, rank`

        * Cuối file ghi thêm 1 dòng summary đếm số record theo từng xếp loại
        * Số thực dùng `fixed << setprecision(2)`

- **Output:**
    - File `result.csv`
    - Mỗi dòng dữ liệu có dạng:

`id,total,rank`

    - Dòng cuối là summary:

`SUMMARY,count_gioi,count_kha,count_trung_binh,count_yeu`

1.5.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <string>
#include <iomanip>
#include <cctype>

using namespace std;

struct Record {
    string id;
    double math;
    double lit;
    double sci;
    bool missingMath;
    bool missingLit;
    bool missingSci;
    double total;
    string rank;
};

// MODULE: Trim
```

```
string trim(const string& s) {
    size_t start = s.find_first_not_of(" \t\r\n");
    size_t end = s.find_last_not_of(" \t\r\n");

    if (start == string::npos) return "";
    return s.substr(start, end - start + 1);
}

// MODULE: To lower
string toLowerStr(string s) {
    for (char& c : s) {
        c = (char)tolower((unsigned char)c);
    }
    return s;
}

// MODULE: Check missing token
bool isMissingToken(string t) {
    t = toLowerStr(trim(t));
    return t.empty() || t == "na" || t == "nan";
}

// MODULE: Parse double safely
bool parseDouble(const string& s, double& value) {
    string t = trim(s);
    if (t.empty()) return false;

    try {
        size_t idx;
        value = stod(t, &idx);
        return idx == t.size();
    } catch (...) {
        return false;
    }
}

// MODULE 1: Split CSV
void splitCSV(const string& line, vector<string>& cells) {
    cells.clear();
    stringstream ss(line);
    string cell;

    while (getline(ss, cell, ',')) {
        cells.push_back(cell);
    }
}

// MODULE 2: Parse one score cell
bool parseScore(const string& cell, double& value, bool& isMissing) {
```

```
        if (isMissingToken(cell)) {
            value = 0.0;
            isMissing = true;
            return true;
        }

        isMissing = false;
        return parseDouble(cell, value);
    }

    // MODULE 3: Compute rank
    string computeRank(double total) {
        if (total >= 24.0) return "Gioi";
        if (total >= 18.0) return "Kha";
        if (total >= 12.0) return "Trung binh";
        return "Yeu";
    }

    // MODULE 4: Parse one row -> Record
    bool parseRow(const vector<string>& cells, Record& rec) {
        if (cells.size() != 4) return false;

        rec.id = trim(cells[0]);
        if (rec.id.empty()) rec.id = "UNKNOWN";

        if (!parseScore(cells[1], rec.math, rec.missingMath)) return false;
        if (!parseScore(cells[2], rec.lit, rec.missingLit)) return false;
        if (!parseScore(cells[3], rec.sci, rec.missingSci)) return false;

        rec.total = rec.math + rec.lit + rec.sci;
        rec.rank = computeRank(rec.total);

        return true;
    }

    // MODULE 5: Read whole file
    bool readScores(string path, vector<Record>& records) {
        ifstream file(path);
        if (!file.is_open()) return false;

        string line;
        while (getline(file, line)) {
            if (trim(line).empty()) continue;

            vector<string> cells;
            splitCSV(line, cells);

            Record rec;
```

```
            if (parseRow(cells, rec)) {
                records.push_back(rec);
            }
        }

        file.close();
        return true;
    }

    // MODULE 6: Write result.csv
    bool writeResult(string path, const vector<Record>& records) {
        ofstream file(path);
        if (!file.is_open()) return false;

        int countGioi = 0;
        int countKha = 0;
        int countTrungBinh = 0;
        int countYeu = 0;

        file << "id,total,rank\n";
        file << fixed << setprecision(2);

        for (size_t i = 0; i < records.size(); i++) {
            file << records[i].id << ","
                << records[i].total << ","
                << records[i].rank << "\n";

            if (records[i].rank == "Gioi") countGioi++;
            else if (records[i].rank == "Kha") countKha++;
            else if (records[i].rank == "Trung binh") countTrungBinh++;
            else countYeu++;
        }

        file << "SUMMARY,"
            << countGioi << ","
            << countKha << ","
            << countTrungBinh << ","
            << countYeu;

        file.close();
        return true;
    }

    // MAIN
    int main() {
        vector<Record> records;

        if (!readScores("scores.csv", records)) {
            cerr << "Khong mo duoc file scores.csv\n";
```

```
            return 1;
        }

        if (!writeResult("result.csv", records)) {
            cerr << "Khong ghi duoc file result.csv\n";
            return 1;
        }

        return 0;
    }
```

1.5.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (`scores.csv`):

```
SV01,8,9,8
SV02,6,6,7
SV03,4,5,4
```

Expected Output (`result.csv`):

```
id,total,rank
SV01,25.00,Gioi
SV02,19.00,Kha
SV03,13.00,Trung binh
SUMMARY,1,1,1,0
```

Test Case 2 (Có missing)
Input (`scores.csv`):

```
SV01,8,NA,8
SV02,,6,7
,NA,NaN,
```

Expected Output (`result.csv`):

```
id,total,rank
SV01,16.00,Trung binh
SV02,13.00,Trung binh
UNKNOWN,0.00,Yeu
SUMMARY,0,0,2,1
```

Test Case 3 (Case biên – file rỗng)
Input (`scores.csv`):

```

```

Expected Output (`result.csv`):

```
id,total,rank
SUMMARY,0,0,0,0
```

1.6 Đếm từ trong file text [Cơ bản]

1.6.1 Phân tích & Thiết kế (M0)

- **Input:** File `words.txt` gồm nhiều dòng văn bản
- **Xử lý:**
  - **Case biên:**
    - File rỗng → `total_words = 0`, `unique_words = 0`
    - Dòng rỗng hoặc chỉ chứa khoảng trắng → không tạo ra từ nào
  - **Module 1 (Đọc file):**
    - Hàm `readLines(path, vector<string>&)`
    - Đọc toàn bộ file bằng `getline`
  - **Module 2 (Tách từ):**
    - Hàm `splitWords(line, vector<string>&)`
    - Dùng `stringstream` để tách từ theo khoảng trắng
    - Không normalize: không lowercase, không bỏ dấu câu
  - **Module 3 (Đếm từ):**
    - `total_words`: tổng số từ tách được từ toàn bộ file
    - `unique_words`: số từ phân biệt
    - Dùng `set<string>` để lưu các từ duy nhất
  - **Module 4 (Ghi file):**
    - Hàm `writeOutput(path, total_words, unique_words)`
- **Output:**

```
total_words=<n> / unique_words=<m>
```

1.6.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <sstream>
#include <set>

using namespace std;

// MODULE 1: Read file
bool readLines(string path, vector<string>& lines) {
    ifstream file(path);
    if (!file.is_open()) return false;
```

```

        string line;
        while (getline(file, line)) {
            lines.push_back(line);
        }

        file.close();
        return true;
    }

    // MODULE 2: Split words by whitespace
    void splitWords(const string& line, vector<string>& words) {
        stringstream ss(line);
        string word;

        while (ss >> word) {
            words.push_back(word);
        }
    }

    // MODULE 3: Count total and unique words
    void countWords(const vector<string>& lines, int& total_words, int&
    unique_words) {
        total_words = 0;
        unique_words = 0;

        set<string> distinct;
        vector<string> words;

        for (const string& line : lines) {
            words.clear();
            splitWords(line, words);

            for (const string& word : words) {
                total_words++;
                distinct.insert(word);
            }
        }

        unique_words = (int)distinct.size();
    }

    // MODULE 4: Write output
    bool writeOutput(string path, int total_words, int unique_words) {
        ofstream file(path);
        if (!file.is_open()) return false;

        file << "total_words=" << total_words
            << " / unique_words=" << unique_words;
```

```

        file.close();
        return true;
    }

    // MAIN
    int main() {
        vector<string> lines;
        int total_words, unique_words;

        if (!readLines("words.txt", lines)) {
            cerr << "Khong mo duoc file words.txt\n";
            return 1;
        }

        countWords(lines, total_words, unique_words);

        if (!writeOutput("output.txt", total_words, unique_words)) {
            cerr << "Khong ghi duoc file output.txt\n";
            return 1;
        }

        return 0;
    }
```

1.6.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (`words.txt`):

```
hello world
world !
```

Expected Output:

```
total_words=4 / unique_words=3
```

Test Case 2 (Case biên – file rỗng)
Input (`words.txt`):

```

```

Expected Output:

```
total_words=0 / unique_words=0
```

Test Case 3 (Dòng rỗng và nhiều khoảng trắng)
Input (`words.txt`):

```
apple   banana

```

```
   apple
banana
```

Expected Output:

```
total_words=4 / unique_words=2
```

1.7 Kiểm tra file CSV có header [Cơ bản]

1.7.1 Phân tích & Thiết kế (M0)

- **Input:** File CSV bất kỳ (ví dụ: `data.csv`), mỗi dòng gồm nhiều cột phân tách bởi dấu phẩy
- **Xử lý:**
  - **Case biên:**
    - File rỗng → `has_header=0`
    - Dòng đầu rỗng → `has_header=0`
  - **Module 1 (Đọc dòng đầu):**
    - Hàm `readFirstLine(path, string&)`
    - Đọc dòng đầu tiên bằng `getline`
  - **Module 2 (Tách CSV):**
    - Hàm `splitCSV(line, vector<string>&)`
    - Tách theo dấu phẩy
  - **Module 3 (Kiểm tra số):**
    - Hàm `isNumber(string)`
    - Nếu **tất cả cell không parse được thành số** → header
  - **Module 4 (Xác định header):**
    - Nếu mọi cell đều **không phải số** → `has_header=1`
    - Ngược lại → `has_header=0`
  - **Module 5 (Ghi output):**
    - Hàm `writeOutput(path, has_header, columns)`
    - Nếu có header → in thêm danh sách cột
- **Output:**

  `has_header=1` hoặc `has_header=0`

  Nếu có header:

  `columns: col1, col2, ...`

1.7.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <sstream>

using namespace std;

// MODULE: Trim
string trim(const string& s) {
    size_t start = s.find_first_not_of(" \t\r\n");
    size_t end = s.find_last_not_of(" \t\r\n");

    if (start == string::npos) return "";
    return s.substr(start, end - start + 1);
}

// MODULE: Check number
bool isNumber(const string& s) {
    if (s.empty()) return false;

    try {
        size_t idx;
        stod(s, &idx);
        return idx == s.size();
    } catch (...) {
        return false;
    }
}

// MODULE 1: Read first line
bool readFirstLine(string path, string& line) {
    ifstream file(path);
    if (!file.is_open()) return false;

    if (!getline(file, line)) {
        file.close();
        return true; // file rong
    }

    file.close();
    return true;
}

// MODULE 2: Split CSV
void splitCSV(const string& line, vector<string>& cells) {
    stringstream ss(line);
```

```
        string temp;

        while (getline(ss, temp, ',')) {
            cells.push_back(trim(temp));
        }
    }

    // MODULE 3: Detect header
    bool detectHeader(const vector<string>& cells) {
        if (cells.empty()) return false;

        for (const string& c : cells) {
            if (isNumber(c)) return false;
        }
        return true;
    }

    // MODULE 4: Write output
    bool writeOutput(string path, bool has_header, const vector<string>&
    cols) {
        ofstream file(path);
        if (!file.is_open()) return false;

        file << "has_header=" << (has_header ? 1 : 0);

        if (has_header) {
            file << "\ncolumns: ";
            for (size_t i = 0; i < cols.size(); i++) {
                file << cols[i];
                if (i != cols.size() - 1) file << ", ";
            }
        }

        file.close();
        return true;
    }

    // MAIN
    int main() {
        string line;
        vector<string> cells;

        if (!readFirstLine("data.csv", line)) {
            cerr << "Khong mo duoc file data.csv\n";
            return 1;
        }

        if (!line.empty()) {
            splitCSV(line, cells);
```

```
        }

        bool has_header = detectHeader(cells);

        if (!writeOutput("output.txt", has_header, cells)) {
            cerr << "Khong ghi duoc file output.txt\n";
            return 1;
        }

        return 0;
    }
```

1.7.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường – có header)
Input (data.csv):

```
name, age, score
A, 20, 8.5
```

Expected Output:

```
has_header=1
columns: name, age, score
```

Test Case 2 (Không có header)
Input (data.csv):

```
1, 2, 3
4, 5, 6
```

Expected Output:

```
has_header=0
```

Test Case 3 (Case biên – file rỗng)
Input (data.csv):

```

```

Expected Output:

```
has_header=0
```

1.8 Lọc dòng theo điều kiện [Trung bình]

1.8.1 Phân tích & Thiết kế (M0)

- **Input:**

  - File `data.csv`: mỗi dòng dạng `id, value`
  - File `config.txt`: chứa 2 số thực `lo` và `hi`

- **Xử lý:**
  - **Case biên:**
    * Value missing (rỗng, `NA`/`NaN`) → bỏ (dropped)
    * Value không parse được → bỏ
    * `lo > hi` → không có dòng nào được giữ
    * File rỗng → output rỗng, `kept=0`, `dropped=0`
  - **Module 1 (Chuẩn hóa token):**
    * `trim`, `toLowerStr`, `isMissingToken`
  - **Module 2 (Đọc config):**
    * Hàm `readConfig(path, lo, hi)`
    * Parse 2 số thực từ file
  - **Module 3 (Parse CSV):**
    * `splitCSV(line, cells)`
    * Mỗi dòng phải có đúng 2 cột
  - **Module 4 (Lọc dữ liệu):**
    * Nếu value hợp lệ và `lo` $\leq$ `value` $\leq$ `hi` → kept
    * Ngược lại → dropped
  - **Module 5 (Ghi file):**
    * Ghi `filtered.csv`: các dòng được giữ
    * Cuối file ghi: `kept=<n> / dropped=<m>`

- **Output:**
  - File `filtered.csv`
  - Dòng cuối:

```
kept=<n> / dropped=<m>
```

1.8.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <string>

using namespace std;

// MODULE: Trim
```

```
string trim(const string& s) {
    size_t start = s.find_first_not_of(" \t\r\n");
    size_t end = s.find_last_not_of(" \t\r\n");

    if (start == string::npos) return "";
    return s.substr(start, end - start + 1);
}

// MODULE: To lower
string toLowerStr(string s) {
    for (char& c : s) {
        c = (char)tolower((unsigned char)c);
    }
    return s;
}

// MODULE: Check missing
bool isMissingToken(string t) {
    t = toLowerStr(trim(t));
    return t.empty() || t == "na" || t == "nan";
}

// MODULE: Parse double
bool parseDouble(const string& s, double& value) {
    string t = trim(s);
    if (t.empty()) return false;

    try {
        size_t idx;
        value = stod(t, &idx);
        return idx == t.size();
    } catch (...) {
        return false;
    }
}

// MODULE 1: Read config
bool readConfig(string path, double& lo, double& hi) {
    ifstream file(path);
    if (!file.is_open()) return false;

    file >> lo >> hi;

    file.close();
    return true;
}

// MODULE 2: Split CSV
void splitCSV(const string& line, vector<string>& cells) {
```

```
        cells.clear();
        stringstream ss(line);
        string cell;

        while (getline(ss, cell, ',')) {
            cells.push_back(cell);
        }
    }

    // MODULE 3: Filter data
    bool filterData(string inPath, string outPath, double lo, double hi,
                    int& kept, int& dropped) {

        ifstream inFile(inPath);
        if (!inFile.is_open()) return false;

        ofstream outFile(outPath);
        if (!outFile.is_open()) return false;

        kept = 0;
        dropped = 0;

        string line;
        while (getline(inFile, line)) {
            if (trim(line).empty()) continue;

            vector<string> cells;
            splitCSV(line, cells);

            if (cells.size() != 2) {
                dropped++;
                continue;
            }

            string id = trim(cells[0]);
            string valStr = cells[1];

            if (isMissingToken(valStr)) {
                dropped++;
                continue;
            }

            double value;
            if (!parseDouble(valStr, value)) {
                dropped++;
                continue;
            }

            if (value >= lo && value <= hi) {
```

```
                outFile << id << "," << value << "\n";
                kept++;
            } else {
                dropped++;
            }
        }

        outFile << "kept=" << kept << " / dropped=" << dropped;

        inFile.close();
        outFile.close();
        return true;
    }

    // MAIN
    int main() {
        double lo, hi;
        int kept, dropped;

        if (!readConfig("config.txt", lo, hi)) {
            cerr << "Khong mo duoc file config.txt\n";
            return 1;
        }

        if (!filterData("data.csv", "filtered.csv", lo, hi, kept, dropped) ) {
            cerr << "Loi xu ly file\n";
            return 1;
        }

        return 0;
    }
```

1.8.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)

Input (`config.txt`):

```
10 20
```

Input (`data.csv`):

```
A,5
B,10
C,15
D,25
```

Expected Output (`filtered.csv`):

```
B,10
C,15
```

```
kept=2 / dropped=2
```

Test Case 2 (Có missing)
Input (`config.txt`):

```
0 100
```

Input (`data.csv`):

```
A,NA
B,
C,50
D,abc
```

Expected Output:

```
C,50
kept=1 / dropped=3
```

Test Case 3 (Case biên – file rỗng)
Input (`data.csv`):

```

```

Expected Output:

```
kept=0 / dropped=0
```

1.9 Merge 2 file CSV theo id [Trung bình]

1.9.1 Phân tích & Thiết kế (M0)

- **Input:**
  - File `fileA.csv`: mỗi dòng có dạng `id, score_A`
  - File `fileB.csv`: mỗi dòng có dạng `id, score_B`
- **Xử lý:**
  - **Case biên:**
    * Một trong hai file rỗng → không join được dòng nào
    * Có id chỉ xuất hiện ở đúng một file → không ghi vào `merged.csv`, nhưng vẫn tính vào thống kê `id_only_one`
    * Missing được hiểu là ô trống hoặc `NA`/`NaN` (không phân biệt hoa/thường); các dòng thiếu dữ liệu hợp lệ sẽ bị bỏ qua
  - **Module 1 (Chuẩn hóa token):**
    * Hàm `trim(string)` để bỏ khoảng trắng đầu/cuối
    * Hàm `toLowerStr(string)` để đưa token về chữ thường khi kiểm tra missing

    * Hàm `isMissingToken(string)` để nhận diện ô trống, `NA`, `NaN`
  - **Module 2 (Parse 1 dòng CSV):**
    * Hàm `parseLine(line, id, score)`
    * Tách dòng theo dấu phẩy thành 2 cột: `id`, `score`
    * Nếu thiếu cột, thiếu dữ liệu, hoặc `score` không parse được thành số → dòng không hợp lệ
  - **Module 3 (Đọc file CSV vào map):**
    * Hàm `readCSV(path, map<string, double>&)`
    * Đọc từng dòng bằng `getline`
    * Với mỗi dòng hợp lệ, lưu `id` → `score`
    * Nếu trùng id trong cùng một file thì giữ giá trị xuất hiện sau cùng
  - **Module 4 (Inner Join theo id):**
    * Duyệt các id của file A
    * Nếu id cũng có trong file B → đưa vào danh sách kết quả join
    * Tính `diff = score_A - score_B`
  - **Module 5 (Đếm thống kê):**
    * `joined_rows`: số id xuất hiện ở cả hai file
    * `id_only_one`: số id chỉ xuất hiện ở đúng một file
    * Công thức:
      $$\texttt{id_only_one} = |\texttt{A only}| + |\texttt{B only}|$$
  - **Module 6 (Ghi file + in báo cáo):**
    * Hàm `writeMerged(path, rows)` ghi file `merged.csv`
    * Định dạng:
      ```
      id, score_A, score_B, diff
      ```
    * Dùng `fixed << setprecision(6)` cho số thực
    * In ra màn hình:
      ```
      joined_rows=<n> / id_only_one=<m>
      ```

- **Output:**
  - File `merged.csv` chứa các dòng inner join:
    ```
    id,score_A,score_B,diff
    ```
  - Màn hình in:
    ```
    joined_rows=<n> / id_only_one=<m>
    ```

1.9.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>
#include <map>
#include <iomanip>
#include <cctype>

using namespace std;

struct JoinRow {
    string id;
    double scoreA;
    double scoreB;
    double diff;
};

// MODULE: Trim string
string trim(const string& s) {
    size_t start = s.find_first_not_of(" \t\r\n");
    size_t end = s.find_last_not_of(" \t\r\n");

    if (start == string::npos) return "";
    return s.substr(start, end - start + 1);
}

// MODULE: To lower
string toLowerStr(string s) {
    for (char& c : s) {
        c = (char)tolower((unsigned char)c);
    }
    return s;
}

// MODULE: Check missing token
bool isMissingToken(const string& s) {
    string t = toLowerStr(trim(s));
    return t.empty() || t == "na" || t == "nan";
}

// MODULE: Parse double safely
bool parseDouble(const string& s, double& value) {
    string t = trim(s);
    if (t.empty()) return false;

    try {
```

```
            size_t idx;
            value = stod(t, &idx);
            return idx == t.size();
        } catch (...) {
            return false;
        }
    }

    // MODULE 1: Parse one CSV line -> id, score
    bool parseLine(const string& line, string& id, double& score) {
        stringstream ss(line);
        string cell1, cell2, extra;

        if (!getline(ss, cell1, ',')) return false;
        if (!getline(ss, cell2, ',')) return false;

        // Neu con du cot thi dong khong dung format id,score
        if (getline(ss, extra, ',')) return false;

        id = trim(cell1);
        string scoreToken = trim(cell2);

        if (isMissingToken(id) || isMissingToken(scoreToken)) return false;
        if (!parseDouble(scoreToken, score)) return false;

        return true;
    }

    // MODULE 2: Read CSV into map<id, score>
    bool readCSV(string path, map<string, double>& data) {
        ifstream file(path);
        if (!file.is_open()) return false;

        string line;
        while (getline(file, line)) {
            line = trim(line);
            if (line.empty()) continue;

            string id;
            double score;

            if (parseLine(line, id, score)) {
                data[id] = score; // Neu trung id, giu gia tri xuat hien sau
            }
        }

        file.close();
```

```
        return true;
    }

    // MODULE 3: Inner join by id
    void innerJoin(const map<string, double>& a,
                const map<string, double>& b,
                vector<JoinRow>& rows,
                int& joined_rows,
                int& id_only_one) {
        joined_rows = 0;
        id_only_one = 0;

        for (map<string, double>::const_iterator it = a.begin(); it != a.end(); ++it) {
            map<string, double>::const_iterator found = b.find(it->first);
            if (found != b.end()) {
                JoinRow row;
                row.id = it->first;
                row.scoreA = it->second;
                row.scoreB = found->second;
                row.diff = row.scoreA - row.scoreB;
                rows.push_back(row);
                joined_rows++;
            } else {
                id_only_one++;
            }
        }

        for (map<string, double>::const_iterator it = b.begin(); it != b.end(); ++it) {
            if (a.find(it->first) == a.end()) {
                id_only_one++;
            }
        }
    }

    // MODULE 4: Write merged.csv
    bool writeMerged(string path, const vector<JoinRow>& rows) {
        ofstream file(path);
        if (!file.is_open()) return false;

        file << "id,score_A,score_B,diff\n";
        file << fixed << setprecision(6);

        for (size_t i = 0; i < rows.size(); i++) {
            file << rows[i].id << ","
                << rows[i].scoreA << ","
                << rows[i].scoreB << ","
                << rows[i].diff << "\n";
```

```
        }

        file.close();
        return true;
    }

    // MAIN
    int main() {
        map<string, double> fileAData;
        map<string, double> fileBData;
        vector<JoinRow> rows;
        int joined_rows = 0;
        int id_only_one = 0;

        if (!readCSV("fileA.csv", fileAData)) {
            cerr << "Khong mo duoc file fileA.csv\n";
            return 1;
        }

        if (!readCSV("fileB.csv", fileBData)) {
            cerr << "Khong mo duoc file fileB.csv\n";
            return 1;
        }

        innerJoin(fileAData, fileBData, rows, joined_rows, id_only_one);

        if (!writeMerged("merged.csv", rows)) {
            cerr << "Khong ghi duoc file merged.csv\n";
            return 1;
        }

        cout << "joined_rows=" << joined_rows
            << " / id_only_one=" << id_only_one << "\n";

        return 0;
    }
```

1.9.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (fileA.csv):

```
A01,8.5
A02,7.0
A03,9.0
```

Input (fileB.csv):

```
A02,6.5
A03,9.5
```

```
A04,8.0
```

Expected Output (merged.csv):

```
id,score_A,score_B,diff
A02,7.000000,6.500000,0.500000
A03,9.000000,9.500000,-0.500000
```

Expected Console Output:

```
joined_rows=2 / id_only_one=2
```

Test Case 2 (Case biên – một file rỗng)
Input (fileA.csv):

```

```

Input (fileB.csv):

```
B01,5.0
B02,6.0
```

Expected Output (merged.csv):

```
id,score_A,score_B,diff
```

Expected Console Output:

```
joined_rows=0 / id_only_one=2
```

Test Case 3 (Có missing / dòng lỗi)
Input (fileA.csv):

```
A01,8.0
A02,NA
A03,9.5
,7.0
```

Input (fileB.csv):

```
A01,7.5
A03,abc
A04,6.0
```

Expected Output (merged.csv):

```
id,score_A,score_B,diff
A01,8.000000,7.500000,0.500000
```

Expected Console Output:

```
joined_rows=1 / id_only_one=2
```

1.10 Phân tích log file [Nâng cao]

1.10.1 Phân tích & Thiết kế (M0)

- **Input:**
  - File `log.txt`, mỗi dòng có dạng:

    `YYYY-MM-DD HH:MM:SS [LEVEL] message`

    với LEVEL $\in$ {`INFO, WARN, ERROR`}
- **Xử lý:**
  - **Case biên:**
    * File rỗng $\rightarrow$ tất cả count = 0, `first_error = NONE`, `last_error = NONE`
    * Dòng sai format $\rightarrow$ bỏ qua
    * Không có dòng ERROR $\rightarrow$ `first_error = NONE`, `last_error = NONE`
  - **Module 1 (Đọc file):**
    * Hàm `readLines(path, vector<string>&)`
    * Đọc toàn bộ dòng bằng `getline`
  - **Module 2 (Parse dòng log):**
    * Hàm `parseLog(line, timestamp, level)`
    * Tách:
      · `timestamp`: 19 ký tự đầu
      · `level`: nằm trong dấu `[ ]`
    * Nếu không đúng format $\rightarrow$ return false
  - **Module 3 (Thống kê):**
    * Đếm số dòng theo từng LEVEL:

      $$\texttt{INFO}, \texttt{WARN}, \texttt{ERROR}$$

    * Nếu gặp ERROR:
      · Nếu là ERROR đầu tiên $\rightarrow$ lưu `first_error`
      · Luôn cập nhật `last_error`
  - **Module 4 (Ghi file):**
    * Hàm `writeSummary(path, info, warn, error, first, last)`
- **Output:**

```
INFO=<n> / WARN=<n> / ERROR=<n> / first_error=<timestamp> /
last_error=<timestamp>
```

1.10.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

using namespace std;

// MODULE 1: Read file
bool readLines(string path, vector<string>& lines) {
    ifstream file(path);
    if (!file.is_open()) return false;

    string line;
    while (getline(file, line)) {
        lines.push_back(line);
    }

    file.close();
    return true;
}

// MODULE 2: Parse log line
bool parseLog(const string& line, string& timestamp, string& level) {
    if (line.size() < 22) return false;

    // timestamp = 19 ky tu dau
    timestamp = line.substr(0, 19);

    // tim [LEVEL]
    size_t l = line.find('[');
    size_t r = line.find(']');

    if (l == string::npos || r == string::npos || r <= l + 1) return false;

    level = line.substr(l + 1, r - l - 1);

    if (level != "INFO" && level != "WARN" && level != "ERROR")
        return false;

    return true;
}

// MODULE 3: Analyze
void analyzeLogs(const vector<string>& lines,
                 int& info, int& warn, int& error,
                 string& first_error, string& last_error) {
```

```

        info = warn = error = 0;
        first_error = "NONE";
        last_error = "NONE";

        for (const string& line : lines) {
            string timestamp, level;

            if (!parseLog(line, timestamp, level)) continue;

            if (level == "INFO") info++;
            else if (level == "WARN") warn++;
            else if (level == "ERROR") {
                error++;
                if (first_error == "NONE") first_error = timestamp;
                last_error = timestamp;
            }
        }
    }

    // MODULE 4: Write output
    bool writeSummary(string path,
                      int info, int warn, int error,
                      const string& first_error,
                      const string& last_error) {

        ofstream file(path);
        if (!file.is_open()) return false;

        file << "INFO=" << info
            << " / WARN=" << warn
            << " / ERROR=" << error
            << " / first_error=" << first_error
            << " / last_error=" << last_error;

        file.close();
        return true;
    }

    // MAIN
    int main() {
        vector<string> lines;

        if (!readLines("log.txt", lines)) {
            cerr << "Khong mo duoc file log.txt\n";
            return 1;
        }

        int info, warn, error;
```

```
        string first_error, last_error;

        analyzeLogs(lines, info, warn, error, first_error, last_error);

        if (!writeSummary("summary.txt", info, warn, error, first_error,
    last_error)) {
            cerr << "Khong ghi duoc file summary.txt\n";
            return 1;
        }

        return 0;
    }
```

1.10.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (log.txt):

```
2026-04-01 10:00:00 [INFO] Start system
2026-04-01 10:01:00 [WARN] Low memory
2026-04-01 10:02:00 [ERROR] Crash detected
2026-04-01 10:03:00 [ERROR] Restart failed
```

Expected Output:

```
INFO=1 / WARN=1 / ERROR=2 / first_error=2026-04-01 10:02:00 / last_error
    =2026-04-01 10:03:00
```

Test Case 2 (Không có ERROR)
Input (log.txt):

```
2026-04-01 10:00:00 [INFO] Start
2026-04-01 10:01:00 [WARN] Something
```

Expected Output:

```
INFO=1 / WARN=1 / ERROR=0 / first_error=NONE / last_error=NONE
```

Test Case 3 (Case biên – file rỗng)
Input (log.txt):

Expected Output:

```
INFO=0 / WARN=0 / ERROR=0 / first_error=NONE / last_error=NONE
```

2 Tuần 2: Mảng 1D + Đa thức + Thống kê mô tả

2.1 Tìm min, max, mean mảng [Cơ bản]

2.1.1 Phân tích & Thiết kế (M0)

- **Input:** File `array.txt` chứa:
  - Số nguyên n
  - n số thực
- **Xử lý:**
  - **Case biên:**
    * $n = 0 \rightarrow$ in `EMPTY`
    * $n = 1 \rightarrow \min = \max = \text{mean} = a[0]$
  - **Module 1 (Đọc file):**
    * Hàm `readArray(path, vector<double>&)`
    * Kiểm tra mở file (fail $\rightarrow$ `cerr` + `return false`)
  - **Module 2 (Tính mean):**
    * Dùng `long double` để tránh sai số
  - **Module 3 (Tìm min, max):**
    * Duyệt mảng 1 lần
- **Output:**

```
min=<f> / max=<f> / mean=<f>
```

(với `setprecision(4)`)

2.1.2 Mã nguồn C++11 (Tách module)

```
#include <iostream>
#include <fstream>
#include <vector>
```

```
#include <iomanip>

using namespace std;

// MODULE 1: Doc du lieu tu file
bool readArray(string path, vector<double>& a) {
    ifstream file(path);
    if (!file.is_open()) {
        cerr << "Khong mo duoc file\n";
        return false;
    }

    int n;
    file >> n;

    double x;
    for (int i = 0; i < n; i++) {
        file >> x;
        a.push_back(x);
    }

    file.close();
    return true;
}

// MODULE 2: Tinh mean
double calcMean(const vector<double>& a) {
    if (a.empty()) return 0.0;

    long double sum = 0;
    for (double x : a) sum += x;

    return (double)(sum / a.size());
}

// MODULE 3: Ghi output
bool writeOutput(string path, const vector<double>& a) {
    ofstream file(path);
    if (!file.is_open()) {
        cerr << "Khong ghi duoc file\n";
        return false;
    }

    int n = a.size();

    if (n == 0) {
        file << "EMPTY";
        file.close();
        return true;
```

```
        }

        double mn = a[0], mx = a[0];

        for (int i = 1; i < n; i++) {
            if (a[i] < mn) mn = a[i];
            if (a[i] > mx) mx = a[i];
        }

        double mean = calcMean(a);

        file << fixed << setprecision(4);
        file << "min=" << mn
            << " / max=" << mx
            << " / mean=" << mean;

        file.close();
        return true;
    }

    // MAIN
    int main() {
        vector<double> a;

        if (!readArray("array.txt", a)) {
            return 1;
        }

        if (!writeOutput("output.txt", a)) {
            return 1;
        }

        return 0;
    }
```

2.1.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (array.txt):

```
5 10.5 -2.0 4.0 0.0 5.5
```

Expected Output:

```
min=-2.0000 / max=10.5000 / mean=3.6000
```

Test Case 2 (Case biên – n=0)
Input (array.txt):

```
0
```

Expected Output:

```
EMPTY
```

Test Case 3 (Case biên – n=1)

Input (array.txt):

```
1 42.5
```

Expected Output:

```
min=42.5000 / max=42.5000 / mean=42.5000
```

2.2 Đếm phần tử thỏa điều kiện [Cơ bản]

2.2.1 Phân tích & Thiết kế (M0)

- **Input:**
  - File `array.txt` chứa mảng số nguyên: số lượng n và n phần tử.
  - File `config.txt` chứa ngưỡng: một số nguyên k.
- **Xử lý:**
  - **Case biên:**
    * $n = 0 \rightarrow$ mảng rỗng, cả hai biến đếm đều bằng 0.
    * Tất cả phần tử trong mảng đều bằng $k \rightarrow$ `count_ge_k` $= n$, `count_lt_k` $= 0$.
  - **Module 1 (Đọc mảng):**
    * Hàm `readArray(path, vector<int>&)`
    * Kiểm tra mở file (fail $\rightarrow$ `cerr` + `return false`)
  - **Module 2 (Đọc cấu hình k):**
    * Hàm `readConfig(path, int&)`
    * Kiểm tra mở file (fail $\rightarrow$ `cerr` + `return false`)
  - **Module 3 (Đếm theo điều kiện):**
    * Hàm `countIf(vector<int>, k, geq)` với `geq` là biến `bool` để cờ hiệu đếm lớn hơn hoặc bằng (`true`) hay nhỏ hơn (`false`).
  - **Module 4 (Ghi output):**
    * Hàm `writeOutput(path, count_ge_k, count_lt_k)`
- **Output:**

```
count_ge_k=<i> / count_lt_k=<i>
```

2.2.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <vector>

using namespace std;

// MODULE 1: Doc mang tu file
bool readArray(string path, vector<int>& a) {
    ifstream file(path);
    if (!file.is_open()) {
        cerr << "Khong mo duoc file " << path << "\n";
        return false;
    }

    int n;
    if (file >> n) {
        int x;
        for (int i = 0; i < n; i++) {
            file >> x;
            a.push_back(x);
        }
    }

    file.close();
    return true;
}

// MODULE 2: Doc nguong k tu file config
bool readConfig(string path, int& k) {
    ifstream file(path);
    if (!file.is_open()) {
        cerr << "Khong mo duoc file " << path << "\n";
        return false;
    }

    file >> k;

    file.close();
    return true;
}

// MODULE 3: Dem so luong phan tu thoa man
int countIf(const vector<int>& a, int k, bool geq) {
    int count = 0;
    for (int x : a) {
        if (geq) {
            if (x >= k) count++;
```

```
            } else {
                if (x < k) count++;
            }
        }
        return count;
    }

    // MODULE 4: Ghi output
    bool writeOutput(string path, int count_ge_k, int count_lt_k) {
        ofstream file(path);
        if (!file.is_open()) {
            cerr << "Khong ghi duoc file " << path << "\n";
            return false;
        }

        file << "count_ge_k=" << count_ge_k
            << " / count_lt_k=" << count_lt_k;

        file.close();
        return true;
    }

    // MAIN
    int main() {
        vector<int> a;
        int k;

        if (!readArray("array.txt", a)) {
            return 1;
        }

        if (!readConfig("config.txt", k)) {
            return 1;
        }

        int count_ge = countIf(a, k, true);
        int count_lt = countIf(a, k, false);

        if (!writeOutput("output.txt", count_ge, count_lt)) {
            return 1;
        }

        return 0;
    }

```

2.2.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (array.txt):

```
6 10 -5 8 15 3 8
```

Input (config.txt):

```
8
```

Expected Output (output.txt):

```
count_ge_k=4 / count_lt_k=2
```

Test Case 2 (Case biên – n=0)
Input (array.txt):

```
0
```

Input (config.txt):

```
5
```

Expected Output (output.txt):

```
count_ge_k=0 / count_lt_k=0
```

Test Case 3 (Case biên – Tất cả bằng k)
Input (array.txt):

```
4 7 7 7 7
```

Input (config.txt):

```
7
```

Expected Output (output.txt):

```
count_ge_k=4 / count_lt_k=0
```

2.3 Tính đa thức tại nhiều điểm [Cơ bản]

2.3.1 Phân tích & Thiết kế (M0)

- **Input:**
  - File `poly.csv` chứa 1 dòng: `degree, a0, a1, ..., an` (phân tách bằng dấu phẩy).
  - File `x_values.csv` chứa nhiều giá trị x, mỗi dòng 1 giá trị.
- **Xử lý:**

- **Case biên:**
  * `degree = 0` → Đa thức là hằng số $P(x) = a_0$.
  * File `x_values.csv` rỗng → File output chỉ in ra đúng dòng header `x,P(x)`.
  * Gặp giá trị khuyết (`NA`, `NaN`, `nan`, hoặc dòng trống) → Bỏ qua không đưa vào tính toán.
- **Module 1 (Đọc hệ số đa thức):**
  * Hàm `readPoly(path, int&, vector<double>&)`
  * Tách chuỗi bằng dấu phẩy (`,`) thông qua `stringstream`.
  * Bắt lỗi mở file (fail → `cerr` + `return false`).
- **Module 2 (Đọc giá trị x):**
  * Hàm `readXValues(path, vector<double>&)`
  * Đọc từng dòng, kiểm tra missing values.
  * Bắt lỗi mở file.
- **Module 3 (Tính giá trị đa thức):**
  * Hàm `evalHorner(vector<double> a, double x)`
  * Áp dụng thuật toán Horner: `res = a[n]; for i=n-1..0: res = res*x + a[i]`.
- **Module 4 (Ghi output):**
  * Hàm `writeOutput(path, vector<double>, vector<double>)`
  * Ghi header `x,P(x)`. Định dạng số thực bằng `fixed` và `setprecision(6)`.

- **Output:** File `output.csv` với định dạng:

```
x,P(x)
<f>,<f>
```

2.3.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <sstream>
#include <iomanip>

using namespace std;

// MODULE 1: Doc da thuc tu file poly.csv
bool readPoly(string path, int& degree, vector<double>& a) {
    ifstream file(path);
    if (!file.is_open()) {
        cerr << "Khong mo duoc file " << path << "\n";
```

```
        return false;
    }

    string line;
    if (getline(file, line)) {
        stringstream ss(line);
        string token;

        // Doc degree
        if (getline(ss, token, ',')) {
            degree = stoi(token);
        }

        // Doc cac he so a0, a1, ..., an
        while (getline(ss, token, ',')) {
            // Kiem tra missing value theo quy dinh (da ngat dong de tranh tran le PDF)
            if (token == "NA" || token == "NaN" ||
                token == "nan" || token.empty()) {
                a.push_back(0.0); // He so khuyet xem nhu bang 0
            } else {
                a.push_back(stod(token));
            }
        }
    }

    file.close();
    return true;
}

// MODULE 2: Doc cac diem x tu x_values.csv
bool readXValues(string path, vector<double>& x_vals) {
    ifstream file(path);
    if (!file.is_open()) {
        cerr << "Khong mo duoc file " << path << "\n";
        return false;
    }

    string line;
    while (getline(file, line)) {
        // Xu ly gia tri missing / o trong
        if (line.empty() || line == "NA" || line == "NaN" || line == "nan") {
            continue;
        }
        x_vals.push_back(stod(line));
    }

    file.close();
```

```
        return true;
    }

    // MODULE 3: Tinh P(x) bang thuat toan Horner
    double evalHorner(vector<double> a, double x) {
        if (a.empty()) return 0.0;

        int n = a.size() - 1;
        double res = a[n];
        for (int i = n - 1; i >= 0; i--) {
            res = res * x + a[i];
        }
        return res;
    }

    // MODULE 4: Ghi ket qua ra output.csv
    bool writeOutput(string path, const vector<double>& x_vals, const
    vector<double>& a) {
        ofstream file(path);
        if (!file.is_open()) {
            cerr << "Khong ghi duoc file " << path << "\n";
            return false;
        }

        // Ghi dong header
        file << "x,P(x)\n";

        // Ghi du lieu
        file << fixed << setprecision(6);
        for (double x : x_vals) {
            double px = evalHorner(a, x);
            file << x << "," << px << "\n";
        }

        file.close();
        return true;
    }

    // MAIN
    int main() {
        int degree = 0;
        vector<double> a;
        vector<double> x_vals;

        if (!readPoly("poly.csv", degree, a)) {
            return 1;
        }

        if (!readXValues("x_values.csv", x_vals)) {
```

```
            return 1;
        }

        if (!writeOutput("output.csv", x_vals, a)) {
            return 1;
        }

        return 0;
    }

```

2.3.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường - Bậc 2)
$P(x) = 1 + 2x + 3x^2$ (degree=2)
Input (poly.csv):

```
2,1.0,2.0,3.0
```

Input (x_values.csv):

```
1.5
-2.0
NaN
```

Expected Output (output.csv):

```
x,P(x)
1.500000,10.750000
-2.000000,9.000000
```

Test Case 2 (Case biên – Hàm hằng, degree = 0)
$P(x) = 5.5$
Input (poly.csv):

```
0,5.5
```

Input (x_values.csv):

```
10.0
0.0
```

Expected Output (output.csv):

```
x,P(x)
10.000000,5.500000
0.000000,5.500000
```

Test Case 3 (Case biên – x_values rỗng)
Input (poly.csv):

```
3,1,0,0,5
```

Input (x_values.csv):

```

```

(File trống)

Expected Output (output.csv):

```
x,P(x)
```

2.4 Variance và std deviation [Trung bình]

2.4.1 Phân tích & Thiết kế (M0)

- **Input:**
  - File `array.txt` chứa:
    * Số nguyên n (số lượng phần tử).
    * n số thực.
- **Xử lý:**
  - **Case biên:**
    * $n = 0 \rightarrow$ Ghi ra file chữ `EMPTY`.
    * $n = 1 \rightarrow$ variance $= 0$, std $= 0$.
    * Tất cả các phần tử bằng nhau $\rightarrow$ variance $= 0$, std $= 0$.
  - **Module 1 (Đọc mảng):**
    * Hàm `readArray(path, vector<double>&)`
    * Bắt lỗi mở file (fail $\rightarrow$ `cerr` + `return false`).
  - **Module 2 (Tính trung bình - Mean):**
    * Hàm `calcMean(vector<double>)`
    * Dùng kiểu `long double` để cộng dồn tránh sai số.
  - **Module 3 (Tính phương sai và độ lệch chuẩn):**
    * Hàm `calcVariance(vector<double>, double mean)`
    * Tính phương sai quần thể (chia cho n).
    * Tích lũy tổng bình phương độ lệch $\sum(x_i - \text{mean})^2$ bằng `long double` để tránh hiện tượng *catastrophic cancellation* (sai số triệt tiêu).
    * Độ lệch chuẩn (std deviation) $\sigma = \sqrt{\sigma^2}$. Sử dụng hàm `sqrt()` trong thư viện `<cmath>`.
  - **Module 4 (Ghi output):**
    * Hàm `writeOutput(path, vector<double>)`
    * Định dạng số thực bằng `fixed` và `setprecision(6)`.

- **Output:** File `output.txt` với định dạng:

```
mean=<f> / variance=<f> / std=<f>
```

2.4.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <iomanip>
#include <cmath>

using namespace std;

// MODULE 1: Doc mang tu file array.txt
bool readArray(string path, vector<double>& a) {
    ifstream file(path);
    if (!file.is_open()) {
        cerr << "Khong mo duoc file " << path << "\n";
        return false;
    }

    int n;
    if (file >> n) {
        double x;
        for (int i = 0; i < n; i++) {
            file >> x;
            a.push_back(x);
        }
    }

    file.close();
    return true;
}

// MODULE 2: Tinh mean (trung binh)
double calcMean(const vector<double>& a) {
    if (a.empty()) return 0.0;

    long double sum = 0.0;
    for (double x : a) {
        sum += x;
    }
    return (double)(sum / a.size());
}

// MODULE 3: Tinh variance (phuong sai quan the)
double calcVariance(const vector<double>& a, double mean) {
    if (a.size() <= 1) return 0.0;
```

```

    long double sum_sq_diff = 0.0;
    for (double x : a) {
        double diff = x - mean;
        // Dung long double tich luy de tranh catastrophic cancellation
        sum_sq_diff += (long double)(diff * diff);
    }

    // Phuong sai quan the chia cho n
    return (double)(sum_sq_diff / a.size());
}

// MODULE 4: Ghi ket qua ra output.txt
bool writeOutput(string path, const vector<double>& a) {
    ofstream file(path);
    if (!file.is_open()) {
        cerr << "Khong ghi duoc file " << path << "\n";
        return false;
    }

    if (a.empty()) {
        file << "EMPTY";
        file.close();
        return true;
    }

    double mean = calcMean(a);
    double variance = calcVariance(a, mean);
    double std_dev = sqrt(variance);

    file << fixed << setprecision(6);
    file << "mean=" << mean
        << " / variance=" << variance
        << " / std=" << std_dev;

    file.close();
    return true;
}

// MAIN
int main() {
    vector<double> a;

    if (!readArray("array.txt", a)) {
        return 1;
    }

    if (!writeOutput("output.txt", a)) {
```

```
            return 1;
        }

        return 0;
    }

```

2.4.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (array.txt):

```
5 10.0 20.0 30.0 40.0 50.0
```

Expected Output (output.txt):

```
mean=30.000000 / variance=200.000000 / std=14.142136
```

Test Case 2 (Case biên – n=0)
Input (array.txt):

```
0
```

Expected Output (output.txt):

```
EMPTY
```

Test Case 3 (Case biên – n=1)
Input (array.txt):

```
1 42.5
```

Expected Output (output.txt):

```
mean=42.500000 / variance=0.000000 / std=0.000000
```

Test Case 4 (Case biên – Tất cả phần tử bằng nhau)
Input (array.txt):

```
4 7.2 7.2 7.2 7.2
```

Expected Output (output.txt):

```
mean=7.200000 / variance=0.000000 / std=0.000000
```

2.5 Đạo hàm đa thức [Trung bình]

2.5.1 Phân tích & Thiết kế (M0)

- **Input:**

- File `poly.csv` chứa 1 dòng: `degree, a0, a1, ..., an`.
- File `x_values.csv` chứa các giá trị x cần tính đạo hàm (mỗi dòng 1 số).

- **Xử lý:**
  - **Case biên:**
    * `degree = 0` → Đa thức hằng $P(x) = a_0$. Đạo hàm $P'(x) = 0$ (bậc 0, hệ số 0).
    * Giá trị khuyết (`NA`, `NaN`) trong file hệ số → coi là 0. Trong file x → bỏ qua.
  - **Module 1 (Đọc dữ liệu):**
    * Tái sử dụng logic đọc mảng hệ số phân tách bằng dấu phẩy và đọc mảng x từng dòng (đã xử lý missing values).
  - **Module 2 (Tính hệ số đạo hàm):**
    * Hàm `computeDerivCoeffs(vector<double> a)`
    * Áp dụng công thức: `deriv[i] = (i + 1) * a[i + 1]` với i chạy từ 0 đến $n - 1$.
    * Nếu mảng `a` có kích thước ≤ 1 (tương đương `degree = 0`), trả về mảng con chỉ chứa `{0.0}`.
  - **Module 3 (Tính giá trị đa thức):**
    * Tái sử dụng hàm `evalHorner` để tính $P'(x)$ tại các điểm x dựa trên mảng hệ số đạo hàm vừa tìm được.
  - **Module 4 (Ghi outputs):**
    * Hàm `writeDerivative(path, vector<double> deriv)`: Ghi file `derivative.csv` định dạng `degree_deriv, b0, b1...`
    * Hàm `writeOutput(path, vector<double> x_vals, vector<double> deriv)`: Ghi file `output.csv` định dạng `x,P'(x)`.
    * Tuân thủ quy định dùng `fixed` và `setprecision(6)`.

- **Output:**
  - File `derivative.csv`
  - File `output.csv`

2.5.2 Mã nguồn C++11 (Tách module)

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <sstream>
#include <iomanip>

```

```
using namespace std;

// MODULE 1: Doc da thuc tu file poly.csv
bool readPoly(string path, int& degree, vector<double>& a) {
    ifstream file(path);
    if (!file.is_open()) {
        cerr << "Khong mo duoc file " << path << "\n";
        return false;
    }

    string line;
    if (getline(file, line)) {
        stringstream ss(line);
        string token;

        if (getline(ss, token, ',')) {
            degree = stoi(token);
        }

        while (getline(ss, token, ',')) {
            if (token == "NA" || token == "NaN" ||
                token == "nan" || token.empty()) {
                a.push_back(0.0);
            } else {
                a.push_back(stod(token));
            }
        }
    }
    file.close();
    return true;
}

// MODULE 2: Doc cac diem x tu x_values.csv
bool readXValues(string path, vector<double>& x_vals) {
    ifstream file(path);
    if (!file.is_open()) {
        cerr << "Khong mo duoc file " << path << "\n";
        return false;
    }

    string line;
    while (getline(file, line)) {
        if (line.empty() || line == "NA" || line == "NaN" || line == "nan") {
            continue;
        }
        x_vals.push_back(stod(line));
    }
    file.close();
```

```
        return true;
    }

    // MODULE 3: Tinh he so da thuc dao ham
    vector<double> computeDerivCoeffs(const vector<double>& a) {
        vector<double> deriv;

        // Case bien: da thuc hang (bac 0) -> dao ham = 0
        if (a.size() <= 1) {
            deriv.push_back(0.0);
            return deriv;
        }

        // Cong thuc: b[i] = (i + 1) * a[i + 1]
        for (size_t i = 0; i < a.size() - 1; i++) {
            deriv.push_back((i + 1) * a[i + 1]);
        }

        return deriv;
    }

    // MODULE 4: Tinh gia tri da thuc bang Horner
    double evalHorner(const vector<double>& a, double x) {
        if (a.empty()) return 0.0;

        int n = a.size() - 1;
        double res = a[n];
        for (int i = n - 1; i >= 0; i--) {
            res = res * x + a[i];
        }
        return res;
    }

    // MODULE 5: Ghi file derivative.csv
    bool writeDerivative(string path, const vector<double>& deriv) {
        ofstream file(path);
        if (!file.is_open()) {
            cerr << "Khong ghi duoc file " << path << "\n";
            return false;
        }

        int degree_deriv = deriv.empty() ? 0 : deriv.size() - 1;
        file << degree_deriv;

        file << fixed << setprecision(6);
        for (double coeff : deriv) {
            file << "," << coeff;
        }
        file << "\n";
```

```
        file.close();
        return true;
    }

    // MODULE 6: Ghi file output.csv
    bool writeOutput(string path, const vector<double>& x_vals, const vector<double>& deriv) {
        ofstream file(path);
        if (!file.is_open()) {
            cerr << "Khong ghi duoc file " << path << "\n";
            return false;
        }

        file << "x,P'(x)\n";
        file << fixed << setprecision(6);

        for (double x : x_vals) {
            double px_prime = evalHorner(deriv, x);
            file << x << "," << px_prime << "\n";
        }

        file.close();
        return true;
    }

    // MAIN
    int main() {
        int degree = 0;
        vector<double> a;
        vector<double> x_vals;

        if (!readPoly("poly.csv", degree, a)) return 1;
        if (!readXValues("x_values.csv", x_vals)) return 1;

        vector<double> deriv = computeDerivCoeffs(a);

        if (!writeDerivative("derivative.csv", deriv)) return 1;
        if (!writeOutput("output.csv", x_vals, deriv)) return 1;

        return 0;
    }

```

2.5.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường - Bậc 3)
$P(x) = 1 + 2x + 3x^2 + 4x^3 \Rightarrow P'(x) = 2 + 6x + 12x^2$

Input (poly.csv):

```
3,1.0,2.0,3.0,4.0
```

Input (x_values.csv):

```
1.0
-1.0
```

Expected Output (derivative.csv):

```
2,2.000000,6.000000,12.000000
```

Expected Output (output.csv):

```
x,P'(x)
1.000000,20.000000
-1.000000,8.000000
```

Test Case 2 (Case biên – Đa thức hằng, degree = 0)
$P(x) = 5.5 \Rightarrow P'(x) = 0$
Input (poly.csv):

```
0,5.5
```

Input (x_values.csv):

```
10.0
```

Expected Output (derivative.csv):

```
0,0.000000
```

Expected Output (output.csv):

```
x,P'(x)
10.000000,0.000000
```

2.6 Tìm vị trí phần tử lớn nhất [Cơ bản]

2.6.1 Phân tích & Thiết kế (M0)

- **Input:**
  - File `array.txt` chứa: số nguyên n và mảng n số nguyên.
- **Xử lý:**
  - **Case biên:**
    - $n = 0 \rightarrow$ ghi `EMPTY`.
    - $n = 1 \rightarrow$ `max_val = a[0]`, `max_idx = 0`.
  - **Thuật toán:**

    * Khởi tạo `max_val = a[0]`, `max_idx = 0`.
    * Duyệt mảng từ $i = 1$ đến $n - 1$:
      · Nếu `a[i] > max_val` thì cập nhật.
    * Dùng dấu `>` để đảm bảo khi có nhiều phần tử lớn nhất bằng nhau thì lấy chỉ số nhỏ nhất.
  - **Module 1 (Đọc dữ liệu):** Hàm `readArray`. Bắt lỗi mở file (`return false`).
  - **Module 2 (Xử lý & Ghi kết quả):** Hàm `writeOutput` tìm max và ghi ra file.

- **Output:** File `output.txt`:

```
max_val=<i> / max_idx=<i>
```

2.6.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <vector>

using namespace std;

// MODULE 1: Doc mang tu file
bool readArray(string path, vector<int>& a) {
    ifstream file(path);
    if (!file.is_open()) {
        cerr << "Khong mo duoc file " << path << "\n";
        return false;
    }

    int n;
    file >> n;

    int x;
    for (int i = 0; i < n; i++) {
        file >> x;
        a.push_back(x);
    }

    file.close();
    return true;
}

// MODULE 2: Tim max va ghi ket qua
bool writeOutput(string path, const vector<int>& a) {
    ofstream file(path);
    if (!file.is_open()) {
```

```
            cerr << "Khong ghi duoc file " << path << "\n";
            return false;
        }

        if (a.empty()) {
            file << "EMPTY";
            file.close();
            return true;
        }

        int max_val = a[0];
        int max_idx = 0;

        for (int i = 1; i < (int)a.size(); i++) {
            if (a[i] > max_val) {
                max_val = a[i];
                max_idx = i+1;
            }
        }

        file << "max_val=" << max_val
            << " / max_idx=" << max_idx;

        file.close();
        return true;
    }

    // MAIN
    int main() {
        vector<int> a;

        if (!readArray("array.txt", a)) return 1;
        if (!writeOutput("output.txt", a)) return 1;

        return 0;
    }
```

2.6.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (array.txt):

```
5 1 5 3 9 2
```

Expected Output (output.txt):

```
max_val=9 / max_idx=4
```

Test Case 2 (Case biên – nhiều phần tử lớn nhất bằng nhau)

Input (array.txt):

```
6 4 8 2 8 1 8
```

Expected Output (output.txt):

```
max_val=8 / max_idx=2
```

Test Case 3 (Case biên – n = 0)
Input (array.txt):

```
0
```

Expected Output (output.txt):

```
EMPTY
```

Test Case 4 (Case biên – n = 1)
Input (array.txt):

```
1 42
```

Expected Output (output.txt):

```
max_val=42 / max_idx=1
```

2.7 Bài 7: Chuẩn hoá mảng Min-Max [Cơ bản]

2.7.1 Phân tích & Thiết kế (M0)

- **Input:**
  - File `array.txt` chứa: số nguyên n và mảng n số thực (không có missing).
- **Xử lý:**
  - **Công thức chuẩn hoá:**

$$x' = \frac{x - \min}{\max - \min}$$

  - **Case biên:**
    * $n = 0 \rightarrow$ file output chỉ có header.
    * Nếu $\max = \min \rightarrow$ mọi giá trị `scaled = 0`.
  - **Module 1 (Đọc dữ liệu):** Hàm `readArray`. Bắt lỗi mở file (`return false`).
  - **Module 2 (Tìm min, max):** Duyệt mảng để xác định min và max.
  - **Module 3 (Chuẩn hoá & Ghi kết quả):** Hàm `writeOutput` tính giá trị chuẩn hoá và ghi ra file.

- **Output:** File `normalized.csv`:

```
i,original,scaled
```

2.7.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <iomanip>

using namespace std;

// MODULE 1: Doc mang tu file
bool readArray(string path, vector<double>& a) {
    ifstream file(path);
    if (!file.is_open()) {
        cerr << "Khong mo duoc file " << path << "\n";
        return false;
    }

    int n;
    file >> n;

    double x;
    for (int i = 0; i < n; i++) {
        file >> x;
        a.push_back(x);
    }

    file.close();
    return true;
}

// MODULE 2 + 3: Chuan hoa va ghi ket qua
bool writeOutput(string path, const vector<double>& a) {
    ofstream file(path);
    if (!file.is_open()) {
        cerr << "Khong ghi duoc file " << path << "\n";
        return false;
    }

    file << "i,original,scaled\n";
    file << fixed << setprecision(6);

    int n = a.size();
    if (n == 0) {
        file.close();
        return true;
```

```
        }

        double mn = a[0], mx = a[0];
        for (double x : a) {
            if (x < mn) mn = x;
            if (x > mx) mx = x;
        }

        for (int i = 0; i < n; i++) {
            double scaled;
            if (mx == mn) scaled = 0.0;
            else scaled = (a[i] - mn) / (mx - mn);

            file << i << "," << a[i] << "," << scaled << "\n";
        }

        file.close();
        return true;
    }

    // MAIN
    int main() {
        vector<double> a;

        if (!readArray("array.txt", a)) return 1;
        if (!writeOutput("normalized.csv", a)) return 1;

        return 0;
    }
```

2.7.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (array.txt):

```
5 10 20 30 40 50
```

Expected Output (normalized.csv):

```
i,original,scaled
0,10.000000,0.000000
1,20.000000,0.250000
2,30.000000,0.500000
3,40.000000,0.750000
4,50.000000,1.000000
```

Test Case 2 (Case biên – max = min)
Input (array.txt):

```
3 5 5 5
```

Expected Output (normalized.csv):

```
i,original,scaled
0,5.000000,0.000000
1,5.000000,0.000000
2,5.000000,0.000000
```

Test Case 3 (Case biên – n = 0)
Input (array.txt):

```
0
```

Expected Output (normalized.csv):

```
i,original,scaled
```

2.8 Impute missing bằng median [Trung bình]

2.8.1 Phân tích & Thiết kế (M0)

- **Input:**
  - File `data.csv` chứa mảng số thực có missing (`NA`, `NaN`, hoặc dòng trống).
- **Xử lý:**
  - **Quy ước missing:**
    * Dòng trống hoặc `NA`/`NaN` (không phân biệt hoa thường) được xem là missing.
    * Không dùng `-1` làm missing.
  - **Cách tính median:**
    * Lấy các giá trị hợp lệ, sắp xếp tăng dần.
    * Nếu số phần tử hợp lệ lẻ → median là phần tử giữa.
    * Nếu số phần tử hợp lệ chẵn → median là trung bình của 2 phần tử giữa.
    * Nếu không có giá trị hợp lệ nào → median = 0.
  - **Case biên:**
    * File rỗng → `count_valid = 0`, `count_missing = 0`, `median = 0`.
    * Tất cả phần tử đều missing → mọi giá trị impute bằng 0.
  - **Module 1 (Hỗ trợ xử lý chuỗi):**
    * Hàm `trim` để xóa khoảng trắng đầu/cuối.
    * Hàm `toLowerStr` để đưa chuỗi về chữ thường.

    * Hàm `isMissingToken` để kiểm tra missing.
  - **Module 2 (Đọc dữ liệu):** Hàm `loadData`. Đọc từng dòng, lưu lại trạng thái missing và danh sách giá trị hợp lệ. Bắt lỗi mở file (`return false`).
  - **Module 3 (Tính median):** Hàm `getMedian`.
  - **Module 4 (Ghi output):** Hàm `writeOutput` ghi file `output.csv`.
  - **Module 5 (Ghi summary):** Hàm `writeSummary` ghi file `summary.txt`.

- **Output:**
  - File `output.csv`

```
i,value_imputed
```

  - File `summary.txt`

```
count_valid=<i>
count_missing=<i>
median=<f>
```

2.8.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <algorithm>
#include <iomanip>
#include <cctype>

using namespace std;

struct DataNode {
    double value;
    bool isMissing;
};

// MODULE: Trim
string trim(const string& s) {
    size_t start = s.find_first_not_of(" \t\r\n");
    size_t end = s.find_last_not_of(" \t\r\n");

    if (start == string::npos) return "";
    return s.substr(start, end - start + 1);
}

// MODULE: ToLower
string toLowerStr(string s) {
    for (char& c : s) {
```

```
            c = (char)tolower((unsigned char)c);
        }
        return s;
    }

    // MODULE 1: Kiem tra missing token
    bool isMissingToken(string t) {
        t = trim(t);
        t = toLowerStr(t);

        if (t.empty()) return true;
        return (t == "na" || t == "nan");
    }

    // MODULE 2: Doc du lieu tu file
    bool loadData(string path, vector<DataNode>& data, vector<double>&
    valid) {
        ifstream file(path);
        if (!file.is_open()) {
            cerr << "Khong mo duoc file " << path << "\n";
            return false;
        }

        string line;
        while (getline(file, line)) {
            if (isMissingToken(line)) {
                data.push_back({0.0, true});
            } else {
                double val = stod(trim(line));
                data.push_back({val, false});
                valid.push_back(val);
            }
        }

        file.close();
        return true;
    }

    // MODULE 3: Tinh median
    double getMedian(vector<double> valid) {
        if (valid.empty()) return 0.0;

        sort(valid.begin(), valid.end());
        int n = (int)valid.size();

        if (n % 2 == 1) return valid[n / 2];
        return (valid[n / 2 - 1] + valid[n / 2]) / 2.0;
    }

```

```
// MODULE 4: Ghi output.csv
bool writeOutput(string path, const vector<DataNode>& data, double median) {
    ofstream file(path);
    if (!file.is_open()) {
        cerr << "Khong ghi duoc file " << path << "\n";
        return false;
    }

    file << "i,value_imputed\n";
    file << fixed << setprecision(6);

    for (int i = 0; i < (int)data.size(); i++) {
        double outVal = data[i].isMissing ? median : data[i].value;
        file << i << "," << outVal << "\n";
    }

    file.close();
    return true;
}

// MODULE 5: Ghi summary.txt
bool writeSummary(string path, int countValid, int countMissing, double median) {
    ofstream file(path);
    if (!file.is_open()) {
        cerr << "Khong ghi duoc file " << path << "\n";
        return false;
    }

    file << "count_valid=" << countValid << "\n";
    file << "count_missing=" << countMissing << "\n";
    file << fixed << setprecision(6);
    file << "median=" << median << "\n";

    file.close();
    return true;
}

// MAIN
int main() {
    vector<DataNode> data;
    vector<double> valid;

    if (!loadData("data.csv", data, valid)) return 1;

    double median = getMedian(valid);
    int countValid = (int)valid.size();
    int countMissing = (int)data.size() - countValid;
```

```

        if (!writeOutput("output.csv", data, median)) return 1;
        if (!writeSummary("summary.txt", countValid, countMissing, median)
    ) return 1;

        return 0;
    }
```

2.8.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (data.csv):

```
10
NA
30

20
```

Expected Output (output.csv):

```
i,value_imputed
0,10.000000
1,20.000000
2,30.000000
3,20.000000
4,20.000000
```

Expected Output (summary.txt):

```
count_valid=3
count_missing=2
median=20.000000
```

Test Case 2 (Case biên – tất cả đều missing)
Input (data.csv):

```
NA

NaN
```

Expected Output (output.csv):

```
i,value_imputed
0,0.000000
1,0.000000
2,0.000000
```

Expected Output (summary.txt):

```
count_valid=0
count_missing=3
```

```
median=0.000000
```

Test Case 3 (Case biên – không có missing)
Input (data.csv):

```
5
15
25
35
```

Expected Output (output.csv):

```
i,value_imputed
0,5.000000
1,15.000000
2,25.000000
3,35.000000
```

Expected Output (summary.txt):

```
count_valid=4
count_missing=0
median=20.000000
```

2.9 Histogram đơn giản [Trung bình]

2.9.1 Phân tích & Thiết kế (M0)

- **Input:**
  - File `array.txt` chứa: số nguyên n và mảng n số thực.
  - File `config.txt` chứa: số nguyên k là số lượng bin.
- **Xử lý:**
  - **Ý tưởng:**
    * Chia đều khoảng $[\min, \max]$ thành k bin.
    * Độ rộng mỗi bin:
    $$w = \frac{\max - \min}{k}$$
    * Với mỗi phần tử x, chỉ số bin:
    $$idx = \left\lfloor \frac{x - \min}{w} \right\rfloor$$
    * Nếu $x = \max$ thì ép vào bin cuối cùng.
  - **Case biên:**
    * $n = 0$ hoặc $k \leq 0 \rightarrow$ file output rỗng.

    * Nếu max = min → đưa toàn bộ phần tử vào bin đầu tiên, các bin còn lại bằng 0.
  - **Module 1 (Đọc dữ liệu):** Hàm `readArray` và `readK`. Bắt lỗi mở file (`return false`).
  - **Module 2 (Tìm min, max):** Hàm `findRange`.
  - **Module 3 (Đếm bin):** Hàm `countBins`.
  - **Module 4 (Ghi output):** Hàm `writeHistogram` ghi ra file `histogram.txt`.

- **Output:** File `histogram.txt`:

```
[lo, hi): count
```

2.9.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <iomanip>
#include <cmath>

using namespace std;

// MODULE 1a: Doc mang tu file
bool readArray(string path, vector<double>& a) {
    ifstream file(path);
    if (!file.is_open()) {
        cerr << "Khong mo duoc file " << path << "\n";
        return false;
    }

    int n;
    file >> n;

    double x;
    for (int i = 0; i < n; i++) {
        file >> x;
        a.push_back(x);
    }

    file.close();
    return true;
}

// MODULE 1b: Doc so bin k
bool readK(string path, int& k) {
    ifstream file(path);
    if (!file.is_open()) {
```

```
            cerr << "Khong mo duoc file " << path << "\n";
            return false;
        }

        file >> k;
        file.close();
        return true;
    }

    // MODULE 2: Tim min va max
    void findRange(const vector<double>& a, double& mn, double& mx) {
        mn = mx = a[0];
        for (double x : a) {
            if (x < mn) mn = x;
            if (x > mx) mx = x;
        }
    }

    // MODULE 3: Dem so phan tu trong moi bin
    vector<int> countBins(const vector<double>& a, int k, double mn,
    double mx) {
        vector<int> counts(k, 0);

        if (mx == mn) {
            counts[0] = (int)a.size();
            return counts;
        }

        double width = (mx - mn) / k;

        for (double x : a) {
            int idx = (int)((x - mn) / width);
            if (idx >= k) idx = k - 1;
            counts[idx]++;
        }

        return counts;
    }

    // MODULE 4: Ghi histogram.txt
    bool writeHistogram(string path, int k, double mn, double mx, const
    vector<int>& counts) {
        ofstream file(path);
        if (!file.is_open()) {
            cerr << "Khong ghi duoc file " << path << "\n";
            return false;
        }

        file << fixed << setprecision(2);
```

```

        double width = (mx - mn) / k;
        for (int i = 0; i < k; i++) {
            double lo = mn + i * width;
            double hi = (i == k - 1) ? mx : (mn + (i + 1) * width);
            file << "[" << lo << ", " << hi << "): " << counts[i] << "\n";
        }

        file.close();
        return true;
    }

    // MAIN
    int main() {
        vector<double> a;
        int k;

        if (!readArray("array.txt", a)) return 1;
        if (!readK("config.txt", k)) return 1;

        ofstream testCreate("histogram.txt");
        if (!testCreate.is_open()) {
            cerr << "Khong ghi duoc file histogram.txt\n";
            return 1;
        }
        testCreate.close();

        if (a.empty() || k <= 0) {
            return 0;
        }

        double mn, mx;
        findRange(a, mn, mx);

        vector<int> counts = countBins(a, k, mn, mx);

        if (!writeHistogram("histogram.txt", k, mn, mx, counts)) return 1;

        return 0;
    }
```

2.9.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (array.txt):

```
5 1.0 2.5 3.0 4.8 5.0
```

Input (config.txt):

```
2
```

Expected Output (histogram.txt):

```
[1.00, 3.00): 2
[3.00, 5.00): 3
```

Test Case 2 (Case biên – max = min)
Input (array.txt):

```
4 5 5 5 5
```

Input (config.txt):

```
3
```

Expected Output (histogram.txt):

```
[5.00, 5.00): 4
[5.00, 5.00): 0
[5.00, 5.00): 0
```

Test Case 3 (Case biên – n = 0)
Input (array.txt):

```
0
```

Input (config.txt):

```
4
```

Expected Output (histogram.txt):

```

```

2.10 Nội suy tuyến tính [Nâng cao]

2.10.1 Phân tích & Thiết kế (M0)

- **Input:**
  - File `points.csv` chứa các điểm `x,y` đã sắp xếp tăng theo x (không có x trùng).
  - File `queries.csv` chứa các giá trị x_q cần nội suy, mỗi dòng 1 giá trị.
- **Xử lý:**
  - **Nguyên tắc nội suy:**
    - Với mỗi x_q:
      - Nếu $x_q < x_{\min}$ hoặc $x_q > x_{\max}$ → ghi `OUT_OF_RANGE`.

- Nếu x_q trùng đúng một giá trị x_i trong dữ liệu → trả về đúng y_i.
- Nếu $x_i < x_q < x_{i+1}$ → nội suy tuyến tính theo công thức:

$$y_q = y_i + (x_q - x_i) \cdot \frac{y_{i+1} - y_i}{x_{i+1} - x_i}$$

– **Case biên:**
  * File `points.csv` rỗng → chương trình báo lỗi và `return 1`.
  * File `queries.csv` rỗng → file output rỗng.
  * Truy vấn nằm ngoài đoạn $[x_{\min}, x_{\max}]$ → ghi `OUT_OF_RANGE`.

– **Module 1 (Đọc dữ liệu điểm):** Hàm `loadPoints`. Bắt lỗi mở file (`return false`).

– **Module 2 (Nội suy 1 truy vấn):** Hàm `interpolateOne`. Tìm khoảng phù hợp và tính giá trị nội suy.

– **Module 3 (Xử lý toàn bộ truy vấn):** Hàm `processQueries`. Đọc `queries.csv`, xử lý từng dòng và ghi ra `interpolated.csv`.

- **Output:** File `interpolated.csv`:

```
x_q,y_interp
x_q,OUT_OF_RANGE
```

2.10.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <sstream>
#include <iomanip>

using namespace std;

struct Point {
    double x;
    double y;
};

// MODULE 1: Doc cac diem tu points.csv
bool loadPoints(string path, vector<Point>& pts) {
    ifstream file(path);
    if (!file.is_open()) {
        cerr << "Khong mo duoc file " << path << "\n";
        return false;
    }

```

```
        string line;
        while (getline(file, line)) {
            if (line.empty()) continue;

            stringstream ss(line);
            string sx, sy;

            if (getline(ss, sx, ',') && getline(ss, sy)) {
                Point p;
                p.x = stod(sx);
                p.y = stod(sy);
                pts.push_back(p);
            }
        }

        file.close();
        return true;
    }

    // MODULE 2: Noi suy 1 gia tri xq
    string interpolateOne(const vector<Point>& pts, double xq) {
        if (pts.empty()) return "OUT_OF_RANGE";

        if (xq < pts.front().x || xq > pts.back().x) {
            return "OUT_OF_RANGE";
        }

        for (int i = 0; i < (int)pts.size(); i++) {
            if (xq == pts[i].x) {
                stringstream ss;
                ss << fixed << setprecision(6) << pts[i].y;
                return ss.str();
            }
        }

        for (int i = 0; i < (int)pts.size() - 1; i++) {
            if (pts[i].x < xq && xq < pts[i + 1].x) {
                double xi = pts[i].x;
                double yi = pts[i].y;
                double xj = pts[i + 1].x;
                double yj = pts[i + 1].y;

                double yq = yi + (xq - xi) * (yj - yi) / (xj - xi);

                stringstream ss;
                ss << fixed << setprecision(6) << yq;
                return ss.str();
            }
        }
```

```

        return "OUT_OF_RANGE";
    }

    // MODULE 3: Xu ly danh sach truy van va ghi file
    bool processQueries(string queryPath, string outPath, const vector<Point>& pts) {
        ifstream fin(queryPath);
        if (!fin.is_open()) {
            cerr << "Khong mo duoc file " << queryPath << "\n";
            return false;
        }

        ofstream fout(outPath);
        if (!fout.is_open()) {
            cerr << "Khong ghi duoc file " << outPath << "\n";
            return false;
        }

        fout << fixed << setprecision(6);

        string line;
        while (getline(fin, line)) {
            if (line.empty()) continue;

            double xq = stod(line);
            string ans = interpolateOne(pts, xq);

            fout << xq << "," << ans << "\n";
        }

        fin.close();
        fout.close();
        return true;
    }

    // MAIN
    int main() {
        vector<Point> pts;

        if (!loadPoints("points.csv", pts)) return 1;
        if (pts.empty()) {
            cerr << "File points.csv rong\n";
            return 1;
        }

        if (!processQueries("queries.csv", "interpolated.csv", pts)) return 1;

```

```
        return 0;
    }
```

2.10.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (points.csv):

```
1,2
3,6
5,10
```

Input (queries.csv):

```
2
4
```

Expected Output (interpolated.csv):

```
2.000000,4.000000
4.000000,8.000000
```

Test Case 2 (Case biên – truy vấn trùng điểm có sẵn)
Input (points.csv):

```
0,1
2,5
4,9
```

Input (queries.csv):

```
2
4
```

Expected Output (interpolated.csv):

```
2.000000,5.000000
4.000000,9.000000
```

Test Case 3 (Case biên – ngoài khoảng)
Input (points.csv):

```
1,2
3,6
5,10
```

Input (queries.csv):

```
0
6
```

Expected Output (interpolated.csv):

```
0.000000,OUT_OF_RANGE
6.000000,OUT_OF_RANGE
```

Test Case 4 (Case biên – queries.csv rỗng)
Input (points.csv):

```
1,2
3,6
5,10
```

Input (queries.csv):

```

```

Expected Output (interpolated.csv):

```

```

3 Tuần 3: Prefix Sum 1D + Tối ưu truy vấn

3.1 Xây dựng prefix sum [Cơ bản]

3.1.1 Phân tích & Thiết kế (M0)

- **Input:** File `array.txt` chứa:
  - Số nguyên n
  - n số thực
- **Xử lý:**
  - **Case biên:**
    - $n = 0 \rightarrow$ in `EMPTY`
    - $n = 1 \rightarrow \min = \max = \text{mean} = a[0]$
  - **Module 1 (Đọc file):**
    - Hàm `readArray(path, vector<double>&)`
    - Kiểm tra mở file (fail $\rightarrow$ `cerr` + `return false`)
  - **Module 2 (Tính mean):**
    - Dùng `long double` để tránh sai số
  - **Module 3 (Tìm min, max):**
    - Duyệt mảng 1 lần
- **Output:**

```
min=<f> / max=<f> / mean=<f>
```

(với `setprecision(4)`)

3.1.2 Mã nguồn C++11 (Tách module)

```
#include <iostream>
#include <vector>
#include <fstream>
```

```cpp

using namespace std;

bool readArray(const string& path, vector<long long>& a) {
    ifstream fin(path);
    if (!fin) return false;
    int n;
    if (fin >> n) {
        a.resize(n);
        for (int i = 0; i < n; ++i) fin >> a[i];
    }
    return true;
}

vector<long long> buildPrefix(const vector<long long>& a) {
    if (a.empty()) return {};
    vector<long long> pre(a.size());
    pre[0] = a[0];
    for (size_t i = 1; i < a.size(); ++i) pre[i] = pre[i - 1] + a[i];
    return pre;
}

bool writePrefix(const string& path, const vector<long long>& pre) {
    ofstream fout(path);
    if (!fout) return false;
    for (size_t i = 0; i < pre.size(); ++i) {
        fout << i << ": " << pre[i] << "\n";
    }
    return true;
}

int main() {
    vector<long long> a;
    if (!readArray("input.txt", a)) {
        cerr << "Loi mo file input!\n";
        return 1;
    }
    vector<long long> pre = buildPrefix(a);
    if (!writePrefix("prefix.txt", pre)) {
        cerr << "Loi mo file output!\n";
        return 1;
    }
    return 0;
}
```

3.1.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)

Input (array.txt):

```
5 10.5 -2.0 4.0 0.0 5.5
```

Expected Output:

```
min=-2.0000 / max=10.5000 / mean=3.6000
```

Test Case 2 (Case biên – n=0)
Input (array.txt):

```
0
```

Expected Output:

```
EMPTY
```

Test Case 3 (Case biên – n=1)
Input (array.txt):

```
1 42.5
```

Expected Output:

```
min=42.5000 / max=42.5000 / mean=42.5000
```

3.2 Trả lời truy vấn tổng đoạn [Cơ bản]

3.2.1 Phân tích & Thiết kế (M0)

- **Input:** File `array.txt` chứa:
  - Số nguyên n
  - n số thực
- **Xử lý:**
  - **Case biên:**
    - $n = 0 \rightarrow$ in `EMPTY`
    - $n = 1 \rightarrow \text{min} = \text{max} = \text{mean} = a[0]$
  - **Module 1 (Đọc file):**
    - Hàm `readArray(path, vector<double>&)`
    - Kiểm tra mở file (fail $\rightarrow$ `cerr` + `return false`)
  - **Module 2 (Tính mean):**
    - Dùng `long double` để tránh sai số
  - **Module 3 (Tìm min, max):**
    - Duyệt mảng 1 lần

- **Output:**

```
min=<f> / max=<f> / mean=<f>
```

(với `setprecision(4)`)

3.2.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>

using namespace std;

bool sumRange(const vector<long long>& pre, int l, int r, long long& ans) {
    if (l < 0 || r >= pre.size() || l > r) return false;
    ans = pre[r] - (l > 0 ? pre[l - 1] : 0);
    return true;
}

int main() {
    ifstream fin_arr("array.txt"), fin_q("queries.txt");
    ofstream fout("output.txt");
    if (!fin_arr || !fin_q || !fout) {
        cerr << "Loi mo file!\n";
        return 1;
    }

    int n; fin_arr >> n;
    vector<long long> a(n), pre(n);
    for (int i = 0; i < n; ++i) {
        fin_arr >> a[i];
        pre[i] = a[i] + (i > 0 ? pre[i - 1] : 0);
    }

    int q; fin_q >> q;
    while (q--) {
        int l, r; fin_q >> l >> r;
        long long ans;
        if (sumRange(pre, l, r, ans)) fout << ans << "\n";
        else fout << "INVALID\n";
    }
    return 0;
}
```

3.2.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)

Input (array.txt):

```
5 10.5 -2.0 4.0 0.0 5.5
```

Expected Output:

```
min=-2.0000 / max=10.5000 / mean=3.6000
```

Test Case 2 (Case biên – n=0)
Input (array.txt):

```
0
```

Expected Output:

```
EMPTY
```

Test Case 3 (Case biên – n=1)
Input (array.txt):

```
1 42.5
```

Expected Output:

```
min=42.5000 / max=42.5000 / mean=42.5000
```

3.3 Đếm phần tử không âm trong đoạn [Cơ bản]

3.3.1 Phân tích & Thiết kế (M0)

- **Input:** File `array.txt` chứa:
  - Số nguyên n
  - n số thực
- **Xử lý:**
  - **Case biên:**
    - $n = 0 \rightarrow$ in `EMPTY`
    - $n = 1 \rightarrow \text{min} = \text{max} = \text{mean} = a[0]$
  - **Module 1 (Đọc file):**
    - Hàm `readArray(path, vector<double>&)`
    - Kiểm tra mở file (fail $\rightarrow$ `cerr` + `return false`)
  - **Module 2 (Tính mean):**
    - Dùng `long double` để tránh sai số
  - **Module 3 (Tìm min, max):**
    - Duyệt mảng 1 lần

- **Output:**

```
min=<f> / max=<f> / mean=<f>
```

(với `setprecision(4)`)

3.3.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>

using namespace std;

bool countNonNegative(const vector<int>& pre_b, int l, int r, int& ans) {
    if (l < 0 || r >= pre_b.size() || l > r) return false;
    ans = pre_b[r] - (l > 0 ? pre_b[l - 1] : 0);
    return true;
}

int main() {
    ifstream fin_arr("array.txt"), fin_q("queries.txt");
    ofstream fout("output.txt");
    if (!fin_arr || !fin_q || !fout) { cerr << "Loi mo file!\n"; return 1; }

    int n; fin_arr >> n;
    vector<int> pre_b(n);
    for (int i = 0; i < n; ++i) {
        long long val; fin_arr >> val;
        int b = (val >= 0 ? 1 : 0);
        pre_b[i] = b + (i > 0 ? pre_b[i - 1] : 0);
    }

    int q; fin_q >> q;
    while (q--) {
        int l, r; fin_q >> l >> r;
        int ans;
        if (countNonNegative(pre_b, l, r, ans)) fout << ans << "\n";
        else fout << "INVALID\n";
    }
    return 0;
}
```

3.3.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (array.txt):

```
5 10.5 -2.0 4.0 0.0 5.5
```

Expected Output:

```
min=-2.0000 / max=10.5000 / mean=3.6000
```

Test Case 2 (Case biên – n=0)
Input (array.txt):

```
0
```

Expected Output:

```
EMPTY
```

Test Case 3 (Case biên – n=1)
Input (array.txt):

```
1 42.5
```

Expected Output:

```
min=42.5000 / max=42.5000 / mean=42.5000
```

3.4 Tổng đoạn trung bình [Trung bình]

3.4.1 Phân tích & Thiết kế (M0)

- **Input:** File `array.txt` chứa:
  - Số nguyên n
  - n số thực
- **Xử lý:**
  - **Case biên:**
    - $n = 0 \rightarrow$ in `EMPTY`
    - $n = 1 \rightarrow \text{min} = \text{max} = \text{mean} = a[0]$
  - **Module 1 (Đọc file):**
    - Hàm `readArray(path, vector<double>&)`
    - Kiểm tra mở file (fail $\rightarrow$ `cerr` + `return false`)
  - **Module 2 (Tính mean):**
    - Dùng `long double` để tránh sai số
  - **Module 3 (Tìm min, max):**
    - Duyệt mảng 1 lần
- **Output:**

```
min=<f> / max=<f> / mean=<f>
```

(với `setprecision(4)`)

3.4.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>
#include <iomanip>

using namespace std;

double rangeAverage(const vector<long long>& pre, int l, int r) {
    long long sum = pre[r] - (l > 0 ? pre[l - 1] : 0);
    return (double)sum / (r - l + 1);
}

int main() {
    ifstream fin_arr("array.txt"), fin_q("queries.txt");
    ofstream fout("output.txt");
    if (!fin_arr || !fin_q || !fout) { cerr << "Loi file!\n"; return 1; }

    int n; fin_arr >> n;
    vector<long long> pre(n);
    for (int i = 0; i < n; ++i) {
        long long val; fin_arr >> val;
        pre[i] = val + (i > 0 ? pre[i - 1] : 0);
    }

    fout << fixed << setprecision(6);
    int q; fin_q >> q;
    while (q--) {
        int l, r; fin_q >> l >> r;
        if (l < 0 || r >= n || l > r) fout << "INVALID\n";
        else fout << rangeAverage(pre, l, r) << "\n";
    }
    return 0;
}
```

3.4.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (array.txt):

```
5 10.5 -2.0 4.0 0.0 5.5
```

Expected Output:

```
min=-2.0000 / max=10.5000 / mean=3.6000
```

Test Case 2 (Case biên – n=0)

Input (array.txt):

```
0
```

Expected Output:

```
EMPTY
```

Test Case 3 (Case biên – n=1)

Input (array.txt):

```
1 42.5
```

Expected Output:

```
min=42.5000 / max=42.5000 / mean=42.5000
```

3.5 Đoạn có tổng bằng K [Trung bình]

3.5.1 Phân tích & Thiết kế (M0)

- **Input:** File `array.txt` chứa:
  - Số nguyên n
  - n số thực
- **Xử lý:**
  - **Case biên:**
    - $n = 0 \rightarrow$ in `EMPTY`
    - $n = 1 \rightarrow \text{min} = \text{max} = \text{mean} = a[0]$
  - **Module 1 (Đọc file):**
    - Hàm `readArray(path, vector<double>&)`
    - Kiểm tra mở file (fail $\rightarrow$ `cerr` + `return false`)
  - **Module 2 (Tính mean):**
    - Dùng `long double` để tránh sai số
  - **Module 3 (Tìm min, max):**
    - Duyệt mảng 1 lần
- **Output:**

```
min=<f> / max=<f> / mean=<f>
```

(với `setprecision(4)`)

3.5.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>
#include <unordered_map>

using namespace std;

int countRangesEqK(const vector<long long>& a, long long K, int& firstL, int& firstR) {
    unordered_map<long long, int> freq;
    unordered_map<long long, int> first_idx;
    long long pre = 0;
    int count = 0;
    firstL = -1; firstR = -1;

    freq[0] = 1;
    first_idx[0] = -1;

    for (int i = 0; i < (int)a.size(); ++i) {
        pre += a[i];
        if (freq.count(pre - K)) {
            count += freq[pre - K];
            if (firstL == -1) {
                firstL = first_idx[pre - K] + 1;
                firstR = i;
            }
        }
        freq[pre]++;
        if (!first_idx.count(pre)) first_idx[pre] = i;
    }
    return count;
}

int main() {
    ifstream fin("input.txt");
    ofstream fout("output.txt");
    if (!fin || !fout) { cerr << "Loi file!\n"; return 1; }

    int n; long long K;
    fin >> n >> K;
    vector<long long> a(n);
    for (int i = 0; i < n; ++i) fin >> a[i];

    int firstL, firstR;
    int count = countRangesEqK(a, K, firstL, firstR);
    fout << "count=" << count << "\n";
    if (count > 0) fout << "l=" << firstL << " r=" << firstR << "\n";
```

```

        return 0;
    }
```

3.5.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (array.txt):

```
5 10.5 -2.0 4.0 0.0 5.5
```

Expected Output:

```
min=-2.0000 / max=10.5000 / mean=3.6000
```

Test Case 2 (Case biên – n=0)
Input (array.txt):

```
0
```

Expected Output:

```
EMPTY
```

Test Case 3 (Case biên – n=1)
Input (array.txt):

```
1 42.5
```

Expected Output:

```
min=42.5000 / max=42.5000 / mean=42.5000
```

3.6 Phần tử xuất hiện nhiều nhất trong đoạn [Trung bình]

3.6.1 Phân tích & Thiết kế (M0)

- **Input:** File `array.txt` chứa:
  - Số nguyên n
  - n số thực
- **Xử lý:**
  - **Case biên:**
    - $n = 0 \rightarrow$ in `EMPTY`
    - $n = 1 \rightarrow \text{min} = \text{max} = \text{mean} = a[0]$

- **Module 1 (Đọc file):**
  - Hàm `readArray(path, vector<double>&)`
  - Kiểm tra mở file (fail → `cerr` + `return false`)
- **Module 2 (Tính mean):**
  - Dùng `long double` để tránh sai số
- **Module 3 (Tìm min, max):**
  - Duyệt mảng 1 lần

- **Output:**

```
min=<f> / max=<f> / mean=<f>
```

(với `setprecision(4)`)

3.6.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>

using namespace std;

int main() {
    ifstream fin_arr("array.txt"), fin_q("queries.txt");
    ofstream fout("output.txt");
    if (!fin_arr || !fin_q || !fout) { cerr << "Loi file!\n"; return 1; }

    int n; fin_arr >> n;
    vector<vector<int>> pre(101, vector<int>(n + 1, 0));

    for (int i = 1; i <= n; ++i) {
        int val; fin_arr >> val;
        for (int v = 0; v <= 100; ++v) pre[v][i] = pre[v][i - 1];
        if (val >= 0 && val <= 100) pre[val][i]++;
    }

    int q; fin_q >> q;
    while (q--) {
        int l, r; fin_q >> l >> r;
        if (l < 0 || r >= n || l > r) { fout << "INVALID\n"; continue; }

        int max_freq = -1, best_val = -1;
        for (int v = 0; v <= 100; ++v) {
            int freq = pre[v][r + 1] - pre[v][l];
            if (freq > max_freq) {
```

```
                    max_freq = freq;
                    best_val = v;
                }
            }
            fout << best_val << "\n";
        }
        return 0;
    }
```

3.6.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (array.txt):

```
5 10.5 -2.0 4.0 0.0 5.5
```

Expected Output:

```
min=-2.0000 / max=10.5000 / mean=3.6000
```

Test Case 2 (Case biên – n=0)
Input (array.txt):

```
0
```

Expected Output:

```
EMPTY
```

Test Case 3 (Case biên – n=1)
Input (array.txt):

```
1 42.5
```

Expected Output:

```
min=42.5000 / max=42.5000 / mean=42.5000
```

3.7 Tổng đoạn lớn nhất (Prefix) [Trung bình]

3.7.1 Phân tích & Thiết kế (M0)

- **Input:** File `array.txt` chứa:
  - Số nguyên n
  - n số thực
- **Xử lý:**
  - **Case biên:**

    * $n = 0 \rightarrow$ in `EMPTY`
    * $n = 1 \rightarrow \min = \max = \text{mean} = a[0]$
  - **Module 1 (Đọc file):**
    * Hàm `readArray(path, vector<double>&)`
    * Kiểm tra mở file (fail $\rightarrow$ `cerr` + `return false`)
  - **Module 2 (Tính mean):**
    * Dùng `long double` để tránh sai số
  - **Module 3 (Tìm min, max):**
    * Duyệt mảng 1 lần

- **Output:**

```
min=<f> / max=<f> / mean=<f>
```

(với `setprecision(4)`)

3.7.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>

using namespace std;

int main() {
    ifstream fin("input.txt");
    ofstream fout("output.txt");
    if (!fin || !fout) { cerr << "Loi file!\n"; return 1; }

    int n; fin >> n;
    if (n == 0) return 0;

    vector<long long> pre(n);
    long long val; fin >> val;
    pre[0] = val;
    for (int i = 1; i < n; ++i) {
        fin >> val;
        pre[i] = pre[i - 1] + val;
    }

    long long max_sum = pre[0];
    int bestL = 0, bestR = 0;

    long long min_pre = 0;
    int min_pre_idx = -1;

    for (int r = 0; r < n; ++r) {
```

```
            long long current_sum = pre[r] - min_pre;
            if (current_sum > max_sum) {
                max_sum = current_sum;
                bestL = min_pre_idx + 1;
                bestR = r;
            }
            if (pre[r] < min_pre) {
                min_pre = pre[r];
                min_pre_idx = r;
            }
        }

        fout << "max_sum=" << max_sum << " l=" << bestL << " r=" << bestR << "\n";
        return 0;
    }
```

3.7.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (array.txt):

```
5 10.5 -2.0 4.0 0.0 5.5
```

Expected Output:

```
min=-2.0000 / max=10.5000 / mean=3.6000
```

Test Case 2 (Case biên – n=0)
Input (array.txt):

```
0
```

Expected Output:

```
EMPTY
```

Test Case 3 (Case biên – n=1)
Input (array.txt):

```
1 42.5
```

Expected Output:

```
min=42.5000 / max=42.5000 / mean=42.5000
```

3.8 Prefix product modulo M [Trung bình]

3.8.1 Phân tích & Thiết kế (M0)

- **Input:** File `array.txt` chứa:

  - Số nguyên n
  - n số thực

- **Xử lý:**
  - **Case biên:**
    * $n = 0 \rightarrow$ in `EMPTY`
    * $n = 1 \rightarrow \text{min} = \text{max} = \text{mean} = a[0]$
  - **Module 1 (Đọc file):**
    * Hàm `readArray(path, vector<double>&)`
    * Kiểm tra mở file (fail $\rightarrow$ `cerr` + `return false`)
  - **Module 2 (Tính mean):**
    * Dùng `long double` để tránh sai số
  - **Module 3 (Tìm min, max):**
    * Duyệt mảng 1 lần

- **Output:**

```
min=<f> / max=<f> / mean=<f>
```

(với `setprecision(4)`)

3.8.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>

using namespace std;

long long power(long long base, long long exp, long long mod) {
    long long res = 1;
    base %= mod;
    while (exp > 0) {
        if (exp % 2 == 1) res = (res * base) % mod;
        base = (base * base) % mod;
        exp /= 2;
    }
    return res;
}

long long modInverse(long long n, long long M) {
    return power(n, M - 2, M);
}

int main() {
```

```
    ifstream fin_arr("array.txt"), fin_q("queries.txt");
    ofstream fout("output.txt");
    if (!fin_arr || !fin_q || !fout) { cerr << "Loi file!\n"; return 1; }

    int n; long long M;
    fin_arr >> n >> M;
    vector<long long> pre(n + 1, 1);
    vector<int> zeros(n + 1, 0);

    for (int i = 1; i <= n; ++i) {
        long long val; fin_arr >> val;
        zeros[i] = zeros[i - 1] + (val == 0 ? 1 : 0);
        val = (val == 0 ? 1 : val % M); // ữGi pre khác 0 để tránh chia 0
        pre[i] = (pre[i - 1] * val) % M;
    }

    int q; fin_q >> q;
    while (q--) {
        int l, r; fin_q >> l >> r;
        if (l < 0 || r >= n || l > r) { fout << "INVALID\n"; continue; }

        int zero_count = zeros[r + 1] - zeros[l];
        if (zero_count > 0) {
            fout << "INVALID\n";
        } else {
            long long ans = (pre[r + 1] * modInverse(pre[l], M)) % M;
            fout << ans << "\n";
        }
    }
    return 0;
}
```

3.8.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (array.txt):

```
5 10.5 -2.0 4.0 0.0 5.5
```

Expected Output:

```
min=-2.0000 / max=10.5000 / mean=3.6000
```

Test Case 2 (Case biên – n=0)
Input (array.txt):

```
0
```

Expected Output:

```
EMPTY
```

Test Case 3 (Case biên – n=1)

Input (array.txt):

```
1 42.5
```

Expected Output:

```
min=42.5000 / max=42.5000 / mean=42.5000
```

3.9 Sliding prefix: cập nhật 1 điểm [Nâng cao]

3.9.1 Phân tích & Thiết kế (M0)

- **Input:** File `array.txt` chứa:
  - Số nguyên n
  - n số thực
- **Xử lý:**
  - **Case biên:**
    - $n = 0 \rightarrow$ in `EMPTY`
    - $n = 1 \rightarrow \text{min} = \text{max} = \text{mean} = a[0]$
  - **Module 1 (Đọc file):**
    - Hàm `readArray(path, vector<double>&)`
    - Kiểm tra mở file (fail $\rightarrow$ `cerr` + `return false`)
  - **Module 2 (Tính mean):**
    - Dùng `long double` để tránh sai số
  - **Module 3 (Tìm min, max):**
    - Duyệt mảng 1 lần
- **Output:**

```
min=<f> / max=<f> / mean=<f>
```

(với `setprecision(4)`)

3.9.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>

using namespace std;

void rebuildPrefix(const vector<long long>& a, vector<long long>& pre) {
    if (a.empty()) return;
    pre[0] = a[0];
    for (size_t i = 1; i < a.size(); ++i) pre[i] = pre[i - 1] + a[i];
}

int main() {
    ifstream fin("input.txt");
    ofstream fout("output.txt");
    if (!fin || !fout) { cerr << "Loi file!\n"; return 1; }

    int n; fin >> n;
    vector<long long> a(n), pre(n);
    for (int i = 0; i < n; ++i) fin >> a[i];
    rebuildPrefix(a, pre);

    int q; fin >> q;
    while (q--) {
        char type; fin >> type;
        if (type == 'U') {
            int i; long long val;
            fin >> i >> val;
            if (i >= 0 && i < n) {
                a[i] = val;
                rebuildPrefix(a, pre);
            }
        } else if (type == 'Q') {
            int l, r; fin >> l >> r;
            if (l < 0 || r >= n || l > r) fout << "INVALID\n";
            else fout << pre[r] - (l > 0 ? pre[l - 1] : 0) << "\n";
        }
    }
    return 0;
}
```

3.9.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (array.txt):

```
5 10.5 -2.0 4.0 0.0 5.5
```

Expected Output:

```
min=-2.0000 / max=10.5000 / mean=3.6000
```

Test Case 2 (Case biên – n=0)
Input (array.txt):

```
0
```

Expected Output:

```
EMPTY
```

Test Case 3 (Case biên – n=1)
Input (array.txt):

```
1 42.5
```

Expected Output:

```
min=42.5000 / max=42.5000 / mean=42.5000
```

3.10 Đoạn đẹp nhất theo nhiều tiêu chí [Nâng cao]

3.10.1 Phân tích & Thiết kế (M0)

- **Input:** File `array.txt` chứa:
  - Số nguyên n
  - n số thực
- **Xử lý:**
  - **Case biên:**
    - $n = 0 \rightarrow$ in `EMPTY`
    - $n = 1 \rightarrow \min = \max = \text{mean} = a[0]$
  - **Module 1 (Đọc file):**
    - Hàm `readArray(path, vector<double>&)`
    - Kiểm tra mở file (fail $\rightarrow$ `cerr` + `return false`)
  - **Module 2 (Tính mean):**
    - Dùng `long double` để tránh sai số
  - **Module 3 (Tìm min, max):**
    - Duyệt mảng 1 lần
- **Output:**

```
min=<f> / max=<f> / mean=<f>
```

(với `setprecision(4)`)

3.10.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>
#include <iomanip>

using namespace std;

int main() {
    ifstream fin_arr("array.txt"), fin_cfg("config.txt");
    ofstream fout("output.txt");
    if (!fin_arr || !fin_cfg || !fout) { cerr << "Loi file!\n"; return 1; }

    int n; fin_arr >> n;
    vector<double> pre(n + 1, 0.0);
    for (int i = 1; i <= n; ++i) {
        double val; fin_arr >> val;
        pre[i] = pre[i - 1] + val;
    }

    int minLen; fin_cfg >> minLen;

    double max_mean = -1e18;
    int bestL = -1, bestR = -1, bestLen = -1;

    for (int l = 0; l < n; ++l) {
        for (int r = l + minLen - 1; r < n; ++r) {
            int len = r - l + 1;
            double mean = (pre[r + 1] - pre[l]) / len;

            bool better = false;
            // Độ chính xác float epsilon
            if (mean > max_mean + 1e-9) better = true;
            else if (abs(mean - max_mean) <= 1e-9) {
                if (len > bestLen) better = true;
                else if (len == bestLen && l < bestL) better = true;
            }

            if (better) {
                max_mean = mean;
                bestLen = len;
                bestL = l;
                bestR = r;
            }
        }
    }

```

```
        if (bestL != -1) {
            fout << fixed << setprecision(6);
            fout << max_mean << " " << bestL << " " << bestR << " " << bestLen << "\n";
        }
        return 0;
    }
```

3.10.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (array.txt):

```
5 10.5 -2.0 4.0 0.0 5.5
```

Expected Output:

```
min=-2.0000 / max=10.5000 / mean=3.6000
```

Test Case 2 (Case biên – n=0)
Input (array.txt):

```
0
```

Expected Output:

```
EMPTY
```

Test Case 3 (Case biên – n=1)
Input (array.txt):

```
1 42.5
```

Expected Output:

```
min=42.5000 / max=42.5000 / mean=42.5000
```

4 Tuần 4: Kadane + LIS O(n²) + Tư duy dãy con

4.1 Max subarray sum (Kadane cơ bản) [Cơ bản]

4.1.1 Phân tích & Thiết kế (M0)

- **Input:** File `array.txt` chứa:
  - Số nguyên n
  - n số thực
- **Xử lý:**
  - **Case biên:**
    - $n = 0 \to$ in `EMPTY`
    - $n = 1 \to \min = \max = \text{mean} = a[0]$
  - **Module 1 (Đọc file):**
    - Hàm `readArray(path, vector<double>&)`
    - Kiểm tra mở file (fail $\to$ `cerr` + `return false`)
  - **Module 2 (Tính mean):**
    - Dùng `long double` để tránh sai số
  - **Module 3 (Tìm min, max):**
    - Duyệt mảng 1 lần
- **Output:**

```
min=<f> / max=<f> / mean=<f>
```

(với `setprecision(4)`)

4.1.2 Mã nguồn C++11 (Tách module)

```
#include <iostream>
#include <vector>
#include <fstream>
```

```
#include <algorithm>

using namespace std;

bool readArray(const string& path, vector<long long>& a) {
    ifstream fin(path);
    if (!fin) return false;
    int n;
    if (fin >> n) {
        a.resize(n);
        for (int i = 0; i < n; ++i) fin >> a[i];
    }
    return true;
}

long long kadane(const vector<long long>& a) {
    long long bestEnding = a[0];
    long long bestSoFar = a[0];
    for (size_t i = 1; i < a.size(); ++i) {
        bestEnding = max(a[i], bestEnding + a[i]);
        bestSoFar = max(bestSoFar, bestEnding);
    }
    return bestSoFar;
}

int main() {
    vector<long long> a;
    if (!readArray("array.txt", a)) {
        cerr << "Loi mo file input!\n";
        return 1;
    }

    ofstream fout("output.txt");
    if (!fout) { cerr << "Loi mo file output!\n"; return 1; }

    if (a.empty()) {
        fout << "EMPTY\n";
    } else {
        fout << "max_sum=" << kadane(a) << "\n";
    }
    return 0;
}
```

4.1.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (array.txt):

```
5 10.5 -2.0 4.0 0.0 5.5
```

Expected Output:

```
min=-2.0000 / max=10.5000 / mean=3.6000
```

Test Case 2 (Case biên – n=0)
Input (array.txt):

```
0
```

Expected Output:

```
EMPTY
```

Test Case 3 (Case biên – n=1)
Input (array.txt):

```
1 42.5
```

Expected Output:

```
min=42.5000 / max=42.5000 / mean=42.5000
```

4.2 Kadane có truy vết l, r [Cơ bản]

4.2.1 Phân tích & Thiết kế (M0)

- **Input:** File `array.txt` chứa:
  - Số nguyên n
  - n số thực
- **Xử lý:**
  - **Case biên:**
    - $n = 0 \rightarrow$ in `EMPTY`
    - $n = 1 \rightarrow \text{min} = \text{max} = \text{mean} = a[0]$
  - **Module 1 (Đọc file):**
    - Hàm `readArray(path, vector<double>&)`
    - Kiểm tra mở file (fail $\rightarrow$ `cerr` + `return false`)
  - **Module 2 (Tính mean):**
    - Dùng `long double` để tránh sai số
  - **Module 3 (Tìm min, max):**
    - Duyệt mảng 1 lần
- **Output:**

```
min=<f> / max=<f> / mean=<f>
```

(với `setprecision(4)`)

4.2.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>

using namespace std;

struct SubarrayResult {
    long long sum;
    int l, r;
};

SubarrayResult kadaneWithTrace(const vector<long long>& a) {
    long long bestEnding = a[0];
    long long bestSoFar = a[0];
    int start = 0, bestL = 0, bestR = 0;

    for (int i = 1; i < (int)a.size(); ++i) {
        if (a[i] > bestEnding + a[i]) {
            bestEnding = a[i];
            start = i;
        } else {
            bestEnding += a[i];
        }

        if (bestEnding > bestSoFar) {
            bestSoFar = bestEnding;
            bestL = start;
            bestR = i;
        }
    }
    return {bestSoFar, bestL, bestR};
}

int main() {
    ifstream fin("array.txt");
    ofstream fout("output.txt");
    if (!fin || !fout) { cerr << "Loi mo file!\n"; return 1; }

    int n; fin >> n;
    if (n == 0) { fout << "EMPTY\n"; return 0; }

    vector<long long> a(n);
    for (int i = 0; i < n; ++i) fin >> a[i];

    SubarrayResult res = kadaneWithTrace(a);
    fout << "max_sum=" << res.sum << " / l=" << res.l << " / r=" << res.r << "\n";
```

```
        return 0;
    }
```

4.2.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (array.txt):

```
5 10.5 -2.0 4.0 0.0 5.5
```

Expected Output:

```
min=-2.0000 / max=10.5000 / mean=3.6000
```

Test Case 2 (Case biên – n=0)
Input (array.txt):

```
0
```

Expected Output:

```
EMPTY
```

Test Case 3 (Case biên – n=1)
Input (array.txt):

```
1 42.5
```

Expected Output:

```
min=42.5000 / max=42.5000 / mean=42.5000
```

4.3 LIS độ dài O(n²) [Cơ bản]

4.3.1 Phân tích & Thiết kế (M0)

- **Input:** File `array.txt` chứa:
  - Số nguyên n
  - n số thực
- **Xử lý:**
  - **Case biên:**
    - $n = 0 \rightarrow$ in `EMPTY`
    - $n = 1 \rightarrow \min = \max = \text{mean} = a[0]$
  - **Module 1 (Đọc file):**
    - Hàm `readArray(path, vector<double>&)`

    * Kiểm tra mở file (fail → `cerr` + `return false`)
  - **Module 2 (Tính mean):**
    * Dùng `long double` để tránh sai số
  - **Module 3 (Tìm min, max):**
    * Duyệt mảng 1 lần

- **Output:**

```
min=<f> / max=<f> / mean=<f>
```

(với `setprecision(4)`)

4.3.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>
#include <algorithm>

using namespace std;

int lisLength(const vector<int>& a) {
    if (a.empty()) return 0;
    vector<int> dp(a.size(), 1);
    int max_len = 1;

    for (int i = 1; i < (int)a.size(); ++i) {
        for (int j = 0; j < i; ++j) {
            if (a[j] < a[i]) {
                dp[i] = max(dp[i], dp[j] + 1);
            }
        }
        max_len = max(max_len, dp[i]);
    }
    return max_len;
}

int main() {
    ifstream fin("array.txt");
    ofstream fout("output.txt");
    if (!fin || !fout) { cerr << "Loi file!\n"; return 1; }

    int n;
    if (!(fin >> n) || n == 0) {
        fout << "lis_len=0\n";
        return 0;
    }

```

```
        vector<int> a(n);
        for (int i = 0; i < n; ++i) fin >> a[i];

        fout << "lis_len=" << lisLength(a) << "\n";
        return 0;
    }
```

4.3.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (array.txt):

```
5 10.5 -2.0 4.0 0.0 5.5
```

Expected Output:

```
min=-2.0000 / max=10.5000 / mean=3.6000
```

Test Case 2 (Case biên – n=0)
Input (array.txt):

```
0
```

Expected Output:

```
EMPTY
```

Test Case 3 (Case biên – n=1)
Input (array.txt):

```
1 42.5
```

Expected Output:

```
min=42.5000 / max=42.5000 / mean=42.5000
```

4.4 Min subarray sum [Trung bình]

4.4.1 Phân tích & Thiết kế (M0)

- **Input:** File `array.txt` chứa:
  - Số nguyên n
  - n số thực
- **Xử lý:**
  - **Case biên:**
    - $n = 0 \rightarrow$ in `EMPTY`

    - $n = 1 \rightarrow \text{min} = \text{max} = \text{mean} = a[0]$
  - **Module 1 (Đọc file):**
    - Hàm `readArray(path, vector<double>&)`
    - Kiểm tra mở file (fail → `cerr` + `return false`)
  - **Module 2 (Tính mean):**
    - Dùng `long double` để tránh sai số
  - **Module 3 (Tìm min, max):**
    - Duyệt mảng 1 lần

- **Output:**

```
min=<f> / max=<f> / mean=<f>
```

(với `setprecision(4)`)

4.4.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>

using namespace std;

struct SubarrayResult {
    long long sum;
    int l, r;
};

SubarrayResult minSubarray(const vector<long long>& a) {
    long long bestEnding = a[0];
    long long bestSoFar = a[0];
    int start = 0, bestL = 0, bestR = 0;

    for (int i = 1; i < (int)a.size(); ++i) {
        if (a[i] < bestEnding + a[i]) {
            bestEnding = a[i];
            start = i;
        } else {
            bestEnding += a[i];
        }

        if (bestEnding < bestSoFar) {
            bestSoFar = bestEnding;
            bestL = start;
            bestR = i;
        }
    }
```

```
        return {bestSoFar, bestL, bestR};
    }

    int main() {
        ifstream fin("array.txt");
        ofstream fout("output.txt");
        if (!fin || !fout) { cerr << "Loi file!\n"; return 1; }

        int n; fin >> n;
        if (n == 0) { fout << "EMPTY\n"; return 0; }

        vector<long long> a(n);
        for (int i = 0; i < n; ++i) fin >> a[i];

        SubarrayResult res = minSubarray(a);
        fout << "min_sum=" << res.sum << " / l=" << res.l << " / r=" << res.r << "\n";
        return 0;
    }
```

4.4.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (array.txt):

```
5 10.5 -2.0 4.0 0.0 5.5
```

Expected Output:

```
min=-2.0000 / max=10.5000 / mean=3.6000
```

Test Case 2 (Case biên – n=0)
Input (array.txt):

```
0
```

Expected Output:

```
EMPTY
```

Test Case 3 (Case biên – n=1)
Input (array.txt):

```
1 42.5
```

Expected Output:

```
min=42.5000 / max=42.5000 / mean=42.5000
```

4.5 LIS truy vết dãy con [Trung bình]

4.5.1 Phân tích & Thiết kế (M0)

- **Input:** File `array.txt` chứa:
  - Số nguyên n
  - n số thực
- **Xử lý:**
  - **Case biên:**
    - $n = 0 \rightarrow$ in `EMPTY`
    - $n = 1 \rightarrow \min = \max = \text{mean} = a[0]$
  - **Module 1 (Đọc file):**
    - Hàm `readArray(path, vector<double>&)`
    - Kiểm tra mở file (fail $\rightarrow$ `cerr` + `return false`)
  - **Module 2 (Tính mean):**
    - Dùng `long double` để tránh sai số
  - **Module 3 (Tìm min, max):**
    - Duyệt mảng 1 lần
- **Output:**

```
min=<f> / max=<f> / mean=<f>
```

(với `setprecision(4)`)

4.5.2 Mã nguồn C++11 (Tách module)

```cpp
    #include <iostream>
#include <vector>
#include <fstream>
#include <algorithm>

using namespace std;

void traceLIS(const vector<int>& a, vector<int>& indices) {
    if (a.empty()) return;
    int n = a.size();
    vector<int> dp(n, 1), parent(n, -1);

    int max_len = 1, end_idx = 0;

    for (int i = 1; i < n; ++i) {
        for (int j = 0; j < i; ++j) {
```

```
            if (a[j] < a[i] && dp[j] + 1 > dp[i]) {
                dp[i] = dp[j] + 1;
                parent[i] = j;
            }
        }
        if (dp[i] > max_len) {
            max_len = dp[i];
            end_idx = i;
        }
    }

    int curr = end_idx;
    while (curr != -1) {
        indices.push_back(curr);
        curr = parent[curr];
    }
    reverse(indices.begin(), indices.end());
}

int main() {
    ifstream fin("array.txt");
    ofstream fout("output.txt");
    if (!fin || !fout) { cerr << "Loi file!\n"; return 1; }

    int n; fin >> n;
    if (n == 0) { fout << "lis_len=0 / indices=\n"; return 0; }

    vector<int> a(n);
    for (int i = 0; i < n; ++i) fin >> a[i];

    vector<int> indices;
    traceLIS(a, indices);

    fout << "lis_len=" << indices.size() << " / indices=";
    for (size_t i = 0; i < indices.size(); ++i) {
        fout << indices[i] << (i + 1 == indices.size() ? "" : " ");
    }
    fout << "\n";
    return 0;
}
```

4.5.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (array.txt):

```
5 10.5 -2.0 4.0 0.0 5.5
```

Expected Output:

```
min=-2.0000 / max=10.5000 / mean=3.6000
```

Test Case 2 (Case biên – n=0)
Input (array.txt):

```
0
```

Expected Output:

```
EMPTY
```

Test Case 3 (Case biên – n=1)
Input (array.txt):

```
1 42.5
```

Expected Output:

```
min=42.5000 / max=42.5000 / mean=42.5000
```

4.6 Đoạn có tổng lớn nhất độ dài đúng k [Trung bình]

4.6.1 Phân tích & Thiết kế (M0)

- **Input:** File `array.txt` chứa:
  - Số nguyên n
  - n số thực
- **Xử lý:**
  - **Case biên:**
    - $n = 0 \rightarrow$ in `EMPTY`
    - $n = 1 \rightarrow \min = \max = \text{mean} = a[0]$
  - **Module 1 (Đọc file):**
    - Hàm `readArray(path, vector<double>&)`
    - Kiểm tra mở file (fail $\rightarrow$ `cerr` + `return false`)
  - **Module 2 (Tính mean):**
    - Dùng `long double` để tránh sai số
  - **Module 3 (Tìm min, max):**
    - Duyệt mảng 1 lần
- **Output:**

```
min=<f> / max=<f> / mean=<f>
```

(với `setprecision(4)`)

4.6.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>

using namespace std;

int main() {
    ifstream fin_arr("array.txt"), fin_cfg("config.txt");
    ofstream fout("output.txt");
    if (!fin_arr || !fin_cfg || !fout) { cerr << "Loi mo file!\n"; return 1; }

    int n; fin_arr >> n;
    vector<long long> a(n);
    for (int i = 0; i < n; ++i) fin_arr >> a[i];

    int k; fin_cfg >> k;

    if (k > n) {
        fout << "IMPOSSIBLE\n";
        return 0;
    }

    long long current_sum = 0;
    for (int i = 0; i < k; ++i) current_sum += a[i];

    long long max_sum = current_sum;
    int bestL = 0, bestR = k - 1;

    for (int i = k; i < n; ++i) {
        current_sum = current_sum + a[i] - a[i - k];
        if (current_sum > max_sum) {
            max_sum = current_sum;
            bestL = i - k + 1;
            bestR = i;
        }
    }

    fout << "max_sum=" << max_sum << " / l=" << bestL << " / r=" << bestR << "\n";
    return 0;
}
```

4.6.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)

Input (array.txt):

```
5 10.5 -2.0 4.0 0.0 5.5
```

Expected Output:

```
min=-2.0000 / max=10.5000 / mean=3.6000
```

Test Case 2 (Case biên – n=0)
Input (array.txt):

```
0
```

Expected Output:

```
EMPTY
```

Test Case 3 (Case biên – n=1)
Input (array.txt):

```
1 42.5
```

Expected Output:

```
min=42.5000 / max=42.5000 / mean=42.5000
```

4.7 Kadane với nhiều bộ test (Prefix) [Trung bình]

4.7.1 Phân tích & Thiết kế (M0)

- **Input:** File `array.txt` chứa:
  - Số nguyên n
  - n số thực
- **Xử lý:**
  - **Case biên:**
    - $n = 0 \rightarrow$ in `EMPTY`
    - $n = 1 \rightarrow \text{min} = \text{max} = \text{mean} = a[0]$
  - **Module 1 (Đọc file):**
    - Hàm `readArray(path, vector<double>&)`
    - Kiểm tra mở file (fail $\rightarrow$ `cerr` + `return false`)
  - **Module 2 (Tính mean):**
    - Dùng `long double` để tránh sai số
  - **Module 3 (Tìm min, max):**
    - Duyệt mảng 1 lần

- **Output:**

```
min=<f> / max=<f> / mean=<f>
```

(với `setprecision(4)`)

4.7.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>
#include <algorithm>

using namespace std;

long long kadane(const vector<long long>& a) {
    long long bestEnding = a[0], bestSoFar = a[0];
    for (size_t i = 1; i < a.size(); ++i) {
        bestEnding = max(a[i], bestEnding + a[i]);
        bestSoFar = max(bestSoFar, bestEnding);
    }
    return bestSoFar;
}

int main() {
    ifstream fin("multi_test.txt");
    ofstream fout("output.txt");
    if (!fin || !fout) { cerr << "Loi mo file!\n"; return 1; }

    int T;
    if (!(fin >> T)) return 0;

    while (T--) {
        int n; fin >> n;
        if (n == 0) {
            fout << "EMPTY\n";
            continue;
        }
        vector<long long> a(n);
        for (int i = 0; i < n; ++i) fin >> a[i];

        fout << kadane(a) << "\n";
    }
    return 0;
}
```

4.7.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)

Input (array.txt):

```
5 10.5 -2.0 4.0 0.0 5.5
```

Expected Output:

```
min=-2.0000 / max=10.5000 / mean=3.6000
```

Test Case 2 (Case biên – n=0)
Input (array.txt):

```
0
```

Expected Output:

```
EMPTY
```

Test Case 3 (Case biên – n=1)
Input (array.txt):

```
1 42.5
```

Expected Output:

```
min=42.5000 / max=42.5000 / mean=42.5000
```

4.8 Longest Non-Decreasing Subsequence [Trung bình]

4.8.1 Phân tích & Thiết kế (M0)

- **Input:** File `array.txt` chứa:
  - Số nguyên n
  - n số thực
- **Xử lý:**
  - **Case biên:**
    - $n = 0 \rightarrow$ in `EMPTY`
    - $n = 1 \rightarrow \text{min} = \text{max} = \text{mean} = a[0]$
  - **Module 1 (Đọc file):**
    - Hàm `readArray(path, vector<double>&)`
    - Kiểm tra mở file (fail $\rightarrow$ `cerr` + `return false`)
  - **Module 2 (Tính mean):**
    - Dùng `long double` để tránh sai số
  - **Module 3 (Tìm min, max):**

∗ Duyệt mảng 1 lần

- **Output:**

```
min=<f> / max=<f> / mean=<f>
```

(với `setprecision(4)`)

4.8.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>
#include <algorithm>

using namespace std;

int lndsLength(const vector<int>& a) {
    if (a.empty()) return 0;
    vector<int> dp(a.size(), 1);
    int max_len = 1;

    for (int i = 1; i < (int)a.size(); ++i) {
        for (int j = 0; j < i; ++j) {
            if (a[j] <= a[i]) {
                dp[i] = max(dp[i], dp[j] + 1);
            }
        }
        max_len = max(max_len, dp[i]);
    }
    return max_len;
}

int main() {
    ifstream fin("array.txt");
    ofstream fout("output.txt");
    if (!fin || !fout) { cerr << "Loi file!\n"; return 1; }

    int n;
    if (!(fin >> n) || n == 0) { fout << "lnds_len=0\n"; return 0; }

    vector<int> a(n);
    for (int i = 0; i < n; ++i) fin >> a[i];

    fout << "lnds_len=" << lndsLength(a) << "\n";
    return 0;
}
```

4.8.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (array.txt):

```
5 10.5 -2.0 4.0 0.0 5.5
```

Expected Output:

```
min=-2.0000 / max=10.5000 / mean=3.6000
```

Test Case 2 (Case biên – n=0)
Input (array.txt):

```
0
```

Expected Output:

```
EMPTY
```

Test Case 3 (Case biên – n=1)
Input (array.txt):

```
1 42.5
```

Expected Output:

```
min=42.5000 / max=42.5000 / mean=42.5000
```

4.9 Maximum circular subarray [Nâng cao]

4.9.1 Phân tích & Thiết kế (M0)

- **Input:** File `array.txt` chứa:
  - Số nguyên n
  - n số thực
- **Xử lý:**
  - **Case biên:**
    - $n = 0 \rightarrow$ in `EMPTY`
    - $n = 1 \rightarrow \min = \max = \text{mean} = a[0]$
  - **Module 1 (Đọc file):**
    - Hàm `readArray(path, vector<double>&)`
    - Kiểm tra mở file (fail $\rightarrow$ `cerr` + `return false`)
  - **Module 2 (Tính mean):**
    - Dùng `long double` để tránh sai số

– **Module 3 (Tìm min, max):**

  * Duyệt mảng 1 lần

- **Output:**

```
min=<f> / max=<f> / mean=<f>
```

(với `setprecision(4)`)

4.9.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>
#include <algorithm>

using namespace std;

long long kadaneMax(const vector<long long>& a) {
    long long bestEnding = a[0], bestSoFar = a[0];
    for (size_t i = 1; i < a.size(); ++i) {
        bestEnding = max(a[i], bestEnding + a[i]);
        bestSoFar = max(bestSoFar, bestEnding);
    }
    return bestSoFar;
}

long long kadaneMin(const vector<long long>& a) {
    long long bestEnding = a[0], bestSoFar = a[0];
    for (size_t i = 1; i < a.size(); ++i) {
        bestEnding = min(a[i], bestEnding + a[i]);
        bestSoFar = min(bestSoFar, bestEnding);
    }
    return bestSoFar;
}

int main() {
    ifstream fin("array.txt");
    ofstream fout("output.txt");
    if (!fin || !fout) { cerr << "Loi file!\n"; return 1; }

    int n; fin >> n;
    if (n == 0) { fout << "EMPTY\n"; return 0; }

    vector<long long> a(n);
    long long total_sum = 0;
    for (int i = 0; i < n; ++i) {
        fin >> a[i];
        total_sum += a[i];
```

```
        }

        long long max_k = kadaneMax(a);
        long long min_k = kadaneMin(a);

        long long result;
        if (max_k < 0) {
            result = max_k;
        } else {
            result = max(max_k, total_sum - min_k);
        }

        fout << "max_circular=" << result << "\n";
        return 0;
    }
```

4.9.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (array.txt):

```
5 10.5 -2.0 4.0 0.0 5.5
```

Expected Output:

```
min=-2.0000 / max=10.5000 / mean=3.6000
```

Test Case 2 (Case biên – n=0)
Input (array.txt):

```
0
```

Expected Output:

```
EMPTY
```

Test Case 3 (Case biên – n=1)
Input (array.txt):

```
1 42.5
```

Expected Output:

```
min=42.5000 / max=42.5000 / mean=42.5000
```

4.10 Đếm số dãy con tăng có độ dài = L [Nâng cao]

4.10.1 Phân tích & Thiết kế (M0)

- **Input:** File `array.txt` chứa:

  - Số nguyên n
  - n số thực

- **Xử lý:**
  - **Case biên:**
    * $n = 0 \rightarrow$ in `EMPTY`
    * $n = 1 \rightarrow \text{min} = \text{max} = \text{mean} = a[0]$
  - **Module 1 (Đọc file):**
    * Hàm `readArray(path, vector<double>&)`
    * Kiểm tra mở file (fail $\rightarrow$ `cerr` + `return false`)
  - **Module 2 (Tính mean):**
    * Dùng `long double` để tránh sai số
  - **Module 3 (Tìm min, max):**
    * Duyệt mảng 1 lần

- **Output:**

```
min=<f> / max=<f> / mean=<f>
```

(với `setprecision(4)`)

4.10.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>

using namespace std;

int main() {
    ifstream fin_arr("array.txt"), fin_cfg("config.txt");
    ofstream fout("output.txt");
    if (!fin_arr || !fin_cfg || !fout) { cerr << "Loi mo file!\n"; return 1; }

    int n; fin_arr >> n;
    vector<int> a(n);
    for (int i = 0; i < n; ++i) fin_arr >> a[i];

    int L; fin_cfg >> L;

    if (L <= 0 || L > n) {
        fout << "count=0\n";
        return 0;
    }
```

```

        // dp[i][len]: ốS dãy con ătng độ dài len, ếkt thúc ạti a[i]
        vector<vector<long long>> dp(n, vector<long long>(L + 1, 0));

        for (int i = 0; i < n; ++i) dp[i][1] = 1;

        for (int len = 2; len <= L; ++len) {
            for (int i = 0; i < n; ++i) {
                for (int j = 0; j < i; ++j) {
                    if (a[j] < a[i]) {
                        dp[i][len] += dp[j][len - 1];
                    }
                }
            }
        }

        long long total_count = 0;
        for (int i = 0; i < n; ++i) {
            total_count += dp[i][L];
        }

        fout << "count=" << total_count << "\n";
        return 0;
    }
```

4.10.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (array.txt):

```
5 10.5 -2.0 4.0 0.0 5.5
```

Expected Output:

```
min=-2.0000 / max=10.5000 / mean=3.6000
```

Test Case 2 (Case biên – n=0)
Input (array.txt):

```
0
```

Expected Output:

```
EMPTY
```

Test Case 3 (Case biên – n=1)
Input (array.txt):

```
1 42.5
```

Expected Output:

```
    min=42.5000 / max=42.5000 / mean=42.5000
```

5 Tuần 5: Two Pointers / Sliding Window + Rolling Mean

5.1 Đoạn ngắn nhất tổng $\geq S$ [Cơ bản]

5.1.1 Phân tích & Thiết kế (M0)

- **Input:** File `input.txt` chứa:
  - Số nguyên n
  - n số nguyên không âm
  - Số nguyên S
- **Xử lý:**
  - **Case biên:**
    * $n = 0 \rightarrow$ in `NONE`
    * $S = 0 \rightarrow$ in `len=1 / l=0 / r=0` (nếu tồn tại phần tử)
    * Không tồn tại đoạn nào có tổng $\geq S \rightarrow$ in `NONE`
  - **Module 1 (Đọc file):**
    * Hàm `readInput(path, vector<long long>&, long long&)`
    * Kiểm tra mở file (`fail` $\rightarrow$ `cerr + return false`)
  - **Module 2 (Tìm đoạn ngắn nhất):**
    * Dùng kỹ thuật `Sliding Window`
    * Tăng `right`, cộng dồn tổng
    * Khi tổng $\geq S$ thì thu hẹp từ trái
  - **Module 3 (Ghi file):**
    * Nếu tồn tại: ghi `len=<v> / l=<i> / r=<j>`
    * Ngược lại ghi `NONE`
- **Output:**

```
len=<v> / l=<i> / r=<j>
```

Hoặc:

```
NONE
```

5.1.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>
#include <climits>

using namespace std;

struct ShortestAns {
    bool ok;
    int len, l, r;
};

bool readInput(const string& path, vector<long long>& a, long long& S) {
    ifstream fin(path);
    if (!fin) return false;
    int n;
    if (fin >> n) {
        a.resize(n);
        for (int i = 0; i < n; ++i) fin >> a[i];
        fin >> S;
    }
    return true;
}

ShortestAns shortestAtLeastS(const vector<long long>& a, long long S) {
    int n = (int)a.size();
    if (n == 0) return {false, 0, -1, -1};
    if (S == 0) return {true, 1, 0, 0};

    long long sum = 0;
    int left = 0;
    int bestLen = INT_MAX, bestL = -1, bestR = -1;

    for (int right = 0; right < n; ++right) {
        sum += a[right];

        while (sum >= S) {
            if (right - left + 1 < bestLen) {
                bestLen = right - left + 1;
                bestL = left;
                bestR = right;
            }
            sum -= a[left];
            ++left;
        }
    }

```

```
    if (bestLen == INT_MAX) return {false, 0, -1, -1};
    return {true, bestLen, bestL, bestR};
}

bool writeOutput(const string& path, const ShortestAns& ans) {
    ofstream fout(path);
    if (!fout) return false;

    if (!ans.ok) fout << "NONE";
    else fout << "len=" << ans.len
              << " / l=" << ans.l
              << " / r=" << ans.r;
    return true;
}

int main() {
    vector<long long> a;
    long long S;

    if (!readInput("input.txt", a, S)) {
        cerr << "Loi mo file input!\n";
        return 1;
    }

    ShortestAns ans = shortestAtLeastS(a, S);

    if (!writeOutput("output.txt", ans)) {
        cerr << "Loi mo file output!\n";
        return 1;
    }

    return 0;
}
```

5.1.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (input.txt):

```
6
2 3 1 2 4 3
7
```

Expected Output:

```
len=2 / l=4 / r=5
```

Test Case 2 (Không tồn tại)
Input (input.txt):

```
4
1 1 1 1
10
```

Expected Output:

```
NONE
```

Test Case 3 (Case biên – S=0)
Input (input.txt):

```
3
5 2 8
0
```

Expected Output:

```
len=1 / l=0 / r=0
```

5.2 Rolling mean cửa sổ k [Cơ bản]

5.2.1 Phân tích & Thiết kế (M0)

- **Input:** File `input.txt` chứa:
  - Số nguyên n
  - n số thực
  - Số nguyên k
- **Xử lý:**
  - **Case biên:**
    - $n = 0 \rightarrow$ ghi chỉ header
    - $k > n \rightarrow$ không có cửa sổ nào, ghi chỉ header
    - $k = n \rightarrow$ đúng 1 cửa sổ
    - $k = 1 \rightarrow mean[i] = a[i]$
  - **Module 1 (Đọc file):**
    - Hàm `readInput(path, vector<double>&, int&)`
    - Kiểm tra mở file (`fail` → `cerr + return false`)
  - **Module 2 (Tính rolling mean):**
    - Tính tổng cửa sổ đầu tiên
    - Mỗi bước cập nhật:

```
sum = sum + a[i+k] - a[i]
```

    * Mean = sum / k
  - **Module 3 (Ghi file CSV):**
    * Ghi file `rolling.csv`
    * Header: `i,mean`
    * Mỗi dòng: chỉ số và giá trị mean

- **Output:**

```
i,mean
0,...
1,...
```

(với `setprecision(6)`)

5.2.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>
#include <iomanip>

using namespace std;

bool readInput(const string& path, vector<double>& a, int& k) {
    ifstream fin(path);
    if (!fin) return false;

    int n;
    if (fin >> n) {
        a.resize(n);
        for (int i = 0; i < n; ++i) fin >> a[i];
        fin >> k;
    }
    return true;
}

vector<double> rollingMean(const vector<double>& a, int k) {
    int n = (int)a.size();
    vector<double> ans;

    if (n == 0 || k > n || k <= 0) return ans;

    long double sum = 0;
    for (int i = 0; i < k; ++i) sum += a[i];

    ans.push_back((double)(sum / k));

    for (int i = 0; i + k < n; ++i) {
```

```
        sum = sum + a[i + k] - a[i];
        ans.push_back((double)(sum / k));
    }

    return ans;
}

bool writeCSV(const string& path, const vector<double>& mean) {
    ofstream fout(path);
    if (!fout) return false;

    fout << "i,mean\n";
    fout << fixed << setprecision(6);

    for (size_t i = 0; i < mean.size(); ++i) {
        fout << i << "," << mean[i] << "\n";
    }
    return true;
}

int main() {
    vector<double> a;
    int k;

    if (!readInput("input.txt", a, k)) {
        cerr << "Loi mo file input!\n";
        return 1;
    }

    vector<double> mean = rollingMean(a, k);

    if (!writeCSV("rolling.csv", mean)) {
        cerr << "Loi mo file output!\n";
        return 1;
    }

    return 0;
}
```

5.2.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (input.txt):

```
5
1 2 3 4 5
3
```

Expected Output (rolling.csv):

```
i,mean
0,2.000000
1,3.000000
2,4.000000
```

Test Case 2 (Case biên – k=n)
Input (input.txt):

```
4
2 4 6 8
4
```

Expected Output (rolling.csv):

```
i,mean
0,5.000000
```

Test Case 3 (Case biên – k>n)
Input (input.txt):

```
3
1 2 3
5
```

Expected Output (rolling.csv):

```
i,mean
```

5.3 Đoạn dài nhất không vượt quá S [Cơ bản]

5.3.1 Phân tích & Thiết kế (M0)

- **Input:** File `input.txt` chứa:
  - Số nguyên n
  - n số nguyên không âm
  - Số nguyên S
- **Xử lý:**
  - **Case biên:**
    - $n = 0 \rightarrow max_len = 0$
    - Không có đoạn nào hợp lệ $\rightarrow max_len = 0$
    - Nếu $a[0] > S$ và mọi phần tử đều lớn hơn $S \rightarrow max_len = 0$
  - **Module 1 (Đọc file):**
    - Hàm `readInput(path, vector<long long>&, long long&)`

    * Kiểm tra mở file (`fail` → `cerr + return false`)
  - **Module 2 (Tìm đoạn dài nhất):**
    * Dùng kỹ thuật `Sliding Window`
    * Tăng `right`, cộng vào tổng
    * Khi tổng $> S$ thì tăng `left` đến khi tổng $\leq S$
    * Sau mỗi bước cập nhật độ dài lớn nhất
  - **Module 3 (Ghi file):**
    * Ghi:

```
max_len=<v> / l=<i> / r=<j>
```

- **Output:**

```
max_len=<v> / l=<i> / r=<j>
```

5.3.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>

using namespace std;

struct LongestAns {
    int len, l, r;
};

bool readInput(const string& path, vector<long long>& a, long long& S) {
    ifstream fin(path);
    if (!fin) return false;

    int n;
    if (fin >> n) {
        a.resize(n);
        for (int i = 0; i < n; ++i) fin >> a[i];
        fin >> S;
    }
    return true;
}

LongestAns longestAtMostS(const vector<long long>& a, long long S) {
    int n = (int)a.size();
    if (n == 0) return {0, -1, -1};

    long long sum = 0;
    int left = 0;
```

```
    int bestLen = 0, bestL = -1, bestR = -1;

    for (int right = 0; right < n; ++right) {
        sum += a[right];

        while (left <= right && sum > S) {
            sum -= a[left];
            ++left;
        }

        if (sum <= S && right - left + 1 > bestLen) {
            bestLen = right - left + 1;
            bestL = left;
            bestR = right;
        }
    }

    return {bestLen, bestL, bestR};
}

bool writeOutput(const string& path, const LongestAns& ans) {
    ofstream fout(path);
    if (!fout) return false;

    fout << "max_len=" << ans.len
         << " / l=" << ans.l
         << " / r=" << ans.r;
    return true;
}

int main() {
    vector<long long> a;
    long long S;

    if (!readInput("input.txt", a, S)) {
        cerr << "Loi mo file input!\n";
        return 1;
    }

    LongestAns ans = longestAtMostS(a, S);

    if (!writeOutput("output.txt", ans)) {
        cerr << "Loi mo file output!\n";
        return 1;
    }

    return 0;
}
```

5.3.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (input.txt):

```
5
1 2 1 1 3
4
```

Expected Output:

```
max_len=4 / l=0 / r=3
```

Test Case 2 (Không có đoạn nào)
Input (input.txt):

```
3
5 6 7
4
```

Expected Output:

```
max_len=0 / l=-1 / r=-1
```

Test Case 3 (Case biên – n=0)
Input (input.txt):

```
0
4
```

Expected Output:

```
max_len=0 / l=-1 / r=-1
```

5.4 Rolling max cửa sổ k [Trung bình]

5.4.1 Phân tích & Thiết kế (M0)

- **Input:** File `input.txt` chứa:
  - Số nguyên n
  - n số thực
  - Số nguyên k
- **Xử lý:**
  - **Case biên:**
    - $n = 0 \rightarrow$ ghi chỉ header
    - $k > n \rightarrow$ không có cửa sổ nào, ghi chỉ header

    * $k = n \rightarrow$ đúng 1 cửa sổ
    * $k = 1 \rightarrow$ max[i] = a[i]
  - **Module 1 (Đọc file):**

```
readInput(path, vector<double>&, int&)
```

  - **Module 2 (Tính rolling max):**
    * Dùng `deque<int>` lưu chỉ số
    * Bỏ chỉ số ra ngoài cửa sổ
    * Bỏ các phần tử nhỏ hơn hoặc bằng phần tử mới
    * Phần tử đầu deque luôn là max
    * Độ phức tạp $O(n)$
  - **Module 3 (Ghi file CSV):**
    * File `rolling_max.csv`
    * Header: `i,max`

- **Output:**

```
i,max
0,...
1,...
```

5.4.2 Mã nguồn C++11 (Tách module)

```
#include <iostream>
#include <vector>
#include <fstream>
#include <iomanip>
#include <deque>
using namespace std;

bool readInput(const string& path, vector<double>& a, int& k) {
    ifstream fin(path);
    if (!fin) return false;

    int n;
    fin >> n;
    a.resize(n);

    for (int i = 0; i < n; i++) fin >> a[i];
    fin >> k;
    return true;
}

vector<double> rollingMax(const vector<double>& a, int k) {
    int n = a.size();
```

```
    vector<double> ans;

    if (n == 0 || k <= 0 || k > n) return ans;

    deque<int> dq;

    for (int i = 0; i < n; i++) {
        while (!dq.empty() && dq.front() <= i - k)
            dq.pop_front();

        while (!dq.empty() && a[dq.back()] <= a[i])
            dq.pop_back();

        dq.push_back(i);

        if (i >= k - 1)
            ans.push_back(a[dq.front()]);
    }

    return ans;
}

bool writeCSV(const string& path, const vector<double>& mx) {
    ofstream fout(path);
    if (!fout) return false;

    fout << "i,max\n";
    fout << fixed << setprecision(6);

    for (int i = 0; i < (int)mx.size(); i++) {
        fout << i << "," << mx[i] << "\n";
    }
    return true;
}

int main() {
    vector<double> a;
    int k;

    if (!readInput("input.txt", a, k)) {
        cerr << "Loi mo file input!\n";
        return 1;
    }

    vector<double> mx = rollingMax(a, k);

    if (!writeCSV("rolling_max.csv", mx)) {
        cerr << "Loi mo file output!\n";
        return 1;
```

```
    }

    return 0;
}
```

5.4.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1
Input (input.txt):

```
8
1 3 -1 -3 5 3 6 7
3
```

Expected Output (rolling_max.csv):

```
i,max
0,3.000000
1,3.000000
2,5.000000
3,5.000000
4,6.000000
5,7.000000
```

Test Case 2
Input (input.txt):

```
4
2 8 1 6
1
```

Expected Output (rolling_max.csv):

```
i,max
0,2.000000
1,8.000000
2,1.000000
3,6.000000
```

Test Case 3
Input (input.txt):

```
3
1 2 3
5
```

Expected Output (rolling_max.csv):

```
i,max
```

5.5 Hai con trỏ: tổng 2 phần tử bằng K [Trung bình]

5.5.1 Phân tích & Thiết kế (M0)

- **Input:** File `input.txt` chứa:
  - Số nguyên n
  - n số nguyên đã sắp xếp tăng
  - Số nguyên K
- **Xử lý:**
  - **Case biên:**
    * $n = 0 \rightarrow$ in `NONE`
    * $n = 1 \rightarrow$ in `NONE`
    * Chỉ in cặp khi $i < j$
    * Nếu $a[i] = a[j] = K/2$ thì vẫn hợp lệ khi $i \neq j$
  - **Module 1 (Đọc file):**
    * Hàm `readInput(path, vector<long long>&, long long&)`
    * Kiểm tra mở file (`fail` $\rightarrow$ `cerr + return false`)
  - **Module 2 (Tìm các cặp):**
    * Dùng kỹ thuật `Two Pointers`
    * `left = 0, right = n-1`
    * Nếu tổng $= K \rightarrow$ ghi cặp, tăng trái giảm phải
    * Nếu tổng $< K \rightarrow$ tăng `left`
    * Nếu tổng $> K \rightarrow$ giảm `right`
  - **Module 3 (Ghi file):**
    * Mỗi cặp ghi:

```
i j a[i] a[j]
```

    * Nếu không có cặp nào ghi `NONE`
- **Output:**

```
i j a[i] a[j]
```

Hoặc:

```
NONE
```

5.5.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>

using namespace std;

struct PairAns {
    int i, j;
    long long x, y;
};

bool readInput(const string& path, vector<long long>& a, long long& K) {
    ifstream fin(path);
    if (!fin) return false;

    int n;
    if (fin >> n) {
        a.resize(n);
        for (int i = 0; i < n; ++i) fin >> a[i];
        fin >> K;
    }
    return true;
}

vector<PairAns> findPairs(const vector<long long>& a, long long K) {
    vector<PairAns> ans;
    int n = (int)a.size();

    int left = 0, right = n - 1;

    while (left < right) {
        long long sum = a[left] + a[right];

        if (sum == K) {
            ans.push_back({left, right, a[left], a[right]});
            ++left;
            --right;
        }
        else if (sum < K) {
            ++left;
        }
        else {
            --right;
        }
    }

    return ans;
```

```
}

bool writeOutput(const string& path, const vector<PairAns>& ans) {
    ofstream fout(path);
    if (!fout) return false;

    if (ans.empty()) {
        fout << "NONE";
    } else {
        for (size_t i = 0; i < ans.size(); ++i) {
            fout << ans[i].i << " "
                 << ans[i].j << " "
                 << ans[i].x << " "
                 << ans[i].y << "\n";
        }
    }
    return true;
}

int main() {
    vector<long long> a;
    long long K;

    if (!readInput("input.txt", a, K)) {
        cerr << "Loi mo file input!\n";
        return 1;
    }

    vector<PairAns> ans = findPairs(a, K);

    if (!writeOutput("output.txt", ans)) {
        cerr << "Loi mo file output!\n";
        return 1;
    }

    return 0;
}
```

5.5.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (input.txt):

```
6
1 2 3 4 5 6
7
```

Expected Output:

```
0 5 1 6
```

```
1 4 2 5
2 3 3 4
```

Test Case 2 (Không có cặp)
Input (input.txt):

```
4
1 2 3 4
20
```

Expected Output:

```
NONE
```

Test Case 3 (Case biên – giá trị giống nhau)
Input (input.txt):

```
4
5 5 5 5
10
```

Expected Output:

```
0 3 5 5
1 2 5 5
```

5.6 Rolling standard deviation [Trung bình]

5.6.1 Phân tích & Thiết kế (M0)

- **Input:** File `input.txt` chứa:
  - Số nguyên n
  - n số thực
  - Số nguyên k
- **Xử lý:**
  - **Case biên:**
    - $n = 0 \rightarrow$ ghi chỉ header
    - $k > n \rightarrow$ không có cửa sổ nào, ghi chỉ header
    - $k = n \rightarrow$ đúng 1 cửa sổ
    - $k = 1 \rightarrow \text{std} = 0$
  - **Module 1 (Đọc file):**

```
readInput(path, vector<double>&, int&)
```

- **Module 2 (Tính rolling std):**
  * Công thức:

```
Var = E[X^2] - (E[X])^2
```

  * Lưu `sum` và `sumSq`
  * Cập nhật mỗi bước $O(1)$
  * `std = sqrt(var)`
- **Module 3 (Ghi file CSV):**
  * File `rolling_std.csv`
  * Header: `i,std`

- **Output:**

```
i,std
0,...
1,...
```

5.6.2 Mã nguồn C++11 (Tách module)

```
#include <iostream>
#include <vector>
#include <fstream>
#include <iomanip>
#include <cmath>
using namespace std;

bool readInput(const string& path, vector<double>& a, int& k) {
    ifstream fin(path);
    if (!fin) return false;

    int n;
    fin >> n;
    a.resize(n);

    for (int i = 0; i < n; i++) fin >> a[i];
    fin >> k;
    return true;
}

vector<double> rollingStd(const vector<double>& a, int k) {
    int n = a.size();
    vector<double> ans;

    if (n == 0 || k <= 0 || k > n) return ans;

    if (k == 1) {
        for (int i = 0; i < n; i++) ans.push_back(0.0);
```

```
        return ans;
    }

    long double sum = 0.0, sumSq = 0.0;

    for (int i = 0; i < k; i++) {
        sum += a[i];
        sumSq += a[i] * a[i];
    }

    for (int start = 0; start + k <= n; start++) {
        long double mean = sum / k;
        long double var = sumSq / k - mean * mean;

        if (var < 0) var = 0;
        ans.push_back((double)sqrt((double)var));

        if (start + k < n) {
            double oldVal = a[start];
            double newVal = a[start + k];

            sum += newVal - oldVal;
            sumSq += newVal * newVal - oldVal * oldVal;
        }
    }

    return ans;
}

bool writeCSV(const string& path, const vector<double>& v) {
    ofstream fout(path);
    if (!fout) return false;

    fout << "i,std\n";
    fout << fixed << setprecision(6);

    for (int i = 0; i < (int)v.size(); i++) {
        fout << i << "," << v[i] << "\n";
    }

    return true;
}

int main() {
    vector<double> a;
    int k;

    if (!readInput("input.txt", a, k)) {
        cerr << "Loi mo file input!\n";
```

```
        return 1;
    }

    vector<double> ans = rollingStd(a, k);

    if (!writeCSV("rolling_std.csv", ans)) {
        cerr << "Loi mo file output!\n";
        return 1;
    }

    return 0;
}
```

5.6.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1
Input (input.txt):

```
5
1 2 3 4 5
2
```

Expected Output (rolling_std.csv):

```
i,std
0,0.500000
1,0.500000
2,0.500000
3,0.500000
```

Test Case 2
Input (input.txt):

```
4
2 8 1 6
1
```

Expected Output (rolling_std.csv):

```
i,std
0,0.000000
1,0.000000
2,0.000000
3,0.000000
```

Test Case 3
Input (input.txt):

```
3
1 2 3
5
```

Expected Output (rolling_std.csv):

```
i,std
```

5.7 Đoạn không âm dài nhất [Cơ bản]

5.7.1 Phân tích & Thiết kế (M0)

- **Input:** File `input.txt` chứa:
  - Số nguyên n
  - n số nguyên
- **Xử lý:**
  - **Case biên:**
    - $n = 0 \rightarrow max_len = 0$
    - Toàn số âm $\rightarrow max_len = 0$
    - Nếu tất cả phần tử $\geq 0 \rightarrow$ lấy toàn mảng
  - **Module 1 (Đọc file):**
    - Hàm `readInput(path, vector<long long>&)`
    - Kiểm tra mở file (`fail` $\rightarrow$ `cerr + return false`)
  - **Module 2 (Tìm đoạn không âm dài nhất):**
    - Duyệt mảng 1 lần
    - Nếu phần tử ≥ 0 thì mở rộng đoạn hiện tại
    - Nếu phần tử < 0 thì kết thúc đoạn hiện tại
    - Cập nhật đoạn dài nhất
  - **Module 3 (Ghi file):**
    - Ghi:

```
max_len=<v> / l=<i> / r=<j>
```

- **Output:**

```
max_len=<v> / l=<i> / r=<j>
```

5.7.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>

using namespace std;
```

```

struct SegmentAns {
    int len, l, r;
};

bool readInput(const string& path, vector<long long>& a) {
    ifstream fin(path);
    if (!fin) return false;

    int n;
    if (fin >> n) {
        a.resize(n);
        for (int i = 0; i < n; ++i) fin >> a[i];
    }
    return true;
}

SegmentAns longestNonNegative(const vector<long long>& a) {
    int n = (int)a.size();
    if (n == 0) return {0, -1, -1};

    int bestLen = 0, bestL = -1, bestR = -1;
    int curL = -1, curLen = 0;

    for (int i = 0; i < n; ++i) {
        if (a[i] >= 0) {
            if (curLen == 0) curL = i;
            ++curLen;

            if (curLen > bestLen) {
                bestLen = curLen;
                bestL = curL;
                bestR = i;
            }
        } else {
            curLen = 0;
        }
    }

    return {bestLen, bestL, bestR};
}

bool writeOutput(const string& path, const SegmentAns& ans) {
    ofstream fout(path);
    if (!fout) return false;

    fout << "max_len=" << ans.len
         << " / l=" << ans.l
         << " / r=" << ans.r;
```

```
    return true;
}

int main() {
    vector<long long> a;

    if (!readInput("input.txt", a)) {
        cerr << "Loi mo file input!\n";
        return 1;
    }

    SegmentAns ans = longestNonNegative(a);

    if (!writeOutput("output.txt", ans)) {
        cerr << "Loi mo file output!\n";
        return 1;
    }

    return 0;
}
```

5.7.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (input.txt):

```
8
1 2 -3 4 5 6 -1 2
```

Expected Output:

```
max_len=3 / l=3 / r=5
```

Test Case 2 (Toàn số âm)
Input (input.txt):

```
4
-1 -2 -3 -4
```

Expected Output:

```
max_len=0 / l=-1 / r=-1
```

Test Case 3 (Tất cả không âm)
Input (input.txt):

```
5
0 2 4 1 3
```

Expected Output:

```
max_len=5 / l=0 / r=4
```

5.8 Cặp tổng gần K nhất [Trung bình]

5.8.1 Phân tích & Thiết kế (M0)

- **Input:** File `input.txt` chứa:
  - Số nguyên n
  - n số nguyên đã sắp xếp tăng
  - Số nguyên K
- **Xử lý:**
  - **Case biên:**
    * $n < 2 \rightarrow$ không tồn tại cặp
    * Nếu nhiều cặp cùng sai khác nhỏ nhất $\rightarrow$ chọn cặp có i nhỏ nhất
  - **Module 1 (Đọc file):**
    * Hàm `readInput(path, vector<long long>&, long long&)`
    * Kiểm tra mở file (`fail` $\rightarrow$ `cerr + return false`)
  - **Module 2 (Tìm cặp gần nhất):**
    * Dùng kỹ thuật `Two Pointers`
    * `left = 0, right = n-1`
    * Tính:

      ```
      abs(a[left] + a[right] - K)
      ```

    * Cập nhật đáp án tốt nhất
    * Nếu tổng $< K$ tăng `left`, ngược lại giảm `right`
  - **Module 3 (Ghi file):**
    * Ghi:

      ```
      i, j, a[i]+a[j]
      ```

- **Output:**

```
i, j, a[i]+a[j]
```

5.8.2 Mã nguồn C++11 (Tách module)

```
#include <iostream>
#include <vector>
#include <fstream>
#include <cstdlib>
#include <climits>

using namespace std;

struct ClosestAns {
    int i, j;
    long long sum;
};

bool readInput(const string& path, vector<long long>& a, long long& K) {
    ifstream fin(path);
    if (!fin) return false;

    int n;
    if (fin >> n) {
        a.resize(n);
        for (int t = 0; t < n; ++t) fin >> a[t];
        fin >> K;
    }
    return true;
}

ClosestAns closestPair(const vector<long long>& a, long long K) {
    int n = (int)a.size();
    if (n < 2) return {-1, -1, 0};

    int left = 0, right = n - 1;
    long long bestDiff = LLONG_MAX;
    ClosestAns ans = {-1, -1, 0};

    while (left < right) {
        long long sum = a[left] + a[right];
        long long diff = llabs(sum - K);

        if (diff < bestDiff ||
           (diff == bestDiff && left < ans.i)) {
            bestDiff = diff;
            ans = {left, right, sum};
        }

        if (sum < K) ++left;
        else --right;
    }
```

```

    return ans;
}

bool writeOutput(const string& path, const ClosestAns& ans) {
    ofstream fout(path);
    if (!fout) return false;

    fout << ans.i << ", "
         << ans.j << ", "
         << ans.sum;
    return true;
}

int main() {
    vector<long long> a;
    long long K;

    if (!readInput("input.txt", a, K)) {
        cerr << "Loi mo file input!\n";
        return 1;
    }

    ClosestAns ans = closestPair(a, K);

    if (!writeOutput("output.txt", ans)) {
        cerr << "Loi mo file output!\n";
        return 1;
    }

    return 0;
}
```

5.8.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (input.txt):

```
6
1 2 4 7 10 12
9
```

Expected Output:

```
0, 3, 8
```

Test Case 2 (Khớp chính xác)
Input (input.txt):

```
5
1 3 5 7 9
10
```

Expected Output:

```
0, 4, 10
```

Test Case 3 (Case biên – n<2)
Input (input.txt):

```
1
5
7
```

Expected Output:

```
-1, -1, 0
```

5.9 Dãy không giảm dài nhất liên tiếp [Cơ bản]

5.9.1 Phân tích & Thiết kế (M0)

- **Input:** File `input.txt` chứa:
  - Số nguyên n
  - n số nguyên
- **Xử lý:**
  - **Case biên:**
    * $n = 0 \rightarrow max_len = 0$
    * $n = 1 \rightarrow max_len = 1$
    * Nếu toàn mảng không giảm → lấy toàn mảng
  - **Module 1 (Đọc file):**
    * Hàm `readInput(path, vector<long long>&)`
    * Kiểm tra mở file (`fail` → `cerr + return false`)
  - **Module 2 (Tìm dãy không giảm dài nhất):**
    * Duyệt mảng 1 lần
    * Nếu:

      ```
      a[i-1] <= a[i]
      ```

      thì tiếp tục mở rộng đoạn hiện tại
    * Ngược lại bắt đầu đoạn mới
    * Cập nhật đoạn dài nhất

– **Module 3 (Ghi file):**

  * Ghi:

```
max_len=<v> / l=<i> / r=<j>
```

- **Output:**

```
max_len=<v> / l=<i> / r=<j>
```

5.9.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>

using namespace std;

struct SegmentAns {
    int len, l, r;
};

bool readInput(const string& path, vector<long long>& a) {
    ifstream fin(path);
    if (!fin) return false;

    int n;
    if (fin >> n) {
        a.resize(n);
        for (int i = 0; i < n; ++i) fin >> a[i];
    }
    return true;
}

SegmentAns longestNonDecreasing(const vector<long long>& a) {
    int n = (int)a.size();
    if (n == 0) return {0, -1, -1};
    if (n == 1) return {1, 0, 0};

    int bestLen = 1, bestL = 0, bestR = 0;
    int curL = 0, curLen = 1;

    for (int i = 1; i < n; ++i) {
        if (a[i - 1] <= a[i]) {
            ++curLen;
        } else {
            curLen = 1;
            curL = i;
        }
```

```

        if (curLen > bestLen) {
            bestLen = curLen;
            bestL = curL;
            bestR = i;
        }
    }

    return {bestLen, bestL, bestR};
}

bool writeOutput(const string& path, const SegmentAns& ans) {
    ofstream fout(path);
    if (!fout) return false;

    fout << "max_len=" << ans.len
         << " / l=" << ans.l
         << " / r=" << ans.r;
    return true;
}

int main() {
    vector<long long> a;

    if (!readInput("input.txt", a)) {
        cerr << "Loi mo file input!\n";
        return 1;
    }

    SegmentAns ans = longestNonDecreasing(a);

    if (!writeOutput("output.txt", ans)) {
        cerr << "Loi mo file output!\n";
        return 1;
    }

    return 0;
}
```

5.9.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (input.txt):

```
7
1 2 2 1 3 4 4
```

Expected Output:

```
max_len=4 / l=3 / r=6
```

Test Case 2 (Case biên – n=1)
Input (input.txt):

```
1
5
```

Expected Output:

```
max_len=1 / l=0 / r=0
```

Test Case 3 (Giảm hoàn toàn)
Input (input.txt):

```
5
9 7 5 3 1
```

Expected Output:

```
max_len=1 / l=0 / r=0
```

5.10 Smallest window chứa tất cả K giá trị phân biệt [Nâng cao]

5.10.1 Phân tích & Thiết kế (M0)

- **Input:** File `input.txt` chứa:
  - Số nguyên n
  - n số nguyên
  - Số nguyên K
- **Xử lý:**
  - **Case biên:**
    * $n = 0 \rightarrow$ `IMPOSSIBLE`
    * $K \leq 0 \rightarrow$ `IMPOSSIBLE`
    * Nếu K lớn hơn số giá trị phân biệt của mảng $\rightarrow$ `IMPOSSIBLE`
  - **Module 1 (Đọc file):**
    * Hàm `readInput(path, vector<long long>&, int&)`
    * Kiểm tra mở file (`fail` $\rightarrow$ `cerr + return false`)
  - **Module 2 (Tìm cửa sổ ngắn nhất):**
    * Dùng kỹ thuật `Sliding Window`
    * Dùng `unordered_map` lưu tần suất phần tử trong cửa sổ
    * Mở rộng `right` cho đến khi đủ K giá trị phân biệt
    * Thu hẹp từ trái để tối ưu độ dài

    * Cập nhật đáp án ngắn nhất
  - **Module 3 (Ghi file):**
    * Nếu tồn tại:

```
len=<v> / l=<i> / r=<j>
```

    * Ngược lại ghi IMPOSSIBLE

- **Output:**

```
len=<v> / l=<i> / r=<j>
```

Hoặc:

```
IMPOSSIBLE
```

5.10.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>
#include <unordered_map>
#include <unordered_set>
#include <climits>

using namespace std;

struct WindowAns {
    bool ok;
    int len, l, r;
};

bool readInput(const string& path, vector<long long>& a, int& K) {
    ifstream fin(path);
    if (!fin) return false;

    int n;
    if (fin >> n) {
        a.resize(n);
        for (int i = 0; i < n; ++i) fin >> a[i];
        fin >> K;
    }
    return true;
}

WindowAns smallestWindowKDistinct(const vector<long long>& a, int K) {
    int n = (int)a.size();
    if (n == 0 || K <= 0) return {false, 0, -1, -1};

```

```
    unordered_set<long long> st(a.begin(), a.end());
    if (K > (int)st.size()) return {false, 0, -1, -1};

    unordered_map<long long, int> freq;
    int left = 0, distinct = 0;
    int bestLen = INT_MAX, bestL = -1, bestR = -1;

    for (int right = 0; right < n; ++right) {
        if (++freq[a[right]] == 1) ++distinct;

        while (distinct >= K) {
            if (right - left + 1 < bestLen) {
                bestLen = right - left + 1;
                bestL = left;
                bestR = right;
            }

            --freq[a[left]];
            if (freq[a[left]] == 0) {
                freq.erase(a[left]);
                --distinct;
            }
            ++left;
        }
    }

    if (bestLen == INT_MAX) return {false, 0, -1, -1};
    return {true, bestLen, bestL, bestR};
}

bool writeOutput(const string& path, const WindowAns& ans) {
    ofstream fout(path);
    if (!fout) return false;

    if (!ans.ok) {
        fout << "IMPOSSIBLE";
    } else {
        fout << "len=" << ans.len
             << " / l=" << ans.l
             << " / r=" << ans.r;
    }
    return true;
}

int main() {
    vector<long long> a;
    int K;

    if (!readInput("input.txt", a, K)) {
```

```
        cerr << "Loi mo file input!\n";
        return 1;
    }

    WindowAns ans = smallestWindowKDistinct(a, K);

    if (!writeOutput("output.txt", ans)) {
        cerr << "Loi mo file output!\n";
        return 1;
    }

    return 0;
}
```

5.10.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (input.txt):

```
7
1 2 1 3 2 1 4
3
```

Expected Output:

```
len=3 / l=1 / r=3
```

Test Case 2 (Không thể)
Input (input.txt):

```
5
1 1 1 1 1
2
```

Expected Output:

```
IMPOSSIBLE
```

Test Case 3 (Case biên – n=0)
Input (input.txt):

```
0
2
```

Expected Output:

```
IMPOSSIBLE
```

6 Tuần 6: Ma trận 2D + Prefix Sum 2D

6.1 Tổng hàng và cột ma trận [Cơ bản]

6.1.1 Phân tích & Thiết kế (M0)

- **Input:** File `matrix.txt` chứa:
  - Số nguyên m
  - Số nguyên n
  - Ma trận $m \times n$ số nguyên
- **Xử lý:**
  - **Case biên:**
    * Ma trận 1×1
    * $m = 0$ hoặc $n = 0 \rightarrow$ file output rỗng
  - **Module 1 (Đọc file):**

```
readMatrix(path, vector<vector<long long>>&)
```

  - **Module 2 (Tính tổng hàng):**

```
vector<long long> rowSums(matrix)
```

  - **Module 3 (Tính tổng cột):**

```
vector<long long> colSums(matrix)
```

  - **Module 4 (Ghi file):**
    * File `row_sums.txt`
    * File `col_sums.txt`
    * Mỗi dòng:

```
i, tong
```

- **Output:**

```
0, ...
1, ...
```

6.1.2 Mã nguồn C++11 (Tách module)

```
#include <iostream>
#include <vector>
#include <fstream>

using namespace std;

bool readMatrix(const string& path, vector<vector<long long>>& a) {
    ifstream fin(path);
    if (!fin) return false;

    int m, n;
    fin >> m >> n;

    a.assign(m, vector<long long>(n));

    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++)
            fin >> a[i][j];

    return true;
}

vector<long long> rowSums(const vector<vector<long long>>& a) {
    vector<long long> ans;

    for (int i = 0; i < (int)a.size(); i++) {
        long long sum = 0;
        for (int j = 0; j < (int)a[i].size(); j++)
            sum += a[i][j];
        ans.push_back(sum);
    }

    return ans;
}

vector<long long> colSums(const vector<vector<long long>>& a) {
    vector<long long> ans;

    if (a.empty()) return ans;

    int m = a.size();
    int n = a[0].size();

    ans.assign(n, 0);

    for (int j = 0; j < n; j++)
        for (int i = 0; i < m; i++)
```

```
            ans[j] += a[i][j];

    return ans;
}

bool writeFile(const string& path, const vector<long long>& v) {
    ofstream fout(path);
    if (!fout) return false;

    for (int i = 0; i < (int)v.size(); i++)
        fout << i << ", " << v[i] << "\n";

    return true;
}

int main() {
    vector<vector<long long>> a;

    if (!readMatrix("matrix.txt", a)) {
        cerr << "Loi mo file input!\n";
        return 1;
    }

    vector<long long> rs = rowSums(a);
    vector<long long> cs = colSums(a);

    if (!writeFile("row_sums.txt", rs)) {
        cerr << "Loi ghi row_sums!\n";
        return 1;
    }

    if (!writeFile("col_sums.txt", cs)) {
        cerr << "Loi ghi col_sums!\n";
        return 1;
    }

    return 0;
}
```

6.1.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1
Input (matrix.txt):

```
2 3
1 2 3
4 5 6
```

Expected Output (row_sums.txt):

```
0, 6
1, 15
```

Expected Output (col_sums.txt):

```
0, 5
1, 7
2, 9
```

Test Case 2
Input (matrix.txt):

```
1 1
9
```

Expected Output (row_sums.txt):

```
0, 9
```

Expected Output (col_sums.txt):

```
0, 9
```

Test Case 3
Input (matrix.txt):

```
0 0
```

Expected Output:
Hai file rỗng.

6.2 Kiểm tra ma trận đối xứng [Cơ bản]

6.2.1 Phân tích & Thiết kế (M0)

- **Input:** File `matrix.txt` chứa:
  - Số nguyên n
  - Ma trận $n \times n$ số nguyên
- **Xử lý:**
  - **Case biên:**
    * $n = 1 \rightarrow$ luôn đối xứng
    * $n = 0 \rightarrow$ ma trận rỗng, đối xứng
    * Không vuông $\rightarrow$ `is_symmetric=0`
  - **Module 1 (Đọc file):**

```
readMatrix(path, vector<vector<long long>>&)
```

  - **Module 2 (Kiểm tra đối xứng):**

* Chỉ duyệt nửa trên:

```
i < j
```

* Nếu tồn tại:

```
a[i][j] != a[j][i]
```

thì không đối xứng

* Độ phức tạp $O(n^2/2)$

– **Module 3 (Ghi file):**

```
is_symmetric=1
```

hoặc

```
is_symmetric=0
```

- **Output:**

```
is_symmetric=1
```

hoặc

```
is_symmetric=0
```

6.2.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>

using namespace std;

bool readMatrix(const string& path, vector<vector<long long>>& a) {
    ifstream fin(path);
    if (!fin) return false;

    int n;
    fin >> n;

    a.assign(n, vector<long long>(n));

    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            fin >> a[i][j];

    return true;
}

```

```
bool isSymmetric(const vector<vector<long long>>& a) {
    int n = a.size();

    if (n == 0) return true;
    if (n == 1) return true;

    for (int i = 0; i < n; i++) {
        if ((int)a[i].size() != n) return false;
    }

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (a[i][j] != a[j][i])
                return false;
        }
    }

    return true;
}

bool writeOutput(const string& path, bool ok) {
    ofstream fout(path);
    if (!fout) return false;

    fout << "is_symmetric=" << (ok ? 1 : 0);
    return true;
}

int main() {
    vector<vector<long long>> a;

    if (!readMatrix("matrix.txt", a)) {
        cerr << "Loi mo file input!\n";
        return 1;
    }

    bool ans = isSymmetric(a);

    if (!writeOutput("output.txt", ans)) {
        cerr << "Loi mo file output!\n";
        return 1;
    }

    return 0;
}
```

6.2.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1
Input (matrix.txt):

```
3
1 2 3
2 4 5
3 5 6
```

Expected Output:

```
is_symmetric=1
```

Test Case 2
Input (matrix.txt):

```
2
1 2
3 4
```

Expected Output:

```
is_symmetric=0
```

Test Case 3
Input (matrix.txt):

```
0
```

Expected Output:

```
is_symmetric=1
```

6.3 Xây dựng prefix sum 2D [Cơ bản]

6.3.1 Phân tích & Thiết kế (M0)

- **Input:** File `matrix.txt` chứa:
  - Số nguyên m
  - Số nguyên n
  - Ma trận $m \times n$ số nguyên
- **Xử lý:**
  - **Case biên:**
    - $1 \times 1 \rightarrow pre[0][0] = a[0][0]$
    - $m = 0$ hoặc $n = 0 \rightarrow$ file output rỗng
  - **Module 1 (Đọc file):**

```
readMatrix(path, vector<vector<long long>>&)
```

- **Module 2 (Xây dựng prefix sum 2D):**
  * Công thức:

```
pre[i][j] = a[i][j]
          + pre[i-1][j]
          + pre[i][j-1]
          - pre[i-1][j-1]
```

  * Nếu $i = 0$ thì phần trên bằng 0
  * Nếu $j = 0$ thì phần trái bằng 0
- **Module 3 (Ghi file):**
  * File `pre2D.txt`
  * Mỗi dòng:

```
i j pre[i][j]
```

- **Output:**

```
0 0 ...
0 1 ...
...
```

6.3.2 Mã nguồn C++11 (Tách module)

```
#include <iostream>
#include <vector>
#include <fstream>

using namespace std;

typedef vector<vector<long long>> Matrix;

bool readMatrix(const string& path, Matrix& a) {
    ifstream fin(path);
    if (!fin) return false;

    int m, n;
    fin >> m >> n;

    a.assign(m, vector<long long>(n));

    for (int i = 0; i < m; i++)
        for (int j = 0; j < n; j++)
            fin >> a[i][j];

```

```
    return true;
}

Matrix buildPre2D(const Matrix& a) {
    int m = a.size();
    Matrix pre;

    if (m == 0) return pre;

    int n = a[0].size();
    pre.assign(m, vector<long long>(n, 0));

    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            long long up = (i > 0) ? pre[i - 1][j] : 0;
            long long left = (j > 0) ? pre[i][j - 1] : 0;
            long long diag = (i > 0 && j > 0) ? pre[i - 1][j - 1] : 0;

            pre[i][j] = a[i][j] + up + left - diag;
        }
    }

    return pre;
}

bool writeFile(const string& path, const Matrix& pre) {
    ofstream fout(path);
    if (!fout) return false;

    for (int i = 0; i < (int)pre.size(); i++) {
        for (int j = 0; j < (int)pre[i].size(); j++) {
            fout << i << " " << j << " " << pre[i][j] << "\n";
        }
    }

    return true;
}

int main() {
    Matrix a;

    if (!readMatrix("matrix.txt", a)) {
        cerr << "Loi mo file input!\n";
        return 1;
    }

    Matrix pre = buildPre2D(a);

    if (!writeFile("pre2D.txt", pre)) {
```

```
        cerr << "Loi mo file output!\n";
        return 1;
    }

    return 0;
}
```

6.3.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1
Input (matrix.txt):

```
2 3
1 2 3
4 5 6
```

Expected Output (pre2D.txt):

```
0 0 1
0 1 3
0 2 6
1 0 5
1 1 12
1 2 21
```

Test Case 2
Input (matrix.txt):

```
1 1
9
```

Expected Output (pre2D.txt):

```
0 0 9
```

Test Case 3
Input (matrix.txt):

```
0 0
```

Expected Output:
File rỗng.

6.4 Trả lời truy vấn tổng vùng [Trung bình]

6.4.1 Phân tích & Thiết kế (M0)

- **Input:** File `input.txt` chứa:
  - Hai số nguyên m, n
  - Ma trận $m \times n$

- Số nguyên q
- q truy vấn dạng $(x1, y1, x2, y2)$

- **Xử lý:**
  - **Case biên:**
    - $m = 0$ hoặc $n = 0 \rightarrow$ không xử lý
    - Truy vấn 1 ô: $x1 = x2, y1 = y2 \rightarrow$ tổng chính là $a[x1][y1]$
  - **Module 1 (Đọc file):**
    - Hàm `readMatrix(path, vector<vector<long long>>&, vector<Query>&)`
    - Kiểm tra mở file (`fail` $\rightarrow$ `return false`)
  - **Module 2 (Xây prefix 2D):**
    - `pre[i][j] = a[i][j] + pre[i-1][j] + pre[i][j-1] - pre[i-1][j-1]`
  - **Module 3 (Trả lời truy vấn):**
    - Dùng công thức:

```
sum = pre[x2][y2] - pre[x1-1][y2]
    - pre[x2][y1-1]
    + pre[x1-1][y1-1]

```

    - Nếu chỉ số sai hoặc $x1 > x2$ hoặc $y1 > y2 \rightarrow$ in `INVALID`
    - Mỗi truy vấn xử lý $O(1)$

- **Output:**

```
<ket qua moi truy van tren 1 dong>

```

6.4.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>

using namespace std;

struct Query {
    int x1, y1, x2, y2;
};

bool readMatrix(const string& path,
                vector<vector<long long>>& a,
                vector<Query>& queries) {
    ifstream fin(path);
    if (!fin) return false;
```

```

    int m, n;
    fin >> m >> n;
    a.assign(m, vector<long long>(n));

    for (int i = 0; i < m; ++i)
        for (int j = 0; j < n; ++j)
            fin >> a[i][j];

    int q;
    fin >> q;
    queries.resize(q);

    for (int i = 0; i < q; ++i)
        fin >> queries[i].x1 >> queries[i].y1
            >> queries[i].x2 >> queries[i].y2;

    return true;
}

vector<vector<long long>> buildPrefix2D(
        const vector<vector<long long>>& a) {
    int m = a.size();
    if (m == 0) return {};

    int n = a[0].size();
    vector<vector<long long>> pre(m, vector<long long>(n, 0));

    for (int i = 0; i < m; ++i) {
        for (int j = 0; j < n; ++j) {
            pre[i][j] = a[i][j];
            if (i > 0) pre[i][j] += pre[i - 1][j];
            if (j > 0) pre[i][j] += pre[i][j - 1];
            if (i > 0 && j > 0) pre[i][j] -= pre[i - 1][j - 1];
        }
    }
    return pre;
}

bool sumRect(const vector<vector<long long>>& pre,
             int x1, int y1, int x2, int y2,
             long long& ans) {
    int m = pre.size();
    int n = pre[0].size();

    if (x1 < 0 || y1 < 0 || x2 < 0 || y2 < 0 ||
        x1 >= m || x2 >= m || y1 >= n || y2 >= n ||
        x1 > x2 || y1 > y2)
        return false;
```

```

    ans = pre[x2][y2];
    if (x1 > 0) ans -= pre[x1 - 1][y2];
    if (y1 > 0) ans -= pre[x2][y1 - 1];
    if (x1 > 0 && y1 > 0) ans += pre[x1 - 1][y1 - 1];

    return true;
}

int main() {
    vector<vector<long long>> a;
    vector<Query> queries;

    if (!readMatrix("input.txt", a, queries)) {
        cerr << "Loi mo file input!\n";
        return 1;
    }

    if (a.empty() || a[0].empty()) return 0;

    vector<vector<long long>> pre = buildPrefix2D(a);

    for (size_t i = 0; i < queries.size(); ++i) {
        long long ans;
        if (sumRect(pre,
                    queries[i].x1, queries[i].y1,
                    queries[i].x2, queries[i].y2,
                    ans))
            cout << ans << "\n";
        else
            cout << "INVALID\n";
    }

    return 0;
}
```

6.4.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (input.txt):

```
3 3
1 2 3
4 5 6
7 8 9
3
0 0 1 1
1 1 2 2
0 2 2 2
```

Expected Output:

```
12
28
18
```

Test Case 2 (Case biên – truy vấn 1 ô)
Input (input.txt):

```
2 2
5 1
2 7
1
1 1 1 1
```

Expected Output:

```
7
```

Test Case 3 (Truy vấn lỗi)
Input (input.txt):

```
2 2
1 2
3 4
2
1 1 0 0
0 0 2 1
```

Expected Output:

```
INVALID
INVALID
```

6.5 Ma trận đường chéo [Trung bình]

6.5.1 Phân tích & Thiết kế (M0)

- **Input:** File `input.txt` chứa:
  - Số nguyên n
  - Ma trận vuông $n \times n$
- **Xử lý:**
  - **Case biên:**
    * $n = 0 \rightarrow$ không xử lý
    * $n = 1 \rightarrow$ đường chéo chính = đường chéo phụ = $a[0][0]$
    * `is_diagonal = 1`

- **Module 1 (Đọc file):**
  - Hàm `readMatrix(path, vector<vector<long long>>&)`
  - Kiểm tra mở file (`fail` → `return false`)
- **Module 2 (Tính tổng đường chéo):**
  - Với mọi $i = 0..n-1$

```
mainSum += a[i][i]
antiSum += a[i][n - 1 - i]

```

- **Module 3 (Kiểm tra ma trận đường chéo):**
  - Với mọi $i \neq j$ thì $a[i][j] = 0$
  - Nếu tồn tại phần tử khác 0 → `is_diagonal = 0`

- **Output:**

```
main_diag_sum=<v> / anti_diag_sum=<v> / is_diagonal=<0/1>

```

6.5.2 Mã nguồn C++11 (Tách module)

```
#include <iostream>
#include <vector>
#include <fstream>

using namespace std;

bool readMatrix(const string& path, vector<vector<long long>>& a) {
    ifstream fin(path);
    if (!fin) return false;

    int n;
    fin >> n;
    a.assign(n, vector<long long>(n));

    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            fin >> a[i][j];

    return true;
}

long long mainDiagSum(const vector<vector<long long>>& a) {
    int n = a.size();
    long long sum = 0;

    for (int i = 0; i < n; ++i)
```

```
        sum += a[i][i];

    return sum;
}

long long antiDiagSum(const vector<vector<long long>>& a) {
    int n = a.size();
    long long sum = 0;

    for (int i = 0; i < n; ++i)
        sum += a[i][n - 1 - i];

    return sum;
}

bool isDiagonalMatrix(const vector<vector<long long>>& a) {
    int n = a.size();

    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            if (i != j && a[i][j] != 0)
                return false;

    return true;
}

int main() {
    vector<vector<long long>> a;

    if (!readMatrix("input.txt", a)) {
        cerr << "Loi mo file input!\n";
        return 1;
    }

    if (a.empty()) return 0;

    long long mainSum = mainDiagSum(a);
    long long antiSum = antiDiagSum(a);
    bool diagonal = isDiagonalMatrix(a);

    cout << "main_diag_sum=" << mainSum
         << " / anti_diag_sum=" << antiSum
         << " / is_diagonal=" << diagonal;

    return 0;
}
```

6.5.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (input.txt):

```
3
1 0 0
0 5 0
0 0 9
```

Expected Output:

```
main_diag_sum=15 / anti_diag_sum=5 / is_diagonal=1
```

Test Case 2 (Không phải ma trận đường chéo)
Input (input.txt):

```
3
1 2 0
0 3 0
0 0 4
```

Expected Output:

```
main_diag_sum=8 / anti_diag_sum=3 / is_diagonal=0
```

Test Case 3 (Case biên – n=1)
Input (input.txt):

```
1
7
```

Expected Output:

```
main_diag_sum=7 / anti_diag_sum=7 / is_diagonal=1
```

6.6 Nhân hai ma trận [Trung bình]

6.6.1 Phân tích & Thiết kế (M0)

- **Input:** File `input.txt` chứa:
  - Hai số nguyên m, k và ma trận $A(m \times k)$
  - Hai số nguyên r, n và ma trận $B(r \times n)$
- **Xử lý:**
  - **Case biên:**
    * Nếu $k \neq r \rightarrow$ in `INCOMPATIBLE`
    * Nếu ma trận có kích thước 1×1 vẫn xử lý bình thường

- **Module 1 (Đọc file):**
  - Hàm `readMatrices(path, A, B)`
  - Kiểm tra mở file (`fail` → `return false`)
- **Module 2 (Kiểm tra kích thước):**
  - Điều kiện nhân được: số cột của A bằng số hàng của B
- **Module 3 (Nhân ma trận):**
  - Công thức:

```
C[i][j] += A[i][t] * B[t][j]

```

  - Duyệt 3 vòng lặp

- **Output:**

```
ỗ
Mi hàng là 1 dòng, các ầphn ửt cách nhau ởbi space

```

6.6.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>

using namespace std;

bool readMatrices(const string& path,
                  vector<vector<long long>>& A,
                  vector<vector<long long>>& B) {
    ifstream fin(path);
    if (!fin) return false;

    int m, k;
    fin >> m >> k;
    A.assign(m, vector<long long>(k));

    for (int i = 0; i < m; ++i)
        for (int j = 0; j < k; ++j)
            fin >> A[i][j];

    int r, n;
    fin >> r >> n;
    B.assign(r, vector<long long>(n));

    for (int i = 0; i < r; ++i)
        for (int j = 0; j < n; ++j)
```

```
            fin >> B[i][j];

    return true;
}

vector<vector<long long>> multiplyMatrix(
        const vector<vector<long long>>& A,
        const vector<vector<long long>>& B) {
    int m = A.size();
    int k = A[0].size();
    int n = B[0].size();

    vector<vector<long long>> C(m, vector<long long>(n, 0));

    for (int i = 0; i < m; ++i)
        for (int t = 0; t < k; ++t)
            for (int j = 0; j < n; ++j)
                C[i][j] += A[i][t] * B[t][j];

    return C;
}

void printMatrix(const vector<vector<long long>>& C) {
    for (size_t i = 0; i < C.size(); ++i) {
        for (size_t j = 0; j < C[i].size(); ++j) {
            if (j > 0) cout << " ";
            cout << C[i][j];
        }
        cout << "\n";
    }
}

int main() {
    vector<vector<long long>> A, B;

    if (!readMatrices("input.txt", A, B)) {
        cerr << "Loi mo file input!\n";
        return 1;
    }

    if (A.empty() || B.empty()) return 0;

    if (A[0].size() != B.size()) {
        cout << "INCOMPATIBLE";
        return 0;
    }

    vector<vector<long long>> C = multiplyMatrix(A, B);
    printMatrix(C);
```

```

    return 0;
}
```

6.6.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (input.txt):

```
2 3
1 2 3
4 5 6
3 2
7 8
9 10
11 12
```

Expected Output:

```
58 64
139 154
```

Test Case 2 (Không tương thích kích thước)
Input (input.txt):

```
2 2
1 2
3 4
3 2
1 2
3 4
5 6
```

Expected Output:

```
INCOMPATIBLE
```

Test Case 3 (Case biên – ma trận 1x1)
Input (input.txt):

```
1 1
5
1 1
7
```

Expected Output:

```
35
```

6.7 Vùng con tổng lớn nhất (brute force) [Trung bình]

6.7.1 Phân tích & Thiết kế (M0)

- **Input:** File `input.txt` chứa:
  - Hai số nguyên m, n
  - Ma trận $m \times n$ ($m, n \leq 50$)
- **Xử lý:**
  - **Case biên:**
    * $m = 0$ hoặc $n = 0 \rightarrow$ không xử lý
    * Ma trận 1 ô $\rightarrow$ đáp án là chính ô đó
    * Ma trận toàn số âm $\rightarrow$ chọn vùng có giá trị lớn nhất
  - **Module 1 (Đọc file):**
    * Hàm `readMatrix(path, vector<vector<long long>>&)`
    * Kiểm tra mở file (`fail` $\rightarrow$ `return false`)
  - **Module 2 (Xây prefix 2D):**
    *
    ```
    pre[i][j] = a[i][j] + pre[i-1][j]
              + pre[i][j-1] - pre[i-1][j-1]

    ```
  - **Module 3 (Duyệt mọi hình chữ nhật):**
    * Duyệt mọi bộ $(x1, y1, x2, y2)$
    * Tính tổng vùng trong $O(1)$ bằng prefix 2D
    * Cập nhật kết quả lớn nhất
    * Độ phức tạp: $O(m^2n^2)$
- **Output:**

```
max_sum x1 y1 x2 y2

```

6.7.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>
#include <climits>

using namespace std;

```

```
bool readMatrix(const string& path, vector<vector<long long>>& a) {
    ifstream fin(path);
    if (!fin) return false;

    int m, n;
    fin >> m >> n;
    a.assign(m, vector<long long>(n));

    for (int i = 0; i < m; ++i)
        for (int j = 0; j < n; ++j)
            fin >> a[i][j];

    return true;
}

vector<vector<long long>> buildPrefix2D(
        const vector<vector<long long>>& a) {
    int m = a.size();
    if (m == 0) return {};

    int n = a[0].size();
    vector<vector<long long>> pre(m, vector<long long>(n, 0));

    for (int i = 0; i < m; ++i)
        for (int j = 0; j < n; ++j) {
            pre[i][j] = a[i][j];
            if (i > 0) pre[i][j] += pre[i - 1][j];
            if (j > 0) pre[i][j] += pre[i][j - 1];
            if (i > 0 && j > 0) pre[i][j] -= pre[i - 1][j - 1];
        }

    return pre;
}

long long sumRect(const vector<vector<long long>>& pre,
                  int x1, int y1, int x2, int y2) {
    long long ans = pre[x2][y2];

    if (x1 > 0) ans -= pre[x1 - 1][y2];
    if (y1 > 0) ans -= pre[x2][y1 - 1];
    if (x1 > 0 && y1 > 0) ans += pre[x1 - 1][y1 - 1];

    return ans;
}

int main() {
    vector<vector<long long>> a;

    if (!readMatrix("input.txt", a)) {
```

```
        cerr << "Loi mo file input!\n";
        return 1;
    }

    if (a.empty() || a[0].empty()) return 0;

    int m = a.size();
    int n = a[0].size();

    vector<vector<long long>> pre = buildPrefix2D(a);

    long long best = LLONG_MIN;
    int bx1 = 0, by1 = 0, bx2 = 0, by2 = 0;

    for (int x1 = 0; x1 < m; ++x1)
        for (int y1 = 0; y1 < n; ++y1)
            for (int x2 = x1; x2 < m; ++x2)
                for (int y2 = y1; y2 < n; ++y2) {
                    long long cur =
                        sumRect(pre, x1, y1, x2, y2);

                    if (cur > best) {
                        best = cur;
                        bx1 = x1;
                        by1 = y1;
                        bx2 = x2;
                        by2 = y2;
                    }
                }

    cout << best << " "
         << bx1 << " " << by1 << " "
         << bx2 << " " << by2;

    return 0;
}
```

6.7.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (input.txt):

```
3 3
1 -2 3
4 5 -1
-2 6 2
```

Expected Output:

```
17 1 0 2 2
```

Test Case 2 (Toàn số âm)
Input (input.txt):

```
2 2
-5 -2
-7 -1
```

Expected Output:

```
-1 1 1 1 1
```

Test Case 3 (Case biên – 1 ô)
Input (input.txt):

```
1 1
9
```

Expected Output:

```
9 0 0 0 0
```

6.8 Kiểm tra ma trận tam giác [Trung bình]

6.8.1 Phân tích & Thiết kế (M0)

- **Input:** File `input.txt` chứa:
  - Số nguyên n
  - Ma trận vuông $n \times n$
- **Xử lý:**
  - **Case biên:**
    - $n = 0 \rightarrow$ không xử lý
    - $n = 1 \rightarrow$ vừa là ma trận tam giác trên, vừa là tam giác dưới
  - **Module 1 (Đọc file):**
    - Hàm `readMatrix(path, vector<vector<long long>>&)`
    - Kiểm tra mở file (`fail` $\rightarrow$ `return false`)
  - **Module 2 (Kiểm tra tam giác trên):**
    - Với mọi $i > j$ thì $a[i][j] = 0$
    - Nếu tồn tại phần tử khác 0 $\rightarrow$ `is_upper = 0`
  - **Module 3 (Kiểm tra tam giác dưới):**
    - Với mọi $i < j$ thì $a[i][j] = 0$
    - Nếu tồn tại phần tử khác 0 $\rightarrow$ `is_lower = 0`
- **Output:**

```
is_upper=<0/1> / is_lower=<0/1>

```

6.8.2 Mã nguồn C++11 (Tách module)

```
#include <iostream>
#include <vector>
#include <fstream>

using namespace std;

bool readMatrix(const string& path, vector<vector<long long>>& a) {
    ifstream fin(path);
    if (!fin) return false;

    int n;
    fin >> n;
    a.assign(n, vector<long long>(n));

    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            fin >> a[i][j];

    return true;
}

bool isUpperTriangular(const vector<vector<long long>>& a) {
    int n = a.size();

    for (int i = 0; i < n; ++i)
        for (int j = 0; j < i; ++j)
            if (a[i][j] != 0)
                return false;

    return true;
}

bool isLowerTriangular(const vector<vector<long long>>& a) {
    int n = a.size();

    for (int i = 0; i < n; ++i)
        for (int j = i + 1; j < n; ++j)
            if (a[i][j] != 0)
                return false;

    return true;
}

int main() {
    vector<vector<long long>> a;

    if (!readMatrix("input.txt", a)) {
```

```
        cerr << "Loi mo file input!\n";
        return 1;
    }

    if (a.empty()) return 0;

    bool upper = isUpperTriangular(a);
    bool lower = isLowerTriangular(a);

    cout << "is_upper=" << upper
         << " / is_lower=" << lower;

    return 0;
}
```

6.8.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Tam giác trên)
Input (input.txt):

```
3
1 2 3
0 4 5
0 0 6
```

Expected Output:

```
is_upper=1 / is_lower=0
```

Test Case 2 (Tam giác dưới)
Input (input.txt):

```
3
1 0 0
2 3 0
4 5 6
```

Expected Output:

```
is_upper=0 / is_lower=1
```

Test Case 3 (Case biên – n=1)
Input (input.txt):

```
1
9
```

Expected Output:

```
is_upper=1 / is_lower=1
```

6.9 Xoay ma trận 90 độ theo chiều kim đồng hồ [Trung bình]

6.9.1 Phân tích & Thiết kế (M0)

- **Input:** File `input.txt` chứa:
  - Hai số nguyên m, n
  - Ma trận $m \times n$
- **Xử lý:**
  - **Case biên:**
    * $m = 0$ hoặc $n = 0 \rightarrow$ không xử lý
    * Ma trận $1 \times 1 \rightarrow$ giữ nguyên
    * Ma trận kết quả có kích thước $n \times m$
  - **Module 1 (Đọc file):**
    * Hàm `readMatrix(path, vector<vector<long long>>&)`
    * Kiểm tra mở file (`fail` $\rightarrow$ `return false`)
  - **Module 2 (Xoay ma trận):**
    * Với mỗi phần tử (i, j) của ma trận cũ:

```
new[j][m - 1 - i] = a[i][j]

```

  - **Module 3 (Ghi file):**
    * Hàm `writeMatrix(path, rotated)`
    * Mỗi hàng là 1 dòng, các phần tử cách nhau bởi space
- **Output:**

```
Ghi ma ậṇtrn ếkt ả̉qu ra file rotated.txt

```

6.9.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>

using namespace std;

bool readMatrix(const string& path, vector<vector<long long>>& a) {
    ifstream fin(path);
    if (!fin) return false;
```

```

    int m, n;
    fin >> m >> n;
    a.assign(m, vector<long long>(n));

    for (int i = 0; i < m; ++i)
        for (int j = 0; j < n; ++j)
            fin >> a[i][j];

    return true;
}

vector<vector<long long>> rotateClockwise(
        const vector<vector<long long>>& a) {
    int m = a.size();
    if (m == 0) return {};

    int n = a[0].size();
    vector<vector<long long>> b(n, vector<long long>(m));

    for (int i = 0; i < m; ++i)
        for (int j = 0; j < n; ++j)
            b[j][m - 1 - i] = a[i][j];

    return b;
}

bool writeMatrix(const string& path,
                 const vector<vector<long long>>& a) {
    ofstream fout(path);
    if (!fout) return false;

    for (size_t i = 0; i < a.size(); ++i) {
        for (size_t j = 0; j < a[i].size(); ++j) {
            if (j > 0) fout << " ";
            fout << a[i][j];
        }
        fout << "\n";
    }

    return true;
}

int main() {
    vector<vector<long long>> a;

    if (!readMatrix("input.txt", a)) {
        cerr << "Loi mo file input!\n";
        return 1;
```

```
    }

    if (a.empty() || a[0].empty()) return 0;

    vector<vector<long long>> rotated =
        rotateClockwise(a);

    if (!writeMatrix("rotated.txt", rotated)) {
        cerr << "Loi mo file output!\n";
        return 1;
    }

    return 0;
}
```

6.9.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (input.txt):

```
2 3
1 2 3
4 5 6
```

Expected Output (rotated.txt):

```
4 1
5 2
6 3
```

Test Case 2 (Ma trận vuông)
Input (input.txt):

```
3 3
1 2 3
4 5 6
7 8 9
```

Expected Output (rotated.txt):

```
7 4 1
8 5 2
9 6 3
```

Test Case 3 (Case biên – 1x1)
Input (input.txt):

```
1 1
9
```

Expected Output (rotated.txt):

```
9
```

6.10 Đếm vùng con tổng bằng K [Nâng cao]

6.10.1 Phân tích & Thiết kế (M0)

- **Input:** File `input.txt` chứa:
  - Hai số nguyên m, n ($m, n \leq 100$)
  - Ma trận $m \times n$
  - Giá trị K
- **Xử lý:**
  - **Case biên:**
    * $m = 0$ hoặc $n = 0 \rightarrow$ không xử lý
    * Ma trận 1 ô: kết quả là 1 nếu phần tử bằng K, ngược lại là 0
  - **Module 1 (Đọc file):**
    * Hàm `readMatrix(path, vector<vector<long long>>&, long long& K)`
    * Kiểm tra mở file (`fail` $\rightarrow$ `return false`)
  - **Module 2 (Duyệt cặp hàng):**
    * Chọn hai hàng `top` và `bottom`
    * Gom tổng từng cột giữa hai hàng thành mảng 1 chiều
  - **Module 3 (Đếm subarray sum = K):**
    * Dùng prefix sum 1D + `map`
    * Nếu prefix hiện tại là `sum`, cần số lần xuất hiện của `sum - K`
    * Cộng vào đáp án
  - **Độ phức tạp:**
    * $O(m^2 \cdot n)$
- **Output:**

```
count=<n>

```

6.10.2 Mã nguồn C++11 (Tách module)

```
#include <iostream>
#include <vector>
#include <fstream>
#include <map>

using namespace std;

bool readMatrix(const string& path,
                vector<vector<long long>>& a,
                long long& K) {
    ifstream fin(path);
    if (!fin) return false;

    int m, n;
    fin >> m >> n;
    a.assign(m, vector<long long>(n));

    for (int i = 0; i < m; ++i)
        for (int j = 0; j < n; ++j)
            fin >> a[i][j];

    fin >> K;
    return true;
}

long long countSubarraySumK(
        const vector<long long>& arr,
        long long K) {
    map<long long, long long> freq;
    freq[0] = 1;

    long long sum = 0;
    long long cnt = 0;

    for (size_t i = 0; i < arr.size(); ++i) {
        sum += arr[i];

        if (freq.count(sum - K))
            cnt += freq[sum - K];

        freq[sum]++;
    }

    return cnt;
}

long long countRectangles(
```

```
        const vector<vector<long long>>& a,
        long long K) {
    int m = a.size();
    int n = a[0].size();

    long long ans = 0;

    for (int top = 0; top < m; ++top) {
        vector<long long> col(n, 0);

        for (int bottom = top; bottom < m; ++bottom) {
            for (int j = 0; j < n; ++j)
                col[j] += a[bottom][j];

            ans += countSubarraySumK(col, K);
        }
    }

    return ans;
}

int main() {
    vector<vector<long long>> a;
    long long K;

    if (!readMatrix("input.txt", a, K)) {
        cerr << "Loi mo file input!\n";
        return 1;
    }

    if (a.empty() || a[0].empty()) return 0;

    long long result = countRectangles(a, K);

    cout << "count=" << result;

    return 0;
}
```

6.10.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (input.txt):

```
2 2
1 2
3 4
3
```

Expected Output:

```
count=2
```

Giải thích: Hai vùng có tổng bằng 3:

- Ô $(1, 0)$ có giá trị 3
- Hàng đầu: $1 + 2 = 3$

Test Case 2 (Ma trận toàn 0)
Input (input.txt):

```
2 2
0 0
0 0
0
```

Expected Output:

```
count=9
```

Test Case 3 (Case biên – 1 ô)
Input (input.txt):

```
1 1
5
5
```

Expected Output:

```
count=1
```

7 Tuần 7: Struct Toán: Phân số (Fraction)

7.1 Chuẩn hoá phân số [Cơ bản]

7.1.1 Phân tích & Thiết kế (M0)

- **Input:** File `fractions.csv` chứa nhiều dòng:
  - Mỗi dòng có dạng: `num,den`
- **Xử lý:**
  - **Case biên:**
    * `den = 0` → phân số không hợp lệ
    * `num = 0` → chuẩn hoá thành `0,1`
    * `den < 0` → đổi dấu cả tử và mẫu
  - **Module 1 (Đọc file):**
    * Hàm `readFractions(path, vector<Fraction>&)`
    * Đọc từng dòng dạng CSV
    * Kiểm tra mở file (`fail` → `return false`)
  - **Module 2 (Ước chung lớn nhất):**
    * Thuật toán Euclid:

```
while (b != 0) {
    t = a % b;
    a = b;
    b = t;
}

```

  - **Module 3 (Chuẩn hoá phân số):**
    * Nếu mẫu bằng 0 → trả `false`
    * Đưa mẫu về dương
    * Rút gọn theo gcd
  - **Module 4 (Ghi file):**
    * Ghi các phân số hợp lệ vào `normalized.csv`
    * Định dạng: `num,den`

- **Output:**

```
Ghi file normalized.csv
invalid=<n>

```

7.1.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>
#include <sstream>
#include <cstdlib>

using namespace std;

struct Fraction {
    long long num, den;
};

long long gcd(long long a, long long b) {
    a = llabs(a);
    b = llabs(b);

    while (b != 0) {
        long long t = a % b;
        a = b;
        b = t;
    }

    return a;
}

bool readFractions(const string& path,
                   vector<Fraction>& arr) {
    ifstream fin(path);
    if (!fin) return false;

    string line;
    while (getline(fin, line)) {
        if (line.empty()) continue;

        stringstream ss(line);
        string s1, s2;

        if (getline(ss, s1, ',') &&
            getline(ss, s2)) {
            Fraction f;
            f.num = atoll(s1.c_str());
```

```
            f.den = atoll(s2.c_str());
            arr.push_back(f);
        }
    }

    return true;
}

bool normalize(Fraction& f) {
    if (f.den == 0) return false;

    if (f.num == 0) {
        f.num = 0;
        f.den = 1;
        return true;
    }

    if (f.den < 0) {
        f.num = -f.num;
        f.den = -f.den;
    }

    long long g = gcd(f.num, f.den);
    f.num /= g;
    f.den /= g;

    return true;
}

bool writeFractions(const string& path,
                    const vector<Fraction>& arr) {
    ofstream fout(path);
    if (!fout) return false;

    for (size_t i = 0; i < arr.size(); ++i)
        fout << arr[i].num
             << "," << arr[i].den << "\n";

    return true;
}

int main() {
    vector<Fraction> input, output;

    if (!readFractions("fractions.csv", input)) {
        cerr << "Loi mo file input!\n";
        return 1;
    }

```

```
    int invalid = 0;

    for (size_t i = 0; i < input.size(); ++i) {
        Fraction f = input[i];

        if (normalize(f))
            output.push_back(f);
        else
            invalid++;
    }

    if (!writeFractions("normalized.csv", output)) {
        cerr << "Loi mo file output!\n";
        return 1;
    }

    cout << "invalid=" << invalid;

    return 0;
}
```

7.1.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (fractions.csv):

```
2,4
3,-9
0,7
```

Expected Output (normalized.csv):

```
1,2
-1,3
0,1
```

Console Output:

```
invalid=0
```

Test Case 2 (Có phân số lỗi)
Input (fractions.csv):

```
1,0
5,10
7,0
```

Expected Output (normalized.csv):

```
1,2
```

Console Output:

```
invalid=2
```

Test Case 3 (Case biên – num=0)
Input (fractions.csv):

```
0,-5
```

Expected Output (normalized.csv):

```
0,1
```

Console Output:

```
invalid=0
```

7.2 Cộng và trừ 2 phân số [Cơ bản]

7.2.1 Phân tích & Thiết kế (M0)

- **Input:** File `pairs.csv` chứa nhiều dòng:
  - Mỗi dòng gồm 4 số: `n1,d1,n2,d2`
  - Tương ứng hai phân số $\frac{n1}{d1}$ và $\frac{n2}{d2}$
- **Xử lý:**
  - **Case biên:**
    - Hai phân số nghịch nhau → tổng bằng `0,1`
    - Nếu tử số bằng 0 → chuẩn hoá thành `0,1`
    - Nếu mẫu âm → đổi dấu cả tử và mẫu
  - **Module 1 (Đọc file):**
    - Hàm `readPairs(path, vector<PairFrac>&)`
    - Đọc từng dòng CSV
    - Kiểm tra mở file (`fail` → `return false`)
  - **Module 2 (Ước chung lớn nhất):**
    - Thuật toán Euclid để rút gọn phân số
  - **Module 3 (Chuẩn hoá phân số):**
    - Đưa mẫu về dương
    - Rút gọn gcd
    - $0/x \to 0/1$
  - **Module 4 (Cộng và trừ):**

*

```
a/b + c/d = (a*d + c*b)/(b*d)
a/b - c/d = (a*d - c*b)/(b*d)

```

* Sau đó chuẩn hoá kết quả

– **Module 5 (Ghi file):**

* Ghi `results.csv`
* Định dạng: `sum_num,sum_den,diff_num,diff_den`

- **Output:**

```
Ghi file results.csv

```

7.2.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>
#include <sstream>
#include <cstdlib>

using namespace std;

struct Fraction {
    long long num, den;
};

struct PairFrac {
    Fraction a, b;
};

long long gcd(long long a, long long b) {
    a = llabs(a);
    b = llabs(b);

    while (b != 0) {
        long long t = a % b;
        a = b;
        b = t;
    }

    return a;
}

void normalize(Fraction& f) {
    if (f.num == 0) {
```

```
        f.num = 0;
        f.den = 1;
        return;
    }

    if (f.den < 0) {
        f.num = -f.num;
        f.den = -f.den;
    }

    long long g = gcd(f.num, f.den);
    f.num /= g;
    f.den /= g;
}

bool readPairs(const string& path,
               vector<PairFrac>& arr) {
    ifstream fin(path);
    if (!fin) return false;

    string line;
    while (getline(fin, line)) {
        if (line.empty()) continue;

        stringstream ss(line);
        string s[4];

        for (int i = 0; i < 4; ++i)
            getline(ss, s[i], i == 3 ? '\n' : ',');

        PairFrac p;
        p.a.num = atoll(s[0].c_str());
        p.a.den = atoll(s[1].c_str());
        p.b.num = atoll(s[2].c_str());
        p.b.den = atoll(s[3].c_str());

        arr.push_back(p);
    }

    return true;
}

Fraction addFrac(Fraction x, Fraction y) {
    Fraction r;
    r.num = x.num * y.den + y.num * x.den;
    r.den = x.den * y.den;
    normalize(r);
    return r;
}
```

```

Fraction subFrac(Fraction x, Fraction y) {
    Fraction r;
    r.num = x.num * y.den - y.num * x.den;
    r.den = x.den * y.den;
    normalize(r);
    return r;
}

bool writeResults(const string& path,
                  const vector<PairFrac>& arr) {
    ofstream fout(path);
    if (!fout) return false;

    for (size_t i = 0; i < arr.size(); ++i) {
        Fraction s = addFrac(arr[i].a, arr[i].b);
        Fraction d = subFrac(arr[i].a, arr[i].b);

        fout << s.num << "," << s.den << ","
             << d.num << "," << d.den << "\n";
    }

    return true;
}

int main() {
    vector<PairFrac> arr;

    if (!readPairs("pairs.csv", arr)) {
        cerr << "Loi mo file input!\n";
        return 1;
    }

    if (!writeResults("results.csv", arr)) {
        cerr << "Loi mo file output!\n";
        return 1;
    }

    return 0;
}
```

7.2.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (pairs.csv):

```
1,2,1,3
3,4,1,4
```

Expected Output (results.csv):

```
5,6,1,6
1,1,1,2
```

Test Case 2 (Hai phân số nghịch nhau)
Input (pairs.csv):

```
1,5,-1,5
```

Expected Output (results.csv):

```
0,1,2,5
```

Test Case 3 (Case biên – mẫu âm)
Input (pairs.csv):

```
1,-2,1,2
```

Expected Output (results.csv):

```
0,1,-1,1
```

7.3 Sắp xếp danh sách phân số [Cơ bản]

7.3.1 Phân tích & Thiết kế (M0)

- **Input:** File `fractions.csv` chứa nhiều dòng:
  - Mỗi dòng có dạng: `num,den`
- **Xử lý:**
  - **Case biên:**
    - `den = 0` → phân số không hợp lệ, bỏ qua
    - `num = 0` → chuẩn hoá thành `0,1`
    - `den < 0` → đổi dấu cả tử và mẫu
  - **Module 1 (Đọc file):**
    - Hàm `readFractions(path, vector<Fraction>&)`
    - Đọc từng dòng CSV
    - Kiểm tra mở file (`fail` → `return false`)
  - **Module 2 (Chuẩn hoá phân số):**
    - Nếu mẫu bằng 0 → loại bỏ
    - Đưa mẫu về dương
    - Rút gọn gcd
  - **Module 3 (So sánh phân số):**

```
a/b < c/d <=> a*d < c*b

```

    * Dùng `__int128` để tránh tràn số
    * Hàm `cmpFrac(a,b)` trả về `-1/0/1`
  - **Module 4 (Sắp xếp):**
    * Dùng `std::sort` với lambda comparator
  - **Module 5 (Ghi file):**
    * Ghi `sorted.csv`
    * Định dạng: `num,den`

- **Output:**

```
Ghi file sorted.csv

```

7.3.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>
#include <sstream>
#include <algorithm>
#include <cstdlib>

using namespace std;

struct Fraction {
    long long num, den;
};

long long gcd(long long a, long long b) {
    a = llabs(a);
    b = llabs(b);

    while (b != 0) {
        long long t = a % b;
        a = b;
        b = t;
    }

    return a;
}

bool normalize(Fraction& f) {
    if (f.den == 0) return false;
```

```

    if (f.num == 0) {
        f.num = 0;
        f.den = 1;
        return true;
    }

    if (f.den < 0) {
        f.num = -f.num;
        f.den = -f.den;
    }

    long long g = gcd(f.num, f.den);
    f.num /= g;
    f.den /= g;

    return true;
}

bool readFractions(const string& path,
                   vector<Fraction>& arr) {
    ifstream fin(path);
    if (!fin) return false;

    string line;
    while (getline(fin, line)) {
        if (line.empty()) continue;

        stringstream ss(line);
        string s1, s2;

        if (getline(ss, s1, ',') &&
            getline(ss, s2)) {
            Fraction f;
            f.num = atoll(s1.c_str());
            f.den = atoll(s2.c_str());
            arr.push_back(f);
        }
    }

    return true;
}

int cmpFrac(Fraction a, Fraction b) {
    __int128 left = (__int128)a.num * b.den;
    __int128 right = (__int128)b.num * a.den;

    if (left < right) return -1;
    if (left > right) return 1;
```

```
    return 0;
}

bool writeFractions(const string& path,
                    const vector<Fraction>& arr) {
    ofstream fout(path);
    if (!fout) return false;

    for (size_t i = 0; i < arr.size(); ++i)
        fout << arr[i].num
             << "," << arr[i].den << "\n";

    return true;
}

int main() {
    vector<Fraction> input, valid;

    if (!readFractions("fractions.csv", input)) {
        cerr << "Loi mo file input!\n";
        return 1;
    }

    for (size_t i = 0; i < input.size(); ++i) {
        Fraction f = input[i];

        if (normalize(f))
            valid.push_back(f);
    }

    sort(valid.begin(), valid.end(),
        [](Fraction a, Fraction b) {
            return cmpFrac(a, b) < 0;
        });

    if (!writeFractions("sorted.csv", valid)) {
        cerr << "Loi mo file output!\n";
        return 1;
    }

    return 0;
}
```

7.3.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (fractions.csv):

```
1,2
```

```
3,4
1,3
```

Expected Output (sorted.csv):

```
1,3
1,2
3,4
```

Test Case 2 (Có invalid)
Input (fractions.csv):

```
2,0
5,10
-3,6
```

Expected Output (sorted.csv):

```
-1,2
1,2
```

Test Case 3 (Case biên – num=0)
Input (fractions.csv):

```
0,7
2,3
```

Expected Output (sorted.csv):

```
0,1
2,3
```

7.4 Tổng tất cả phân số hợp lệ [Trung bình]

7.4.1 Phân tích & Thiết kế (M0)

- **Input:** File `fractions.csv` chứa nhiều dòng:
  - Mỗi dòng có dạng: `num,den`
- **Xử lý:**
  - **Case biên:**
    * `den = 0` → phân số không hợp lệ
    * Không có phân số hợp lệ → `sum = 0/1`
    * `num = 0` → chuẩn hoá thành `0/1`
    * `den < 0` → đổi dấu cả tử và mẫu
  - **Module 1 (Đọc file):**

    * Hàm `readFractions(path, vector<Fraction>&)`
    * Đọc từng dòng CSV
    * Kiểm tra mở file (`fail` → `return false`)
  - **Module 2 (Chuẩn hoá phân số):**
    * Nếu mẫu bằng 0 → loại bỏ
    * Đưa mẫu về dương
    * Rút gọn gcd
  - **Module 3 (Cộng phân số):**
    * Khởi tạo:

```
sum = {0,1}

```

    * Cộng từng phân số hợp lệ bằng `addFrac`
    * Sau mỗi lần cộng gọi `normalize`
  - **Module 4 (Thống kê):**
    * Đếm số lượng hợp lệ và không hợp lệ

- **Output:**

```
valid=<n> / invalid=<m> / sum=<num>/<den>

```

7.4.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>
#include <sstream>
#include <cstdlib>

using namespace std;

struct Fraction {
    long long num, den;
};

long long gcd(long long a, long long b) {
    a = llabs(a);
    b = llabs(b);

    while (b != 0) {
        long long t = a % b;
        a = b;
        b = t;
    }
```

```

    return a;
}

bool normalize(Fraction& f) {
    if (f.den == 0) return false;

    if (f.num == 0) {
        f.num = 0;
        f.den = 1;
        return true;
    }

    if (f.den < 0) {
        f.num = -f.num;
        f.den = -f.den;
    }

    long long g = gcd(f.num, f.den);
    f.num /= g;
    f.den /= g;

    return true;
}

bool readFractions(const string& path,
                   vector<Fraction>& arr) {
    ifstream fin(path);
    if (!fin) return false;

    string line;
    while (getline(fin, line)) {
        if (line.empty()) continue;

        stringstream ss(line);
        string s1, s2;

        if (getline(ss, s1, ',') &&
            getline(ss, s2)) {
            Fraction f;
            f.num = atoll(s1.c_str());
            f.den = atoll(s2.c_str());
            arr.push_back(f);
        }
    }

    return true;
}

```

```
Fraction addFrac(Fraction a, Fraction b) {
    Fraction r;
    r.num = a.num * b.den + b.num * a.den;
    r.den = a.den * b.den;
    normalize(r);
    return r;
}

int main() {
    vector<Fraction> arr;

    if (!readFractions("fractions.csv", arr)) {
        cerr << "Loi mo file input!\n";
        return 1;
    }

    int valid = 0, invalid = 0;
    Fraction sum = {0, 1};

    for (size_t i = 0; i < arr.size(); ++i) {
        Fraction f = arr[i];

        if (normalize(f)) {
            valid++;
            sum = addFrac(sum, f);
        } else {
            invalid++;
        }
    }

    cout << "valid=" << valid
         << " / invalid=" << invalid
         << " / sum=" << sum.num
         << "/" << sum.den;

    return 0;
}
```

7.4.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (fractions.csv):

```
1,2
1,3
1,6
```

Expected Output:

```
valid=3 / invalid=0 / sum=1/1
```

Test Case 2 (Có invalid)
Input (fractions.csv):

```
2,0
3,4
5,0
```

Expected Output:

```
valid=1 / invalid=2 / sum=3/4
```

Test Case 3 (Không có phân số hợp lệ)
Input (fractions.csv):

```
1,0
2,0
```

Expected Output:

```
valid=0 / invalid=2 / sum=0/1
```

7.5 Lọc phân số trong khoảng [a, b][Trung bình]

7.5.1 Phân tích & Thiết kế (M0)

- **Input:**
  - File `fractions.csv`: mỗi dòng dạng `num,den`
  - File `config.txt`: `a_num,a_den,b_num,b_den`
- **Xử lý:**
  - **Case biên:**
    * `den = 0` → phân số không hợp lệ, bỏ qua
    * `num = 0` → chuẩn hoá thành `0/1`
    * `den < 0` → đổi dấu cả tử và mẫu
    * Nếu `lo > hi` → khoảng rỗng, `in_range = 0`
  - **Module 1 (Đọc file):**
    * Hàm `readFractions(path, vector<Fraction>&)`
    * Hàm `readConfig(path, lo, hi)`
    * Kiểm tra mở file (`fail` → `return false`)
  - **Module 2 (Chuẩn hoá phân số):**
    * Lọc invalid
    * Đưa mẫu về dương
    * Rút gọn gcd

– **Module 3 (So sánh phân số):**

```
a/b < c/d <=> a*d < c*b

```

  * Dùng `__int128` tránh tràn số

– **Module 4 (Lọc trong khoảng):**

```
f   [lo,hi] <=> cmpFrac(lo,f)<=0
              && cmpFrac(f,hi)<=0

```

– **Module 5 (Sắp xếp và ghi file):**

  * Sắp xếp tăng dần
  * Ghi `filtered.csv`: `num,den`

– **Module 6 (Thống kê):**

  * `total_valid`
  * `in_range`
  * `out_of_range`

- **Output:**

```
Ghi file filtered.csv
total_valid=<n> / in_range=<m> / out_of_range=<k>

```

7.5.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>
#include <sstream>
#include <algorithm>
#include <cstdlib>

using namespace std;

struct Fraction {
    long long num, den;
};

long long gcd(long long a, long long b) {
    a = llabs(a);
    b = llabs(b);

    while (b != 0) {
        long long t = a % b;
```

```
        a = b;
        b = t;
    }
    return a;
}

bool normalize(Fraction& f) {
    if (f.den == 0) return false;

    if (f.num == 0) {
        f.num = 0;
        f.den = 1;
        return true;
    }

    if (f.den < 0) {
        f.num = -f.num;
        f.den = -f.den;
    }

    long long g = gcd(f.num, f.den);
    f.num /= g;
    f.den /= g;
    return true;
}

bool readFractions(const string& path,
                   vector<Fraction>& arr) {
    ifstream fin(path);
    if (!fin) return false;

    string line;
    while (getline(fin, line)) {
        if (line.empty()) continue;

        stringstream ss(line);
        string s1, s2;

        if (getline(ss, s1, ',') &&
            getline(ss, s2)) {
            Fraction f;
            f.num = atoll(s1.c_str());
            f.den = atoll(s2.c_str());
            arr.push_back(f);
        }
    }
    return true;
}

```

```
bool readConfig(const string& path,
                Fraction& lo,
                Fraction& hi) {
    ifstream fin(path);
    if (!fin) return false;

    string line;
    getline(fin, line);

    stringstream ss(line);
    string s[4];

    for (int i = 0; i < 4; ++i)
        getline(ss, s[i], i == 3 ? '\n' : ',');

    lo.num = atoll(s[0].c_str());
    lo.den = atoll(s[1].c_str());
    hi.num = atoll(s[2].c_str());
    hi.den = atoll(s[3].c_str());

    return normalize(lo) && normalize(hi);
}

int cmpFrac(Fraction a, Fraction b) {
    __int128 left = (__int128)a.num * b.den;
    __int128 right = (__int128)b.num * a.den;

    if (left < right) return -1;
    if (left > right) return 1;
    return 0;
}

bool inRange(Fraction f,
             Fraction lo,
             Fraction hi) {
    return cmpFrac(lo, f) <= 0 &&
           cmpFrac(f, hi) <= 0;
}

bool writeFractions(const string& path,
                    const vector<Fraction>& arr) {
    ofstream fout(path);
    if (!fout) return false;

    for (size_t i = 0; i < arr.size(); ++i)
        fout << arr[i].num
             << "," << arr[i].den << "\n";

    return true;
```

```
}

int main() {
    vector<Fraction> input, valid, filtered;
    Fraction lo, hi;

    if (!readFractions("fractions.csv", input)) {
        cerr << "Loi mo file fractions!\n";
        return 1;
    }

    if (!readConfig("config.txt", lo, hi)) {
        cerr << "Loi mo file config!\n";
        return 1;
    }

    for (size_t i = 0; i < input.size(); ++i) {
        Fraction f = input[i];

        if (normalize(f))
            valid.push_back(f);
    }

    int total_valid = valid.size();
    int in_range = 0, out_of_range = 0;

    if (cmpFrac(lo, hi) <= 0) {
        for (size_t i = 0; i < valid.size(); ++i) {
            if (inRange(valid[i], lo, hi)) {
                filtered.push_back(valid[i]);
                in_range++;
            } else {
                out_of_range++;
            }
        }
    } else {
        out_of_range = total_valid;
    }

    sort(filtered.begin(), filtered.end(),
        [](Fraction a, Fraction b) {
            return cmpFrac(a, b) < 0;
        });

    if (!writeFractions("filtered.csv", filtered)) {
        cerr << "Loi mo file output!\n";
        return 1;
    }

```

```
    cout << "total_valid=" << total_valid
         << " / in_range=" << in_range
         << " / out_of_range=" << out_of_range;

    return 0;
}
```

7.5.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (fractions.csv):

```
1,2
3,4
1,3
5,6
```

Input (config.txt):

```
1,3,3,4
```

Expected Output (filtered.csv):

```
1,3
1,2
3,4
```

Console Output:

```
total_valid=4 / in_range=3 / out_of_range=1
```

Test Case 2 (Có invalid)
Input (fractions.csv):

```
2,0
1,2
4,8
```

Input (config.txt):

```
1,2,1,1
```

Expected Output (filtered.csv):

```
1,2
1,2
```

Console Output:

```
total_valid=2 / in_range=2 / out_of_range=0
```

Test Case 3 (Khoảng rỗng)
Input (fractions.csv):

```
1,2
2,3
```

Input (config.txt):

```
3,4,1,2
```

Expected Output (filtered.csv):

Console Output:

```
total_valid=2 / in_range=0 / out_of_range=2
```

7.6 Nhân và chia 2 phân số [Cơ bản]

7.6.1 Phân tích & Thiết kế (M0)

- **Input:** File `pairs.csv` chứa nhiều dòng:
  - Mỗi dòng gồm 4 số: `n1,d1,n2,d2`
  - Tương ứng hai phân số $\frac{n1}{d1}$ và $\frac{n2}{d2}$
- **Xử lý:**
  - **Case biên:**
    - Nếu mẫu số bằng 0 → phân số không hợp lệ
    - Nếu `d2 = 0` hoặc `n2 = 0` → chia cho 0
    - Nếu tử số bằng 0 → chuẩn hoá thành `0/1`
    - Nếu mẫu âm → đổi dấu cả tử và mẫu
  - **Module 1 (Đọc file):**
    - Hàm `readPairs(path, vector<PairFrac>&)`
    - Đọc từng dòng CSV
    - Kiểm tra mở file (`fail` → `return false`)
  - **Module 2 (Ước chung lớn nhất):**
    - Thuật toán Euclid để rút gọn
  - **Module 3 (Chuẩn hoá):**
    - Đưa mẫu về dương
    - Rút gọn gcd
  - **Module 4 (Nhân và chia):**
    -

```
(a/b) * (c/d) = (a*c)/(b*d)
(a/b) / (c/d) = (a*d)/(b*c)

```

    * Nếu chia cho 0 → ghi `DIV_ZERO`
  - **Module 5 (Ghi file):**
    * Ghi `results.csv`
    * Định dạng: `prod_num,prod_den,quot_num,quot_den`

- **Output:**

```
Ghi file results.csv

```

7.6.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>
#include <sstream>
#include <cstdlib>

using namespace std;

struct Fraction {
    long long num, den;
};

struct PairFrac {
    Fraction a, b;
};

long long gcd(long long a, long long b) {
    a = llabs(a);
    b = llabs(b);

    while (b != 0) {
        long long t = a % b;
        a = b;
        b = t;
    }

    return a;
}

bool normalize(Fraction& f) {
    if (f.den == 0) return false;

    if (f.num == 0) {
        f.num = 0;
        f.den = 1;
        return true;
```

```
    }

    if (f.den < 0) {
        f.num = -f.num;
        f.den = -f.den;
    }

    long long g = gcd(f.num, f.den);
    f.num /= g;
    f.den /= g;

    return true;
}

bool readPairs(const string& path,
               vector<PairFrac>& arr) {
    ifstream fin(path);
    if (!fin) return false;

    string line;
    while (getline(fin, line)) {
        if (line.empty()) continue;

        stringstream ss(line);
        string s[4];

        for (int i = 0; i < 4; ++i)
            getline(ss, s[i], i == 3 ? '\n' : ',');

        PairFrac p;
        p.a.num = atoll(s[0].c_str());
        p.a.den = atoll(s[1].c_str());
        p.b.num = atoll(s[2].c_str());
        p.b.den = atoll(s[3].c_str());

        arr.push_back(p);
    }

    return true;
}

Fraction mulFrac(Fraction x, Fraction y) {
    Fraction r;
    r.num = x.num * y.num;
    r.den = x.den * y.den;
    normalize(r);
    return r;
}

```

```
bool divFrac(Fraction x, Fraction y, Fraction& r) {
    if (y.num == 0 || y.den == 0) return false;

    r.num = x.num * y.den;
    r.den = x.den * y.num;
    return normalize(r);
}

bool writeResults(const string& path,
                  const vector<PairFrac>& arr) {
    ofstream fout(path);
    if (!fout) return false;

    for (size_t i = 0; i < arr.size(); ++i) {
        Fraction p = mulFrac(arr[i].a, arr[i].b);
        fout << p.num << "," << p.den << ",";

        Fraction q;
        if (divFrac(arr[i].a, arr[i].b, q))
            fout << q.num << "," << q.den;
        else
            fout << "DIV_ZERO";

        fout << "\n";
    }

    return true;
}

int main() {
    vector<PairFrac> arr;

    if (!readPairs("pairs.csv", arr)) {
        cerr << "Loi mo file input!\n";
        return 1;
    }

    if (!writeResults("results.csv", arr)) {
        cerr << "Loi mo file output!\n";
        return 1;
    }

    return 0;
}
```

7.6.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)

Input (pairs.csv):

```
1,2,3,4
2,3,5,6
```

Expected Output (results.csv):

```
3,8,2,3
5,9,4,5
```

Test Case 2 (Chia cho 0)
Input (pairs.csv):

```
1,2,0,5
3,4,7,0
```

Expected Output (results.csv):

```
0,1,DIV_ZERO
0,1,DIV_ZERO
```

Test Case 3 (Case biên – mẫu âm)
Input (pairs.csv):

```
1,-2,1,3
```

Expected Output (results.csv):

```
-1,6,-3,2
```

7.7 Phân số lớn nhất và nhỏ nhất [Cơ bản]

7.7.1 Phân tích & Thiết kế (M0)

- **Input:** File `fractions.csv` chứa nhiều dòng:
  - Mỗi dòng có dạng: `num,den`
- **Xử lý:**
  - **Case biên:**
    - `den = 0` → phân số không hợp lệ, bỏ qua
    - `num = 0` → chuẩn hoá thành `0/1`
    - `den < 0` → đổi dấu cả tử và mẫu
    - Không có phân số hợp lệ → in `EMPTY`
  - **Module 1 (Đọc file):**
    - Hàm `readFractions(path, vector<Fraction>&)`
    - Đọc từng dòng CSV

    * Kiểm tra mở file (`fail` → `return false`)
  - **Module 2 (Chuẩn hoá phân số):**
    * Lọc invalid
    * Đưa mẫu về dương
    * Rút gọn gcd
  - **Module 3 (So sánh phân số):**
    *
    ```
    a/b < c/d <=> a*d < c*b

    ```
    * Dùng `__int128` tránh tràn số
  - **Module 4 (Tìm min, max):**
    * Duyệt danh sách hợp lệ 1 lần
    * Cập nhật nhỏ nhất và lớn nhất

- **Output:**

```
min=<num>/<den> / max=<num>/<den>

```

Hoặc:

```
EMPTY

```

7.7.2 Mã nguồn C++11 (Tách module)

```
#include <iostream>
#include <vector>
#include <fstream>
#include <sstream>
#include <cstdlib>

using namespace std;

struct Fraction {
    long long num, den;
};

long long gcd(long long a, long long b) {
    a = llabs(a);
    b = llabs(b);

    while (b != 0) {
        long long t = a % b;
        a = b;
        b = t;
```

```
    }

    return a;
}

bool normalize(Fraction& f) {
    if (f.den == 0) return false;

    if (f.num == 0) {
        f.num = 0;
        f.den = 1;
        return true;
    }

    if (f.den < 0) {
        f.num = -f.num;
        f.den = -f.den;
    }

    long long g = gcd(f.num, f.den);
    f.num /= g;
    f.den /= g;

    return true;
}

bool readFractions(const string& path,
                   vector<Fraction>& arr) {
    ifstream fin(path);
    if (!fin) return false;

    string line;
    while (getline(fin, line)) {
        if (line.empty()) continue;

        stringstream ss(line);
        string s1, s2;

        if (getline(ss, s1, ',') &&
            getline(ss, s2)) {
            Fraction f;
            f.num = atoll(s1.c_str());
            f.den = atoll(s2.c_str());
            arr.push_back(f);
        }
    }

    return true;
}
```

```

int cmpFrac(Fraction a, Fraction b) {
    __int128 left = (__int128)a.num * b.den;
    __int128 right = (__int128)b.num * a.den;

    if (left < right) return -1;
    if (left > right) return 1;
    return 0;
}

int main() {
    vector<Fraction> input, valid;

    if (!readFractions("fractions.csv", input)) {
        cerr << "Loi mo file input!\n";
        return 1;
    }

    for (size_t i = 0; i < input.size(); ++i) {
        Fraction f = input[i];

        if (normalize(f))
            valid.push_back(f);
    }

    if (valid.empty()) {
        cout << "EMPTY";
        return 0;
    }

    Fraction mn = valid[0];
    Fraction mx = valid[0];

    for (size_t i = 1; i < valid.size(); ++i) {
        if (cmpFrac(valid[i], mn) < 0)
            mn = valid[i];

        if (cmpFrac(valid[i], mx) > 0)
            mx = valid[i];
    }

    cout << "min=" << mn.num << "/" << mn.den
         << " / max=" << mx.num << "/" << mx.den;

    return 0;
}
```

7.7.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (fractions.csv):

```
1,2
3,4
-1,3
```

Expected Output:

```
min=-1/3 / max=3/4
```

Test Case 2 (Có invalid)
Input (fractions.csv):

```
2,0
5,10
7,0
```

Expected Output:

```
min=1/2 / max=1/2
```

Test Case 3 (Không có phân số hợp lệ)
Input (fractions.csv):

```
1,0
2,0
```

Expected Output:

```
EMPTY
```

7.8 Trung bình các phân số [Trung bình]

7.8.1 Phân tích & Thiết kế (M0)

- **Input:** File `fractions.csv` chứa nhiều dòng:
  - Mỗi dòng có dạng: `num,den`
- **Xử lý:**
  - **Case biên:**
    * `den = 0` → phân số không hợp lệ, bỏ qua
    * `num = 0` → chuẩn hoá thành `0/1`
    * `den < 0` → đổi dấu cả tử và mẫu
    * Không có phân số hợp lệ ($n = 0$) → in `EMPTY`

- **Module 1 (Đọc file):**
  * Hàm `readFractions(path, vector<Fraction>&)`
  * Đọc từng dòng CSV
  * Kiểm tra mở file (`fail` → `return false`)
- **Module 2 (Chuẩn hoá phân số):**
  * Lọc invalid
  * Đưa mẫu về dương
  * Rút gọn gcd
- **Module 3 (Tính tổng):**
  * Khởi tạo:

```
sum = {0,1}

```

  * Cộng từng phân số hợp lệ bằng `addFrac`
- **Module 4 (Tính mean):**
  *

```
mean = sum / n

```

  * Kết quả tiếp tục chuẩn hoá

- **Output:**

```
mean=<num>/<den>

```

Hoặc:

```
EMPTY

```

7.8.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>
#include <sstream>
#include <cstdlib>

using namespace std;

struct Fraction {
    long long num, den;
};

long long gcd(long long a, long long b) {
```

```
    a = llabs(a);
    b = llabs(b);

    while (b != 0) {
        long long t = a % b;
        a = b;
        b = t;
    }

    return a;
}

bool normalize(Fraction& f) {
    if (f.den == 0) return false;

    if (f.num == 0) {
        f.num = 0;
        f.den = 1;
        return true;
    }

    if (f.den < 0) {
        f.num = -f.num;
        f.den = -f.den;
    }

    long long g = gcd(f.num, f.den);
    f.num /= g;
    f.den /= g;

    return true;
}

bool readFractions(const string& path,
                   vector<Fraction>& arr) {
    ifstream fin(path);
    if (!fin) return false;

    string line;
    while (getline(fin, line)) {
        if (line.empty()) continue;

        stringstream ss(line);
        string s1, s2;

        if (getline(ss, s1, ',') &&
            getline(ss, s2)) {
            Fraction f;
            f.num = atoll(s1.c_str());
```

```
            f.den = atoll(s2.c_str());
            arr.push_back(f);
        }
    }

    return true;
}

Fraction addFrac(Fraction a, Fraction b) {
    Fraction r;
    r.num = a.num * b.den + b.num * a.den;
    r.den = a.den * b.den;
    normalize(r);
    return r;
}

Fraction divInt(Fraction a, long long n) {
    Fraction r;
    r.num = a.num;
    r.den = a.den * n;
    normalize(r);
    return r;
}

int main() {
    vector<Fraction> input;
    Fraction sum = {0, 1};

    if (!readFractions("fractions.csv", input)) {
        cerr << "Loi mo file input!\n";
        return 1;
    }

    int cnt = 0;

    for (size_t i = 0; i < input.size(); ++i) {
        Fraction f = input[i];

        if (normalize(f)) {
            sum = addFrac(sum, f);
            cnt++;
        }
    }

    if (cnt == 0) {
        cout << "EMPTY";
        return 0;
    }

```

```
    Fraction mean = divInt(sum, cnt);

    cout << "mean=" << mean.num
         << "/" << mean.den;

    return 0;
}
```

7.8.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (fractions.csv):

```
1,2
1,4
3,4
```

Expected Output:

```
mean=1/2
```

Test Case 2 (Có invalid)
Input (fractions.csv):

```
2,0
1,2
3,2
```

Expected Output:

```
mean=1/1
```

Test Case 3 (Không có phân số hợp lệ)
Input (fractions.csv):

```
1,0
5,0
```

Expected Output:

```
EMPTY
```

7.9 Kiểm tra phân số là số nguyên [Trung bình]

7.9.1 Phân tích & Thiết kế (M0)

- **Input:** File `fractions.csv` chứa nhiều dòng:
  - Mỗi dòng có dạng: `num,den`

- **Xử lý:**
  - **Case biên:**
    * `den = 0` → phân số không hợp lệ, bỏ qua
    * `num = 0` → chuẩn hoá thành `0/1` và là số nguyên
    * `den < 0` → đổi dấu cả tử và mẫu
    * Phân số là số nguyên khi `den = 1`
  - **Module 1 (Đọc file):**
    * Hàm `readFractions(path, vector<Fraction>&)`
    * Đọc từng dòng CSV
    * Kiểm tra mở file (`fail` → `return false`)
  - **Module 2 (Chuẩn hoá phân số):**
    * Lọc invalid
    * Đưa mẫu về dương
    * Rút gọn gcd
  - **Module 3 (Kiểm tra số nguyên):**
    * Sau chuẩn hoá:

```
den == 1

```

  - **Module 4 (Ghi file):**
    * Ghi `integers.csv`
    * Chỉ ghi các phân số nguyên dạng: `num/1`
  - **Module 5 (Thống kê):**
    * `total_valid`
    * `is_integer_count`
- **Output:**

```
Ghi file integers.csv
total_valid=<n> / is_integer_count=<m>

```

7.9.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>
#include <sstream>
#include <cstdlib>

using namespace std;
```

```

struct Fraction {
    long long num, den;
};

long long gcd(long long a, long long b) {
    a = llabs(a);
    b = llabs(b);

    while (b != 0) {
        long long t = a % b;
        a = b;
        b = t;
    }

    return a;
}

bool normalize(Fraction& f) {
    if (f.den == 0) return false;

    if (f.num == 0) {
        f.num = 0;
        f.den = 1;
        return true;
    }

    if (f.den < 0) {
        f.num = -f.num;
        f.den = -f.den;
    }

    long long g = gcd(f.num, f.den);
    f.num /= g;
    f.den /= g;

    return true;
}

bool readFractions(const string& path,
                   vector<Fraction>& arr) {
    ifstream fin(path);
    if (!fin) return false;

    string line;
    while (getline(fin, line)) {
        if (line.empty()) continue;

        stringstream ss(line);
```

```
        string s1, s2;

        if (getline(ss, s1, ',') &&
            getline(ss, s2)) {
            Fraction f;
            f.num = atoll(s1.c_str());
            f.den = atoll(s2.c_str());
            arr.push_back(f);
        }
    }

    return true;
}

bool writeIntegers(const string& path,
                   const vector<Fraction>& arr) {
    ofstream fout(path);
    if (!fout) return false;

    for (size_t i = 0; i < arr.size(); ++i)
        fout << arr[i].num
             << "/1\n";

    return true;
}

int main() {
    vector<Fraction> input, integers;

    if (!readFractions("fractions.csv", input)) {
        cerr << "Loi mo file input!\n";
        return 1;
    }

    int total_valid = 0;
    int is_integer_count = 0;

    for (size_t i = 0; i < input.size(); ++i) {
        Fraction f = input[i];

        if (normalize(f)) {
            total_valid++;

            if (f.den == 1) {
                integers.push_back(f);
                is_integer_count++;
            }
        }
    }
```

```

    if (!writeIntegers("integers.csv", integers)) {
        cerr << "Loi mo file output!\n";
        return 1;
    }

    cout << "total_valid=" << total_valid
         << " / is_integer_count="
         << is_integer_count;

    return 0;
}
```

7.9.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (fractions.csv):

```
4,2
3,1
5,2
```

Expected Output (integers.csv):

```
2/1
3/1
```

Console Output:

```
total_valid=3 / is_integer_count=2
```

Test Case 2 (Có invalid)
Input (fractions.csv):

```
2,0
0,5
6,3
```

Expected Output (integers.csv):

```
0/1
2/1
```

Console Output:

```
total_valid=2 / is_integer_count=2
```

Test Case 3 (Không có số nguyên)
Input (fractions.csv):

```
1,2
3,4
```

Expected Output (integers.csv):

Console Output:

```
total_valid=2 / is_integer_count=0
```

7.10 Dãy Farey bậc N [Nâng cao]

7.10.1 Phân tích & Thiết kế (M0)

- **Input:** File `config.txt` chứa:
  - Một số nguyên `N` ($N \leq 50$)
- **Xử lý:**
  - **Case biên:**
    - `N = 1` → dãy gồm `0/1, 1/1`
    - Nếu `N <= 0` → dãy rỗng
  - **Module 1 (Đọc file):**
    - Hàm `readN(path, int& N)`
    - Kiểm tra mở file (`fail` → `return false`)
  - **Module 2 (Ước chung lớn nhất):**
    - Thuật toán Euclid:

```
gcd(a,b)

```

  - **Module 3 (Sinh phân số):**
    - Duyệt:

```
0 <= p <= q <= N

```

    - Chỉ nhận khi:

```
gcd(p,q) = 1

```

    - Khi đó phân số đã tối giản
  - **Module 4 (Sắp xếp):**
    - So sánh:

```
a/b < c/d <=> a*d < c*b

```

    - Dùng `__int128` tránh tràn số

- **Module 5 (Ghi file):**
  - Ghi `farey.csv`: `num,den`
- **Module 6 (Thống kê):**
  - In số lượng phần tử `count`

- **Output:**

```
Ghi file farey.csv
count=<n>

```

7.10.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>
#include <algorithm>

using namespace std;

struct Fraction {
    int num, den;
};

int gcd(int a, int b) {
    if (a < 0) a = -a;
    if (b < 0) b = -b;

    while (b != 0) {
        int t = a % b;
        a = b;
        b = t;
    }

    return a;
}

bool readN(const string& path, int& N) {
    ifstream fin(path);
    if (!fin) return false;
    fin >> N;
    return true;
}

int cmpFrac(Fraction a, Fraction b) {
    __int128 left = (__int128)a.num * b.den;
    __int128 right = (__int128)b.num * a.den;

```

```
    if (left < right) return -1;
    if (left > right) return 1;
    return 0;
}

bool writeFractions(const string& path,
                    const vector<Fraction>& arr) {
    ofstream fout(path);
    if (!fout) return false;

    for (size_t i = 0; i < arr.size(); ++i)
        fout << arr[i].num
             << "," << arr[i].den << "\n";

    return true;
}

int main() {
    int N;

    if (!readN("config.txt", N)) {
        cerr << "Loi mo file input!\n";
        return 1;
    }

    vector<Fraction> farey;

    if (N > 0) {
        for (int q = 1; q <= N; ++q) {
            for (int p = 0; p <= q; ++p) {
                if (gcd(p, q) == 1) {
                    Fraction f;
                    f.num = p;
                    f.den = q;
                    farey.push_back(f);
                }
            }
        }

        sort(farey.begin(), farey.end(),
            [](Fraction a, Fraction b) {
                return cmpFrac(a, b) < 0;
            });
    }

    if (!writeFractions("farey.csv", farey)) {
        cerr << "Loi mo file output!\n";
        return 1;
    }
```

```

    cout << "count=" << farey.size();

    return 0;
}
```

7.10.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (N = 1)
Input (config.txt):

```
1
```

Expected Output (farey.csv):

```
0,1
1,1
```

Console Output:

```
count=2
```

Test Case 2 (N = 3)
Input (config.txt):

```
3
```

Expected Output (farey.csv):

```
0,1
1,3
1,2
2,3
1,1
```

Console Output:

```
count=5
```

Test Case 3 (N <= 0)
Input (config.txt):

```
0
```

Expected Output (farey.csv):

Console Output:

```
count=0
```

8 Tuần 8: File + Struct: Đa thức Record + Ma trận Phân số

8.1 Đọc danh sách đa thức từ CSV [Cơ bản]

8.1.1 Phân tích & Thiết kế (M0)

- **Input:** File `polynomials.csv` chứa nhiều dòng:
  - Mỗi dòng có dạng: `id,degree,a0,a1,...,an`
  - Trong đó:
    * `id`: mã đa thức
    * `degree`: bậc đa thức
    * `a[i]`: hệ số của x^i
- **Xử lý:**
  - **Case biên:**
    * `degree < 0` → invalid
    * `degree = 0` → đa thức hằng
    * Thiếu hệ số → gán bằng 0
    * Nếu số hệ số ít hơn bậc yêu cầu vẫn hợp lệ, bổ sung 0
  - **Module 1 (Cấu trúc dữ liệu):**

```
struct Poly {
    string id;
    int deg;
    vector<double> a;
};
```

  - **Module 2 (Đọc file CSV):**
    * Hàm: `readPolysCSV(path, vector<Poly>&, int& invalid)`
    * Đọc từng dòng
    * Tách dữ liệu bằng dấu phẩy
  - **Module 3 (Kiểm tra hợp lệ):**
    * Nếu lỗi mở file → false

    * Nếu `degree < 0` → tăng invalid
  - **Module 4 (Thống kê):**
    * Đếm số đa thức hợp lệ
    * Đếm số đa thức invalid
    * Tìm bậc lớn nhất
  - **Module 5 (Ghi file):**
    * Ghi ra `summary.txt`

- **Output:**

```
valid=<n>
invalid=<m>
max_degree=<d>
```

8.1.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <string>
#include <cstdlib>

using namespace std;

struct Poly {
    string id;
    int deg;
    vector<double> a;
};

bool readPolysCSV(const string& path,
                  vector<Poly>& arr,
                  int& invalid) {
    ifstream fin(path);
    if (!fin) return false;

    string line;
    invalid = 0;

    while (getline(fin, line)) {
        if (line.empty()) continue;

        stringstream ss(line);
        vector<string> token;
        string x;

```

```
        while (getline(ss, x, ',')) {
            token.push_back(x);
        }

        if (token.size() < 2) {
            invalid++;
            continue;
        }

        Poly p;
        p.id = token[0];
        p.deg = atoi(token[1].c_str());

        if (p.deg < 0) {
            invalid++;
            continue;
        }

        p.a.assign(p.deg + 1, 0);

        for (int i = 0; i <= p.deg; ++i) {
            int pos = i + 2;

            if (pos < (int)token.size())
                p.a[i] = atof(token[pos].c_str());
            else
                p.a[i] = 0;
        }

        arr.push_back(p);
    }

    return true;
}

int main() {
    vector<Poly> arr;
    int invalid = 0;

    if (!readPolysCSV("polynomials.csv",
                      arr, invalid)) {
        cerr << "Loi mo file input!\n";
        return 1;
    }

    int valid = arr.size();
    int max_degree = 0;

    if (valid > 0) {
```

```
        max_degree = arr[0].deg;

        for (size_t i = 1; i < arr.size(); ++i)
            if (arr[i].deg > max_degree)
                max_degree = arr[i].deg;
    }

    ofstream fout("summary.txt");

    if (!fout) {
        cerr << "Loi mo file output!\n";
        return 1;
    }

    fout << "valid=" << valid << "\n";
    fout << "invalid=" << invalid << "\n";
    fout << "max_degree=" << max_degree;

    cout << "Done";

    return 0;
}
```

8.1.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (polynomials.csv):

```
P1,2,1,2,3
P2,1,5,6
P3,0,7
```

Expected Output (summary.txt):

```
valid=3
invalid=0
max_degree=2
```

Console Output:

```
Done
```

Test Case 2 (Có invalid)
Input (polynomials.csv):

```
A,-1,1,2
B,3,1,2
```

Expected Output (summary.txt):

```
valid=1
invalid=1
max_degree=3
```

Console Output:

```
Done
```

Test Case 3 (Thiếu hệ số)
Input (polynomials.csv):

```
X,4,1,2
```

Expected Output (summary.txt):

```
valid=1
invalid=0
max_degree=4
```

Console Output:

```
Done
```

8.2 Tính P(x) và ghi report [Cơ bản]

8.2.1 Phân tích & Thiết kế (M0)

- **Input:**
  - File `polynomials.csv`: `id,degree,a0..an`
  - File `x_values.csv`: mỗi dòng là một giá trị `x`
- **Xử lý:**
  - **Case biên:**
    * `degree < 0` → invalid
    * Thiếu hệ số → gán bằng 0
    * `degree = 0` → đa thức hằng
    * `x_values.csv` rỗng → chỉ ghi header + separator + counts
  - **Module 1 (Struct dữ liệu):**

```
struct Poly {
    string id;
    int deg;
    vector<double> a;
};
```

  - **Module 2 (Đọc polynomials.csv):**

    * Hàm `readPolysCSV(...)`
    * Đếm invalid
  - **Module 3 (Đọc x_values.csv):**
    * Hàm `readXValues(...)`
  - **Module 4 (Tính P(x) bằng Horner):**

```
res = a[n];
for (i = n - 1; i >= 0; --i)
    res = res * x + a[i];
```

  - **Module 5 (Ghi report.csv):**
    * Header: `id,x,P(x)`
    * Mỗi dòng: kết quả tính được
    * Cuối file ghi thống kê

- **Output:**

```
report.csv
id,x,P(x)
...
----
valid_count=<n>
invalid_count=<m>
```

8.2.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <string>
#include <cstdlib>
#include <iomanip>

using namespace std;

struct Poly {
    string id;
    int deg;
    vector<double> a;
};

bool readPolysCSV(const string& path,
                  vector<Poly>& arr,
                  int& invalid) {
    ifstream fin(path);
    if (!fin) return false;
```

```

    string line;
    invalid = 0;

    while (getline(fin, line)) {
        if (line.empty()) continue;

        stringstream ss(line);
        vector<string> token;
        string x;

        while (getline(ss, x, ',')) {
            token.push_back(x);
        }

        if (token.size() < 2) {
            invalid++;
            continue;
        }

        Poly p;
        p.id = token[0];
        p.deg = atoi(token[1].c_str());

        if (p.deg < 0) {
            invalid++;
            continue;
        }

        p.a.assign(p.deg + 1, 0);

        for (int i = 0; i <= p.deg; ++i) {
            int pos = i + 2;

            if (pos < (int)token.size())
                p.a[i] = atof(token[pos].c_str());
            else
                p.a[i] = 0;
        }

        arr.push_back(p);
    }

    return true;
}

bool readXValues(const string& path,
                 vector<double>& xs) {
    ifstream fin(path);
```

```
    if (!fin) return false;

    string line;

    while (getline(fin, line)) {
        if (line.empty()) continue;

        xs.push_back(atof(line.c_str()));
    }

    return true;
}

double evalHorner(vector<double> a, double x) {
    int n = a.size() - 1;

    double res = a[n];

    for (int i = n - 1; i >= 0; --i)
        res = res * x + a[i];

    return res;
}

int main() {
    vector<Poly> polys;
    vector<double> xs;
    int invalid = 0;

    if (!readPolysCSV("polynomials.csv",
                      polys, invalid)) {
        cerr << "Loi mo polynomials.csv!\n";
        return 1;
    }

    if (!readXValues("x_values.csv", xs)) {
        cerr << "Loi mo x_values.csv!\n";
        return 1;
    }

    ofstream fout("report.csv");

    if (!fout) {
        cerr << "Loi mo report.csv!\n";
        return 1;
    }

    fout << "id,x,P(x)\n";
    fout << fixed << setprecision(6);
```

```

    for (size_t i = 0; i < polys.size(); ++i) {
        for (size_t j = 0; j < xs.size(); ++j) {
            double val =
                evalHorner(polys[i].a, xs[j]);

            fout << polys[i].id << ","
                 << xs[j] << ","
                 << val << "\n";
        }
    }

    fout << "----\n";
    fout << "valid_count="
         << polys.size() << "\n";
    fout << "invalid_count="
         << invalid;

    cout << "Done";

    return 0;
}
```

8.2.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (polynomials.csv):

```
P1,2,1,2,1
P2,1,3,4
```

Input (x_values.csv):

```
2
3
```

Expected Output (report.csv):

```
id,x,P(x)
P1,2.000000,9.000000
P1,3.000000,16.000000
P2,2.000000,11.000000
P2,3.000000,15.000000
----
valid_count=2
invalid_count=0
```

Test Case 2 (Có invalid)
Input (polynomials.csv):

```
A,-1,1
B,0,5
```

Input (x_values.csv):

```
10
```

Expected Output (report.csv):

```
id,x,P(x)
B,10.000000,5.000000
----
valid_count=1
invalid_count=1
```

Test Case 3 (x rỗng)
Input (polynomials.csv):

```
C,1,1,1
```

Input (x_values.csv):

```

```

Expected Output (report.csv):

```
id,x,P(x)
----
valid_count=1
invalid_count=0
```

8.3 Thống kê hệ số đa thức [Cơ bản]

8.3.1 Phân tích & Thiết kế (M0)

- **Input:** File `polynomials.csv` chứa nhiều dòng:
  - Mỗi dòng có dạng: `id,degree,a0,a1,...,an`
- **Xử lý:**
  - **Case biên:**
    - `degree < 0` → invalid
    - Thiếu hệ số → gán bằng 0
    - `degree = 0` → chỉ có 1 hệ số
    - Nếu tất cả hệ số = 0 → `min_abs = 0`
  - **Module 1 (Struct dữ liệu):**

```
struct Poly {
    string id;
    int deg;
    vector<double> a;
};
```

- **Module 2 (Đọc file CSV):**
  - Hàm `readPolysCSV(...)`
  - Chỉ lưu đa thức hợp lệ
- **Module 3 (Thống kê hệ số):**
  - Với mỗi hệ số:

    ```
    abs_val = fabs(a[i]);
    ```

  - Cập nhật:
    - `max_abs`
    - `min_abs`
    - `zero_count`
- **Module 4 (Ghi file):**
  - Ghi ra `stats.csv`
  - Định dạng: `id,degree,max_abs,min_abs,zero_count`

- **Output:**

```
stats.csv
id,degree,max_abs,min_abs,zero_count
...
```

8.3.2 Mã nguồn C++11 (Tách module)

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <string>
#include <cstdlib>
#include <iomanip>
#include <cmath>

using namespace std;

struct Poly {
    string id;
    int deg;
    vector<double> a;
```

```
};

bool readPolysCSV(const string& path,
                  vector<Poly>& arr) {
    ifstream fin(path);
    if (!fin) return false;

    string line;

    while (getline(fin, line)) {
        if (line.empty()) continue;

        stringstream ss(line);
        vector<string> token;
        string x;

        while (getline(ss, x, ',')) {
            token.push_back(x);
        }

        if (token.size() < 2) continue;

        Poly p;
        p.id = token[0];
        p.deg = atoi(token[1].c_str());

        if (p.deg < 0) continue;

        p.a.assign(p.deg + 1, 0);

        for (int i = 0; i <= p.deg; ++i) {
            int pos = i + 2;

            if (pos < (int)token.size())
                p.a[i] = atof(token[pos].c_str());
            else
                p.a[i] = 0;
        }

        arr.push_back(p);
    }

    return true;
}

int main() {
    vector<Poly> arr;

    if (!readPolysCSV("polynomials.csv", arr)) {
```

```
        cerr << "Loi mo file input!\n";
        return 1;
    }

    ofstream fout("stats.csv");

    if (!fout) {
        cerr << "Loi mo file output!\n";
        return 1;
    }

    fout << "id,degree,max_abs,min_abs,zero_count\n";
    fout << fixed << setprecision(6);

    for (size_t i = 0; i < arr.size(); ++i) {
        double max_abs = fabs(arr[i].a[0]);
        double min_abs = fabs(arr[i].a[0]);
        int zero_count = 0;

        for (int j = 0; j <= arr[i].deg; ++j) {
            double val = fabs(arr[i].a[j]);

            if (val > max_abs)
                max_abs = val;

            if (val < min_abs)
                min_abs = val;

            if (arr[i].a[j] == 0)
                zero_count++;
        }

        fout << arr[i].id << ","
             << arr[i].deg << ","
             << max_abs << ","
             << min_abs << ","
             << zero_count << "\n";
    }

    cout << "Done";

    return 0;
}
```

8.3.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (polynomials.csv):

```
P1,2,1,-5,3
P2,1,0,4
```

Expected Output (stats.csv):

```
id,degree,max_abs,min_abs,zero_count
P1,2,5.000000,1.000000,0
P2,1,4.000000,0.000000,1
```

Test Case 2 (Bậc 0)
Input (polynomials.csv):

```
A,0,-7
```

Expected Output (stats.csv):

```
id,degree,max_abs,min_abs,zero_count
A,0,7.000000,7.000000,0
```

Test Case 3 (Tất cả = 0)
Input (polynomials.csv):

```
Z,3,0,0,0,0
```

Expected Output (stats.csv):

```
id,degree,max_abs,min_abs,zero_count
Z,3,0.000000,0.000000,4
```

8.4 Cộng hai đa thức [Trung bình]

8.4.1 Phân tích & Thiết kế (M0)

- **Input:** File `poly_pairs.csv` chứa nhiều dòng:
  - Mỗi dòng gồm 2 đa thức: `id1,deg1,a0..an,id2,deg2,b0..bm`
- **Xử lý:**
  - **Case biên:**
    * Hai đa thức đều bậc 0
    * Một đa thức rỗng (deg < 0 hoặc thiếu dữ liệu)
    * Thiếu hệ số $\rightarrow$ gán bằng 0
  - **Module 1 (Struct dữ liệu):**

```cpp
struct Poly {
    string id;
    int deg;
    vector<double> a;
};
```

- **Module 2 (Đọc từng cặp đa thức):**
  * Tách dữ liệu CSV
  * Đọc đa thức thứ nhất
  * Đọc đa thức thứ hai
- **Module 3 (Cộng đa thức):**
  * Hàm: `Poly addPoly(Poly p, Poly q)`
  * Bậc mới:

```
max(p.deg, q.deg)
```

  * Cộng hệ số cùng bậc
  * Phần dư giữ nguyên
- **Module 4 (Ghi file):**
  * Ghi ra `sum.csv`
  * Định dạng: `id,degree_sum,c0..cm`

- **Output:**

```
sum.csv
id,degree_sum,c0,c1,...
```

8.4.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <string>
#include <cstdlib>
#include <iomanip>
#include <algorithm>

using namespace std;

struct Poly {
    string id;
    int deg;
    vector<double> a;
};

Poly addPoly(Poly p, Poly q) {
    Poly r;
    r.id = p.id + "_" + q.id;
    r.deg = max(p.deg, q.deg);

    r.a.assign(r.deg + 1, 0);
```

```

    for (int i = 0; i <= r.deg; ++i) {
        if (i <= p.deg)
            r.a[i] += p.a[i];

        if (i <= q.deg)
            r.a[i] += q.a[i];
    }

    return r;
}

bool readOnePoly(vector<string>& token,
                 int& pos,
                 Poly& p) {
    if (pos + 1 >= (int)token.size())
        return false;

    p.id = token[pos++];
    p.deg = atoi(token[pos++].c_str());

    if (p.deg < 0) return false;

    p.a.assign(p.deg + 1, 0);

    for (int i = 0; i <= p.deg; ++i) {
        if (pos < (int)token.size())
            p.a[i] = atof(token[pos++].c_str());
        else
            p.a[i] = 0;
    }

    return true;
}

int main() {
    ifstream fin("poly_pairs.csv");

    if (!fin) {
        cerr << "Loi mo file input!\n";
        return 1;
    }

    ofstream fout("sum.csv");

    if (!fout) {
        cerr << "Loi mo file output!\n";
        return 1;
    }
```

```

    fout << fixed << setprecision(6);

    string line;

    while (getline(fin, line)) {
        if (line.empty()) continue;

        stringstream ss(line);
        vector<string> token;
        string x;

        while (getline(ss, x, ',')) {
            token.push_back(x);
        }

        int pos = 0;
        Poly p, q;

        if (!readOnePoly(token, pos, p))
            continue;

        if (!readOnePoly(token, pos, q))
            continue;

        Poly r = addPoly(p, q);

        fout << r.id << ","
             << r.deg;

        for (int i = 0; i <= r.deg; ++i)
            fout << "," << r.a[i];

        fout << "\n";
    }

    cout << "Done";

    return 0;
}
```

8.4.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (poly_pairs.csv):

```
P1,2,1,2,3,P2,1,4,5
```

Expected Output (sum.csv):

```
P1_P2,2,5.000000,7.000000,3.000000
```

Test Case 2 (Hai đa thức bậc 0)
Input (poly_pairs.csv):

```
A,0,7,B,0,5
```

Expected Output (sum.csv):

```
A_B,0,12.000000
```

Test Case 3 (Thiếu hệ số)
Input (poly_pairs.csv):

```
X,3,1,2,Y,1,4
```

Expected Output (sum.csv):

```
X_Y,3,5.000000,2.000000,0.000000,0.000000
```

8.5 Đạo hàm và đánh giá đa thức [Cơ bản]

8.5.1 Phân tích & Thiết kế (M0)

- **Input:**
  - File `polynomials.csv`: `id,degree,a0..an`
  - File `x_values.csv`: mỗi dòng là một giá trị `x`
- **Xử lý:**
  - **Case biên:**
    - `degree < 0` → invalid
    - Thiếu hệ số → gán bằng 0
    - `degree = 0` → đạo hàm bằng 0
    - `x_values.csv` rỗng → chỉ ghi header
  - **Module 1 (Struct dữ liệu):**

```
struct Poly {
    string id;
    int deg;
    vector<double> a;
};
```

  - **Module 2 (Đọc polynomials.csv):**
    - Chỉ lưu đa thức hợp lệ

- **Module 3 (Đọc x_values.csv):**
  - Đọc các giá trị x
- **Module 4 (Tính P(x) bằng Horner):**

```
res = a[n];
for i = n-1 downto 0:
    res = res*x + a[i];
```

- **Module 5 (Tạo đa thức đạo hàm):**

$$P(x) = a_0 + a_1x + a_2x^2 + \cdots + a_nx^n$$

$$P'(x) = a_1 + 2a_2x + \cdots + na_nx^{n-1}$$

- **Module 6 (Ghi file):**
  - Ghi ra `deriv_report.csv`
  - Định dạng: `id,x,P(x),dP(x)`

- **Output:**

```
deriv_report.csv
id,x,P(x),dP(x)
...
```

8.5.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <string>
#include <cstdlib>
#include <iomanip>

using namespace std;

struct Poly {
    string id;
    int deg;
    vector<double> a;
};

bool readPolysCSV(const string& path,
                  vector<Poly>& arr) {
    ifstream fin(path);
    if (!fin) return false;

    string line;
```

```

    while (getline(fin, line)) {
        if (line.empty()) continue;

        stringstream ss(line);
        vector<string> token;
        string x;

        while (getline(ss, x, ',')) {
            token.push_back(x);
        }

        if (token.size() < 2) continue;

        Poly p;
        p.id = token[0];
        p.deg = atoi(token[1].c_str());

        if (p.deg < 0) continue;

        p.a.assign(p.deg + 1, 0);

        for (int i = 0; i <= p.deg; ++i) {
            int pos = i + 2;

            if (pos < (int)token.size())
                p.a[i] = atof(token[pos].c_str());
            else
                p.a[i] = 0;
        }

        arr.push_back(p);
    }

    return true;
}

bool readXValues(const string& path,
                 vector<double>& xs) {
    ifstream fin(path);
    if (!fin) return false;

    string line;

    while (getline(fin, line)) {
        if (line.empty()) continue;

        xs.push_back(atof(line.c_str()));
    }
```

```

    return true;
}

double evalHorner(vector<double> a,
                  double x) {
    int n = a.size() - 1;

    double res = a[n];

    for (int i = n - 1; i >= 0; --i)
        res = res * x + a[i];

    return res;
}

vector<double> derivative(Poly p) {
    vector<double> d;

    if (p.deg == 0) {
        d.push_back(0);
        return d;
    }

    d.assign(p.deg, 0);

    for (int i = 1; i <= p.deg; ++i)
        d[i - 1] = i * p.a[i];

    return d;
}

int main() {
    vector<Poly> polys;
    vector<double> xs;

    if (!readPolysCSV("polynomials.csv",
                      polys)) {
        cerr << "Loi mo polynomials.csv!\n";
        return 1;
    }

    if (!readXValues("x_values.csv",
                     xs)) {
        cerr << "Loi mo x_values.csv!\n";
        return 1;
    }

    ofstream fout("deriv_report.csv");
```

```

    if (!fout) {
        cerr << "Loi mo output!\n";
        return 1;
    }

    fout << "id,x,P(x),dP(x)\n";
    fout << fixed << setprecision(6);

    for (size_t i = 0; i < polys.size(); ++i) {
        vector<double> d =
            derivative(polys[i]);

        for (size_t j = 0; j < xs.size(); ++j) {
            double val =
                evalHorner(polys[i].a, xs[j]);

            double der =
                evalHorner(d, xs[j]);

            fout << polys[i].id << ","
                 << xs[j] << ","
                 << val << ","
                 << der << "\n";
        }
    }

    cout << "Done";

    return 0;
}
```

8.5.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (polynomials.csv):

```
P1,2,1,2,1
```

Input (x_values.csv):

```
2
```

Expected Output (deriv_report.csv):

```
id,x,P(x),dP(x)
P1,2.000000,9.000000,6.000000
```

Test Case 2 (Bậc 0)
Input (polynomials.csv):

```
A,0,5
```

Input (x_values.csv):

```
3
```

Expected Output (deriv_report.csv):

```
id,x,P(x),dP(x)
A,3.000000,5.000000,0.000000
```

Test Case 3 (x rỗng)
Input (polynomials.csv):

```
B,1,1,1
```

Input (x_values.csv):

```

```

Expected Output (deriv_report.csv):

```
id,x,P(x),dP(x)
```

8.6 Đạo hàm và đánh giá đa thức [Cơ bản]

8.6.1 Phân tích & Thiết kế (M0)

- **Input:**
  - File `polynomials.csv`: `id,degree,a0..an`
  - File `x_values.csv`: mỗi dòng là một giá trị `x`
- **Xử lý:**
  - **Case biên:**
    * `degree < 0` → invalid
    * Thiếu hệ số → gán bằng 0
    * `degree = 0` → đạo hàm bằng 0
    * `x_values.csv` rỗng → chỉ ghi header
  - **Module 1 (Struct dữ liệu):**

```
struct Poly {
    string id;
    int deg;
    vector<double> a;
};
```

  - **Module 2 (Đọc polynomials.csv):**

    * Chỉ lưu đa thức hợp lệ
  - **Module 3 (Đọc x_values.csv):**
    * Đọc các giá trị x
  - **Module 4 (Tính P(x) bằng Horner):**

```
res = a[n];
for i = n-1 downto 0:
    res = res*x + a[i];
```

  - **Module 5 (Tạo đa thức đạo hàm):**

$$P(x) = a_0 + a_1x + a_2x^2 + \cdots + a_nx^n$$

$$P'(x) = a_1 + 2a_2x + \cdots + na_nx^{n-1}$$

  - **Module 6 (Ghi file):**
    * Ghi ra `deriv_report.csv`
    * Định dạng: `id,x,P(x),dP(x)`

- **Output:**

```
deriv_report.csv
id,x,P(x),dP(x)
...
```

8.6.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <string>
#include <cstdlib>
#include <iomanip>

using namespace std;

struct Poly {
    string id;
    int deg;
    vector<double> a;
};

bool readPolysCSV(const string& path,
                  vector<Poly>& arr) {
    ifstream fin(path);
    if (!fin) return false;

```

```
    string line;

    while (getline(fin, line)) {
        if (line.empty()) continue;

        stringstream ss(line);
        vector<string> token;
        string x;

        while (getline(ss, x, ',')) {
            token.push_back(x);
        }

        if (token.size() < 2) continue;

        Poly p;
        p.id = token[0];
        p.deg = atoi(token[1].c_str());

        if (p.deg < 0) continue;

        p.a.assign(p.deg + 1, 0);

        for (int i = 0; i <= p.deg; ++i) {
            int pos = i + 2;

            if (pos < (int)token.size())
                p.a[i] = atof(token[pos].c_str());
            else
                p.a[i] = 0;
        }

        arr.push_back(p);
    }

    return true;
}

bool readXValues(const string& path,
                 vector<double>& xs) {
    ifstream fin(path);
    if (!fin) return false;

    string line;

    while (getline(fin, line)) {
        if (line.empty()) continue;

        xs.push_back(atof(line.c_str()));
```

```
    }

    return true;
}

double evalHorner(vector<double> a,
                  double x) {
    int n = a.size() - 1;

    double res = a[n];

    for (int i = n - 1; i >= 0; --i)
        res = res * x + a[i];

    return res;
}

vector<double> derivative(Poly p) {
    vector<double> d;

    if (p.deg == 0) {
        d.push_back(0);
        return d;
    }

    d.assign(p.deg, 0);

    for (int i = 1; i <= p.deg; ++i)
        d[i - 1] = i * p.a[i];

    return d;
}

int main() {
    vector<Poly> polys;
    vector<double> xs;

    if (!readPolysCSV("polynomials.csv",
                      polys)) {
        cerr << "Loi mo polynomials.csv!\n";
        return 1;
    }

    if (!readXValues("x_values.csv",
                     xs)) {
        cerr << "Loi mo x_values.csv!\n";
        return 1;
    }

```

```
    ofstream fout("deriv_report.csv");

    if (!fout) {
        cerr << "Loi mo output!\n";
        return 1;
    }

    fout << "id,x,P(x),dP(x)\n";
    fout << fixed << setprecision(6);

    for (size_t i = 0; i < polys.size(); ++i) {
        vector<double> d =
            derivative(polys[i]);

        for (size_t j = 0; j < xs.size(); ++j) {
            double val =
                evalHorner(polys[i].a, xs[j]);

            double der =
                evalHorner(d, xs[j]);

            fout << polys[i].id << ","
                 << xs[j] << ","
                 << val << ","
                 << der << "\n";
        }
    }

    cout << "Done";

    return 0;
}
```

8.6.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (polynomials.csv):

```
P1,2,1,2,1
```

Input (x_values.csv):

```
2
```

Expected Output (deriv_report.csv):

```
id,x,P(x),dP(x)
P1,2.000000,9.000000,6.000000
```

Test Case 2 (Bậc 0)

Input (polynomials.csv):

```
A,0,5
```

Input (x_values.csv):

```
3
```

Expected Output (deriv_report.csv):

```
id,x,P(x),dP(x)
A,3.000000,5.000000,0.000000
```

Test Case 3 (x rỗng)
Input (polynomials.csv):

```
B,1,1,1
```

Input (x_values.csv):

```

```

Expected Output (deriv_report.csv):

```
id,x,P(x),dP(x)
```

8.7 Tìm nghiệm nguyên của đa thức [Trung bình]

8.7.1 Phân tích & Thiết kế (M0)

- **Input:** File `polynomials.csv` chứa nhiều dòng:
  - Mỗi dòng có dạng: `id,degree,a0,a1,...,an`
- **Xử lý:**
  - **Case biên:**
    * `degree < 0` → invalid
    * Thiếu hệ số → gán bằng 0
    * Chỉ xét đa thức có hệ số nguyên
    * Tìm nghiệm nguyên trong đoạn `[-50,50]`
    * Không có nghiệm → ghi `id,NONE`
    * Đa thức hằng khác 0 → không có nghiệm
    * Đa thức 0 tuyệt đối → mọi giá trị đều là nghiệm, vẫn duyệt `[-50,50]`
  - **Module 1 (Struct dữ liệu):**

```cpp
struct Poly {
    string id;
    int deg;
    vector<long long> a;
};
```

- **Module 2 (Đọc file CSV):**
  * Chỉ lưu đa thức hợp lệ
  * Chuyển hệ số về kiểu nguyên
- **Module 3 (Tính P(x)):**
  * Dùng Horner

```
res = a[n];
for i = n-1 downto 0:
    res = res*x + a[i];
```

- **Module 4 (Tìm nghiệm):**
  * Duyệt mọi `x` từ -50 đến 50
  * Nếu `P(x)=0` thì ghi nghiệm
- **Module 5 (Ghi file):**
  * File `roots.csv`
  * Mỗi nghiệm một dòng: `id,root`

- **Output:**

```
roots.csv
id,root
...
```

8.7.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <string>
#include <cstdlib>
#include <cmath>

using namespace std;

struct Poly {
    string id;
    int deg;
    vector<long long> a;
};

bool isIntegerValue(double x) {
    return fabs(x - round(x)) < 1e-9;
}

bool readPolysCSV(const string& path,
```

```
                    vector<Poly>& arr) {
    ifstream fin(path);
    if (!fin) return false;

    string line;

    while (getline(fin, line)) {
        if (line.empty()) continue;

        stringstream ss(line);
        vector<string> token;
        string s;

        while (getline(ss, s, ',')) {
            token.push_back(s);
        }

        if (token.size() < 2) continue;

        string id = token[0];
        int deg = atoi(token[1].c_str());

        if (deg < 0) continue;

        vector<long long> coef(deg + 1, 0);
        bool ok = true;

        for (int i = 0; i <= deg; ++i) {
            int pos = i + 2;

            double val = 0;

            if (pos < (int)token.size())
                val = atof(token[pos].c_str());

            if (!isIntegerValue(val)) {
                ok = false;
                break;
            }

            coef[i] = (long long)llround(val);
        }

        if (!ok) continue;

        Poly p;
        p.id = id;
        p.deg = deg;
        p.a = coef;
```

```

        arr.push_back(p);
    }

    return true;
}

long long evalHorner(const vector<long long>& a,
                     long long x) {
    int n = a.size() - 1;

    long long res = a[n];

    for (int i = n - 1; i >= 0; --i)
        res = res * x + a[i];

    return res;
}

int main() {
    vector<Poly> arr;

    if (!readPolysCSV("polynomials.csv", arr)) {
        cerr << "Loi mo file input!\n";
        return 1;
    }

    ofstream fout("roots.csv");

    if (!fout) {
        cerr << "Loi mo file output!\n";
        return 1;
    }

    fout << "id,root\n";

    for (size_t i = 0; i < arr.size(); ++i) {
        bool found = false;

        for (int x = -50; x <= 50; ++x) {
            if (evalHorner(arr[i].a, x) == 0) {
                fout << arr[i].id
                     << "," << x << "\n";
                found = true;
            }
        }

        if (!found) {
            fout << arr[i].id
```

```
                    << ",NONE\n";
            }
        }

        cout << "Done";

        return 0;
}
```

8.7.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (polynomials.csv):

```
P1,2,-1,0,1
```

Expected Output (roots.csv):

```
id,root
P1,-1
P1,1
```

Test Case 2 (Không có nghiệm)
Input (polynomials.csv):

```
A,2,1,0,1
```

Expected Output (roots.csv):

```
id,root
A,NONE
```

Test Case 3 (Đa thức hằng 0)
Input (polynomials.csv):

```
Z,0,0
```

Expected Output (roots.csv):

```
id,root
Z,-50
Z,-49
...
Z,50
```

8.8 Ma trận đơn vị phân số [Trung bình]

8.8.1 Phân tích & Thiết kế (M0)

- **Input:**

- File `config.txt` chứa số nguyên n
- File `matrix.txt` chứa ma trận phân số $n \times n$

- **Xử lý:**
  - **Case biên:**
    - $n = 0 \rightarrow$ ma trận rỗng
    - $n = 1 \rightarrow$ kiểm tra trực tiếp phần tử duy nhất
  - **Module 1 (Đọc config):**
    - Hàm `readConfig(path, n)`
    - Kiểm tra mở file
  - **Module 2 (Đọc ma trận phân số):**
    - Hàm `readMatrix(path, matrix, n)`
    - Mỗi phần tử có dạng `a/b`
  - **Module 3 (Chuẩn hoá phân số):**
    - Rút gọn bằng gcd
    - Đưa mẫu về dương
  - **Module 4 (Tạo ma trận đơn vị):**
    - Đường chéo chính = 1/1
    - Ngoài đường chéo = 0/1
  - **Module 5 (Nhân ma trận):**
    - Tính $A \times I$
    - Tính $I \times A$
  - **Module 6 (So sánh ma trận):**
    - So sánh từng ô sau khi chuẩn hoá phân số

- **Output:**

```
is_identity_product=1
```

hoặc

```
is_identity_product=0
```

8.8.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>

using namespace std;

```

```
struct Fraction {
    long long num, den;
};

long long gcdll(long long a, long long b) {
    if (b == 0) return a >= 0 ? a : -a;
    return gcdll(b, a % b);
}

void normalize(Fraction& f) {
    if (f.den < 0) {
        f.num = -f.num;
        f.den = -f.den;
    }

    long long g = gcdll(f.num, f.den);
    if (g != 0) {
        f.num /= g;
        f.den /= g;
    }
}

Fraction addFraction(const Fraction& a, const Fraction& b) {
    Fraction res;
    res.num = a.num * b.den + b.num * a.den;
    res.den = a.den * b.den;
    normalize(res);
    return res;
}

Fraction multiplyFraction(const Fraction& a, const Fraction& b) {
    Fraction res;
    res.num = a.num * b.num;
    res.den = a.den * b.den;
    normalize(res);
    return res;
}

bool equalFraction(Fraction a, Fraction b) {
    normalize(a);
    normalize(b);
    return a.num == b.num && a.den == b.den;
}

bool readConfig(const string& path, int& n) {
    ifstream fin(path);
    if (!fin) return false;
    fin >> n;
    return true;
```

```
}

bool readMatrix(const string& path,
                vector<vector<Fraction>>& a,
                int n) {
    ifstream fin(path);
    if (!fin) return false;

    a.assign(n, vector<Fraction>(n));

    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            char slash;
            fin >> a[i][j].num >> slash >> a[i][j].den;
            normalize(a[i][j]);
        }
    }

    return true;
}

vector<vector<Fraction>> buildIdentity(int n) {
    vector<vector<Fraction>> I(n, vector<Fraction>(n));

    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            if (i == j)
                I[i][j] = {1, 1};
            else
                I[i][j] = {0, 1};
        }
    }

    return I;
}

vector<vector<Fraction>> multiplyMatrix(
    const vector<vector<Fraction>>& A,
    const vector<vector<Fraction>>& B) {

    int n = A.size();

    vector<vector<Fraction>> C(
        n,
        vector<Fraction>(n, {0, 1})
    );

    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
```

```

            Fraction sum = {0, 1};

            for (int k = 0; k < n; ++k) {
                Fraction t = multiplyFraction(A[i][k], B[k][j]);
                sum = addFraction(sum, t);
            }

            C[i][j] = sum;
        }
    }

    return C;
}

bool equalMatrix(const vector<vector<Fraction>>& A,
                 const vector<vector<Fraction>>& B) {

    int n = A.size();

    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {

            if (!equalFraction(A[i][j], B[i][j]))
                return false;
        }
    }

    return true;
}

int main() {

    int n;

    if (!readConfig("config.txt", n)) {
        cerr << "Loi mo file config!\n";
        return 1;
    }

    vector<vector<Fraction>> A;

    if (!readMatrix("matrix.txt", A, n)) {
        cerr << "Loi mo file matrix!\n";
        return 1;
    }

    vector<vector<Fraction>> I = buildIdentity(n);

```

```
    vector<vector<Fraction>> AI = multiplyMatrix(A, I);
    vector<vector<Fraction>> IA = multiplyMatrix(I, A);

    bool ok =
        equalMatrix(A, AI) &&
        equalMatrix(A, IA);

    cout << "is_identity_product=" << (ok ? 1 : 0);

    return 0;
}
```

8.8.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Ma trận hợp lệ)
Input (config.txt):

```
2
```

Input (matrix.txt):

```
1/2 2/3
3/4 4/5
```

Expected Output:

```
is_identity_product=1
```

Test Case 2 (Case biên – n=1)
Input (config.txt):

```
1
```

Input (matrix.txt):

```
5/7
```

Expected Output:

```
is_identity_product=1
```

Test Case 3 (Case biên – n=0)
Input (config.txt):

```
0
```

Input (matrix.txt):

Expected Output:

```
is_identity_product=1
```

8.9 Đa thức Lagrange nội suy [Nâng cao]

8.9.1 Phân tích & Thiết kế (M0)

- **Input:**

  File `points.csv` chứa:

  - Số nguyên n ($n \leq 8$)
  - n điểm phân biệt (x_i, y_i)

  File `query.csv` chứa:

  - Số nguyên m
  - m giá trị cần tính $P(x_q)$

- **Xử lý:**

  - **Case biên:**
    * $n = 0 \rightarrow$ không nội suy được
    * $n = 1 \rightarrow P(x) = y_0$
  - **Module 1 (Đọc points.csv):**
    * Hàm `readPoints(path, x, y)`
    * Kiểm tra mở file
  - **Module 2 (Đọc query.csv):**
    * Hàm `readQueries(path, queries)`
  - **Module 3 (Tính Lagrange basis):**
    *
    $$L_i(x) = \prod_{j \neq i} \frac{x - x_j}{x_i - x_j}$$
  - **Module 4 (Tính đa thức nội suy):**
    *
    $$P(x) = \sum_{i=0}^{n-1} y_i \cdot L_i(x)$$
    * Không cần tính hệ số đa thức
  - **Module 5 (Ghi kết quả):**
    * Ghi file `result.csv`
    * Format:

```
x_q,P(x_q)
```

- **Output:**

```
x_q,P(x_q)
```

(với `setprecision(6)`)

8.9.2 Mã nguồn C++11 (Tách module)

```
#include <iostream>
#include <vector>
#include <fstream>
#include <iomanip>

using namespace std;

bool readPoints(const string& path,
                vector<double>& x,
                vector<double>& y) {

    ifstream fin(path);

    if (!fin) return false;

    int n;
    fin >> n;

    x.resize(n);
    y.resize(n);

    for (int i = 0; i < n; ++i) {

        char comma;

        fin >> x[i] >> comma >> y[i];
    }

    return true;
}

bool readQueries(const string& path,
                 vector<double>& queries) {

    ifstream fin(path);

    if (!fin) return false;

    int m;
    fin >> m;

    queries.resize(m);

    for (int i = 0; i < m; ++i) {
        fin >> queries[i];
    }

```

```
    return true;
}

double evaluateLagrange(const vector<double>& x,
                        const vector<double>& y,
                        double xq) {

    int n = x.size();

    if (n == 0) return 0.0;

    double result = 0.0;

    for (int i = 0; i < n; ++i) {

        double Li = 1.0;

        for (int j = 0; j < n; ++j) {

            if (i == j) continue;

            Li *= (xq - x[j]) / (x[i] - x[j]);
        }

        result += y[i] * Li;
    }

    return result;
}

bool writeResults(const string& path,
                  const vector<double>& queries,
                  const vector<double>& values) {

    ofstream fout(path);

    if (!fout) return false;

    fout << fixed << setprecision(6);

    for (size_t i = 0; i < queries.size(); ++i) {

        fout << queries[i]
             << ","
             << values[i]
             << "\n";
    }

    return true;
```

```
}

int main() {

    vector<double> x, y;

    if (!readPoints("points.csv", x, y)) {

        cerr << "Loi mo file points!\n";
        return 1;
    }

    vector<double> queries;

    if (!readQueries("query.csv", queries)) {

        cerr << "Loi mo file query!\n";
        return 1;
    }

    vector<double> values(queries.size());

    for (size_t i = 0; i < queries.size(); ++i) {

        values[i] =
            evaluateLagrange(x, y, queries[i]);
    }

    if (!writeResults("result.csv",
                      queries,
                      values)) {

        cerr << "Loi mo file output!\n";
        return 1;
    }

    return 0;
}
```

8.9.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (points.csv):

```
3
0,1
1,3
2,2
```

Input (query.csv):

```
3
0
1.5
3
```

Expected Output (result.csv):

```
0.000000,1.000000
1.500000,2.875000
3.000000,-5.000000
```

Test Case 2 (Case biên – n=1)
Input (points.csv):

```
1
2,5
```

Input (query.csv):

```
2
0
10
```

Expected Output (result.csv):

```
0.000000,5.000000
10.000000,5.000000
```

Test Case 3 (Case biên – n=0)
Input (points.csv):

```
0
```

Input (query.csv):

```
2
1
2
```

Expected Output (result.csv):

```
1.000000,0.000000
2.000000,0.000000
```

8.10 Phân tích bình phương tổng [Nâng cao]

8.10.1 Phân tích & Thiết kế (M0)

- **Input:**

  File `fractions.csv` chứa:

  - Số nguyên n
  - n phân số dạng `num/den`

- **Xử lý:**
  - **Case biên:**
    * $n = 0 \rightarrow$ tổng và variance bằng 0/1
    * Phân số có mẫu âm $\rightarrow$ chuyển mẫu dương
    * Phân số chưa tối giản $\rightarrow$ rút gọn bằng gcd
  - **Module 1 (Đọc file):**
    * Hàm `readFractions(path, vector<Fraction>&)`
    * Kiểm tra mở file
  - **Module 2 (Chuẩn hoá phân số):**
    * Rút gọn bằng gcd
    * Đưa mẫu về dương
  - **Module 3 (Các phép toán phân số):**
    * Cộng phân số
    * Nhân phân số
    * Bình phương phân số
  - **Module 4 (Tính tổng bình phương):**
    * $$S = \sum_{i=1}^{n} \left(\frac{num_i}{den_i} \right)^2$$
  - **Module 5 (Tính variance):**
    * $$E[X] = \frac{1}{n} \sum x_i$$
    * $$E[X^2] = \frac{1}{n} \sum x_i^2$$
    * $$variance = E[X^2] - (E[X])^2$$
  - **Module 6 (Ghi file output):**
    * Ghi file `stats.csv`

- **Output:**

```
sum_squares=<num>/<den> / variance=<num>/<den>
```

8.10.2 Mã nguồn C++11 (Tách module)

```
#include <iostream>
#include <vector>
#include <fstream>

using namespace std;

struct Fraction {
    long long num, den;
};

long long gcdll(long long a, long long b) {

    if (b == 0)
        return a >= 0 ? a : -a;

    return gcdll(b, a % b);
}

void normalize(Fraction& f) {

    if (f.den < 0) {
        f.num = -f.num;
        f.den = -f.den;
    }

    long long g = gcdll(f.num, f.den);

    if (g != 0) {
        f.num /= g;
        f.den /= g;
    }
}

Fraction addFraction(const Fraction& a,
                     const Fraction& b) {

    Fraction res;

    res.num =
        a.num * b.den +
        b.num * a.den;

    res.den =
        a.den * b.den;

    normalize(res);

```

```
    return res;
}

Fraction subtractFraction(const Fraction& a,
                          const Fraction& b) {

    Fraction res;

    res.num =
        a.num * b.den -
        b.num * a.den;

    res.den =
        a.den * b.den;

    normalize(res);

    return res;
}

Fraction multiplyFraction(const Fraction& a,
                          const Fraction& b) {

    Fraction res;

    res.num = a.num * b.num;
    res.den = a.den * b.den;

    normalize(res);

    return res;
}

Fraction divideFraction(const Fraction& a,
                        const Fraction& b) {

    Fraction res;

    res.num = a.num * b.den;
    res.den = a.den * b.num;

    normalize(res);

    return res;
}

Fraction squareFraction(const Fraction& a) {

    Fraction res;
```

```

    res.num = a.num * a.num;
    res.den = a.den * a.den;

    normalize(res);

    return res;
}

bool readFractions(const string& path,
                   vector<Fraction>& a) {

    ifstream fin(path);

    if (!fin) return false;

    int n;

    fin >> n;

    a.resize(n);

    for (int i = 0; i < n; ++i) {

        char slash;

        fin >> a[i].num
            >> slash
            >> a[i].den;

        normalize(a[i]);
    }

    return true;
}

bool writeStats(const string& path,
                const Fraction& sumSquares,
                const Fraction& variance) {

    ofstream fout(path);

    if (!fout) return false;

    fout << "sum_squares="
         << sumSquares.num
         << "/"
         << sumSquares.den
         << " / variance="
```

```
        << variance.num
        << "/"
        << variance.den;

    return true;
}

int main() {

    vector<Fraction> a;

    if (!readFractions("fractions.csv", a)) {

        cerr << "Loi mo file input!\n";
        return 1;
    }

    int n = a.size();

    Fraction sum = {0, 1};
    Fraction sumSquares = {0, 1};

    for (int i = 0; i < n; ++i) {

        sum = addFraction(sum, a[i]);

        Fraction sq = squareFraction(a[i]);

        sumSquares =
            addFraction(sumSquares, sq);
    }

    Fraction variance = {0, 1};

    if (n > 0) {

        Fraction nFrac = {(long long)n, 1};

        Fraction mean =
            divideFraction(sum, nFrac);

        Fraction ex2 =
            divideFraction(sumSquares, nFrac);

        Fraction mean2 =
            squareFraction(mean);

        variance =
            subtractFraction(ex2, mean2);
```

```
    }

    if (!writeStats("stats.csv",
                    sumSquares,
                    variance)) {

        cerr << "Loi mo file output!\n";
        return 1;
    }

    return 0;
}
```

8.10.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (fractions.csv):

```
3
1/2
1/3
1/6
```

Expected Output (stats.csv):

```
sum_squares=7/18 / variance=1/54
```

Test Case 2 (Case biên – n=1)
Input (fractions.csv):

```
1
5/7
```

Expected Output (stats.csv):

```
sum_squares=25/49 / variance=0/1
```

Test Case 3 (Case biên – n=0)
Input (fractions.csv):

```
0
```

Expected Output (stats.csv):

```
sum_squares=0/1 / variance=0/1
```

9 Tuần 9: Chuỗi nâng cao: Normalize, Tokenize, Frequency

9.1 Normalize chuỗi ASCII [Cơ bản]

9.1.1 Phân tích & Thiết kế (M0)

- **Input:**

  File `text.txt` chứa nhiều dòng văn bản ASCII.

- **Xử lý:**
  - **Case biên:**
    * Dòng rỗng → ghi dòng rỗng
    * Dòng chỉ chứa punctuation → ghi dòng rỗng
    * Nhiều dấu cách liên tiếp → gom thành 1 space
  - **Module 1 (Đọc file):**
    * Hàm `readLines(path, vector<string>&)`
    * Kiểm tra mở file
  - **Module 2 (Normalize chuỗi):**
    * Hàm `normalizeLine(const string& s)`
    * Duyệt từng ký tự:
      · `isalnum(c)` → giữ lại và chuyển lowercase
      · Ký tự khác → đổi thành space
      · Không ghi nhiều space liên tiếp
  - **Module 3 (Xoá space đầu/cuối):**
    * Nếu cuối chuỗi là space → xoá
  - **Module 4 (Ghi file output):**
    * Ghi từng dòng vào `normalized.txt`

- **Output:**

```
hello world 123
cpp basic
```

9.1.2 Mã nguồn C++11 (Tách module)

```
#include <iostream>
#include <vector>
#include <fstream>
#include <string>
#include <cctype>

using namespace std;

bool readLines(const string& path,
               vector<string>& lines) {

    ifstream fin(path);

    if (!fin) return false;

    string s;

    while (getline(fin, s)) {
        lines.push_back(s);
    }

    return true;
}

string normalizeLine(const string& s) {

    string result;

    bool lastSpace = true;

    for (char c : s) {

        if (isalnum((unsigned char)c)) {

            result +=
                (char)tolower((unsigned char)c);

            lastSpace = false;
        }
        else {

            if (!lastSpace) {

                result += ' ';
                lastSpace = true;
            }
        }
```

```
    }

    if (!result.empty() &&
        result.back() == ' ') {

        result.pop_back();
    }

    return result;
}

bool writeLines(const string& path,
                const vector<string>& lines) {

    ofstream fout(path);

    if (!fout) return false;

    for (const string& s : lines) {

        fout << s << "\n";
    }

    return true;
}

int main() {

    vector<string> lines;

    if (!readLines("text.txt", lines)) {

        cerr << "Loi mo file input!\n";
        return 1;
    }

    vector<string> normalized;

    for (const string& s : lines) {

        normalized.push_back(
            normalizeLine(s)
        );
    }

    if (!writeLines("normalized.txt",
                    normalized)) {

        cerr << "Loi mo file output!\n";
```

```
        return 1;
    }

    return 0;
}
```

9.1.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (text.txt):

```
Hello, WORLD!! 123
C++   Basic???
```

Expected Output (normalized.txt):

```
hello world 123
c basic
```

Test Case 2 (Case biên – dòng rỗng)
Input (text.txt):

```

Hello!!!
```

Expected Output (normalized.txt):

```

hello
```

Test Case 3 (Case biên – chỉ punctuation)
Input (text.txt):

```
!!!@@@###
```

Expected Output (normalized.txt):

```
```

9.2 Tokenize và đếm từ [Cơ bản]

9.2.1 Phân tích & Thiết kế (M0)

- **Input:**

  File `text.txt` chứa nhiều dòng văn bản ASCII.

- **Xử lý:**

  – **Case biên:**

    - File rỗng → total=0, unique=0
    - Dòng rỗng → không sinh token
    - Dòng chỉ punctuation → không sinh token
  - **Module 1 (Đọc file):**
    - Hàm `readLines(path, vector<string>&)`
    - Kiểm tra mở file
  - **Module 2 (Normalize chuỗi):**
    - Hàm `normalizeLine(const string& s)`
    - Chuyển lowercase
    - Ký tự không phải chữ/số → space
    - Gom nhiều space thành 1
  - **Module 3 (Tokenize):**
    - Hàm `tokenize(const string& normalized)`
    - Tách token theo khoảng trắng
  - **Module 4 (Đếm tần suất):**
    - Dùng:

```
unordered_map<string,int> freq
```

    - Đếm:
      - Tổng số token
      - Số token duy nhất

- **Output:**

```
total_tokens=<n> / unique_tokens=<m>
```

9.2.2 Mã nguồn C++11 (Tách module)

```
#include <iostream>
#include <vector>
#include <fstream>
#include <string>
#include <unordered_map>
#include <sstream>
#include <cctype>

using namespace std;

bool readLines(const string& path,
               vector<string>& lines) {

    ifstream fin(path);
```

```

    if (!fin) return false;

    string s;

    while (getline(fin, s)) {
        lines.push_back(s);
    }

    return true;
}

string normalizeLine(const string& s) {

    string result;

    bool lastSpace = true;

    for (char c : s) {

        if (isalnum((unsigned char)c)) {

            result +=
                (char)tolower((unsigned char)c);

            lastSpace = false;
        }
        else {

            if (!lastSpace) {

                result += ' ';
                lastSpace = true;
            }
        }
    }

    if (!result.empty() &&
        result.back() == ' ') {

        result.pop_back();
    }

    return result;
}

vector<string> tokenize(
    const string& normalized) {

```

```
    vector<string> tokens;

    stringstream ss(normalized);

    string token;

    while (ss >> token) {

        tokens.push_back(token);
    }

    return tokens;
}

int main() {

    vector<string> lines;

    if (!readLines("text.txt", lines)) {

        cerr << "Loi mo file input!\n";
        return 1;
    }

    unordered_map<string, int> freq;

    int totalTokens = 0;

    for (const string& s : lines) {

        string normalized =
            normalizeLine(s);

        vector<string> tokens =
            tokenize(normalized);

        for (const string& token : tokens) {

            freq[token]++;
            totalTokens++;
        }
    }

    cout << "total_tokens="
         << totalTokens
         << " / unique_tokens="
         << freq.size();

    return 0;
```

```
}
```

9.2.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (text.txt):

```
Hello world!!!
Hello C++ world
```

Expected Output:

```
total_tokens=5 / unique_tokens=3
```

Test Case 2 (Case biên – file rỗng)
Input (text.txt):

```

```

Expected Output:

```
total_tokens=0 / unique_tokens=0
```

Test Case 3 (Case biên – chỉ punctuation)
Input (text.txt):

```
!!! ??? ###
```

Expected Output:

```
total_tokens=0 / unique_tokens=0
```

9.3 Top-K từ xuất hiện nhiều nhất [Cơ bản]

9.3.1 Phân tích & Thiết kế (M0)

- **Input:**
  - File `text.txt` chứa nhiều dòng văn bản ASCII
  - File `config.txt` chứa số nguyên K
- **Xử lý:**
  - **Case biên:**
    - File rỗng → không có token
    - $K >$ số token duy nhất → in tất cả
    - Dòng chỉ punctuation → bỏ qua
  - **Module 1 (Đọc file text):**

    * Hàm `readLines(path, vector<string>&)`
    * Kiểm tra mở file
  - **Module 2 (Đọc config):**
    * Hàm `readConfig(path, k)`
  - **Module 3 (Normalize chuỗi):**
    * Hàm `normalizeLine(const string& s)`
    * Chuyển lowercase
    * Ký tự không phải chữ/số → space
    * Gom nhiều space thành 1
  - **Module 4 (Tokenize):**
    * Hàm `tokenize(const string& normalized)`
    * Tách token theo khoảng trắng
  - **Module 5 (Đếm tần suất):**
    * Dùng:

```
unordered_map<string,int> freq
```

  - **Module 6 (Top-K):**
    * Chuyển map sang:

```
vector<pair<string,int>>
```

    * Sort:
      · Giảm dần theo count
      · Nếu bằng count → tăng dần theo từ
    * Hàm:

```
vector<pair<string,int>> topK(...)
```

  - **Module 7 (Ghi output):**
    * Ghi file `freq.csv`
    * Format:

```
word,count
```

- **Output:**

```
word,count
```

9.3.2 Mã nguồn C++11 (Tách module)

```
#include <iostream>
#include <vector>
#include <fstream>
#include <string>
#include <unordered_map>
#include <sstream>
#include <algorithm>
#include <cctype>

using namespace std;

bool readLines(const string& path,
               vector<string>& lines) {

    ifstream fin(path);

    if (!fin) return false;

    string s;

    while (getline(fin, s)) {

        lines.push_back(s);
    }

    return true;
}

bool readConfig(const string& path,
                int& k) {

    ifstream fin(path);

    if (!fin) return false;

    fin >> k;

    return true;
}

string normalizeLine(const string& s) {

    string result;

    bool lastSpace = true;

    for (char c : s) {
```

```

        if (isalnum((unsigned char)c)) {

            result +=
                (char)tolower((unsigned char)c);

            lastSpace = false;
        }
        else {

            if (!lastSpace) {

                result += ' ';
                lastSpace = true;
            }
        }
    }

    if (!result.empty() &&
        result.back() == ' ') {

        result.pop_back();
    }

    return result;
}

vector<string> tokenize(
    const string& normalized) {

    vector<string> tokens;

    stringstream ss(normalized);

    string token;

    while (ss >> token) {

        tokens.push_back(token);
    }

    return tokens;
}

vector<pair<string,int>> topK(
    unordered_map<string,int> freq,
    int k) {

    vector<pair<string,int>> v(
```

```
        freq.begin(),
        freq.end()
    );

    sort(v.begin(),
         v.end(),
         [](const pair<string,int>& a,
            const pair<string,int>& b) {

        if (a.second != b.second)
            return a.second > b.second;

        return a.first < b.first;
    });

    if ((int)v.size() > k) {
        v.resize(k);
    }

    return v;
}

bool writeFreq(
    const string& path,
    const vector<pair<string,int>>& v) {

    ofstream fout(path);

    if (!fout) return false;

    for (const auto& p : v) {

        fout << p.first
             << ","
             << p.second
             << "\n";
    }

    return true;
}

int main() {

    vector<string> lines;

    if (!readLines("text.txt", lines)) {

        cerr << "Loi mo file input!\n";
        return 1;
```

```
    }

    int k;

    if (!readConfig("config.txt", k)) {

        cerr << "Loi mo file config!\n";
        return 1;
    }

    unordered_map<string,int> freq;

    for (const string& s : lines) {

        string normalized =
            normalizeLine(s);

        vector<string> tokens =
            tokenize(normalized);

        for (const string& token : tokens) {

            freq[token]++;
        }
    }

    vector<pair<string,int>> result =
        topK(freq, k);

    if (!writeFreq("freq.csv", result)) {

        cerr << "Loi mo file output!\n";
        return 1;
    }

    return 0;
}
```

9.3.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (text.txt):

```
apple banana apple
banana orange apple
```

Input (config.txt):

```
2
```

Expected Output (freq.csv):

```
apple,3
banana,2
```

Test Case 2 (Case biên – K lớn hơn unique)
Input (text.txt):

```
cat dog
```

Input (config.txt):

```
10
```

Expected Output (freq.csv):

```
cat,1
dog,1
```

Test Case 3 (Case biên – file rỗng)
Input (text.txt):

```

```

Input (config.txt):

```
5
```

Expected Output (freq.csv):

```

```

9.4 Lọc stop-words [Trung bình]

9.4.1 Phân tích & Thiết kế (M0)

- **Input:**
  - File `text.txt` chứa nhiều dòng văn bản ASCII
  - File `stopwords.txt` chứa stop-word
  - Mỗi dòng trong `stopwords.txt` chứa 1 từ
- **Xử lý:**
  - **Case biên:**
    * File text rỗng → tất cả thống kê bằng 0
    * File stopwords rỗng → giữ toàn bộ token
    * Dòng chỉ punctuation → bỏ qua
  - **Module 1 (Đọc text):**
    * Hàm `readLines(path, vector<string>&)`

    * Kiểm tra mở file
  - **Module 2 (Đọc stopwords):**
    * Hàm:

```
readStopwords(path, unordered_set<string>&)
```

    * Lưu stop-word vào:

```
unordered_set<string> stops
```

  - **Module 3 (Normalize chuỗi):**
    * Hàm `normalizeLine(const string& s)`
    * Chuyển lowercase
    * Ký tự không phải chữ/số → space
    * Gom nhiều space thành 1
  - **Module 4 (Tokenize):**
    * Hàm:

```
vector<string> tokenize(...)
```

  - **Module 5 (Lọc stop-word và đếm):**
    * Nếu:

```
stops.count(token) == 0
```

    → đưa vào thống kê
    * Dùng:

```
unordered_map<string,int> freq
```

  - **Module 6 (Ghi output):**
    * File `freq_filtered.csv`
    * Format:

```
word,count
```

    * In summary:

```
total_before=<n>
total_after=<m>
stopwords_removed=<k>
```

- **Output:**

```
total_before=<n>
total_after=<m>
stopwords_removed=<k>
```

9.4.2 Mã nguồn C++11 (Tách module)

```
#include <iostream>
#include <vector>
#include <fstream>
#include <string>
#include <unordered_map>
#include <unordered_set>
#include <sstream>
#include <cctype>

using namespace std;

bool readLines(const string& path,
               vector<string>& lines) {

    ifstream fin(path);

    if (!fin) return false;

    string s;

    while (getline(fin, s)) {

        lines.push_back(s);
    }

    return true;
}

bool readStopwords(
    const string& path,
    unordered_set<string>& stops) {

    ifstream fin(path);

    if (!fin) return false;

    string word;

    while (getline(fin, word)) {

        if (!word.empty()) {

            stops.insert(word);
        }
    }

    return true;
```

```
}

string normalizeLine(const string& s) {

    string result;

    bool lastSpace = true;

    for (char c : s) {

        if (isalnum((unsigned char)c)) {

            result +=
                (char)tolower((unsigned char)c);

            lastSpace = false;
        }
        else {

            if (!lastSpace) {

                result += ' ';
                lastSpace = true;
            }
        }
    }

    if (!result.empty() &&
        result.back() == ' ') {

        result.pop_back();
    }

    return result;
}

vector<string> tokenize(
    const string& normalized) {

    vector<string> tokens;

    stringstream ss(normalized);

    string token;

    while (ss >> token) {

        tokens.push_back(token);
    }
```

```
    return tokens;
}

bool writeFreq(
    const string& path,
    const unordered_map<string,int>& freq) {

    ofstream fout(path);

    if (!fout) return false;

    for (const auto& p : freq) {

        fout << p.first
             << ","
             << p.second
             << "\n";
    }

    return true;
}

int main() {

    vector<string> lines;

    if (!readLines("text.txt", lines)) {

        cerr << "Loi mo file text!\n";
        return 1;
    }

    unordered_set<string> stops;

    if (!readStopwords("stopwords.txt",
                       stops)) {

        cerr << "Loi mo file stopwords!\n";
        return 1;
    }

    unordered_map<string,int> freq;

    int totalBefore = 0;
    int totalAfter = 0;

    for (const string& s : lines) {

```

```
        string normalized =
            normalizeLine(s);

        vector<string> tokens =
            tokenize(normalized);

        for (const string& token : tokens) {

            totalBefore++;

            if (stops.count(token) == 0) {

                freq[token]++;
                totalAfter++;
            }
        }
    }

    int removed =
        totalBefore - totalAfter;

    if (!writeFreq("freq_filtered.csv",
                   freq)) {

        cerr << "Loi mo file output!\n";
        return 1;
    }

    cout << "total_before="
         << totalBefore
         << "\n";

    cout << "total_after="
         << totalAfter
         << "\n";

    cout << "stopwords_removed="
         << removed;

    return 0;
}
```

9.4.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (text.txt):

```
this is a test
this test is simple
```

Input (stopwords.txt):

```
this
is
```

Expected Output (freq_filtered.csv):

```
a,1
test,2
simple,1
```

Expected Summary:

```
total_before=8
total_after=4
stopwords_removed=4
```

Test Case 2 (Case biên – stopwords rỗng)
Input (text.txt):

```
apple banana apple
```

Input (stopwords.txt):

```

```

Expected Output (freq_filtered.csv):

```
apple,2
banana,1
```

Expected Summary:

```
total_before=3
total_after=3
stopwords_removed=0
```

Test Case 3 (Case biên – file rỗng)
Input (text.txt):

```

```

Input (stopwords.txt):

```
a
the
is
```

Expected Output (freq_filtered.csv):

```

```

Expected Summary:

```
total_before=0
total_after=0
stopwords_removed=0
```

9.5 So sánh 2 file văn bản [Trung bình]

9.5.1 Phân tích & Thiết kế (M0)

- **Input:**
  - File `file1.txt`
  - File `file2.txt`
- **Xử lý:**
  - **Case biên:**
    * Một file rỗng → toàn bộ từ nằm ở file còn lại
    * Hai file giống hệt nhau → only_in_1=0, only_in_2=0
    * Dòng chỉ punctuation → bỏ qua
  - **Module 1 (Đọc file):**
    * Hàm:

```
readLines(path, vector<string>&)
```

    * Kiểm tra mở file
  - **Module 2 (Normalize chuỗi):**
    * Hàm:

```
normalizeLine(const string& s)
```

    * Chuyển lowercase
    * Ký tự không phải chữ/số → space
    * Gom nhiều space thành 1
  - **Module 3 (Tokenize):**
    * Hàm:

```
vector<string> tokenize(...)
```

  - **Module 4 (Đếm tần suất):**
    * Dùng:

```
unordered_map<string,int>
```

    cho mỗi file
  - **Module 5 (Merge và so sánh):**
    * Hàm:

```
mergeFreq(map1, map2, output)
```

    * Duyệt tất cả key của 2 map

    * Xác định:
        · Chỉ có ở file1
        · Chỉ có ở file2
        · Có ở cả 2

  – **Module 6 (Ghi output):**
    * Ghi file:

```
comparison.csv
```

    * Format:

```
word,count_file1,count_file2
```

    * In summary:

```
only_in_1=<n>
only_in_2=<m>
in_both=<k>
```

- **Output:**

```
only_in_1=<n>
only_in_2=<m>
in_both=<k>
```

9.5.2 Mã nguồn C++11 (Tách module)

```
#include <iostream>
#include <vector>
#include <fstream>
#include <string>
#include <unordered_map>
#include <unordered_set>
#include <sstream>
#include <cctype>

using namespace std;

struct Row {
    string word;
    int count1;
    int count2;
};

bool readLines(const string& path,
               vector<string>& lines) {

    ifstream fin(path);
```

```

    if (!fin) return false;

    string s;

    while (getline(fin, s)) {

        lines.push_back(s);
    }

    return true;
}

string normalizeLine(const string& s) {

    string result;

    bool lastSpace = true;

    for (char c : s) {

        if (isalnum((unsigned char)c)) {

            result +=
                (char)tolower((unsigned char)c);

            lastSpace = false;
        }
        else {

            if (!lastSpace) {

                result += ' ';
                lastSpace = true;
            }
        }
    }

    if (!result.empty() &&
        result.back() == ' ') {

        result.pop_back();
    }

    return result;
}

vector<string> tokenize(
    const string& normalized) {
```

```

    vector<string> tokens;

    stringstream ss(normalized);

    string token;

    while (ss >> token) {

        tokens.push_back(token);
    }

    return tokens;
}

unordered_map<string,int>
buildFreq(const vector<string>& lines) {

    unordered_map<string,int> freq;

    for (const string& s : lines) {

        string normalized =
            normalizeLine(s);

        vector<string> tokens =
            tokenize(normalized);

        for (const string& token : tokens) {

            freq[token]++;
        }
    }

    return freq;
}

void mergeFreq(
    const unordered_map<string,int>& map1,
    const unordered_map<string,int>& map2,
    vector<Row>& output,
    int& only1,
    int& only2,
    int& both) {

    unordered_set<string> allWords;

    for (const auto& p : map1) {

```

```
        allWords.insert(p.first);
    }

    for (const auto& p : map2) {

        allWords.insert(p.first);
    }

    only1 = 0;
    only2 = 0;
    both = 0;

    for (const string& word : allWords) {

        int c1 = 0;
        int c2 = 0;

        auto it1 = map1.find(word);

        if (it1 != map1.end()) {
            c1 = it1->second;
        }

        auto it2 = map2.find(word);

        if (it2 != map2.end()) {
            c2 = it2->second;
        }

        if (c1 > 0 && c2 == 0)
            only1++;

        else if (c1 == 0 && c2 > 0)
            only2++;

        else
            both++;

        output.push_back({word, c1, c2});
    }
}

bool writeComparison(
    const string& path,
    const vector<Row>& rows) {

    ofstream fout(path);

    if (!fout) return false;
```

```

    for (const Row& r : rows) {

        fout << r.word
             << ","
             << r.count1
             << ","
             << r.count2
             << "\n";
    }

    return true;
}

int main() {

    vector<string> lines1;
    vector<string> lines2;

    if (!readLines("file1.txt", lines1)) {

        cerr << "Loi mo file1!\n";
        return 1;
    }

    if (!readLines("file2.txt", lines2)) {

        cerr << "Loi mo file2!\n";
        return 1;
    }

    unordered_map<string,int> freq1 =
        buildFreq(lines1);

    unordered_map<string,int> freq2 =
        buildFreq(lines2);

    vector<Row> rows;

    int only1, only2, both;

    mergeFreq(freq1,
              freq2,
              rows,
              only1,
              only2,
              both);

    if (!writeComparison(
```

```
            "comparison.csv",
            rows)) {

        cerr << "Loi mo file output!\n";
        return 1;
    }

    cout << "only_in_1="
         << only1
         << "\n";

    cout << "only_in_2="
         << only2
         << "\n";

    cout << "in_both="
         << both;

    return 0;
}
```

9.5.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (file1.txt):

```
apple banana apple
orange
```

Input (file2.txt):

```
banana grape
apple grape
```

Expected Output (comparison.csv):

```
apple,2,1
banana,1,1
orange,1,0
grape,0,2
```

Expected Summary:

```
only_in_1=1
only_in_2=1
in_both=2
```

Test Case 2 (Case biên – hai file giống nhau)
Input (file1.txt):

```
hello world
```

Input (file2.txt):

```
hello world
```

Expected Summary:

```
only_in_1=0
only_in_2=0
in_both=2
```

Test Case 3 (Case biên – file rỗng)
Input (file1.txt):

```

```

Input (file2.txt):

```
test data
```

Expected Summary:

```
only_in_1=0
only_in_2=2
in_both=0
```

9.6 Đếm tần suất bigram [Trung bình]

9.6.1 Phân tích & Thiết kế (M0)

- **Input:**

  File `text.txt` chứa nhiều dòng văn bản ASCII.

- **Xử lý:**

  - **Case biên:**
    - File có ít hơn 2 token → ghi `NO_BIGRAM`
    - Dòng rỗng → bỏ qua
    - Dòng chỉ punctuation → không sinh token
  - **Module 1 (Đọc file):**
    - Hàm:

      ```
      readLines(path, vector<string>&)
      ```

    - Kiểm tra mở file
  - **Module 2 (Normalize chuỗi):**
    - Hàm:

      ```
      normalizeLine(const string& s)
      ```

    * Chuyển lowercase
    * Ký tự không phải chữ/số → space
    * Gom nhiều space thành 1
  - **Module 3 (Tokenize):**
    * Hàm:

```
vector<string> tokenize(...)
```

  - **Module 4 (Sinh bigram):**
    * Với:

```
tokens[i] và tokens[i+1]
```

    sinh:

```
tokens[i] + " " + tokens[i+1]
```

  - **Module 5 (Đếm tần suất):**
    * Dùng:

```
unordered_map<string,int> freq
```

  - **Module 6 (Sort top-20):**
    * Sort:
      · Giảm dần theo count
      · Nếu bằng nhau → tăng dần lexicographic
    * Chỉ in 20 bigram đầu
  - **Module 7 (Ghi output):**
    * File:

```
bigram.csv
```

    * Format:

```
bigram,count
```

- **Output:**

```
bigram,count
```

hoặc

```
NO_BIGRAM
```

9.6.2 Mã nguồn C++11 (Tách module)

```
#include <iostream>
#include <vector>
#include <fstream>
#include <string>
#include <unordered_map>
#include <sstream>
#include <algorithm>
#include <cctype>

using namespace std;

bool readLines(const string& path,
               vector<string>& lines) {

    ifstream fin(path);

    if (!fin) return false;

    string s;

    while (getline(fin, s)) {

        lines.push_back(s);
    }

    return true;
}

string normalizeLine(const string& s) {

    string result;

    bool lastSpace = true;

    for (char c : s) {

        if (isalnum((unsigned char)c)) {

            result +=
                (char)tolower((unsigned char)c);

            lastSpace = false;
        }
        else {

            if (!lastSpace) {

```

```
                result += ' ';
                lastSpace = true;
            }
        }
    }

    if (!result.empty() &&
        result.back() == ' ') {

        result.pop_back();
    }

    return result;
}

vector<string> tokenize(
    const string& normalized) {

    vector<string> tokens;

    stringstream ss(normalized);

    string token;

    while (ss >> token) {

        tokens.push_back(token);
    }

    return tokens;
}

bool writeBigram(
    const string& path,
    const vector<pair<string,int>>& v,
    bool hasBigram) {

    ofstream fout(path);

    if (!fout) return false;

    if (!hasBigram) {

        fout << "NO_BIGRAM";
        return true;
    }

    for (const auto& p : v) {

```

```
        fout << p.first
             << ","
             << p.second
             << "\n";
    }

    return true;
}

int main() {

    vector<string> lines;

    if (!readLines("text.txt", lines)) {

        cerr << "Loi mo file input!\n";
        return 1;
    }

    vector<string> allTokens;

    for (const string& s : lines) {

        string normalized =
            normalizeLine(s);

        vector<string> tokens =
            tokenize(normalized);

        for (const string& token : tokens) {

            allTokens.push_back(token);
        }
    }

    unordered_map<string,int> freq;

    for (size_t i = 0;
         i + 1 < allTokens.size();
         ++i) {

        string bigram =
            allTokens[i]
            + " "
            + allTokens[i + 1];

        freq[bigram]++;
    }

```

```
    vector<pair<string,int>> v(
        freq.begin(),
        freq.end()
    );

    sort(v.begin(),
         v.end(),
         [](const pair<string,int>& a,
            const pair<string,int>& b) {

        if (a.second != b.second)
            return a.second > b.second;

        return a.first < b.first;
    });

    if (v.size() > 20) {

        v.resize(20);
    }

    bool hasBigram =
        allTokens.size() >= 2;

    if (!writeBigram("bigram.csv",
                      v,
                      hasBigram)) {

        cerr << "Loi mo file output!\n";
        return 1;
    }

    return 0;
}
```

9.6.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (text.txt):

```
hello world hello world
hello chatgpt
```

Expected Output (bigram.csv):

```
hello world,2
world hello,2
hello chatgpt,1
```

Test Case 2 (Case biên – chỉ có 1 token)
Input (text.txt):

```
hello
```

Expected Output (bigram.csv):

```
NO_BIGRAM
```

Test Case 3 (Case biên – file rỗng)
Input (text.txt):

```

```

Expected Output (bigram.csv):

```
NO_BIGRAM
```

9.7 Phát hiện từ lặp liên tiếp [Cơ bản]

9.7.1 Phân tích & Thiết kế (M0)

- **Input:**

  File `text.txt` chứa nhiều dòng văn bản ASCII.

- **Xử lý:**

  - **Case biên:**
    * Dòng rỗng → bỏ qua
    * Dòng chỉ có 1 token → không có lặp
    * Không có từ lặp liên tiếp → file output rỗng
  - **Module 1 (Đọc file):**
    * Hàm:

      ```
      readLines(path, vector<string>&)
      ```

    * Kiểm tra mở file
  - **Module 2 (Normalize chuỗi):**
    * Hàm:

      ```
      normalizeLine(const string& s)
      ```

    * Chuyển lowercase
    * Ký tự không phải chữ/số → space
    * Gom nhiều space thành 1
  - **Module 3 (Tokenize):**
    * Hàm:

```
vector<string> tokenize(...)
```

- **Module 4 (Phát hiện lặp liên tiếp):**
  * Duyệt token theo từng dòng
  * Nếu:

```
tokens[i] == tokens[i-1]
```

    → tăng count
  * Nếu count ≥ 2 → ghi kết quả
- **Module 5 (Ghi output):**
  * File:

```
repeats.txt
```

  * Format:

```
line=<i> word=<w> count=<k>
```

  * `i` là số dòng bắt đầu từ 1

- **Output:**

```
line=<i> word=<w> count=<k>
```

9.7.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>
#include <string>
#include <sstream>
#include <cctype>

using namespace std;

struct RepeatInfo {
    int line;
    string word;
    int count;
};

bool readLines(const string& path,
               vector<string>& lines) {

    ifstream fin(path);

    if (!fin) return false;
```

```

    string s;

    while (getline(fin, s)) {

        lines.push_back(s);
    }

    return true;
}

string normalizeLine(const string& s) {

    string result;

    bool lastSpace = true;

    for (char c : s) {

        if (isalnum((unsigned char)c)) {

            result +=
                (char)tolower((unsigned char)c);

            lastSpace = false;
        }
        else {

            if (!lastSpace) {

                result += ' ';
                lastSpace = true;
            }
        }
    }

    if (!result.empty() &&
        result.back() == ' ') {

        result.pop_back();
    }

    return result;
}

vector<string> tokenize(
    const string& normalized) {

    vector<string> tokens;
```

```

    stringstream ss(normalized);

    string token;

    while (ss >> token) {

        tokens.push_back(token);
    }

    return tokens;
}

bool writeRepeats(
    const string& path,
    const vector<RepeatInfo>& repeats) {

    ofstream fout(path);

    if (!fout) return false;

    for (const RepeatInfo& r : repeats) {

        fout << "line="
             << r.line
             << " word="
             << r.word
             << " count="
             << r.count
             << "\n";
    }

    return true;
}

int main() {

    vector<string> lines;

    if (!readLines("text.txt", lines)) {

        cerr << "Loi mo file input!\n";
        return 1;
    }

    vector<RepeatInfo> repeats;

    for (size_t lineIdx = 0;
         lineIdx < lines.size();
```

```
         ++lineIdx) {

        string normalized =
            normalizeLine(lines[lineIdx]);

        vector<string> tokens =
            tokenize(normalized);

        if (tokens.empty()) continue;

        string current = tokens[0];
        int count = 1;

        for (size_t i = 1;
             i < tokens.size();
             ++i) {

            if (tokens[i] == current) {

                count++;
            }
            else {

                if (count >= 2) {

                    repeats.push_back({
                        (int)lineIdx + 1,
                        current,
                        count
                    });
                }

                current = tokens[i];
                count = 1;
            }
        }

        if (count >= 2) {

            repeats.push_back({
                (int)lineIdx + 1,
                current,
                count
            });
        }
    }

    if (!writeRepeats("repeats.txt",
                      repeats)) {
```

```

        cerr << "Loi mo file output!\n";
        return 1;
    }

    return 0;
}
```

9.7.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (text.txt):

```
hello hello world
test test test data
a b c
```

Expected Output (repeats.txt):

```
line=1 word=hello count=2
line=2 word=test count=3
```

Test Case 2 (Không có lặp)
Input (text.txt):

```
one two three
apple banana orange
```

Expected Output (repeats.txt):

```

```

Test Case 3 (Case biên – dòng rỗng)
Input (text.txt):

```

hello hello

wow wow wow
```

Expected Output (repeats.txt):

```
line=2 word=hello count=2
line=4 word=wow count=3
```

9.8 Tần suất ký tự [Cơ bản]

9.8.1 Phân tích & Thiết kế (M0)

- **Input:**

  File `text.txt` chứa nhiều dòng văn bản ASCII.

- **Xử lý:**
  - **Case biên:**
    * File rỗng → tất cả ký tự có count = 0
    * Dòng chỉ chứa số/punctuation → bỏ qua
  - **Module 1 (Đọc file):**
    * Hàm:

```
readLines(path, vector<string>&)
```

    * Kiểm tra mở file
  - **Module 2 (Chuẩn hoá ký tự):**
    * Chuyển lowercase bằng:

```
tolower(...)
```

    * Chỉ xử lý ký tự:

```
a-z
```

  - **Module 3 (Đếm tần suất):**
    * Dùng:

```
vector<int> freq(26, 0)
```

    * Với mỗi ký tự hợp lệ:

```
freq[c - 'a']++
```

  - **Module 4 (Ghi output):**
    * File:

```
char_freq.csv
```

    * Format:

```
char,count
```

    * Luôn ghi đủ 26 dòng
- **Output:**

```
a,0
b,2
...
z,1
```

9.8.2 Mã nguồn C++11 (Tách module)

```
#include <iostream>
#include <vector>
#include <fstream>
#include <string>
#include <cctype>

using namespace std;

bool readLines(const string& path,
               vector<string>& lines) {

    ifstream fin(path);

    if (!fin) return false;

    string s;

    while (getline(fin, s)) {

        lines.push_back(s);
    }

    return true;
}

bool writeFreq(const string& path,
               const vector<int>& freq) {

    ofstream fout(path);

    if (!fout) return false;

    for (int i = 0; i < 26; ++i) {

        char c = 'a' + i;

        fout << c
             << ","
             << freq[i]
             << "\n";
    }

    return true;
}

int main() {

```

```
    vector<string> lines;

    if (!readLines("text.txt", lines)) {

        cerr << "Loi mo file input!\n";
        return 1;
    }

    vector<int> freq(26, 0);

    for (const string& s : lines) {

        for (char c : s) {

            c =
                (char)tolower(
                    (unsigned char)c
                );

            if (c >= 'a' && c <= 'z') {

                freq[c - 'a']++;
            }
        }
    }

    if (!writeFreq("char_freq.csv",
                   freq)) {

        cerr << "Loi mo file output!\n";
        return 1;
    }

    return 0;
}
```

9.8.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (text.txt):

```
Hello World
abc ABC
```

Expected Output (char_freq.csv):

```
a,2
b,2
c,2
d,1
```

```
e,1
f,0
g,0
h,1
i,0
j,0
k,0
l,3
m,0
n,0
o,2
p,0
q,0
r,1
s,0
t,0
u,0
v,0
w,1
x,0
y,0
z,0
```

Test Case 2 (Case biên – file rỗng)
Input (text.txt):

```

```

Expected Output (char_freq.csv):

```
a,0
b,0
c,0
...
z,0
```

Test Case 3 (Case biên – không có chữ)
Input (text.txt):

```
12345 !!! ???
```

Expected Output (char_freq.csv):

```
a,0
b,0
c,0
...
z,0
```

9.9 Nhận diện ngôn ngữ đơn giản [Trung bình]

9.9.1 Phân tích & Thiết kế (M0)

- **Input:**

  File `text.txt` chứa văn bản.

- **Xử lý:**

  - **Case biên:**
    * File rỗng → `UNKNOWN`
    * Chỉ có số/punctuation → `UNKNOWN`
    * Có nhiều ký tự tiếng Việt → `VI_SIMPLE`
  - **Module 1 (Đọc file):**
    * Hàm:

```
readLines(path, vector<string>&)
```

    * Kiểm tra mở file
  - **Module 2 (Đếm ký tự):**
    * Đếm:
      · Tổng ký tự chữ
      · Số ký tự ngoài ASCII
    * ASCII chuẩn:

```
32 -> 126
```

  - **Module 3 (Phân loại):**
    * Nếu:

```
outside_ascii > 5% total_chars
```

    → `VI_SIMPLE`
    * Nếu chỉ có:

```
a-z A-Z
```

    → `EN`
    * Ngược lại:

```
UNKNOWN
```

  - **Module 4 (Ghi output):**
    * File:

```
lang.txt
```

- **Output:**

```
EN
```

hoặc

```
VI_SIMPLE
```

hoặc

```
UNKNOWN
```

9.9.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>
#include <string>
#include <cctype>

using namespace std;

bool readLines(const string& path,
               vector<string>& lines) {

    ifstream fin(path);

    if (!fin) return false;

    string s;

    while (getline(fin, s)) {

        lines.push_back(s);
    }

    return true;
}

bool writeLang(const string& path,
               const string& lang) {

    ofstream fout(path);

    if (!fout) return false;

    fout << lang;

    return true;
}
```

```
37
38 int main() {
39
40     vector<string> lines;
41
42     if (!readLines("text.txt", lines)) {
43
44         cerr << "Loi mo file input!\n";
45         return 1;
46     }
47
48     int totalChars = 0;
49     int outsideASCII = 0;
50     int englishLetters = 0;
51
52     for (const string& s : lines) {
53
54         for (unsigned char c : s) {
55
56             if (isalpha(c)) {
57
58                 totalChars++;
59
60                 if (c >= 32 && c <= 126) {
61
62                     if ((c >= 'a' && c <= 'z') ||
63                         (c >= 'A' && c <= 'Z')) {
64
65                         englishLetters++;
66                     }
67                 }
68                 else {
69
70                     outsideASCII++;
71                 }
72             }
73         }
74     }
75
76     string lang = "UNKNOWN";
77
78     if (totalChars == 0) {
79
80         lang = "UNKNOWN";
81     }
82     else {
83
84         double ratio =
85             (double)outsideASCII
```

```
            / totalChars;

        if (ratio > 0.05) {

            lang = "VI_SIMPLE";
        }
        else if (englishLetters ==
                 totalChars) {

            lang = "EN";
        }
    }

    if (!writeLang("lang.txt", lang)) {

        cerr << "Loi mo file output!\n";
        return 1;
    }

    return 0;
}
```

9.9.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Tiếng Anh)
Input (text.txt):

```
Hello world
This is a simple English text
```

Expected Output (lang.txt):

```
EN
```

Test Case 2 (Tiếng Việt đơn giản)
Input (text.txt):

```
Tôi đang ọhc ậlp trình ở ườtrng đại ọhc
```

Expected Output (lang.txt):

```
VI_SIMPLE
```

Test Case 3 (Case biên – file rỗng)
Input (text.txt):

```

```

Expected Output (lang.txt):

```
UNKNOWN
```

9.10 TF-IDF đơn giản trên tập file [Nâng cao]

9.10.1 Phân tích & Thiết kế (M0)

- **Input:**

  File `index.txt` chứa danh sách file văn bản.

  - Mỗi dòng là tên một file
  - Các file chứa văn bản ASCII

- **Xử lý:**

  - **Case biên:**
    * `index.txt` rỗng → output rỗng
    * File văn bản rỗng → không có token
    * Một từ xuất hiện ở mọi file → IDF = 0
  - **Module 1 (Đọc danh sách file):**
    * Hàm:

```
readIndex(path, vector<string>&)
```

  - **Module 2 (Đọc từng file):**
    * Hàm:

```
readLines(path, vector<string>&)
```

  - **Module 3 (Normalize):**
    * Chuyển lowercase
    * Ký tự không phải chữ/số → space
    * Gom nhiều space thành 1
  - **Module 4 (Tokenize):**
    * Hàm:

```
vector<string> tokenize(...)
```

  - **Module 5 (Tính TF):**
    * :

```
TF(w,d) =
count(w,d) / total_tokens(d)
```

  - **Module 6 (Tính DF và IDF):**
    * :

```
df(w) = ốs file ứcha w
```

    * :

```
IDF(w) = log(N / df(w))
```

    * N: số lượng file

  – **Module 7 (Tính TF-IDF):**

    * :

```
TFIDF = TF * IDF
```

    * Sort:
      · Giảm dần theo TF-IDF
      · Nếu bằng nhau → tăng dần lexicographic
    * Chỉ lấy top-5 từ mỗi file

  – **Module 8 (Ghi output):**

    * File:

```
tfidf.csv
```

    * Format:

```
file,word,tfidf
```

    * Dùng:

```
setprecision(6)
```

- **Output:**

```
file,word,tfidf
```

9.10.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <vector>
#include <fstream>
#include <string>
#include <unordered_map>
#include <unordered_set>
#include <sstream>
#include <algorithm>
#include <iomanip>
#include <cmath>
#include <cctype>

using namespace std;

struct TFIDFRow {
```

```
    string file;
    string word;
    double tfidf;
};

bool readIndex(const string& path,
               vector<string>& files) {

    ifstream fin(path);

    if (!fin) return false;

    string s;

    while (getline(fin, s)) {

        if (!s.empty()) {

            files.push_back(s);
        }
    }

    return true;
}

bool readLines(const string& path,
               vector<string>& lines) {

    ifstream fin(path);

    if (!fin) return false;

    string s;

    while (getline(fin, s)) {

        lines.push_back(s);
    }

    return true;
}

string normalizeLine(const string& s) {

    string result;

    bool lastSpace = true;

    for (char c : s) {
```

```

        if (isalnum((unsigned char)c)) {

            result +=
                (char)tolower((unsigned char)c);

            lastSpace = false;
        }
        else {

            if (!lastSpace) {

                result += ' ';
                lastSpace = true;
            }
        }
    }

    if (!result.empty() &&
        result.back() == ' ') {

        result.pop_back();
    }

    return result;
}

vector<string> tokenize(
    const string& normalized) {

    vector<string> tokens;

    stringstream ss(normalized);

    string token;

    while (ss >> token) {

        tokens.push_back(token);
    }

    return tokens;
}

bool writeTFIDF(
    const string& path,
    const vector<TFIDFRow>& rows) {

    ofstream fout(path);
```

```

    if (!fout) return false;

    fout << fixed
         << setprecision(6);

    for (const TFIDFRow& r : rows) {

        fout << r.file
             << ","
             << r.word
             << ","
             << r.tfidf
             << "\n";
    }

    return true;
}

int main() {

    vector<string> files;

    if (!readIndex("index.txt", files)) {

        cerr << "Loi mo index!\n";
        return 1;
    }

    int N = files.size();

    vector<
        unordered_map<string,int>
    > fileFreqs(N);

    vector<int> totalTokens(N, 0);

    unordered_map<string,int> df;

    for (int i = 0; i < N; ++i) {

        vector<string> lines;

        if (!readLines(files[i], lines)) {

            cerr << "Loi mo file!\n";
            return 1;
        }

```

```
        unordered_set<string> seen;

        for (const string& s : lines) {

            string normalized =
                normalizeLine(s);

            vector<string> tokens =
                tokenize(normalized);

            for (const string& token : tokens) {

                fileFreqs[i][token]++;

                totalTokens[i]++;

                seen.insert(token);
            }
        }

        for (const string& w : seen) {

            df[w]++;
        }
    }

    vector<TFIDFRow> rows;

    for (int i = 0; i < N; ++i) {

        // Tránh chia cho 0 ếnu file õrng
        if (totalTokens[i] == 0) {
            continue;
        }

        vector<pair<string,double>> scores;

        for (const auto& p :
             fileFreqs[i]) {

            const string& word =
                p.first;

            int count =
                p.second;

            double tf =
                (double)count
                / totalTokens[i];
```

```
            double idf =
                log(
                    (double)N
                    / df[word]
                );

            double tfidf =
                tf * idf;

            scores.push_back({
                word,
                tfidf
            });
        }

        sort(scores.begin(),
             scores.end(),
             [](const pair<string,double>& a,
                const pair<string,double>& b) {

            if (a.second != b.second)
                return a.second > b.second;

            return a.first < b.first;
        });

        int limit =
            min(5,
                (int)scores.size());

        for (int j = 0;
             j < limit;
             ++j) {

            rows.push_back({
                files[i],
                scores[j].first,
                scores[j].second
            });
        }
    }

    if (!writeTFIDF("tfidf.csv",
                    rows)) {

        cerr << "Loi mo output!\n";
        return 1;
    }
```

```

    return 0;
}
```

9.10.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (index.txt):

```
doc1.txt
doc2.txt
```

Input (doc1.txt):

```
apple banana apple
```

Input (doc2.txt):

```
banana orange
```

Expected Output (tfidf.csv):

```
doc1.txt,apple,...
doc1.txt,banana,...
doc2.txt,orange,...
doc2.txt,banana,...
```

Test Case 2 (Case biên – file rỗng)
Input (index.txt):

```
empty.txt
```

Input (empty.txt):

Expected Output (tfidf.csv):

Test Case 3 (Từ xuất hiện mọi file)
Input (index.txt):

```
a.txt
b.txt
```

Input (a.txt):

```
hello world
```

Input (b.txt):

```
hello chatgpt
```

Expected:

- `hello` có:

```
idf = log(2/2) = 0
```

- TF-IDF của `hello` bằng 0

10 Tuần 10: Tổng hợp M0: Mini-project + Module hoá

10.1 Pipeline M0 hoàn chỉnh: CSV Cleaning [Cơ bản]

10.1.1 Phân tích & Thiết kế (M0)

- **Input:**
  File `data.csv` chứa:
  - Mỗi dòng gồm: `id,value`
  - Giá trị thiếu có thể là `NA` hoặc rỗng
- **Xử lý:**
  - **Case biên:**
    * Tất cả giá trị đều missing

$$\Rightarrow \text{mean} = 0, \quad \text{std} = 0$$

$$\Rightarrow \text{mọi } z = 0$$

    * `std = 0`

$$\Rightarrow z = 0$$

  - **Module 1 (Đọc CSV):**
    * Hàm `parseCSV(path, ids, values, missing)`
    * Đọc từng dòng bằng `getline()`
    * Kiểm tra missing:
      · `NA`
      · Chuỗi rỗng
  - **Module 2 (Tính thống kê):**
    * Hàm `computeStats()`
    * Tính:
      · count_valid

        - count_missing
        - mean
        - std
    - Bỏ qua các giá trị missing
    - Dùng `long double` để giảm sai số
  - **Module 3 (Impute + Z-score):**
    - Hàm `imputeAndScore()`
    - Missing sẽ được thay bằng mean
    - Công thức:

$$z = \frac{x - mean}{std}$$

- **Output:**

  File `report.csv`

```
id,value_imputed,zscore
```

  File `summary.txt`

```
count_valid=<n>
count_missing=<n>
mean=<f>
std=<f>
```

  (với `setprecision(6)`)

10.1.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <iomanip>
#include <cmath>

using namespace std;

bool parseCSV(
    const string& path,
    vector<string>& ids,
    vector<double>& values,
    vector<bool>& missing
) {
    ifstream fin(path);
    if (!fin) return false;

    string line;
```

```

    while (getline(fin, line)) {
        if (line.empty()) continue;

        stringstream ss(line);

        string id, valStr;

        getline(ss, id, ',');
        getline(ss, valStr);

        ids.push_back(id);

        if (valStr.empty() || valStr == "NA") {
            values.push_back(0);
            missing.push_back(true);
        } else {
            values.push_back(stod(valStr));
            missing.push_back(false);
        }
    }

    return true;
}

void computeStats(
    const vector<double>& values,
    const vector<bool>& missing,
    int& countValid,
    int& countMissing,
    double& mean,
    double& stdev
) {
    countValid = 0;
    countMissing = 0;

    long double sum = 0;

    for (size_t i = 0; i < values.size(); ++i) {
        if (missing[i]) {
            ++countMissing;
        } else {
            ++countValid;
            sum += values[i];
        }
    }

    if (countValid == 0) {
        mean = 0;
```

```
        stdev = 0;
        return;
    }

    mean = sum / countValid;

    long double variance = 0;

    for (size_t i = 0; i < values.size(); ++i) {
        if (!missing[i]) {
            variance += (values[i] - mean) * (values[i] - mean);
        }
    }

    variance /= countValid;

    stdev = sqrt(variance);
}

void imputeAndScore(
    const vector<string>& ids,
    const vector<double>& values,
    const vector<bool>& missing,
    double mean,
    double stdev
) {
    ofstream fout("report.csv");

    fout << fixed << setprecision(6);

    fout << "id,value_imputed,zscore\n";

    for (size_t i = 0; i < ids.size(); ++i) {

        double x = missing[i] ? mean : values[i];

        double z = 0;

        if (stdev != 0) {
            z = (x - mean) / stdev;
        }

        fout << ids[i] << ","
             << x << ","
             << z << "\n";
    }
}

bool writeSummary(
```

```
    const string& path,
    int countValid,
    int countMissing,
    double mean,
    double stdev
) {
    ofstream fout(path);

    if (!fout) return false;

    fout << fixed << setprecision(6);

    fout << "count_valid=" << countValid << "\n";
    fout << "count_missing=" << countMissing << "\n";
    fout << "mean=" << mean << "\n";
    fout << "std=" << stdev << "\n";

    return true;
}

int main() {

    vector<string> ids;
    vector<double> values;
    vector<bool> missing;

    if (!parseCSV("data.csv", ids, values, missing)) {
        cerr << "Loi mo file input!\n";
        return 1;
    }

    int countValid, countMissing;

    double mean, stdev;

    computeStats(
        values,
        missing,
        countValid,
        countMissing,
        mean,
        stdev
    );

    imputeAndScore(
        ids,
        values,
        missing,
        mean,
```

```
        stdev
    );

    if (!writeSummary(
        "summary.txt",
        countValid,
        countMissing,
        mean,
        stdev
    )) {
        cerr << "Loi mo file output!\n";
        return 1;
    }

    return 0;
}
```

10.1.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (data.csv):

```
A,10
B,20
C,NA
D,30
```

Expected Output (summary.txt):

```
count_valid=3
count_missing=1
mean=20.000000
std=8.164966
```

Expected Output (report.csv):

```
id,value_imputed,zscore
A,10.000000,-1.224745
B,20.000000,0.000000
C,20.000000,0.000000
D,30.000000,1.224745
```

Test Case 2 (Tất cả missing)
Input (data.csv):

```
A,NA
B,
C,NA
```

Expected Output (summary.txt):

```
count_valid=0
count_missing=3
mean=0.000000
std=0.000000
```

Expected Output (report.csv):

```
id,value_imputed,zscore
A,0.000000,0.000000
B,0.000000,0.000000
C,0.000000,0.000000
```

Test Case 3 (std = 0)
Input (data.csv):

```
A,5
B,5
C,5
```

Expected Output (summary.txt):

```
count_valid=3
count_missing=0
mean=5.000000
std=0.000000
```

Expected Output (report.csv):

```
id,value_imputed,zscore
A,5.000000,0.000000
B,5.000000,0.000000
C,5.000000,0.000000
```

10.2 Fraction List tổng hợp [Cơ bản]

10.2.1 Phân tích & Thiết kế (M0)

- **Input:**

  File `fractions.csv` chứa:

  - Mỗi dòng gồm:

    ```
    num,den
    ```

- **Xử lý:**

  - **Case biên:**

    * Phân số có mẫu bằng 0
    $$\Rightarrow \text{invalid}$$
    * Tử số bằng 0
    $$\Rightarrow 0/1$$
    * Không có phân số hợp lệ
    $$\Rightarrow \text{sum} = 0/1$$
  - **Module 1 (Đọc file):**
    * Hàm `readFractions()`
    * Đọc dữ liệu từ `fractions.csv`
    * Lưu vào vector phân số
  - **Module 2 (Lọc + Normalize):**
    * Hàm `filterAndNormalize()`
    * Loại bỏ phân số invalid
    * Chuẩn hoá:
      · Mẫu luôn dương
      · Rút gọn bằng gcd
      · `0/x → 0/1`
  - **Module 3 (Sort):**
    * Hàm `sortFractions()`
    * Sắp xếp tăng dần
    * So sánh bằng nhân chéo
  - **Module 4 (Tính tổng):**
    * Hàm `addFrac()`
    * Cộng phân số chính xác
    * Không dùng `double`
  - **Module 5 (Ghi file):**
    * Hàm `writeFractionReport()`
    * Ghi:
      · valid_count
      · invalid_count
      · sum
      · danh sách phân số đã sort

- **Output:**

  File `fraction_out.csv`

```
valid_count=<n>
invalid_count=<n>
sum=num/den

num,den
...
```

10.2.2 Mã nguồn C++11 (Tách module)

```
#include <iostream>
#include <fstream>
#include <vector>
#include <sstream>
#include <algorithm>

using namespace std;

struct Fraction {
    long long num;
    long long den;
};

long long gcdll(long long a, long long b) {
    a = abs(a);
    b = abs(b);

    while (b != 0) {
        long long r = a % b;
        a = b;
        b = r;
    }

    return a;
}

bool normalize(Fraction& f) {

    if (f.den == 0) return false;

    if (f.num == 0) {
        f.den = 1;
        return true;
    }

    if (f.den < 0) {
        f.num = -f.num;
        f.den = -f.den;
    }

    long long g = gcdll(f.num, f.den);

    f.num /= g;
    f.den /= g;

    return true;
}
```

```

bool readFractions(
    const string& path,
    vector<Fraction>& fractions
) {
    ifstream fin(path);

    if (!fin) return false;

    string line;

    while (getline(fin, line)) {

        if (line.empty()) continue;

        stringstream ss(line);

        string sNum, sDen;

        getline(ss, sNum, ',');
        getline(ss, sDen);

        Fraction f;

        f.num = stoll(sNum);
        f.den = stoll(sDen);

        fractions.push_back(f);
    }

    return true;
}

void filterAndNormalize(
    const vector<Fraction>& input,
    vector<Fraction>& validFractions,
    int& invalidCount
) {
    invalidCount = 0;

    for (size_t i = 0; i < input.size(); ++i) {

        Fraction f = input[i];

        if (!normalize(f)) {
            ++invalidCount;
        } else {
            validFractions.push_back(f);
        }
```

```
    }
}

bool fracLess(const Fraction& a, const Fraction& b) {
    return a.num * b.den < b.num * a.den;
}

void sortFractions(vector<Fraction>& a) {
    sort(a.begin(), a.end(), fracLess);
}

Fraction addFrac(const Fraction& a, const Fraction& b) {

    Fraction res;

    res.num = a.num * b.den + b.num * a.den;
    res.den = a.den * b.den;

    normalize(res);

    return res;
}

bool writeFractionReport(
    const string& path,
    const vector<Fraction>& fractions,
    int validCount,
    int invalidCount,
    const Fraction& sum
) {
    ofstream fout(path);

    if (!fout) return false;

    fout << "valid_count=" << validCount << "\n";
    fout << "invalid_count=" << invalidCount << "\n";
    fout << "sum=" << sum.num << "/" << sum.den << "\n\n";

    for (size_t i = 0; i < fractions.size(); ++i) {
        fout << fractions[i].num
             << ","
             << fractions[i].den
             << "\n";
    }

    return true;
}

int main() {
```

```

    vector<Fraction> inputFractions;

    if (!readFractions("fractions.csv", inputFractions)) {
        cerr << "Loi mo file input!\n";
        return 1;
    }

    vector<Fraction> validFractions;

    int invalidCount;

    filterAndNormalize(
        inputFractions,
        validFractions,
        invalidCount
    );

    sortFractions(validFractions);

    Fraction sum = {0, 1};

    for (size_t i = 0; i < validFractions.size(); ++i) {
        sum = addFrac(sum, validFractions[i]);
    }

    if (!writeFractionReport(
        "fraction_out.csv",
        validFractions,
        validFractions.size(),
        invalidCount,
        sum
    )) {
        cerr << "Loi mo file output!\n";
        return 1;
    }

    return 0;
}
```

10.2.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (fractions.csv):

```
1,2
2,4
3,6
5,2
```

```
-4,8
```

Expected Output (fraction_out.csv):

```
valid_count=5
invalid_count=0
sum=4/1

-1,2
1,2
1,2
1,2
5,2
```

Test Case 2 (Có invalid)
Input (fractions.csv):

```
1,0
3,4
5,0
2,8
```

Expected Output (fraction_out.csv):

```
valid_count=2
invalid_count=2
sum=1/1

1,4
3,4
```

Test Case 3 (Không có phân số hợp lệ)
Input (fractions.csv):

```
1,0
2,0
3,0
```

Expected Output (fraction_out.csv):

```
valid_count=0
invalid_count=3
sum=0/1
```

10.3 Polynomial records + evaluate [Cơ bản]

10.3.1 Phân tích & Thiết kế (M0)

- **Input:**

  File `polynomials.csv` chứa:

- Mỗi dòng gồm:

```
id,degree,a0,a1,a2,...
```

- Đa thức:

$$P(x) = a_0 + a_1x + a_2x^2 + \cdots + a_nx^n$$

File `x_values.csv` chứa:

- Mỗi dòng là một giá trị x

- **Xử lý:**
  - **Case biên:**
    * `degree < 0`

      $\Rightarrow$ invalid
    * Thiếu hệ số

      $\Rightarrow 0$
    * File `x_values.csv` rỗng

      $\Rightarrow$ không ghi dữ liệu nhưng vẫn hợp lệ
  - **Module IO:**
    * Hàm `readPoly()`
    * Hàm `readX()`
    * Hàm `writeReport()`
  - **Module Processing:**
    * Hàm `evalHorner()`
    * Dùng thuật toán Horner:

$$P(x) = (((a_nx + a_{n-1})x + a_{n-2})\cdots)x + a_0$$

- **Output:**

  File `report.csv`

```
id,x,P(x)
...
---
valid=<n>
invalid=<n>
```

  Với mọi số thực:

```
setprecision(6)
```

10.3.2 Mã nguồn C++11 (Tách module)

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <iomanip>

using namespace std;

struct Polynomial {
    string id;
    int degree;
    vector<double> coef;
    bool valid;
};

bool readPoly(
    const string& path,
    vector<Polynomial>& polys,
    int& validCount,
    int& invalidCount
) {
    ifstream fin(path);

    if (!fin) return false;

    validCount = 0;
    invalidCount = 0;

    string line;

    while (getline(fin, line)) {

        if (line.empty()) continue;

        stringstream ss(line);

        vector<string> parts;

        string token;

        while (getline(ss, token, ',')) {
            parts.push_back(token);
        }

        if (parts.size() < 2) continue;

        Polynomial p;
```

```

        p.id = parts[0];

        p.degree = stoi(parts[1]);

        p.valid = true;

        if (p.degree < 0) {
            p.valid = false;
            ++invalidCount;
        } else {

            p.coef.assign(p.degree + 1, 0);

            for (int i = 0; i <= p.degree; ++i) {

                int idx = i + 2;

                if (idx < (int)parts.size()
                    && !parts[idx].empty()) {

                    p.coef[i] = stod(parts[idx]);
                } else {
                    p.coef[i] = 0;
                }
            }

            ++validCount;
        }

        polys.push_back(p);
    }

    return true;
}

bool readX(
    const string& path,
    vector<double>& xs
) {
    ifstream fin(path);

    if (!fin) return false;

    double x;

    while (fin >> x) {
        xs.push_back(x);
    }
```

```

    return true;
}

double evalHorner(
    const vector<double>& coef,
    double x
) {
    if (coef.empty()) return 0;

    double result = coef.back();

    for (int i = (int)coef.size() - 2; i >= 0; --i) {
        result = result * x + coef[i];
    }

    return result;
}

bool writeReport(
    const string& path,
    const vector<Polynomial>& polys,
    const vector<double>& xs,
    int validCount,
    int invalidCount
) {
    ofstream fout(path);

    if (!fout) return false;

    fout << fixed << setprecision(6);

    fout << "id,x,P(x)\n";

    for (size_t i = 0; i < polys.size(); ++i) {

        if (!polys[i].valid) continue;

        for (size_t j = 0; j < xs.size(); ++j) {

            double value =
                evalHorner(polys[i].coef, xs[j]);

            fout << polys[i].id << ","
                 << xs[j] << ","
                 << value << "\n";
        }
    }

```

```
    fout << "---\n";

    fout << "valid=" << validCount << "\n";
    fout << "invalid=" << invalidCount << "\n";

    return true;
}

int main() {

    vector<Polynomial> polys;

    int validCount, invalidCount;

    if (!readPoly(
        "polynomials.csv",
        polys,
        validCount,
        invalidCount
    )) {
        cerr << "Loi mo file polynomial!\n";
        return 1;
    }

    vector<double> xs;

    if (!readX("x_values.csv", xs)) {
        cerr << "Loi mo file x_values!\n";
        return 1;
    }

    if (!writeReport(
        "report.csv",
        polys,
        xs,
        validCount,
        invalidCount
    )) {
        cerr << "Loi mo file output!\n";
        return 1;
    }

    return 0;
}
```

10.3.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)

Input (polynomials.csv):

```
P1,2,1,2,3
P2,1,5,4
```

Input (x_values.csv):

```
0
1
2
```

Expected Output (report.csv):

```
id,x,P(x)
P1,0.000000,1.000000
P1,1.000000,6.000000
P1,2.000000,17.000000
P2,0.000000,5.000000
P2,1.000000,9.000000
P2,2.000000,13.000000
---
valid=2
invalid=0
```

Test Case 2 (degree < 0)
Input (polynomials.csv):

```
P1,-1,1,2
P2,2,1,0,1
```

Input (x_values.csv):

```
2
```

Expected Output (report.csv):

```
id,x,P(x)
P2,2.000000,5.000000
---
valid=1
invalid=1
```

Test Case 3 (Thiếu hệ số + x rỗng)
Input (polynomials.csv):

```
P1,3,1,2
```

Input (x_values.csv):

```

```

Expected Output (report.csv):

```
id,x,P(x)
---
valid=1
invalid=0
```

10.4 Multi-query prefix sum tổng hợp [Trung bình]

10.4.1 Phân tích & Thiết kế (M0)

- **Input:**

  File `input.txt` chứa:

  - Số nguyên n
  - n phần tử của mảng
  - Số truy vấn q
  - Danh sách truy vấn:

```
SUM l r
AVG l r
CNT l r v
```

- **Xử lý:**

  - **Case biên:**
    - $l < 0$
    - $r \geq n$
    - $l > r$

$$\Rightarrow \texttt{INVALID}$$

  - **Module 1 (Đọc file):**
    - Hàm `readInput()`
    - Đọc:
      - mảng
      - danh sách truy vấn
  - **Module 2 (Xây prefix sum):**
    - Hàm `buildPrefix()`
    - Prefix kiểu `long long`

$$pre[i] = a[0] + a[1] + \cdots + a[i]$$

  - **Module 3 (SUM Query):**

    * Hàm `handleSUM()`
    * Công thức:

$$sum(l, r) = pre[r] - pre[l - 1]$$

  - **Module 4 (AVG Query):**
    * Hàm `handleAVG()`
    * Công thức:

$$avg = \frac{sum(l, r)}{r - l + 1}$$

  - **Module 5 (CNT Query):**
    * Hàm `handleCNT()`
    * Với $n \leq 1000$:
      · Duyệt brute force $O(n)$
  - **Module 6 (Ghi file):**
    * Hàm `writeOutput()`
    * Ghi kết quả từng truy vấn

- **Output:**

  File `output.txt`

```
<ket qua query 1>
<ket qua query 2>
...
```

  Nếu truy vấn không hợp lệ:

```
INVALID
```

10.4.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <sstream>
#include <iomanip>

using namespace std;

struct Query {
    string type;
    int l, r;
    long long v;
};

bool readInput(
    const string& path,
```

```
    vector<long long>& a,
    vector<Query>& queries
) {
    ifstream fin(path);

    if (!fin) return false;

    int n;

    fin >> n;

    a.resize(n);

    for (int i = 0; i < n; ++i) {
        fin >> a[i];
    }

    int q;

    fin >> q;

    for (int i = 0; i < q; ++i) {

        Query qu;

        fin >> qu.type;

        if (qu.type == "SUM"
            || qu.type == "AVG") {

            fin >> qu.l >> qu.r;

            qu.v = 0;
        }
        else if (qu.type == "CNT") {

            fin >> qu.l >> qu.r >> qu.v;
        }

        queries.push_back(qu);
    }

    return true;
}

vector<long long> buildPrefix(
    const vector<long long>& a
) {
    vector<long long> pre(a.size());
```

```

    if (a.empty()) return pre;

    pre[0] = a[0];

    for (size_t i = 1; i < a.size(); ++i) {
        pre[i] = pre[i - 1] + a[i];
    }

    return pre;
}

bool validRange(
    int l,
    int r,
    int n
) {
    return !(l < 0 || r >= n || l > r);
}

string handleSUM(
    const vector<long long>& pre,
    int l,
    int r
) {
    if (!validRange(l, r, pre.size())) {
        return "INVALID";
    }

    long long sum = pre[r];

    if (l > 0) {
        sum -= pre[l - 1];
    }

    return to_string(sum);
}

string handleAVG(
    const vector<long long>& pre,
    int l,
    int r
) {
    if (!validRange(l, r, pre.size())) {
        return "INVALID";
    }

    long long sum = pre[r];

```

```
    if (l > 0) {
        sum -= pre[l - 1];
    }

    double avg =
        (double)sum / (r - l + 1);

    stringstream ss;

    ss << fixed << setprecision(6)
       << avg;

    return ss.str();
}

string handleCNT(
    const vector<long long>& a,
    int l,
    int r,
    long long v
) {
    if (!validRange(l, r, a.size())) {
        return "INVALID";
    }

    int cnt = 0;

    for (int i = l; i <= r; ++i) {
        if (a[i] == v) {
            ++cnt;
        }
    }

    return to_string(cnt);
}

bool writeOutput(
    const string& path,
    const vector<string>& results
) {
    ofstream fout(path);

    if (!fout) return false;

    for (size_t i = 0; i < results.size(); ++i) {
        fout << results[i] << "\n";
    }

    return true;
```

```
}

int main() {

    vector<long long> a;

    vector<Query> queries;

    if (!readInput(
        "input.txt",
        a,
        queries
    )) {
        cerr << "Loi mo file input!\n";
        return 1;
    }

    vector<long long> pre =
        buildPrefix(a);

    vector<string> results;

    for (size_t i = 0; i < queries.size(); ++i) {

        Query q = queries[i];

        if (q.type == "SUM") {

            results.push_back(
                handleSUM(
                    pre,
                    q.l,
                    q.r
                )
            );
        }
        else if (q.type == "AVG") {

            results.push_back(
                handleAVG(
                    pre,
                    q.l,
                    q.r
                )
            );
        }
        else if (q.type == "CNT") {

            results.push_back(
```

```
                handleCNT(
                    a,
                    q.l,
                    q.r,
                    q.v
                )
            );
        }
    }

    if (!writeOutput(
        "output.txt",
        results
    )) {
        cerr << "Loi mo file output!\n";
        return 1;
    }

    return 0;
}
```

10.4.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (input.txt):

```
5
1 2 3 2 5
4
SUM 1 3
AVG 0 4
CNT 0 4 2
CNT 2 4 5
```

Expected Output (output.txt):

```
7
2.600000
2
1
```

Test Case 2 (INVALID query)
Input (input.txt):

```
4
1 2 3 4
3
SUM -1 2
AVG 1 10
CNT 3 1 4
```

Expected Output (output.txt):

```
INVALID
INVALID
INVALID
```

Test Case 3 (Mảng rỗng)
Input (input.txt):

```
0

2
SUM 0 0
CNT 0 0 1
```

Expected Output (output.txt):

```
INVALID
INVALID
```

10.5 Text analysis pipeline [Trung bình]

10.5.1 Phân tích & Thiết kế (M0)

- **Input:**

  File `text.txt`

  - Chứa nội dung văn bản

  File `stopwords.txt`

  - Mỗi dòng là một stop-word

- **Xử lý:**

  Pipeline:

  $$\text{normalize} \rightarrow \text{tokenize} \rightarrow \text{filter stop-words} \rightarrow \text{count frequency} \rightarrow \text{top-20}$$

  - **Case biên:**
    * File text rỗng
    $$\Rightarrow total = 0,\ unique = 0$$
    * Sau khi lọc stop-words không còn token
    $$\Rightarrow freq.csv \text{ chỉ có header}$$

  - **Module 1 (Đọc file):**

    - Hàm `readText()`
    - Hàm `readStopWords()`
  - **Module 2 (Normalize):**
    - Hàm `normalize()`
    - Chuyển:
      - ký tự hoa → thường
      - ký tự đặc biệt → khoảng trắng
  - **Module 3 (Tokenize):**
    - Hàm `tokenize()`
    - Tách từ bằng `stringstream`
  - **Module 4 (Filter stop-words):**
    - Hàm `filterStopWords()`
    - Loại bỏ token xuất hiện trong stop-word set
  - **Module 5 (Count frequency):**
    - Hàm `countFreq()`
    - Đếm tần suất bằng `map<string,int>`
  - **Module 6 (Sort top-20):**
    - Sắp xếp:
      - frequency giảm dần
      - nếu bằng nhau → sort alphabet
  - **Module 7 (Ghi file):**
    - Hàm `writeFreq()`
    - Hàm `writeSummary()`

- **Output:**

  File `freq.csv`

```
word,count
...
```

  File `summary.txt`

```
total=<n>
unique=<n>

TOP-5
word            count
...
```

  Dùng:

```
setw + left
```

  để căn lề bảng.

10.5.2 Mã nguồn C++11 (Tách module)

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <map>
#include <set>
#include <algorithm>
#include <iomanip>
#include <cctype>

using namespace std;

string readText(const string& path) {

    ifstream fin(path);

    if (!fin) return "";

    string line, text;

    while (getline(fin, line)) {
        text += line + " ";
    }

    return text;
}

set<string> readStopWords(
    const string& path
) {
    ifstream fin(path);

    set<string> stopWords;

    string word;

    while (getline(fin, word)) {

        if (!word.empty()) {
            stopWords.insert(word);
        }
    }

    return stopWords;
}

string normalize(const string& text) {
```

```

    string result = text;

    for (size_t i = 0; i < result.size(); ++i) {

        if (isalnum(result[i])) {
            result[i] = tolower(result[i]);
        } else {
            result[i] = ' ';
        }
    }

    return result;
}

vector<string> tokenize(
    const string& text
) {
    vector<string> tokens;

    stringstream ss(text);

    string token;

    while (ss >> token) {
        tokens.push_back(token);
    }

    return tokens;
}

vector<string> filterStopWords(
    const vector<string>& tokens,
    const set<string>& stopWords
) {
    vector<string> filtered;

    for (size_t i = 0; i < tokens.size(); ++i) {

        if (!stopWords.count(tokens[i])) {
            filtered.push_back(tokens[i]);
        }
    }

    return filtered;
}

map<string, int> countFreq(
    const vector<string>& tokens
```

```
) {
    map<string, int> freq;

    for (size_t i = 0; i < tokens.size(); ++i) {
        ++freq[tokens[i]];
    }

    return freq;
}

vector<pair<string, int>> sortTop(
    const map<string, int>& freq
) {
    vector<pair<string, int>> arr(
        freq.begin(),
        freq.end()
    );

    sort(
        arr.begin(),
        arr.end(),
        [](const pair<string, int>& a,
           const pair<string, int>& b) {

            if (a.second != b.second) {
                return a.second > b.second;
            }

            return a.first < b.first;
        }
    );

    if (arr.size() > 20) {
        arr.resize(20);
    }

    return arr;
}

bool writeFreq(
    const string& path,
    const vector<pair<string, int>>& topWords
) {
    ofstream fout(path);

    if (!fout) return false;

    fout << "word,count\n";

```

```
    for (size_t i = 0; i < topWords.size(); ++i) {

        fout << topWords[i].first
             << ","
             << topWords[i].second
             << "\n";
    }

    return true;
}

bool writeSummary(
    const string& path,
    int total,
    int unique,
    const vector<pair<string, int>>& topWords
) {
    ofstream fout(path);

    if (!fout) return false;

    fout << "total=" << total << "\n";
    fout << "unique=" << unique << "\n\n";

    fout << "TOP-5\n";

    fout << left
         << setw(15) << "word"
         << setw(10) << "count"
         << "\n";

    int limit =
        min(5, (int)topWords.size());

    for (int i = 0; i < limit; ++i) {

        fout << left
             << setw(15)
             << topWords[i].first

             << setw(10)
             << topWords[i].second

             << "\n";
    }

    return true;
}

```

```
int main() {

    string text =
        readText("text.txt");

    set<string> stopWords =
        readStopWords("stopwords.txt");

    string normalized =
        normalize(text);

    vector<string> tokens =
        tokenize(normalized);

    vector<string> filtered =
        filterStopWords(
            tokens,
            stopWords
        );

    map<string, int> freq =
        countFreq(filtered);

    vector<pair<string, int>> topWords =
        sortTop(freq);

    if (!writeFreq(
        "freq.csv",
        topWords
    )) {
        cerr << "Loi mo file freq!\n";
        return 1;
    }

    if (!writeSummary(
        "summary.txt",
        filtered.size(),
        freq.size(),
        topWords
    )) {
        cerr << "Loi mo file summary!\n";
        return 1;
    }

    return 0;
}
```

10.5.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (text.txt):

```
Hello world hello AI world AI AI
```

Input (stopwords.txt):

```
world
```

Expected Output (freq.csv):

```
word,count
ai,3
hello,2
```

Expected Output (summary.txt):

```
total=5
unique=2

TOP-5
word            count
ai              3
hello           2
```

Test Case 2 (File rỗng)
Input (text.txt):

```

```

Input (stopwords.txt):

```
the
is
a
```

Expected Output (freq.csv):

```
word,count
```

Expected Output (summary.txt):

```
total=0
unique=0

TOP-5
word            count
```

Test Case 3 (Chỉ có stop-words)
Input (text.txt):

```
the the is a is the
```

Input (stopwords.txt):

```
the
is
a
```

Expected Output (freq.csv):

```
word,count
```

Expected Output (summary.txt):

```
total=0
unique=0

TOP-5
word            count
```

10.6 Dashboard tổng hợp 1 file [Trung bình]

10.6.1 Phân tích & Thiết kế (M0)

- **Input:**

  File `data.csv` chứa:

  - Dòng đầu là header:

    ```
    id,value1,value2,...,valueN
    ```

  - Các dòng tiếp theo:

    ```
    id,number,number,...
    ```

  - Missing có thể là:
    * ô rỗng
    * `NA`

- **Xử lý:**

  - **Case biên:**
    * Một cột toàn missing

      $$\Rightarrow min = max = mean = std = 0$$

    * File rỗng

      $$\Rightarrow \text{không có dữ liệu}$$

    * Thiếu cột ở một dòng

      $$\Rightarrow \text{coi là missing}$$

- **Module 1 (Đọc file):**
  - Hàm `readCSV()`
  - Đọc:
    - header
    - dữ liệu từng cột
- **Module 2 (Parse dữ liệu):**
  - Hàm `parseRow()`
  - Xử lý missing
- **Module 3 (Tính thống kê):**
  - Hàm `computeStats()`
  - Với mỗi cột tính:
    - count
    - missing
    - min
    - max
    - mean
    - std
  - Dùng `long double` để giảm sai số
- **Module 4 (Ghi dashboard):**
  - Hàm `writeDashboard()`
  - Mỗi hàng ứng với một cột dữ liệu

- **Output:**

  File `dashboard.csv`

```
column,count,missing,min,max,mean,std
...
```

  Dùng:

```
setprecision(6)
```

10.6.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <string>
#include <iomanip>
#include <cmath>

```

```
using namespace std;

struct ColumnStats {

    string name;

    int count;
    int missing;

    double minVal;
    double maxVal;

    double mean;
    double stdev;
};

vector<string> splitCSV(
    const string& line
) {
    vector<string> parts;

    stringstream ss(line);

    string token;

    while (getline(ss, token, ',')) {
        parts.push_back(token);
    }

    return parts;
}

bool readCSV(
    const string& path,
    vector<string>& headers,
    vector<vector<string>>& rows
) {
    ifstream fin(path);

    if (!fin) return false;

    string line;

    if (!getline(fin, line)) {
        return true;
    }

    headers = splitCSV(line);

```

```
    while (getline(fin, line)) {

        rows.push_back(
            splitCSV(line)
        );
    }

    return true;
}

void parseRow(
    const vector<string>& row,
    vector<vector<double>>& cols,
    vector<vector<bool>>& missing
) {
    int totalCols = cols.size();

    for (int i = 0; i < totalCols; ++i) {

        int idx = i + 1;

        if (idx >= (int)row.size()
            || row[idx].empty()
            || row[idx] == "NA") {

            cols[i].push_back(0);

            missing[i].push_back(true);
        }
        else {

            cols[i].push_back(
                stod(row[idx])
            );

            missing[i].push_back(false);
        }
    }
}

ColumnStats computeStats(
    const string& name,
    const vector<double>& values,
    const vector<bool>& missing
) {
    ColumnStats s;

    s.name = name;

```

```
    s.count = 0;
    s.missing = 0;

    long double sum = 0;

    bool first = true;

    for (size_t i = 0; i < values.size(); ++i) {

        if (missing[i]) {

            ++s.missing;
        }
        else {

            ++s.count;

            sum += values[i];

            if (first) {

                s.minVal = values[i];
                s.maxVal = values[i];

                first = false;
            }
            else {

                if (values[i] < s.minVal) {
                    s.minVal = values[i];
                }

                if (values[i] > s.maxVal) {
                    s.maxVal = values[i];
                }
            }
        }
    }

    if (s.count == 0) {

        s.minVal = 0;
        s.maxVal = 0;

        s.mean = 0;
        s.stdev = 0;

        return s;
    }
```

```

    s.mean = sum / s.count;

    long double variance = 0;

    for (size_t i = 0; i < values.size(); ++i) {

        if (!missing[i]) {

            variance +=
                (values[i] - s.mean)
                *
                (values[i] - s.mean);
        }
    }

    variance /= s.count;

    s.stdev = sqrt(variance);

    return s;
}

bool writeDashboard(
    const string& path,
    const vector<ColumnStats>& stats
) {
    ofstream fout(path);

    if (!fout) return false;

    fout << fixed << setprecision(6);

    fout << "column,count,missing,min,max,mean,std\n";

    for (size_t i = 0; i < stats.size(); ++i) {

        fout << stats[i].name << ","
             << stats[i].count << ","
             << stats[i].missing << ","
             << stats[i].minVal << ","
             << stats[i].maxVal << ","
             << stats[i].mean << ","
             << stats[i].stdev << "\n";
    }

    return true;
}

```

```
int main() {

    vector<string> headers;

    vector<vector<string>> rows;

    if (!readCSV(
        "data.csv",
        headers,
        rows
    )) {
        cerr << "Loi mo file input!\n";
        return 1;
    }

    if (headers.size() <= 1) {

        cerr << "Khong co cot du lieu!\n";
        return 1;
    }

    int numCols = headers.size() - 1;

    vector<vector<double>> cols(numCols);

    vector<vector<bool>> missing(numCols);

    for (size_t i = 0; i < rows.size(); ++i) {

        parseRow(
            rows[i],
            cols,
            missing
        );
    }

    vector<ColumnStats> stats;

    for (int i = 0; i < numCols; ++i) {

        stats.push_back(

            computeStats(
                headers[i + 1],
                cols[i],
                missing[i]
            )
        );
    }
```

```

    if (!writeDashboard(
        "dashboard.csv",
        stats
    )) {
        cerr << "Loi mo file output!\n";
        return 1;
    }

    return 0;
}
```

10.6.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (data.csv):

```
id,a,b
A,10,20
B,30,40
C,50,60
```

Expected Output (dashboard.csv):

```
column,count,missing,min,max,mean,std
a,3,0,10.000000,50.000000,30.000000,16.329932
b,3,0,20.000000,60.000000,40.000000,16.329932
```

Test Case 2 (Có missing)
Input (data.csv):

```
id,x,y
A,10,
B,NA,20
C,30,40
```

Expected Output (dashboard.csv):

```
column,count,missing,min,max,mean,std
x,2,1,10.000000,30.000000,20.000000,10.000000
y,2,1,20.000000,40.000000,30.000000,10.000000
```

Test Case 3 (Toàn missing)
Input (data.csv):

```
id,a,b
A,,
B,NA,NA
```

Expected Output (dashboard.csv):

```
column,count,missing,min,max,mean,std
a,0,2,0.000000,0.000000,0.000000,0.000000
b,0,2,0.000000,0.000000,0.000000,0.000000
```

10.7 Sliding window multi-stat [Trung bình]

10.7.1 Phân tích & Thiết kế (M0)

- **Input:**

  File `input.txt` chứa:

  - Số nguyên n
  - n số thực
  - Kích thước window k

- **Xử lý:**

  Với mỗi vị trí i:

$$[i - k + 1, i]$$

  tính:

  - rolling mean
  - rolling max
  - rolling min
  - **Case biên:**
    * $n = 0$

$$\Rightarrow \text{file output chỉ có header}$$

    * $k \leq 0$

$$\Rightarrow \text{INVALID}$$

    * $k > n$

$$\Rightarrow k = n$$

  - **Module 1 (Đọc file):**
    * Hàm `readInput()`
  - **Module 2 (Rolling Mean):**
    * Hàm `rollingMean()`
    * Dùng sliding window sum:

$$sum = sum - a[i - k] + a[i]$$

- **Module 3 (Rolling Max):**
  - Hàm `rollingMax()`
  - Dùng deque để giữ max
- **Module 4 (Rolling Min):**
  - Hàm `rollingMin()`
  - Dùng deque để giữ min
- **Module 5 (Ghi file):**
  - Hàm `writeOutput()`

- **Output:**

  File `multi_rolling.csv`

```
i,mean_i,max_i,min_i
...
```

  Dùng:

```
setprecision(6)
```

10.7.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <deque>
#include <iomanip>

using namespace std;

bool readInput(
    const string& path,
    vector<double>& a,
    int& k
) {
    ifstream fin(path);

    if (!fin) return false;

    int n;

    fin >> n;

    a.resize(n);

    for (int i = 0; i < n; ++i) {
        fin >> a[i];
```

```
    }

    fin >> k;

    return true;
}

vector<double> rollingMean(
    const vector<double>& a,
    int k
) {
    vector<double> result;

    if (a.empty()) return result;

    if (k > (int)a.size()) {
        k = a.size();
    }

    double sum = 0;

    for (int i = 0; i < k; ++i) {
        sum += a[i];
    }

    result.push_back(sum / k);

    for (size_t i = k; i < a.size(); ++i) {

        sum -= a[i - k];

        sum += a[i];

        result.push_back(sum / k);
    }

    return result;
}

vector<double> rollingMax(
    const vector<double>& a,
    int k
) {
    vector<double> result;

    deque<int> dq;

    for (size_t i = 0; i < a.size(); ++i) {

```

```
        while (!dq.empty()
            && dq.front() <= (int)i - k) {

            dq.pop_front();
        }

        while (!dq.empty()
            && a[dq.back()] <= a[i]) {

            dq.pop_back();
        }

        dq.push_back(i);

        if (i >= k - 1) {
            result.push_back(a[dq.front()]);
        }
    }

    return result;
}

vector<double> rollingMin(
    const vector<double>& a,
    int k
) {
    vector<double> result;

    deque<int> dq;

    for (size_t i = 0; i < a.size(); ++i) {

        while (!dq.empty()
            && dq.front() <= (int)i - k) {

            dq.pop_front();
        }

        while (!dq.empty()
            && a[dq.back()] >= a[i]) {

            dq.pop_back();
        }

        dq.push_back(i);

        if (i >= k - 1) {
            result.push_back(a[dq.front()]);
        }
```

```
    }

    return result;
}

bool writeOutput(
    const string& path,
    const vector<double>& means,
    const vector<double>& maxs,
    const vector<double>& mins
) {
    ofstream fout(path);

    if (!fout) return false;

    fout << fixed << setprecision(6);

    fout << "i,mean_i,max_i,min_i\n";

    for (size_t i = 0; i < means.size(); ++i) {

        fout << i << ","
             << means[i] << ","
             << maxs[i] << ","
             << mins[i] << "\n";
    }

    return true;
}

int main() {

    vector<double> a;

    int k;

    if (!readInput(
        "input.txt",
        a,
        k
    )) {
        cerr << "Loi mo file input!\n";
        return 1;
    }

    if (k <= 0) {

        cerr << "INVALID\n";

```

```
        return 1;
    }

    if (k > (int)a.size()) {
        k = a.size();
    }

    vector<double> means =
        rollingMean(a, k);

    vector<double> maxs =
        rollingMax(a, k);

    vector<double> mins =
        rollingMin(a, k);

    if (!writeOutput(
        "multi_rolling.csv",
        means,
        maxs,
        mins
    )) {
        cerr << "Loi mo file output!\n";
        return 1;
    }

    return 0;
}
```

10.7.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (input.txt):

```
5
1 3 2 5 4
3
```

Expected Output (multi_rolling.csv):

```
i,mean_i,max_i,min_i
0,2.000000,3.000000,1.000000
1,3.333333,5.000000,2.000000
2,3.666667,5.000000,2.000000
```

Test Case 2 ($k > n$)
Input (input.txt):

```
4
```

```
10 20 30 40
10
```

Expected Output (multi_rolling.csv):

```
i,mean_i,max_i,min_i
0,25.000000,40.000000,10.000000
```

Test Case 3 (Mảng rỗng)
Input (input.txt):

```
0

3
```

Expected Output (multi_rolling.csv):

```
i,mean_i,max_i,min_i
```

10.8 Sort + search tổng hợp [Trung bình]

10.8.1 Phân tích & Thiết kế (M0)

- **Input:**

  File `records.csv` chứa:

  ```
  id,score
  ```

  Ví dụ:

  ```
  A,9.5
  B,7.0
  C,9.5
  ```

  File `queries.txt` chứa:

  - Mỗi dòng là một `id` cần tra cứu

- **Xử lý:**

  - **Case biên:**
    - File records rỗng

      $\Rightarrow$ mọi query đều NOT_FOUND

    - Query không tồn tại

      $\Rightarrow$ `NOT_FOUND`

    * Các record có cùng score

      ⇒ giữ nguyên thứ tự ban đầu

  - **Module 1 (Đọc file records):**
    * Hàm `readRecords()`
  - **Module 2 (Đọc query):**
    * Hàm `readQueries()`
  - **Module 3 (Sort records):**
    * Hàm `sortRecords()`
    * Dùng:

```
stable_sort()
```

    * Sort theo score giảm dần
  - **Module 4 (Search):**
    * Hàm `findRecord()`
    * Tìm:
      · rank (1-based)
      · id
      · score
  - **Module 5 (Ghi file):**
    * Hàm `writeOutput()`

- **Output:**

  File `output.txt`

```
rank,id,score
NOT_FOUND
...
```

  Dùng:

```
setprecision(6)
```

10.8.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <algorithm>
#include <iomanip>

```

```
using namespace std;

struct Record {

    string id;

    double score;
};

bool readRecords(
    const string& path,
    vector<Record>& records
) {
    ifstream fin(path);

    if (!fin) return false;

    string line;

    while (getline(fin, line)) {

        if (line.empty()) continue;

        stringstream ss(line);

        string id, scoreStr;

        getline(ss, id, ',');
        getline(ss, scoreStr);

        Record r;

        r.id = id;

        r.score = stod(scoreStr);

        records.push_back(r);
    }

    return true;
}

bool readQueries(
    const string& path,
    vector<string>& queries
) {
    ifstream fin(path);

    if (!fin) return false;
```

```

    string id;

    while (getline(fin, id)) {

        if (!id.empty()) {
            queries.push_back(id);
        }
    }

    return true;
}

void sortRecords(
    vector<Record>& records
) {
    stable_sort(
        records.begin(),
        records.end(),

        [](const Record& a,
           const Record& b) {

            return a.score > b.score;
        }
    );
}

int findRecord(
    const vector<Record>& records,
    const string& id
) {
    for (size_t i = 0; i < records.size(); ++i) {

        if (records[i].id == id) {
            return i;
        }
    }

    return -1;
}

bool writeOutput(
    const string& path,
    const vector<Record>& records,
    const vector<string>& queries
) {
    ofstream fout(path);

```

```
    if (!fout) return false;

    fout << fixed << setprecision(6);

    for (size_t i = 0; i < queries.size(); ++i) {

        int pos =
            findRecord(records, queries[i]);

        if (pos == -1) {

            fout << "NOT_FOUND\n";
        }
        else {

            fout << pos + 1 << ","
                 << records[pos].id << ","
                 << records[pos].score
                 << "\n";
        }
    }

    return true;
}

int main() {

    vector<Record> records;

    if (!readRecords(
        "records.csv",
        records
    )) {
        cerr << "Loi mo file records!\n";
        return 1;
    }

    vector<string> queries;

    if (!readQueries(
        "queries.txt",
        queries
    )) {
        cerr << "Loi mo file queries!\n";
        return 1;
    }

    sortRecords(records);

```

```
    if (!writeOutput(
        "output.txt",
        records,
        queries
    )) {
        cerr << "Loi mo file output!\n";
        return 1;
    }

    return 0;
}
```

10.8.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Thông thường)
Input (records.csv):

```
A,9.5
B,7.0
C,9.5
D,6.0
```

Input (queries.txt):

```
C
B
D
```

Expected Output (output.txt):

```
2,C,9.500000
3,B,7.000000
4,D,6.000000
```

Test Case 2 (Không tìm thấy)
Input (records.csv):

```
A,10
B,20
```

Input (queries.txt):

```
C
A
```

Expected Output (output.txt):

```
NOT_FOUND
2,A,10.000000
```

Test Case 3 (File records rỗng)
Input (records.csv):

Input (queries.txt):

```
A
B
```

Expected Output (output.txt):

```
NOT_FOUND
NOT_FOUND
```

10.9 Phát hiện outlier IQR [Nâng cao]

10.9.1 Phân tích & Thiết kế (M0)

- **Input:**

  File `values.csv` chứa:

  - Mỗi dòng là một số thực
  - Không có missing

- **Xử lý:**

  - Sắp xếp dữ liệu tăng dần
  - Tính:
    $$Q1,\ Q3$$
  - Công thức:
    $$IQR = Q3 - Q1$$
  - Outlier nếu:
    $$x < Q1 - 1.5 \times IQR$$
    hoặc
    $$x > Q3 + 1.5 \times IQR$$
  - **Case biên:**
    * File rỗng
      $$\Rightarrow Q1 = Q3 = IQR = 0$$
    * Tất cả giá trị giống nhau
      $$\Rightarrow IQR = 0$$
    * Không có outlier
      $$\Rightarrow \textit{outliers.csv} \text{ chỉ có header}$$

- **Module 1 (Đọc file):**
  - Hàm `readValues()`
- **Module 2 (Tính percentile):**
  - Hàm `computeQuartile()`
  - Tính:

$$Q1,\ Q3$$

- **Module 3 (Phát hiện outlier):**
  - Hàm `detectOutliers()`
- **Module 4 (Ghi outlier):**
  - Hàm `writeOutliers()`
- **Module 5 (Ghi summary):**
  - Hàm `writeSummary()`

- **Output:**

  File `outliers.csv`

```
index,value
...
```

  File `summary.txt`

```
Q1=<f>
Q3=<f>
IQR=<f>
n_outliers=<n>
```

  Dùng:

```
setprecision(6)
```

10.9.2 Mã nguồn C++11 (Tách module)

```cpp
#include <iostream>
#include <fstream>
#include <vector>
#include <algorithm>
#include <iomanip>

using namespace std;

bool readValues(
    const string& path,
    vector<double>& values
) {
```

```
    ifstream fin(path);

    if (!fin) return false;

    double x;

    while (fin >> x) {
        values.push_back(x);
    }

    return true;
}

double computeQuartile(
    vector<double> sorted,
    double q
) {
    if (sorted.empty()) return 0;

    double pos =
        q * (sorted.size() - 1);

    int left = (int)pos;

    int right = left + 1;

    if (right >= (int)sorted.size()) {
        return sorted[left];
    }

    double frac = pos - left;

    return sorted[left]
           +
           frac
           *
           (sorted[right] - sorted[left]);
}

vector<pair<int, double>> detectOutliers(
    const vector<double>& values,
    double Q1,
    double Q3
) {
    vector<pair<int, double>> outliers;

    double IQR = Q3 - Q1;

    double lower =
```

```
        Q1 - 1.5 * IQR;

    double upper =
        Q3 + 1.5 * IQR;

    for (size_t i = 0; i < values.size(); ++i) {

        if (values[i] < lower
            || values[i] > upper) {

            outliers.push_back(
                {i, values[i]}
            );
        }
    }

    return outliers;
}

bool writeOutliers(
    const string& path,
    const vector<pair<int, double>>& outliers
) {
    ofstream fout(path);

    if (!fout) return false;

    fout << fixed << setprecision(6);

    fout << "index,value\n";

    for (size_t i = 0; i < outliers.size(); ++i) {

        fout << outliers[i].first
             << ","
             << outliers[i].second
             << "\n";
    }

    return true;
}

bool writeSummary(
    const string& path,
    double Q1,
    double Q3,
    double IQR,
    int nOutliers
) {
```

```
    ofstream fout(path);

    if (!fout) return false;

    fout << fixed << setprecision(6);

    fout << "Q1=" << Q1 << "\n";

    fout << "Q3=" << Q3 << "\n";

    fout << "IQR=" << IQR << "\n";

    fout << "n_outliers="
         << nOutliers
         << "\n";

    return true;
}

int main() {

    vector<double> values;

    if (!readValues(
        "values.csv",
        values
    )) {
        cerr << "Loi mo file input!\n";
        return 1;
    }

    vector<double> sorted =
        values;

    sort(
        sorted.begin(),
        sorted.end()
    );

    double Q1 =
        computeQuartile(sorted, 0.25);

    double Q3 =
        computeQuartile(sorted, 0.75);

    double IQR =
        Q3 - Q1;

    vector<pair<int, double>> outliers =
```

```
        detectOutliers(
            values,
            Q1,
            Q3
        );

    if (!writeOutliers(
        "outliers.csv",
        outliers
    )) {
        cerr << "Loi mo file outlier!\n";
        return 1;
    }

    if (!writeSummary(
        "summary.txt",
        Q1,
        Q3,
        IQR,
        outliers.size()
    )) {
        cerr << "Loi mo file summary!\n";
        return 1;
    }

    return 0;
}
```

10.9.3 Kế hoạch kiểm thử (Test Cases)

Test Case 1 (Có outlier)
Input (values.csv):

```
10
11
12
13
14
100
```

Expected Output (outliers.csv):

```
index,value
5,100.000000
```

Expected Output (summary.txt):

```
Q1=11.250000
Q3=13.750000
IQR=2.500000
n_outliers=1
```

Test Case 2 (Không có outlier)
Input (values.csv):

```
1
2
3
4
5
```

Expected Output (outliers.csv):

```
index,value
```

Expected Output (summary.txt):

```
Q1=2.000000
Q3=4.000000
IQR=2.000000
n_outliers=0
```

Test Case 3 (File rỗng)
Input (values.csv):

```

```

Expected Output (outliers.csv):

```
index,value
```

Expected Output (summary.txt):

```
Q1=0.000000
Q3=0.000000
IQR=0.000000
n_outliers=0
```

10.10 Mini-project tự do [Nâng cao]

10.10.1 Chọn hướng C (Text Processing Pipeline)

10.10.2 Mục tiêu project

Xây dựng pipeline xử lý văn bản hoàn chỉnh:

$$normalize \rightarrow tokenize \rightarrow TF \rightarrow IDF \rightarrow TF\text{-}IDF \rightarrow similarity$$

Project đọc 2 file văn bản:

- `text1.txt`
- `text2.txt`

Sau đó:

- chuẩn hoá dữ liệu
- tách token
- tính TF-IDF
- tìm top-10 từ quan trọng nhất
- tính cosine similarity giữa 2 văn bản

10.10.3 Checklist M0

✓ Phân tích Input / Output

✓ Chia module rõ ràng

✓ Có xử lý case biên

✓ Tách hàm riêng cho từng bước

✓ Có README

✓ Có test case thường

✓ Có test case biên

✓ Output format rõ ràng

10.10.4 Cấu trúc project

```
project/
 main.cpp
 io.h
 io.cpp
 text_processing.h
 text_processing.cpp
 tfidf.h
 tfidf.cpp
 similarity.h
 similarity.cpp

 text1.txt
 text2.txt

 top10.csv
 similarity.txt
 README.md
```

10.10.5 Phân tích & Thiết kế

- **Input:**
  - Hai file văn bản:

```
text1.txt
text2.txt
```

- **Xử lý:**
  - Normalize:
    - lowercase
    - bỏ ký tự đặc biệt
  - Tokenize:
    - tách từ bằng stringstream
  - TF:
$$TF(t) = \frac{\text{count}(t)}{\text{total terms}}$$
  - IDF:
$$IDF(t) = \log \frac{N}{df(t)}$$
  - TF-IDF:
$$TFIDF(t) = TF(t) \times IDF(t)$$
  - Cosine Similarity:
$$sim(A, B) = \frac{A \cdot B}{|A||B|}$$

- **Case biên:**
  - File rỗng
$$\Rightarrow similarity = 0$$
  - Không có token hợp lệ
$$\Rightarrow top10.csv \text{ chỉ có header}$$
  - Hai file giống hệt nhau
$$\Rightarrow similarity = 1$$

10.10.6 Module 1 – io.h

```
#ifndef IO_H
#define IO_H

#include <string>

using namespace std;

string readText(const string& path);

#endif
```

10.10.7 Module 2 – io.cpp

```
#include "io.h"
#include <fstream>

string readText(const string& path) {

    ifstream fin(path);

    string line, text;

    while (getline(fin, line)) {
        text += line + " ";
    }

    return text;
}
```

10.10.8 Module 3 – text_processing.h

```
#ifndef TEXT_PROCESSING_H
#define TEXT_PROCESSING_H

#include <vector>
#include <string>

using namespace std;

string normalize(const string& text);

vector<string> tokenize(const string& text);

#endif
```

10.10.9 Module 4 – text_processing.cpp

```
#include "text_processing.h"

#include <sstream>
#include <cctype>

string normalize(const string& text) {

    string s = text;

    for (size_t i = 0; i < s.size(); ++i) {

        if (isalnum(s[i])) {
            s[i] = tolower(s[i]);
        }
        else {
            s[i] = ' ';
        }
    }

    return s;
}

vector<string> tokenize(
    const string& text
) {
    vector<string> tokens;

    stringstream ss(text);

    string token;

    while (ss >> token) {
        tokens.push_back(token);
    }

    return tokens;
}
```

10.10.10 Module 5 – tfidf.h

```
#ifndef TFIDF_H
#define TFIDF_H

#include <map>
#include <vector>
#include <string>
```

```

using namespace std;

map<string,double>
computeTF(
    const vector<string>& tokens
);

map<string,double>
computeTFIDF(
    const vector<string>& d1,
    const vector<string>& d2
);

#endif
```

10.10.11 Module 6 – tfidf.cpp

```
#include "tfidf.h"

#include <cmath>

map<string,double>
computeTF(
    const vector<string>& tokens
) {
    map<string,double> tf;

    for (size_t i = 0; i < tokens.size(); ++i) {
        tf[tokens[i]]++;
    }

    for (auto& x : tf) {
        x.second /= tokens.size();
    }

    return tf;
}

map<string,double>
computeTFIDF(
    const vector<string>& d1,
    const vector<string>& d2
) {
    map<string,double> tf1 =
        computeTF(d1);

    map<string,double> tfidf;
```

```

    for (auto x : tf1) {

        int df = 0;

        for (size_t i = 0; i < d1.size(); ++i) {

            if (d1[i] == x.first) {
                df++;
                break;
            }
        }

        for (size_t i = 0; i < d2.size(); ++i) {

            if (d2[i] == x.first) {
                df++;
                break;
            }
        }

        double idf =
            log(2.0 / df);

        tfidf[x.first] =
            x.second * idf;
    }

    return tfidf;
}
```

10.10.12 Module 7 – similarity.h

```
#ifndef SIMILARITY_H
#define SIMILARITY_H

#include <vector>
#include <string>

using namespace std;

double cosineSimilarity(
    const vector<string>& d1,
    const vector<string>& d2
);

#endif
```

10.10.13 Module 8 – similarity.cpp

```
#include "similarity.h"

#include <map>
#include <cmath>

double cosineSimilarity(
    const vector<string>& d1,
    const vector<string>& d2
) {
    map<string,int> f1, f2;

    for (size_t i = 0; i < d1.size(); ++i) {
        f1[d1[i]]++;
    }

    for (size_t i = 0; i < d2.size(); ++i) {
        f2[d2[i]]++;
    }

    double dot = 0;

    for (auto x : f1) {

        dot +=
            x.second
            *
            f2[x.first];
    }

    double norm1 = 0;
    double norm2 = 0;

    for (auto x : f1) {
        norm1 += x.second * x.second;
    }

    for (auto x : f2) {
        norm2 += x.second * x.second;
    }

    norm1 = sqrt(norm1);
    norm2 = sqrt(norm2);

    if (norm1 == 0 || norm2 == 0) {
        return 0;
    }

```

```
    return dot / (norm1 * norm2);
}
```

10.10.14 Module 9 – main.cpp

```
#include <iostream>
#include <fstream>
#include <vector>
#include <algorithm>
#include <iomanip>

#include "io.h"
#include "text_processing.h"
#include "tfidf.h"
#include "similarity.h"

using namespace std;

int main() {

    string text1 =
        readText("text1.txt");

    string text2 =
        readText("text2.txt");

    vector<string> d1 =
        tokenize(
            normalize(text1)
        );

    vector<string> d2 =
        tokenize(
            normalize(text2)
        );

    map<string,double> tfidf =
        computeTFIDF(d1, d2);

    vector<pair<string,double>> arr(
        tfidf.begin(),
        tfidf.end()
    );

    sort(
        arr.begin(),
        arr.end(),

```

```
        [](const pair<string,double>& a,
           const pair<string,double>& b) {

            return a.second > b.second;
        }
    );

    ofstream fout("top10.csv");

    fout << fixed << setprecision(6);

    fout << "word,tfidf\n";

    int limit =
        min(10, (int)arr.size());

    for (int i = 0; i < limit; ++i) {

        fout << arr[i].first
             << ","
             << arr[i].second
             << "\n";
    }

    double sim =
        cosineSimilarity(d1, d2);

    ofstream fs("similarity.txt");

    fs << fixed << setprecision(6);

    fs << "similarity="
       << sim
       << "\n";

    return 0;
}
```

10.10.15 README

```
BUILD

g++ main.cpp io.cpp text_processing.cpp tfidf.cpp similarity.cpp -o app

RUN

./app

```

```
INPUT

text1.txt
text2.txt

OUTPUT

top10.csv
similarity.txt
```

10.10.16 Kế hoạch kiểm thử

Test Case 1 (Thông thường)
Input (text1.txt):

```
AI is powerful AI system
```

Input (text2.txt):

```
AI system for machine learning
```

Expected Output (similarity.txt):

```
similarity=0.447214
```

Test Case 2 (Case biên – file rỗng)
Input (text1.txt):

```

```

Input (text2.txt):

```

```

Expected Output (similarity.txt):

```
similarity=0.000000
```